Model Predictive Control
An Introduction

Lecture Notes

İbrahim Küçükdemiral

March 23, 2026

Contents

Foreword **13**

I Fundamentals **1**

1 Introduction to Model Predictive Control **3**

1.1 From Classical Control to Optimisation-Based Control 3

1.1.1 Classical Control Revisited 3

1.1.2 The MPC Approach: Optimisation-Based Control 4

1.2 Conceptual Framework: The Racing Car Analogy 5

1.3 Receding Horizon Control (RHC) 7

1.3.1 The Algorithm 7

1.4 Why Use Model Predictive Control? 9

1.4.1 Systematic Handling of Constraints 9

1.4.2 Future Knowledge and Preview Capability 9

1.4.3 Multivariable Control and Complex Objectives 11

1.5 Operation of MPC 12

1.5.1 The Prediction Model 12

1.5.2 The Objective (Cost Function) 12

1.5.3 Constraint Handling 14

1.5.4 The Receding Horizon Principle 15

1.6 Applications of Model Predictive Control 17

1.7 Case Study: Energy-Efficient Building Control 19

1.7.1 Model Predictive Control for a One-Zone Building 19

1.8 A First MPC Example in MATLAB 23

1.9 Chapter Summary 26

2 Optimisation: The Engine of MPC 31

2.1 What is Optimisation? 31

2.2 The Anatomy of an Optimisation Problem 32

2.3 Illustrative Examples 33

2.4 Ingredients of Multi-Variable Optimisation 35

2.4.1 Problem Setting: The Decision Vector 35

2.4.2 Understanding Vector-to-Scalar Mapping: The *cost* function 36

2.4.3 Constraints and the Feasible Set 37

2.4.4 Standard Form: Minimisation vs. Maximisation 38

2.5 Terminology in Optimisation 39

2.5.1 Convex Sets 40

2.5.2 Convex Functions 44

2.6 Classification of Optimisation Problems 44

2.6.1 Classification I: Existence of Constraints 45

2.6.2 Classification II: Mathematical Structure 51

2.7 Software Tools for Optimisation 64

2.7.1 Modeling Languages (Parsers) 64

2.7.2 Numerical Solvers 64

2.8 Solving Optimisation Problems: From Theory to Code 65

2.8.1 Economic MPC for Smart Grids 65

2.9 Key Takeaways 68

3 State-Space Representation of Dynamic Systems 73

3.1 What is state-space, and why is it central to MPC? 74

3.2 Nonlinear Systems and Local Linearisation 81

3.2.1 Nonlinear state-space models 81
3.2.2 Equilibrium points 82
3.2.3 Local approximation via Taylor expansion 83
3.3 Obtaining DT Realisation of CT LTI State-space systems 87
3.3.1 Exact Discretisation of LTI Systems 87
3.4 Discretisation of Forced Linear Systems 88
3.5 Numerical Discretisation of Nonlinear Systems 89
3.5.1 Forward Euler Method 89
3.5.2 Runge–Kutta 4 (RK4) Method 89
3.6 Stability of Discrete-Time Systems 91
3.6.1 Geometric Interpretation: The Unit Circle 93
3.6.2 Lyapunov Stability and the Stein Equation 93

4 Controllability & Observability 97

4.1 Controllability 97
4.1.1 Derivation of the Rank Condition 98
4.1.2 Controllability in MATLAB 99
4.2 Observability 100
4.2.1 The Observability Matrix 101
4.2.2 Derivation of the Rank Condition 103
4.2.3 Observability in MATLAB 105

II Optimal Control and Optimal Estimation 109

5 LQR Design via Quadratic Programming 111

5.1 Problem Formulation 111
5.2 Sparse Quadratic Programming Formulation 112
5.3 Solution via KKT Conditions 114
5.3.1 The KKT System 114

5.3.2 Solving the KKT System (The Riccati Recursion) 115

5.4 Infinite-Horizon LQR for Stabilisation 118

5.4.1 Convergence of the Riccati Recursion 118

5.5 The Discrete Algebraic Riccati Equation (DARE) and Infinite Horizon LQR . . . 119

5.5.1 The Optimal Controller and Guaranteed Stability 120

5.6 LQR for Non-Zero Setpoint Tracking 123

5.6.1 Error Dynamics and the Deviation System 123

5.6.2 The Affine Control Law 124

5.7 Chapter Summary 127

6 State Estimation and the Kalman Filter 131

6.1 Background: Stochastic Systems and Probability 131

6.1.1 Random Variables and Expectation 132

6.1.2 The Covariance Matrix 132

6.1.3 The Gaussian Distribution 133

6.1.4 White Noise 133

6.2 Problem Formulation 134

6.2.1 The Stochastic System Model 134

6.2.2 Statistical Assumptions 134

6.2.3 The Estimation Objective 135

6.3 Derivation of the Kalman Filter 135

6.3.1 The Estimator Structure 136

6.4 The Steady-State Kalman Filter 139

6.4.1 Convergence Conditions 139

6.4.2 The Discrete Algebraic Riccati Equation (DARE) 140

6.4.3 The Constant Gain Implementation 140

6.5 Duality between Estimation and Control 140

6.5.1 Implications for Implementation 141

6.6 Practical Tuning: Selecting W and V 141

6.6.1 Measurement Noise (V) 142

6.6.2 Process Noise (W) 142

III Model Predictive Control 147

7 Linear Model Predictive Control 149

7.1 Motivation: The Need for Constraints 149

7.2 Problem Formulation 150

7.3 The Receding Horizon Principle 151

7.4 Efficient Implementation via YALMIP 153

7.5 Stability and the Terminal Cost 153

7.5.1 The YALMIP Workflow 154

7.6 The Effect of Horizon (N) and Terminal Cost (P) 158

7.7 Recursive Feasibility and Formal Stability Guarantees$^{(*)}$ 160

7.7.1 The Gap in the Argument 160

7.7.2 Positive Invariant Sets 161

7.7.3 Computing $\mathcal{X}_f$: A Scalar Example 161

7.7.4 The Complete MPC Problem with Terminal Constraint 162

7.7.5 Recursive Feasibility 163

7.7.6 The Stability Proof 164

7.7.7 What This Means in Practice 165

7.7.8 Why Terminal Set Size Matters 166

7.7.9 Computing $\mathcal{X}_f$ for Multi-State Systems Using MPT3 167

8 Output Feedback and Offset-Free MPC 171

8.1 Introduction 171

8.2 The Disturbance Model 172

8.3 State and Disturbance Estimation 172

8.4 The Target Calculator 173

8.4.1 Method 1: Linear Solution (Unconstrained) 173

8.4.2 Method 2: Constrained Optimisation (Robust) 174

8.5 The MPC Formulation 174

8.5.1 Control Topology 174

8.5.2 Optimisation Problem 176

8.5.3 Handling Infeasibility: Hard vs Soft Constraints 177

9 Simulink Implementation of Model Predictive Control(*) 209

9.1 General Implementation Guidance 209

9.1.1 Why the MPC Toolbox Is Not Required 209

9.1.2 The Code-Generation Barrier and How to Overcome It 210

9.1.3 Managing State Across Time Steps: Persistent Variables 210

9.1.4 Building YALMIP Optimiser Objects: One-Time Compilation 211

9.1.5 Recommended Architecture 212

9.1.6 Eliminating the Algebraic Loop 213

9.1.7 Solver Recommendations 213

9.1.8 Summary of the Implementation Workflow 213

9.2 Case Study: Single-Zone Building Heating 215

9.2.1 System Description 215

9.2.2 File Structure 215

9.2.3 Step 1 — `build_mpc_optimizers.m` 216

9.2.4 Step 2 — `get_profiles.m` 218

9.2.5 Step 3 — `plant_step.m` 219

9.2.6 Step 4 — `mpc_controller_step.m` 219

9.2.7 Step 5 — Building the Simulink Model 221

9.2.8 Step 6 — Running the Simulation and Post-Processing 222

9.2.9 Summary 225

10 Robust MPC and Tube MPC(*) **229**

10.1 The Problem: Disturbances and Constraint Violations 229

10.2 Two Approaches to Robust MPC 230

10.2.1 Min–Max MPC 230

10.2.2 Tube MPC: The Key Idea 231

10.3 Set Operations: Minkowski Sum and Pontryagin Difference 231

10.3.1 Minkowski Sum 231

10.3.2 Pontryagin Difference 232

10.4 Tube MPC: The Full Development 233

10.4.1 Step 1: Decompose the System 233

10.4.2 Step 2: Derive the Error Dynamics 234

10.4.3 Step 3: The Minimal Robust Positively Invariant Set 234

10.5 Step 4: Constraint Tightening 236

10.6 Step 5: The Tube MPC Optimisation Problem 237

10.7 Tube MPC Architecture 237

10.8 Worked Example: Tube MPC for a Scalar System 238

10.9 Worked Example: Two-State System 239

10.10 Guarantees of Tube MPC 241

10.11 Tube MPC vs Standard MPC: A Comparison 242

10.12 Implementation in MATLAB 242

10.13 Computing the mRPI Set with MPT3 244

10.14 Visualising the Tube 245

10.15 Summary and Practical Guidelines 247

11 Course Conclusion and Further Topics(*) **249**

11.1 Rigorous Stability Analysis 249

11.2 Robust MPC 250

11.3 Nonlinear MPC (NMPC) 250

11.4 Real-Time Implementation and Embedded MPC 253

11.5 Data-Driven and Learning-Based MPC 254

11.5.1 Learning Dynamics: Gaussian Processes and Neural Networks 254

11.5.2 Linear Embeddings: Koopman and DMD 255

11.5.3 Direct Data-Driven Control (Data-Enabled MPC) 255

11.5.4 Learning MPC (LMPC) 256

Appendix A Linear Algebra: A Self-Study Reference 259

A.1 Scalars, Vectors, and Matrices 259

A.2 Special Matrices 260

A.3 Matrix Addition and Subtraction 261

A.4 Matrix Transpose 262

A.5 Matrix–Matrix Multiplication 263

A.6 Matrix–Vector Multiplication 265

A.7 The Inverse of a Matrix 266

A.8 The Determinant 268

A.9 The Trace 269

A.10 Eigenvalues and Eigenvectors 269

A.11 Putting It All Together: A State-Space Example 272

A.12 Quick Reference: MATLAB Commands 274

Appendix B Probability and Statistics: A Self-Study Reference 275

B.1 Why Probability in Control? 275

B.2 Sample Spaces, Outcomes, and Events 276

B.3 Conditional Probability and Independence 277

B.4 Bayes' Theorem 278

B.5 Random Variables 278

B.5.1 Discrete Random Variables 278

B.5.2 Continuous Random Variables 279

B.6 Key Probability Distributions 280

B.6.1 Uniform Distribution 280

B.6.2 The Gaussian (Normal) Distribution 280

B.6.3 Exponential and Other Useful Distributions 281

B.7 Expectation, Variance, and Standard Deviation 282

B.8 Covariance and Correlation 283

B.9 The Multivariate Gaussian Distribution 283

B.10 Linear Transformations of Gaussian Variables 285

B.11 Stochastic Processes 286

B.12 The Law of Large Numbers and the Central Limit Theorem 287

B.13 Monte Carlo Methods 289

B.14 Connecting Probability to the Kalman Filter 290

B.15 Probability in Reinforcement Learning 291

B.16 Quick Reference: MATLAB Commands for Probability 292

Foreword

Model Predictive Control (MPC) is an optimisation-based control methodology that has become one of the most widely used advanced control methods. By explicitly exploiting a dynamical model of the system, MPC predicts future behaviour and determines control actions by solving a constrained optimisation problem in real time. This predictive and optimisation-driven nature distinguishes MPC fundamentally from classical feedback controllers such as PID or lead–lag compensators, which rely on fixed control laws designed offline.

Initially developed and deployed in the process industries, MPC has matured into a general-purpose control framework applicable across a wide range of engineering domains. Advances in numerical optimisation, computational power, and embedded hardware have enabled its adoption in robotics, autonomous vehicles, aerospace systems, power electronics, renewable energy systems, and smart infrastructure. In many of these applications, the ability of MPC to handle multivariable dynamics, operational constraints, and future information in a systematic manner is essential.

These lecture notes have been prepared for the Model Predictive Control module delivered as part of the MSc Applied Instrumentation and Control programme at Glasgow Caledonian University. The material has been designed with a strongly practical emphasis. The primary objective is to introduce MPC in a way that allows students to apply the techniques they learn directly to realistic engineering problems, rather than treating MPC as a purely theoretical topic. Wherever possible, abstract concepts are motivated using physical intuition, illustrative examples, and implementation-oriented discussions.

The notes are structured so that they can be followed either as guided lecture material or as a self-contained resource for independent study. A student who works through the notes sequentially should be able to acquire a working knowledge of MPC sufficient to model systems, formulate predictive control problems, and implement controllers using modern computational tools. To accommodate delivery within approximately three weeks, mathematical depth is deliberately balanced against clarity and applicability.

A note on the $^{(*)}$ symbol. Chapters and sections marked with $^{(*)}$ in their title contain supplementary material that is not covered during the timetabled lectures. Specifically, this includes the formal stability theory section in the MPC chapter (recursive feasibility, terminal sets, and the Lyapunov stability proof), the Simulink implementation chapter (Chapter 9), the Robust and Tube MPC chapter (Chapter 10), and the course conclusion and further topics chapter (Chapter 11). These chapters and sections are included for completeness and for students who wish to deepen their understanding independently. The core assessed material is contained in the unmarked chapters and sections.

Beyond the core assessed material, the notes include several supplementary chapters that ex-

tend the scope of the treatment. Robust and Tube MPC is developed in full, including set operations, constraint tightening, worked examples, and MATLAB implementations. The Simulink implementation chapter demonstrates a complete closed-loop MPC simulation for a building energy system. The concluding chapter introduces nonlinear MPC, learning-based MPC, and data-driven predictive control at a conceptual level, contextualising MPC within current research and industrial practice and providing a foundation for further study.

A basic background in SISO control systems, state-space representations, linear algebra, and numerical computation is assumed. Familiarity with MATLAB is strongly recommended, as examples and implementations throughout the notes rely on MATLAB-based workflows. Several examples also use the YALMIP optimisation toolbox, which is freely available at `https://yalmip.github.io`. With this background, the material should be accessible to postgraduate students in control, instrumentation, and related engineering disciplines. Two self-study appendices are included at the end of these notes: one reviewing the required linear algebra and one covering the probability theory used in the Kalman filter chapters. Distance learning students are encouraged to consult these appendices as needed.

The opening chapter introduces Model Predictive Control and contrasts it with classical control methodologies. The motivation for MPC is established by highlighting the limitations of traditional feedback controllers when faced with constraints, multivariable interactions, and the need for anticipation. The receding horizon principle is introduced and explained using intuitive analogies, and the core components of MPC—models, objectives, constraints, and feedback—are discussed. The chapter concludes with a detailed application example drawn from energy-efficient building control, demonstrating how MPC exploits disturbance forecasts and operational constraints in practice.

The second chapter focuses on optimisation, which forms the computational engine of MPC. The fundamental elements of optimisation problems are introduced, including decision variables, cost functions, and constraints. Key concepts such as convex sets, convex functions, and problem classifications are discussed with emphasis on intuition rather than formal proofs. Practical examples illustrate how optimisation problems arise naturally in engineering contexts and how they are solved using modern numerical tools.

The third chapter develops the state-space framework that underpins predictive control. Continuous-time and discrete-time models are reviewed, and methods for discretising linear and nonlinear systems are presented. The treatment includes exact discretisation of linear systems as well as numerical schemes for nonlinear dynamics. Basic notions of discrete-time stability are also introduced, providing essential background for predictive control design.

The fourth chapter revisits controllability and observability from a state-space perspective. These properties are examined in the context of predictive control and state estimation, and their practical significance is emphasised. Rank conditions are derived and interpreted, and computational verification using MATLAB is demonstrated.

The fifth chapter introduces linear quadratic regulation as a finite-horizon optimisation problem. By formulating LQR in this way, the close conceptual relationship between optimal control and MPC becomes more apparent. The solution via the Karush–Kuhn–Tucker conditions and the resulting Riccati recursion are discussed, along with infinite-horizon LQR and stabilisation. The chapter also addresses non-zero setpoint tracking, establishing ideas that are later reused in MPC formulations.

The sixth chapter is devoted to optimal state estimation and the Kalman filter. Stochastic system models and noise assumptions are introduced, followed by the derivation of the Kalman filter as an optimal estimator. The duality between estimation and control is highlighted, and practical issues related to tuning noise covariance matrices are discussed. This material provides the foundation for output-feedback MPC.

The seventh chapter presents the standard formulation of linear Model Predictive Control. The finite-horizon constrained optimisation problem is developed explicitly, and the receding horizon implementation is revisited from a computational standpoint. Efficient implementation strategies using optimisation modelling tools are demonstrated, and the influence of design parameters such as prediction horizon length and terminal cost weighting is discussed.

The eighth chapter extends MPC to output-feedback and offset-free formulations. Disturbance modelling and joint state–disturbance estimation are introduced, along with target calculation methods for reference tracking. Practical issues such as feasibility, soft constraints, and robustness to modelling errors are addressed, reflecting challenges commonly encountered in real-world applications.

The ninth chapter addresses the practical implementation of MPC within the Simulink environment. A general implementation framework is developed, covering the construction of YALMIP optimiser objects, the use of persistent variables for managing state across time steps, and strategies for overcoming code-generation constraints. The chapter explains how to assemble a complete closed-loop simulation without relying on the MATLAB MPC Toolbox, and demonstrates the methodology through a detailed case study of economic MPC applied to a four-zone building heating system. This case study integrates an augmented Kalman filter for joint state and disturbance estimation, an economic steady-state target calculator, and a deviation-form MPC regulator, and includes guidance on common implementation pitfalls and their remedies.

The tenth chapter develops robust MPC for systems subject to bounded disturbances. The limitations of standard MPC in the presence of model uncertainty are discussed, and the Tube MPC framework is presented in detail. The chapter introduces the Minkowski sum and Pontryagin difference as the essential set operations, derives the constraint tightening mechanism, and develops the complete Tube MPC optimisation problem. Worked examples for both scalar and multi-state systems are provided, along with MATLAB implementations using YALMIP and MPT3.

The concluding chapter provides a broader perspective on MPC and outlines directions beyond the core material. Topics such as nonlinear MPC, real-time and embedded implementation, and data-driven and learning-based predictive control are briefly introduced. The aim is to situate the material within the wider current state of control research and practice and to encourage further study of the subject.

Taken as a whole, these notes aim to connect theory and practice in Model Predictive Control. They are intended to equip students not only with an understanding of the underlying principles, but also with the confidence and practical skills required to apply MPC to complex, constrained engineering systems.

Part I

Fundamentals

Chapter 1

Introduction to Model Predictive Control

For over fifty years, control engineering relied on a simple rule: measure the error, then correct it. Classical feedback controllers such as PID react to what has already happened. Model Predictive Control (MPC) takes a fundamentally different approach: it uses a mathematical model of the system to predict future behaviour and computes control actions by solving a constrained optimisation problem at each time step.

MPC originated in the process industries [15] but is now applied across robotics, autonomous vehicles, aerospace, renewable energy, building climate control, biomedical systems, and finance. The common thread in these applications is that the system has multiple interacting variables, hard physical constraints, and benefits from anticipating future conditions rather than merely reacting to past errors.

1.1 From Classical Control to Optimisation-Based Control

1.1.1 Classical Control Revisited

Model Predictive Control (MPC) must first be placed within the framework of classical control theory in order to be understood. The goal of a typical control loop, illustrated in Figure 1.1, is to adjust a system (or plant) so that its outputs follow a desired reference.

In the classical approach, a controller is designed, usually represented as a transfer function $G_c(s)$ in the Laplace domain. On its own, this controller is a dynamic system. In order to produce an input for the plant, it processes measurements and reference signals. The goal of the design process is to create a dynamic system (the controller) with characteristics like stability, quick rise time, and little overshoot in the closed-loop transfer function.

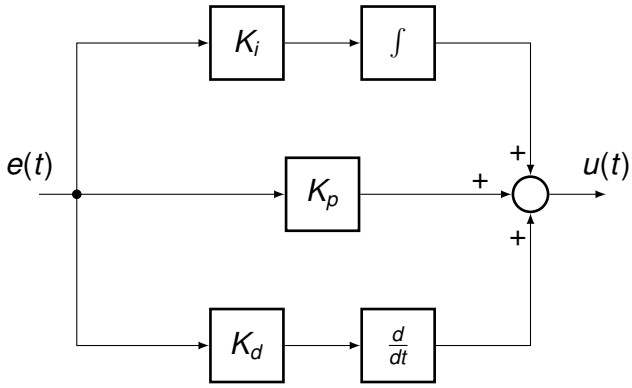


Figure 1.2: The internal structure of a PID controller. The error (e) is processed in parallel: it is integrated, differentiated, and scaled by specific gains before being summed to produce the control input (u).

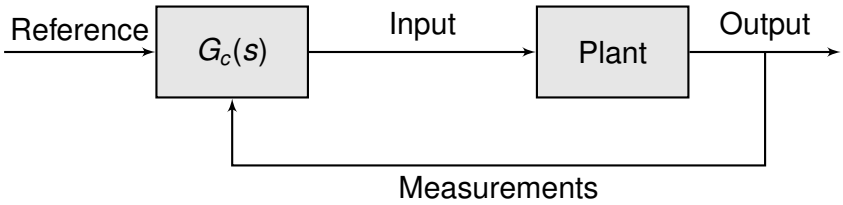


Figure 1.1: The Classical Control Loop. The controller is a pre-designed dynamic block (e.g., PID) that maps error to input of the controlled system called *plant*.

To give an example, in classical control (e.g., Proportional-Integral-Derivative or PID), the engineer explicitly defines *how* the controller calculates the input. The internal structure of a PID controller is shown in Figure 1.2. A PID controller is explicitly instructed to:

1. Calculate the error, $e(t) = r(t) - y(t)$ (Reference - Output).
2. Integrate the error, $\int e(t)dt$.
3. Differentiate the error, $\frac{d}{dt}e(t)$.
4. Multiply these terms by specific gains (K_p, K_i, K_d) and sum them to obtain control signal,

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{d}{dt} e(t)$$

This sum constitutes the control input that is applied to the plant. The behaviour is fixed by the mathematical structure of the controller. That means when one knows $e(t)$ numerically and once the control gains (K_p, K_i, K_d) are selected, the control signal applied to the plant can be precisely calculated in closed form.

1.1.2 The MPC Approach: Optimisation-Based Control

As shown in Figure 1.3, MPC represents a fundamental shift in this philosophy. In an MPC architecture, the explicit controller block is replaced by an *optimisation routine*. Here we call it *Optimiser*. This is mostly an embedded digital computer that runs an algorithm.

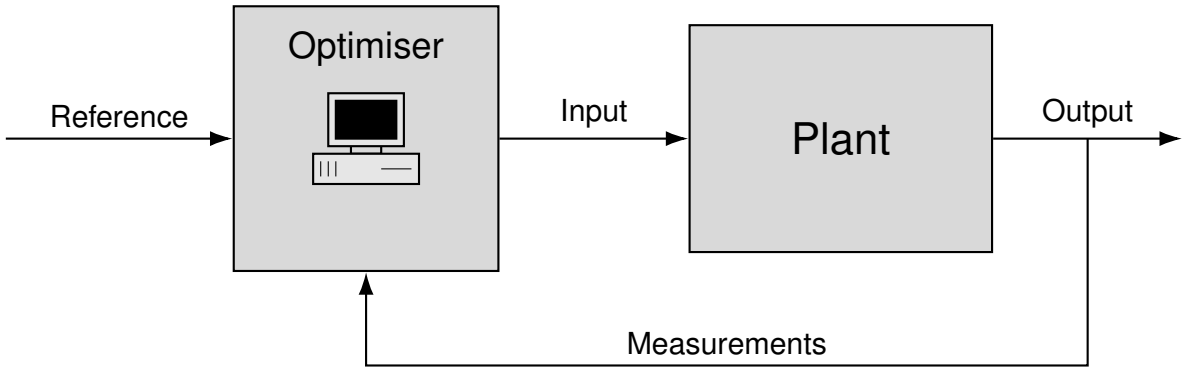


Figure 1.3: The MPC Control Loop. The fixed controller is replaced by an on-line optimiser that uses a model and constraints of the plant as well as predefined objective function to solve for the optimal control input.

Instead of specifying *how* to calculate the input mathematically, MPC specifies *what* the controller should achieve. That means, there is no controller structure specified but the optimiser directly generates the control signal. If the error signal, $e(t)$ is known, we do not know what the controller will generate. The signal generated by the optimiser that goes into the *Plant* depends on many factors and conditions of the controller, measured signal, dynamics of the plant, reference signal, environmental conditions, and their current, past and predicted values.

Generally, the engineer provides the optimiser with four key components:

1. **A Model (Plant dynamics):** Information on how the plant reacts to inputs and disturbances (typically a state-space model derived from first principles like Newton's second law of motion, Kirchoff's voltage and current laws for electrical circuits).
2. **Measurements:** The current state $x(t)$ of the system and historical data. This could be either a measured signal or estimated one.
3. **Objectives:** A mathematical function *cost* representing the goal (e.g., tracking a reference or minimising energy consumption).
4. **Constraints:** Limitations on the system (e.g., maximum valve opening, safety limits, temperature limits, rate bounds of the actuator etc.).

The optimiser processes this information to select the input that brings the plant to ideal form by minimising an objective (cost) while strictly respecting constraints.

1.2 Conceptual Framework: The Racing Car Analogy

To intuitively grasp the mechanics of MPC, consider the challenge of driving a racing car around a circuit in the minimum possible time. To design an effective controller for this vehicle, one must first formalise the environment and the vehicle's behaviour:

- **The Model (Dynamics):** The controller must understand how the car responds to inputs—how steering, acceleration, and braking translate into changes in position and velocity.

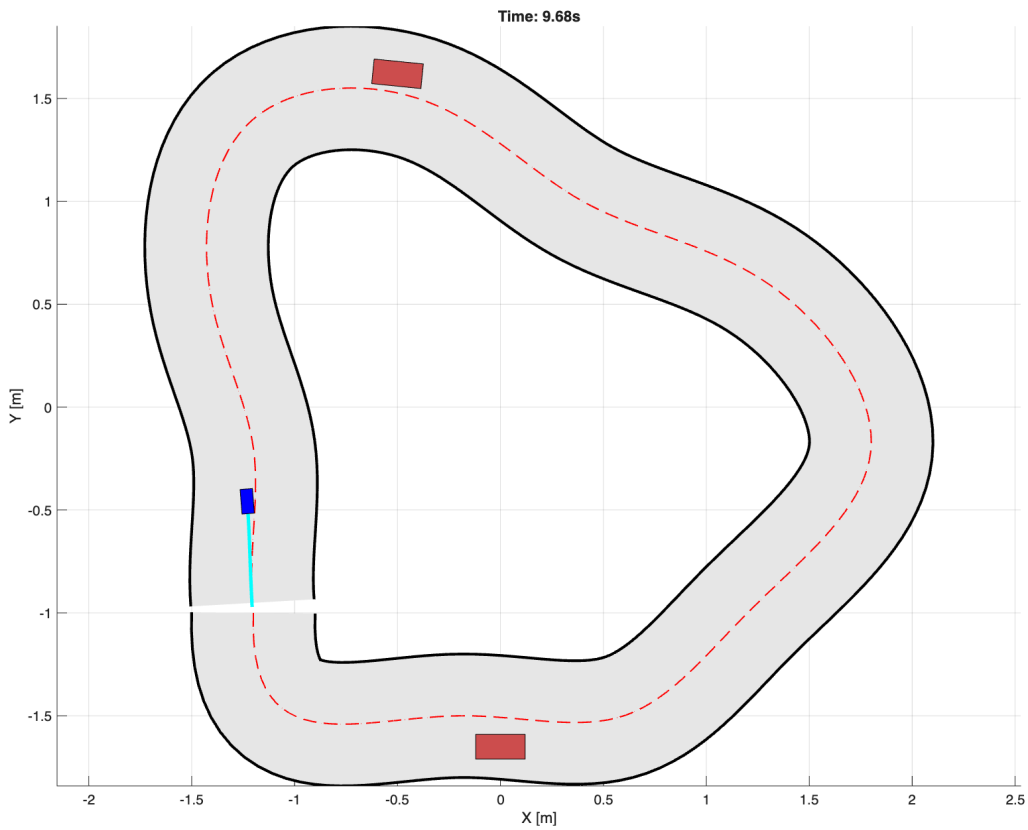


Figure 1.4: Model Predictive Control for trajectory optimisation in a racing scenario. The controller plans an optimal path (cyan) over the prediction horizon that minimises a cost function (tracking the reference line) while satisfying state constraints (track edges) and avoiding obstacles (red cars).

- **The Measurements (Feedback):** To plan the optimal trajectory, the controller must know the car's current state at every instant. This includes the position (x, y) relative to the track boundaries, the velocity (v) to judge braking distances, and the heading angle (θ) to align with the apex of the next corner.
- **The Objective (Cost Function):** The primary goal is to minimise the lap time, often whilst secondary goals, such as minimising fuel consumption or tyre wear, are also considered.
- **The Constraints:** The car must remain within the track boundaries, stay below the engine's maximum power output, and respect the physical limits of tyre traction to avoid skidding.

Consider racing track shown in Figure 1.4. A human driver naturally performs predictive control. Upon approaching a corner, the driver does not simply react to the car's current position. Instead, they look ahead—establishing a *prediction horizon*. They anticipate the car's future trajectory based on its current momentum and adjust their inputs *proactively*. For example, a driver will pull to the outside of the track to flatten the turn radius, anticipating the geometry of the road ahead before they even reach the apex.

Imagine if the driver planned the entire lap at the starting line based on a map, closed their eyes, and executed that plan blindly. This "open-loop" execution would inevitably result in a crash. This failure occurs because the internal model is never perfect. There are several reasons for this failure:

- **Model Mismatch:** Tyre friction changes as the rubber heats up, or oil on the track alters

the grip levels.

- **Disturbances:** External factors, such as sudden wind gusts, push the car off its intended path.
- **Unforeseen Constraints:** A driver might round a blind corner to discover a stationary, crashed car blocking the road.

The original plan calculated at the start line cannot account for these variables. To survive, the driver must use constant visual feedback to discard the old plan and calculate a new trajectory that avoids the obstacle. In MPC, this is known as a **receding horizon** strategy. The controller calculates an optimal sequence of actions for the entire horizon but only executes the immediate first step. A fraction of a second later, it takes a new measurement and repeats the entire optimisation. This constant re-planning makes the system robust to uncertainty and ensures the car remains on the track, even in a changing or unpredictable environment.

1.3 Receding Horizon Control (RHC)

The mathematical framework that brings the racing car analogy to life is known as *Receding Horizon Control* (RHC). This strategy serves as the engine of MPC, bridging the gap between static optimisation and dynamic control.

1.3.1 The Algorithm

The RHC strategy operates in discrete time steps. At each sampling instant t, the controller performs the following sequence, as illustrated in Figure 1.5:

1. **Measure:** Obtain the current state of the system, denoted as $x(t)$ via sensors or estimators. This provides the *starting line* for the current iteration.
2. **Plan (Optimise):** Using an internal model of the plant, calculate an optimal sequence of future control inputs $\{u_0, u_1, \dots, u_{N-1}\}$ over a finite prediction horizon N. This sequence is chosen to minimise a cost function (such as error from a reference) whilst respecting all physical constraints.
3. **Execute:** Although a full sequence of N steps was calculated, the controller applies **only the first element**, u_0, to the plant. The remaining $N-1$ steps of the plan are discarded.
4. **Evolve:** The system evolves for one sample period under the influence of the applied input and any external disturbances.
5. **Repeat:** At the next time step $t+1$, the entire process begins again. The horizon "recedes" (shifts forward by one step), and a new optimisation problem is solved from the new state $x(t+1)$.

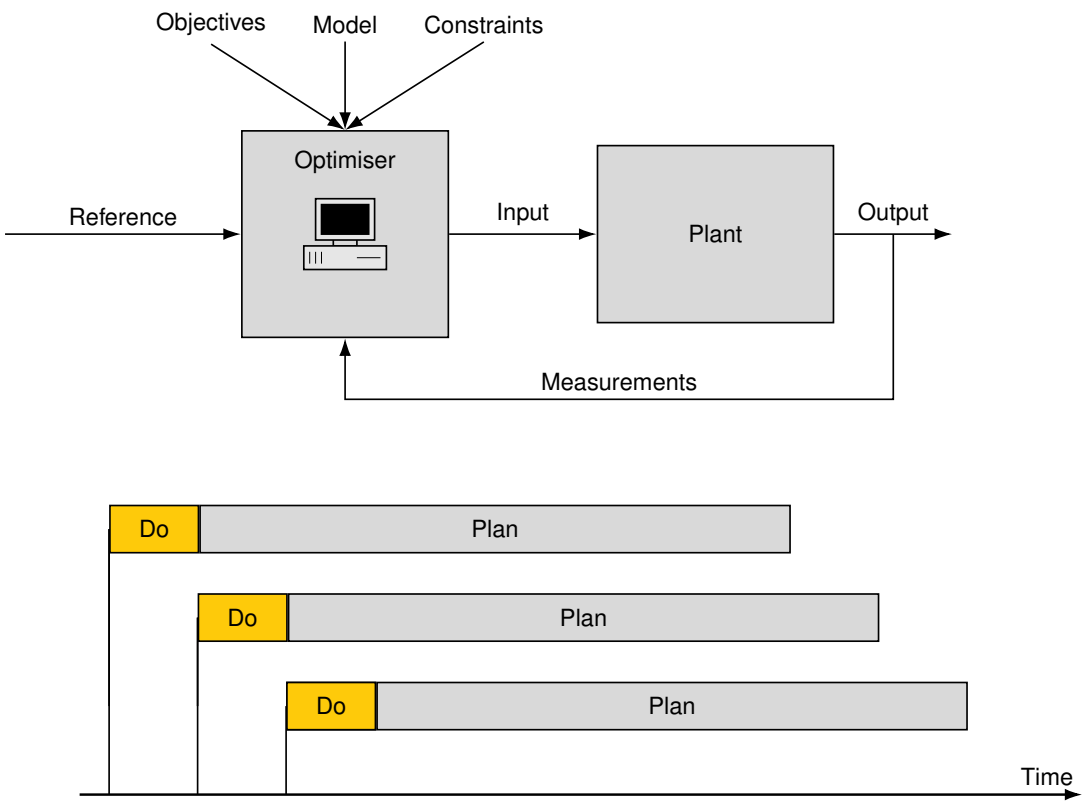


Figure 1.5: **Top**: The closed-loop architecture. The controller uses an internal Model to predict plant behavior, an Optimiser to minimize a cost function (Objectives) subject to safety limits (Constraints), and real-time Measurements to correct prediction errors. **Bottom**: The Receding Horizon strategy. At each time step, an optimal trajectory is computed over a long prediction horizon (*Plan*), but only the first control action is applied (*Do*). The horizon then shifts forward, and the optimisation repeats with updated feedback, turning an open-loop prediction into a closed-loop controller.

The reader may naturally ask: *"If the plan calculated at step t was* optimal*, why discard the rest of it and repeat the work at step $t+1$?"*

The answer lies in the inevitable gap between the controller's internal model and the real world. If we were to execute the entire N-step sequence without looking back, we would be operating in *open-loop*. Any small modelling error that falls into that N instants or unexpected gust of wind would cause the system to drift away from the intended path.

By only executing the first step and then re-measuring the state, we turn a series of open-loop predictions into a *closed-loop* feedback strategy. The *horizon* refers to how far into the future the controller predicts. Since we repeat the optimisation at every step, the horizon moves forward with time. This repetitive re-planning ensures that the control action is always based on the most current information. The *receding* nature of the horizon ensures that the controller is constantly correcting for errors and reacting to new information, much like the driver who constantly adjusts their steering as the road unfolds.

1.4 Why Use Model Predictive Control?

Whilst traditional control methods—like Proportional-Integral-Derivative (PID) control, lead-lag compensation, and the Linear Quadratic Regulator (LQR)—are still workhorses of industrial automation, they often hit their limits when dealing with today's demanding performance requirements. Model Predictive Control has become a leading approach for complex systems by anticipating future behaviour through model-based prediction.

The transition from classical methods to MPC is typically driven by the need to operate closer to the physical limits of a system without compromising safety or stability. Unlike traditional controllers, which are often tuned via trial-and-error to achieve a specific response, MPC provides a systematic framework that accounts for the system's model and its environment. The primary advantages that distinguish MPC from its predecessors are outlined in the following subsections.

1.4.1 Systematic Handling of Constraints

This is the most significant advantage of MPC. In the real world, every engineering system is subject to physical and operational limits. These may include *actuator constraints*, such as the maximum opening of a valve or the voltage limit of a motor, and *state constraints*, such as maintaining a chemical reactor below a critical temperature or ensuring a vehicle remains within track boundaries.

Classical Control: Ignores constraints during design and is inherently *blind* to these boundaries. A PID controller, for instance, may calculate a control action that the hardware cannot physically execute. This often leads to *integrator windup*, where the controller's internal logic becomes saturated, resulting in large overshoots or sluggish recovery.

MPC: Includes constraints directly in the optimisation. The optimiser knows the limits and will find the best control action that resides *on the boundary* of those limits without crossing them. MPC behaves non-linearly: it acts gently when far from a constraint but aggressively when approaching a limit to prevent violation. (See Figure 1.6). MPC treats constraints as fundamental components of the control problem. By predicting future behaviour, the optimiser can anticipate a potential constraint violation and adjust the control signal proactively *before* the limit is reached. This ability to operate safely at the very edge of the feasible operating space is perhaps the most significant economic driver for MPC in industry.

1.4.2 Future Knowledge and Preview Capability

A major drawback of many traditional controllers is that they're inherently *causal*—they can only react to what's already happened or what's happening right now. However, in many modern engineering applications, we often have access to "future knowledge"—information about upcoming changes in what we want the system to do, or predicted external disturbances. MPC can exploit this information directly through its prediction mechanism, via its *look-ahead* or *preview* capability.

To show the advantage of acting on future information, consider the following examples:

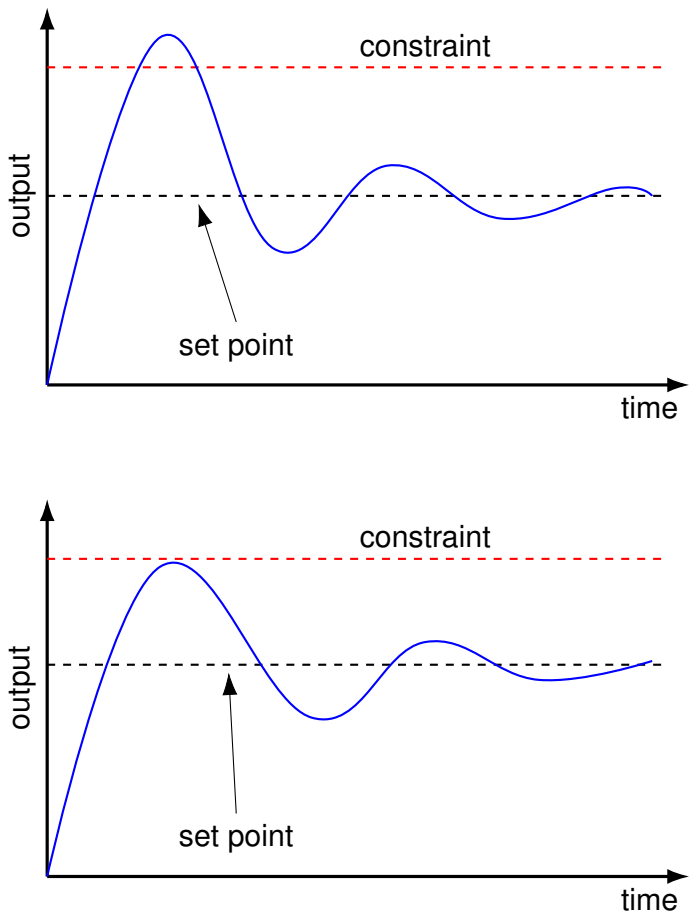


Figure 1.6: **Top:** Classical control showing significant overshoot violating constraints. **Bottom:** Predictive control respecting constraints by adjusting the trajectory.

- **Smart Energy Management:** Think about a building's HVAC (Heating, Ventilation, and Air Conditioning) system. If the weather forecast predicts a scorching heatwave for the afternoon—right when electricity prices hit their peak—an MPC controller can choose to "pre-cool" the building's thermal mass during the cooler morning hours whilst energy is cheaper. A traditional PID controller, being reactive, would only ramp up the cooling once the internal temperature had already started to climb, consuming electricity during the most expensive tariff period.

- **Autonomous Path Following:** In autonomous driving, the vehicle often has map data or sensor information about the road layout ahead. If there's a sharp bend coming up, a reactive controller (like a lane-keeping PID) would only start to steer or slow down once it detects a tracking error at the corner. In contrast, MPC sees the curve in its prediction horizon and begins adjusting the steering angle and reducing speed *before* entering the turn. This produces a smoother, safer, and more "human-like" driving experience.

- **Renewable Energy Integration:** In a microgrid, if the forecast shows a sudden drop in solar irradiance coming (say, due to approaching cloud cover), MPC can proactively fire up a backup power source or dial back non-essential loads. This keeps the grid stable by making up for the power shortfall before the frequency actually drops.

Incorporating forecast information into the optimisation converts a reactive control strategy into a proactive one.

1.4.3 Multivariable Control and Complex Objectives

Whilst classical control theory typically tackles problems by designing individual loops for individual variables (Single-Input Single-Output, or SISO), modern systems are rarely this straightforward. MPC handles Multi-Input Multi-Output (MIMO) systems natively through its state-space formulation, and its flexibility in defining what "optimal" performance actually means.

Handling Cross-Coupling in MIMO Systems

In many real-world systems, the inputs and outputs are "coupled". This means that tweaking one input to control a particular output will inevitably affect other outputs as well. MPC handles this problem very well.

Example 1.1: The Shower Analogy.

Consider the everyday challenge of adjusting a shower. You've got two goals: keeping the water at a specific temperature and achieving your desired flow pressure. You also have two controls at your disposal: the hot tap and the cold tap.

- **The PID Approach (Decentralised):** A traditional strategy might assign one controller to temperature (operating the hot tap) and another to pressure (operating the cold tap). If you want more pressure, the pressure controller opens the cold tap. This increases the flow, but immediately throws off the temperature, causing the water to go freezing cold. The temperature controller then scrambles to fix this, which in turn messes with the pressure. The two loops end up fighting each other, leading to an oscillation and a rather uncomfortable shower.
- **The MPC Approach (Centralised):** MPC uses a multivariable model that understands how everything interacts. It knows that to increase pressure *without* disturbing the temperature, it needs to open *both* the hot and cold taps at the same time in just the right ratio. Because it works out all the inputs together, it accounts for cross-coupling automatically—no need for complicated decoupling networks.

Defining *Optimality* via Cost Functions

Classical controllers such as PID typically have a very narrow notion of what constitutes success: the control objective is simply to drive the tracking error to zero. While this makes PID controllers effective and easy to deploy, it also means that they have no inherent understanding of *cost* or *trade-offs*. Model Predictive Control (MPC), by contrast, is built around an explicit cost function, denoted by J, which allows the engineer to state mathematically what aspects of performance are important for a given application.

By tuning the relative *weights* of the different terms in the cost function, the behaviour of an MPC controller can be shaped to handle more complex and realistic scenarios, where competing objectives must be balanced rather than optimised in isolation.

Consider, for example, a high-precision servo system. A PID controller may react very aggressively in its attempt to eliminate even tiny tracking errors, leading to rapid actuator motion and unwanted vibrations. An MPC controller can be designed to include a penalty on the *rate of change* of the control input, Δu. This effectively tells the controller to prioritise accurate tracking, but only insofar as it does not require excessive or abrupt actuation. The resulting control signal is smoother, reducing mechanical wear and helping to extend the lifespan of the hardware.

A similar principle applies in process control applications such as chemical plants. There may be several different ways to achieve the desired product quality. One strategy might rely more heavily on steam, which is effective but expensive, while another might use additional catalyst, which is cheaper but leads to slower dynamics. Within an MPC framework, such preferences can be encoded directly by assigning different economic weights to each input. The optimiser then determines the specific combination of control actions that satisfies the quality constraints while minimising the overall financial cost.

1.5 Operation of MPC

The Model Predictive Control problem is fundamentally defined as a finite-horizon optimisation problem that is solved repeatedly at each time step. To understand how MPC operates, one must examine its three core mathematical components: Modelling the cost function, imposing constraints, and applying the receding horizon principle.

1.5.1 The Prediction Model

Predicting the future evolution of a system requires an explicit mathematical description of its dynamics. As digital controllers operate at discrete sampling instants, this description is typically expressed in discrete time. For a linear, time-invariant system, the prediction model is written in State–Space form as

$$x_{k+1} = Ax_k + Bu_k. \tag{1.1}$$

Here, $x_k \in \mathbb{R}^n$ denotes the *state vector*, which encapsulates the variables describing the system's current condition (such as position, velocity, or temperature), while $u_k \in \mathbb{R}^m$ denotes the *control input* applied at time step k. The matrices A and B characterise the system dynamics and determine how the state evolves from time step k to $k+1$ under the applied input.

1.5.2 The Objective (Cost Function)

With a model capable of predicting future behaviour, the next step is to define what constitutes desirable performance. In MPC, this is achieved through the definition of a scalar cost function, J, which evaluates the quality of a predicted state and input trajectory over a finite prediction horizon of length N. A widely adopted choice is the quadratic cost function

$$J(x_0, u) = \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right) + x_N^\top Q_N x_N. \tag{1.2}$$

The quadratic structure ensures that larger deviations are penalised disproportionately more than smaller ones, reflecting a least-squares philosophy. The relative importance of competing objectives is governed by the weighting matrices:

- **Performance weight (Q):** penalises deviations of the state from its reference. Increasing Q encourages faster error correction and leads to more aggressive closed-loop behaviour.
- **Input weight (R):** penalises control effort. Larger values of R discourage excessive or rapid changes in the control input, resulting in more conservative actuation.
- **Terminal weight (Q_N):** penalises the terminal state x_N at the end of the horizon and is commonly introduced to promote stability and desirable long-term behaviour.

Although the compact matrix notation $x^\top Qx$ is mathematically convenient, it can obscure the physical interpretation of the tuning parameters. To clarify this intuition, it is instructive to examine a simple example.

Consider a mechanical system, such as a robotic manipulator or a vehicle, with two state variables and a single control input:

- x_1: position error relative to a desired target,
- x_2: velocity,
- u: applied force (for example, motor voltage or throttle input).

The state vector is therefore $x_k = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$. In practice, the state weighting matrix Q is almost always chosen to be diagonal, allowing each state component to be penalised independently:

$$Q = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix}, \tag{1.3}$$

where $q_1 \geq 0$ and $q_2 \geq 0$ are tuning parameters selected by the engineer.

Explicit Expansion of the Cost

Expanding the quadratic term $x_k^\top Qx_k$ reveals its interpretation as a weighted sum of squared state variables:

$$\begin{aligned} x_k^\top Qx_k &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \\ &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} q_1 x_1 \\ q_2 x_2 \end{bmatrix} \\ &= q_1 x_1^2 + q_2 x_2^2. \end{aligned} \tag{1.4}$$

Each state therefore contributes to the overall cost in proportion to both its magnitude and its associated weight. If the control input is scalar, the matrix R reduces to a single non-negative scalar r, and the corresponding term simplifies to

$$u_k^\top Ru_k = ru_k^2. \tag{1.5}$$

This formulation highlights that MPC does not merely seek to eliminate state error, but instead explicitly balances tracking performance against the cost of control effort.

How to Tune the Weights

Once the cost function is written out explicitly, its interpretation becomes quite intuitive. In essence, J acts like a balance sheet: at every time step, the controller adds up a set of penalties and then chooses the control action that makes this total as small as possible:

$$J \approx \sum \Big(\underbrace{q_1 x_1^2}_{\text{Position penalty}} + \underbrace{q_2 x_2^2}_{\text{Velocity penalty}} + \underbrace{r u^2}_{\text{Energy penalty}} \Big). \tag{1.6}$$

The relative size of the weights determines the controller's priorities and, ultimately, its behaviour. Several typical tuning regimes are worth highlighting:

1. **High-precision tuning ($q_1 \gg r$):** If, for instance, $q_1 = 100$ and $r = 1$, the controller treats position errors as far more costly than the use of control effort. As a result, it will act aggressively, applying large inputs to eliminate even small position errors as quickly as possible.

2. **Energy-saving tuning ($r \gg q_1$):** Conversely, if $r = 100$ and $q_1 = 1$, control effort is considered expensive. In this case, the controller is willing to tolerate larger or longer-lasting position errors if that reduces the amount of energy or fuel consumed.

3. **Damping-oriented tuning (large q_2):** If the closed-loop response exhibits excessive oscillations, increasing q_2 can be effective. Since x_2 represents velocity, heavily penalising this term discourages rapid motion, resulting in a smoother, more heavily damped response that behaves much like an electronic damper.

Thus, Q and R are not abstract mathematical abstractions; they are the *knobs* the engineer turns to define the personality of the controller.

1.5.3 Constraint Handling

Unlike classical control approaches (such as PID), which may effectively demand unbounded actuation without regard for physical feasibility, MPC is formulated to explicitly respect real-world limitations. The optimisation problem is therefore solved subject to a set of constraints, typically expressed as linear inequalities. In compact matrix form, these constraints are written as

$$Cx_k + Du_k \leq b. \tag{1.7}$$

At first glance, this expression may appear rather abstract. In practice, however, it is nothing more than a convenient way of collecting many simple and intuitive rules into a single mathematical statement. Each row of the inequality corresponds to a specific restriction—such as limits on actuator saturation, allowable state ranges, or safety margins—that the controller is required to satisfy at every time step.

Example 1.2: Constraints always exist in physical systems

Let us revisit our 2-state vehicle system, where x_1 is position, x_2 is velocity, and u is the acceleration command. Suppose we have the following physical limits:

1. **Actuator Limit:** The engine can provide a maximum acceleration of $+10\ m/s^2$ and a maximum braking (deceleration) of $-10\ m/s^2$.
2. **Safety Limit:** The vehicle must not exceed a speed limit of $50\ m/s$.

Mathematically, these are three separate inequalities:

$$\begin{aligned} &u_k \leq 10 && \text{(Max Accel)} \\ &u_k \geq -10 \implies -u_k \leq 10 && \text{(Max Brake - note the sign flip)} \\ &x_2 \leq 50 && \text{(Speed Limit)} \end{aligned}$$

Note: Standard solvers usually require all constraints to be in the form "$\cdots \leq$ Constant". Therefore, we multiply "greater than" constraints by -1 to flip the sign.

To fit these into the standard form $Cx_k + Du_k \leq b$, we stack them into matrices. Recall that $x_k = [x_1, x_2]^\top$:

$$\underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}}_{C} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \underbrace{\begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}}_{D} u_k \leq \underbrace{\begin{bmatrix} 10 \\ 10 \\ 50 \end{bmatrix}}_{b} \tag{1.8}$$

By reading this matrix row by row, we can see how the algebra represents the physical reality:

- **Row 1:** $0 \cdot x + 1 \cdot u \leq 10 \implies u \leq 10$.
- **Row 2:** $0 \cdot x - 1 \cdot u \leq 10 \implies -u \leq 10 \implies u \geq -10$.
- **Row 3:** $1 \cdot x_2 + 0 \cdot u \leq 50 \implies x_2 \leq 50$.

This matrix formulation defines a **feasible region**—a geometric *safe space* within which the system must operate. If the predicted trajectory tries to leave this space (e.g., predicting a velocity of $51\ m/s$), the optimiser will actively alter the input u_k to prevent the violation, ensuring the car slows down *before* it breaks the speed limit.

1.5.4 The Receding Horizon Principle

The final piece of the MPC puzzle is the strategy of implementation. As illustrated in Figure 1.7, and shown in the flow chart Figure 1.8, the process operates in closed loop, but still relying on an open-loop anticipation.

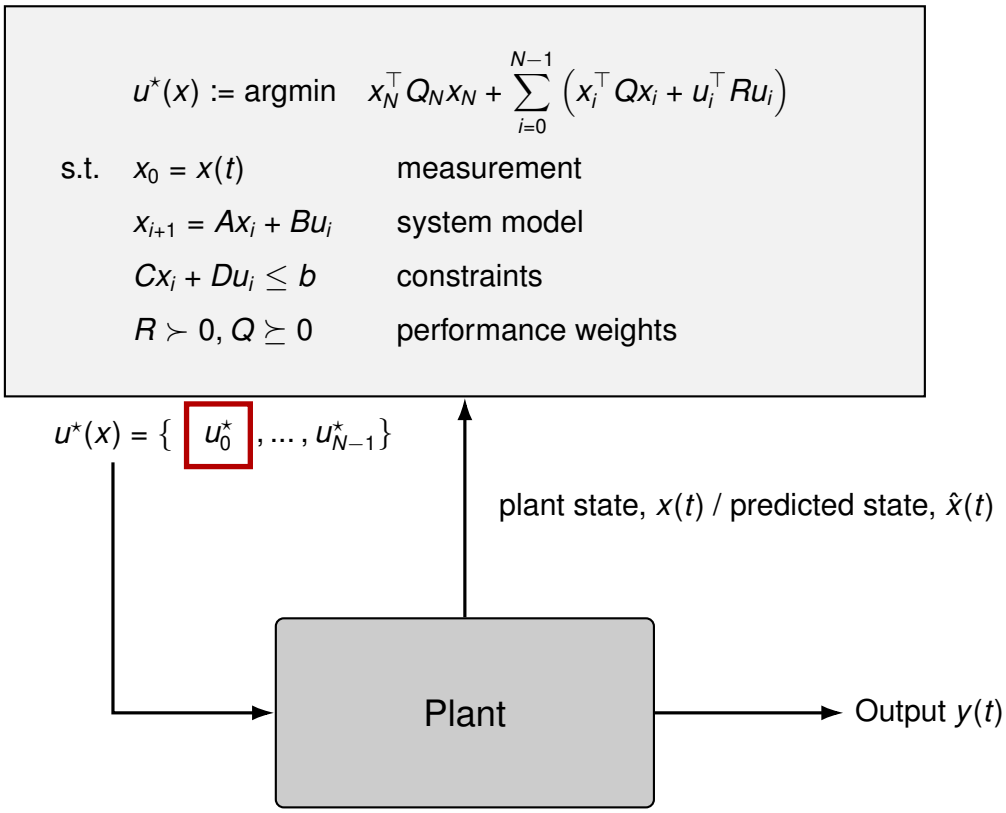


Figure 1.7: Mathematical formulation of the MPC problem and the Receding Horizon implementation.

At every specific time step t, the controller solves the optimisation problem to find the optimal **sequence** of future control inputs, denoted as $U^\star$:

$$U^\star = \{u_0^\star, u_1^\star, \dots, u_{N-1}^\star\} = \arg\min_U J(x_0, U) \tag{1.9}$$

This sequence represents a complete plan for the next N time steps. For example, in a self-driving car, this sequence might represent the steering angles for the next 10 seconds. However, MPC does **not** implement the entire plan but follows the following rules:

1. **Extract:** Take only the first element of the optimal sequence, $u_0^\star$.
2. **Apply:** Send this single signal to the plant actuators.
3. **Discard:** Throw away the remaining calculated inputs ($u_1^\star \dots u_{N-1}^\star$).
4. **Evolve:** Wait for one sample step ($t \to t+1$), measure the new state x_{t+1}, and shift the horizon forward by one step.
5. **Repeat:** Solve the entire optimisation problem again from scratch.

The reader might ask: *"Why waste computational power calculating N steps if we only use the first?"*

The answer is **feedback**. The model used for prediction is never a perfect representation of reality. If we applied the entire sequence $U^\star$ blindly, any unmodelled disturbance (e.g., a gust of wind) or parameter mismatch (e.g., a change in friction) would cause the system to drift off course.

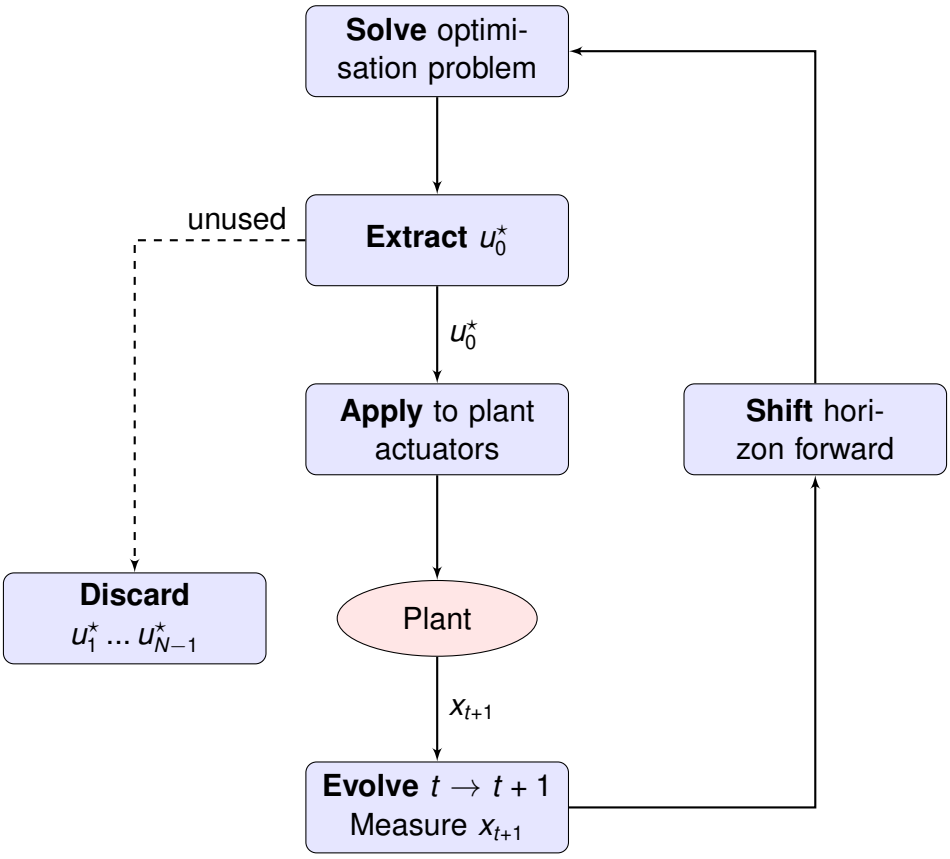


Figure 1.8: Summary of the Receding Horizon Control (RHC) loop.

By re-planning at every single time step based on fresh measurements, MPC converts an "open-loop optimal control" problem into a "closed-loop feedback" strategy, ensuring robustness against uncertainty.

Practical Guideline: When to use MPC?

MPC comes at a cost: it requires a mathematical model and significant computational power to solve the optimisation problem in real-time (often within milliseconds). Therefore,

- If a system is simple (SISO), linear, and operates far from its physical limits, a classical **PID controller** is computationally cheaper and often sufficient.
- MPC should be reserved for problems where its unique ability to handle **constraints**, **multivariable interactions**, and **future look-ahead** justifies the added complexity.

1.6 Applications of Model Predictive Control

Model Predictive Control (MPC) is widely used across an exceptionally broad range of application domains. One of its distinguishing characteristics is its scalability across vastly different time scales. Nowadays, it is applied to different systems ranging from nanoseconds in embedded

electronic systems to weeks in large-scale industrial and logistical decision-making. This diversity reflects MPC's ability to exploit predictions, incorporate constraints, and handle multivariable interactions in a unified framework.

Nanosecond to Millisecond Domains

At the fastest end of the spectrum, MPC is deployed in

- **Computer control (ns scale)**, where extremely rapid decision-making is required, for example in advanced semiconductor manufacturing or embedded hardware processes.
- **Traction control (ms scale)**, where vehicles must respond quickly to rapid variations in tyre–road interaction. MPC allows the controller to anticipate future slip conditions while respecting actuator limits, improving safety and performance.

Second to Hour Domains

For slower mechanical and thermal systems, MPC operates at time scales of seconds to hours:

- **Power systems (μs–s)** benefit from MPC's ability to regulate voltage and frequency, incorporate network constraints, and schedule generation resources.
- **Buildings (seconds)** are a classical MPC application area. The building thermal response, HVAC equipment limitations, and comfort constraints make MPC highly effective for energy-efficient control.
- **Refineries (minutes)** rely on MPC for large multivariable process control. Here MPC has been an industry standard for decades, particularly for maintaining optimal operating points close to constraints.
- **Nurse rostering (hours)** and other personnel scheduling problems use MPC-inspired formulations to accommodate work rules, hospital demand patterns, and staffing constraints.

Day to Week Domains

At even larger horizons, MPC ideas extend into logistical and economic decision-making:

- **Train and airline scheduling (days)** requires the coordination of resources, timetables, and infrastructure. MPC provides a mechanism for repeatedly updating schedules based on predicted passenger demand and network delays.
- **Production planning (weeks)** uses MPC to optimise inventory, production rates, and resource allocation. The receding-horizon structure ensures that plans remain feasible as forecasts and operational conditions evolve.

1.7 Case Study: Energy-Efficient Building Control

Buildings are one of the best places to implement Model Predictive Control. Buildings account for roughly 40% of total energy consumption in many industrialised countries, with HVAC systems representing a large share of that demand. Most of this energy is consumed during day-to-day operation rather than construction, and the systems involved are full of physical and operational constraints that MPC handles well. Better control of heating, cooling, ventilation, and lighting does not just cut costs; it also has a real effect on environmental impact.

What makes buildings particularly interesting is that cutting their carbon emissions can actually be profitable. Figure 1.9 shows a Marginal Abatement Cost Curve for London's building stock. The vertical axis shows what it costs to eliminate one tonne of CO_2, while each bar's width indicates how much carbon a particular technology can cut. Notably, bars below the zero line represent measures that save money over their lifetime—the energy bill reductions more than cover the upfront investment.

Building controls fall within this profitable zone (number 4). Unlike expensive physical retrofits or new renewable infrastructure, improving building control strategies through MPC delivers significant efficiency gains that pay for themselves.

There's another practical reason MPC works so well here: building owners expect upgrades to pay back through lower energy bills. MPC is built for exactly this scenario. It can anticipate weather forecasts and occupancy patterns, handle the slow thermal dynamics of buildings, and respect comfort requirements and equipment limits—all while optimising systematically over time. This means it can shift heating and cooling to more efficient periods, shave peak demand, and keep occupants comfortable.

In short, buildings tick all the boxes that make MPC attractive: multiple interacting systems, real constraints, long-term dynamics, and measurable financial returns. That is why the built environment has become one of the most active areas for MPC research and real-world application [13].

1.7.1 Model Predictive Control for a One-Zone Building

To illustrate how MPC can be applied in building energy management, this section considers a simplified but representative example: a single thermal zone with electrical heating and external disturbances from weather and solar radiation as shown in Figure 1.10. Although simplified, the model captures the essential thermal dynamics and constraints that arise in real buildings, and it provides a clear foundation for constructing an MPC.

The example is based on an EnergyPlus building model that has been linearised and reduced automatically using the *openBuild* tool. The resulting discrete-time state-space model has four states:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}, \qquad x_1 = \text{zone air temperature}, \qquad x_{2..4} = \text{wall temperatures}.$$

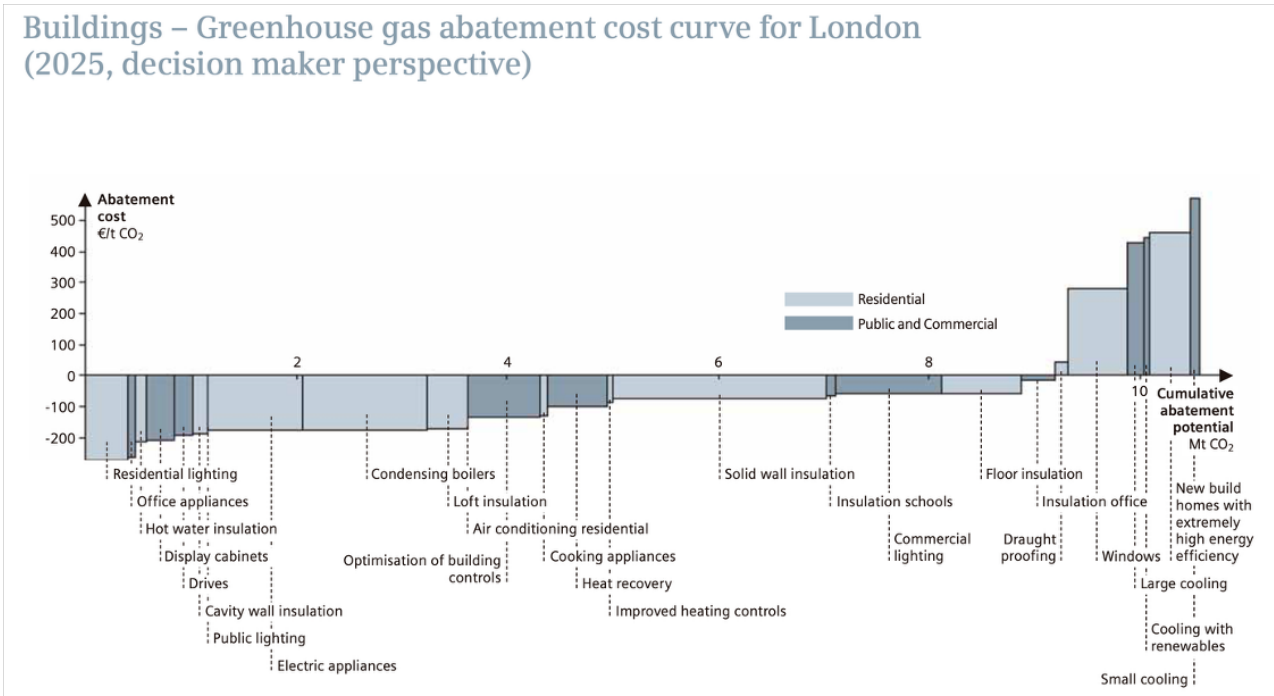


Figure 1.9: Marginal Abatement Cost Curve (MACC) for the residential and commercial sectors. The vertical axis represents the cost to abate one tonne of CO_2, whilst the horizontal axis shows the cumulative reduction potential. Measures below the zero line—such as the *optimisation of building controls*—offer a negative abatement cost, meaning they generate net financial savings over their lifetime. Source: Adapted from Siemens and McKinsey & Company, *Sustainable Urban Infrastructure: London Edition — A view to 2025* (2008).

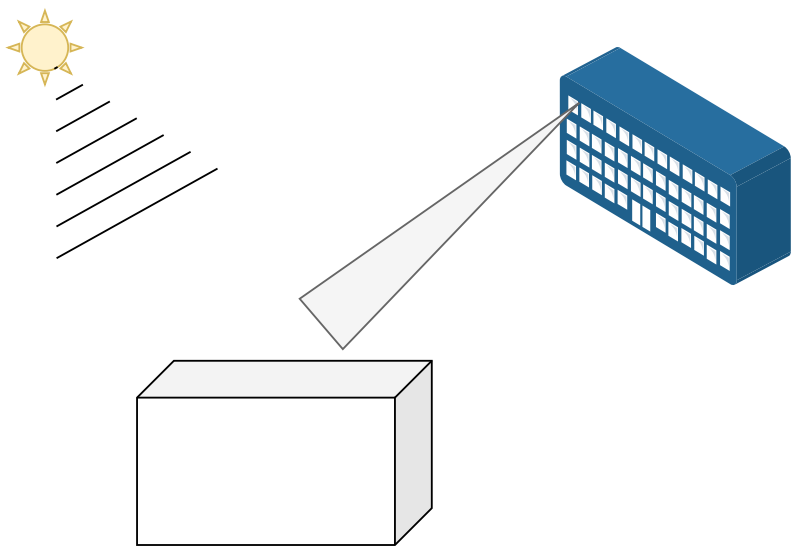


Figure 1.10: Single zone of a building.

The system evolves according to

$$x_{k+1} = Ax_k + Bu_k + E_{\text{rad}}\, v_k^{\text{rad}} + E_{\text{amb}}\, T_k^{\text{amb}}, \tag{1.10}$$

where

- u_k is the control input, representing the heat flux delivered to the zone,
- v_k^{rad} is the incident solar radiation (a disturbance),
- T_k^{amb} is the outdoor ambient temperature (another disturbance),
- $A, B, E_{\text{rad}}, E_{\text{amb}}$ are obtained from linearisation.

The output of interest is the zone air temperature:

$$y_k = c^\top x_k,$$

where $c^\top = \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$ extracts the first state corresponding to x_1.

Control Objectives and Constraints

The primary objective is to maintain the indoor temperature within a predefined comfort band (e.g., around 21°C) while minimising energy use. This naturally leads to the following requirements:

1. **Comfort constraint**:
$$|\, y_k - 21 \,| \leq 2^\circ\text{C}.$$
2. **Actuator constraint** (electric heater):
$$0 \leq u_k \leq 1,$$
where u_k is normalised to the 0–100% heating range.
3. **Disturbance forecasts**: Predictions of future weather (solar radiation and ambient temperature) are incorporated over the MPC prediction horizon. These forecasts are available from standard meteorological data.

MPC Problem Formulation

The MPC problem aims to minimise energy use while satisfying comfort constraints. Over a prediction horizon N, the optimisation problem is:

$$\begin{aligned} \min_{\{u_0,\ldots,u_{N-1}\}} \quad & \sum_{i=0}^{N-1} |u_i| \\ \text{s.t.} \quad & x_{i+1} = Ax_i + Bu_i + E_{\text{rad}}\, v_i^{\text{rad}} + E_{\text{amb}}\, T_i^{\text{amb}}, \\ & y_i = c^\top x_i, \\ & |\, y_i - 21 \,| \leq 2, \\ & 0 \leq u_i \leq 1, \end{aligned} \tag{1.11}$$

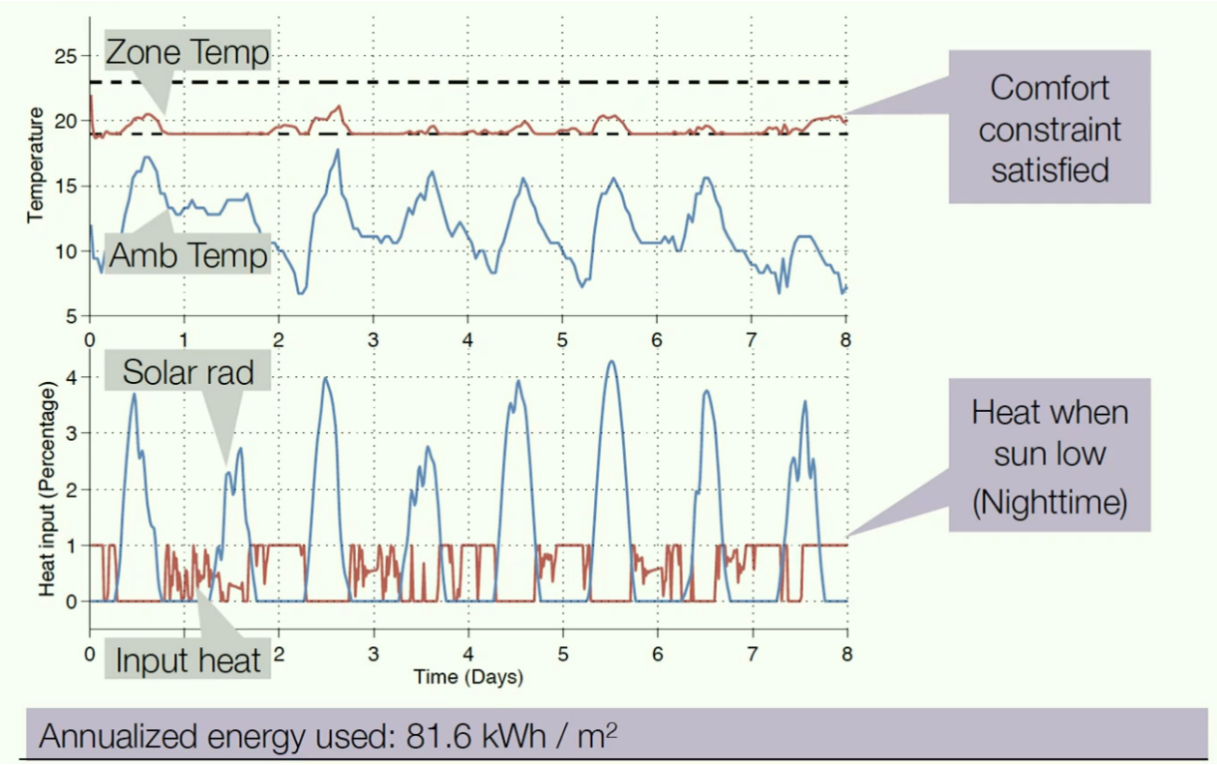


Figure 1.11: Closed-loop simulation results of Model Predictive Control (MPC) for a single-zone building. The top panel demonstrates the controller's ability to maintain the zone temperature (red line) within the specified comfort constraints (dashed lines), despite significant fluctuations in the ambient temperature (blue line). The bottom panel illustrates the relationship between the control input and disturbances; heating (red line) is applied primarily during nighttime or periods of low solar radiation (blue line) to compensate for heat loss, effectively minimizing energy use to an annualized rate of 81.6 kWh/m². (*Taken from lecture slides of ME-425, EPFL with the permission of Prof C. Jones*)

where:

- The cost function $\sum |u_i|$ represents total thermal energy consumption.
- The constraints enforce comfort and equipment limitations.
- Disturbance predictions enter the model explicitly through v_i^{rad} and T_i^{amb}.

This formulation is intentionally simple and is the first step toward more advanced building MPC strategies. Later we shall extend this model to incorporate night-time temperature setbacks, pre-heating strategies, and time-of-use economic considerations while maintaining comfort. Figure 1.11 shows the application of the MPC and its evaluation.

Can we do better with Model Predictive Control with Time-of-Use Pricing?

In its simplest form, the controller's job is to minimise the total thermal energy consumed. But in modern smart grid setups, electricity prices change throughout the day depending on demand. To reflect this reality, we can shift the focus from minimising raw energy use to minimising the actual economic cost of running the system. We do this by incorporating a Time-of-Use tariff into the cost function.

Problem Formulation

We define a time-varying price vector $p \in \mathbb{R}^N$ over the prediction horizon. The tariff structure is defined based on peak and off-peak hours as follows:

$$p_k = \begin{cases} P_{\text{day}} & \text{if } 09:00 \leq t_k < 18:00 \quad \text{(Peak)} \\ P_{\text{night}} & \text{if } 18:00 \leq t_k < 09:00 \quad \text{(Off-Peak)} \end{cases}$$

where $P_{\text{day}} > P_{\text{night}}$. The optimisation problem is now reformulated to minimise the weighted sum of the control inputs. Additionally, to ensure feasibility during aggressive load shifting, we introduce a slack variable σ_i to relax the comfort constraints (soft constraints). If this is not done, the problem might become infeasible. The resulting optimisation problem can be written as:

$$\begin{aligned} \min_{u,\sigma} \quad & \sum_{i=0}^{N-1} p_i |u_i| + \rho \|\sigma_i\| \\ \text{s.t.} \quad & x_{i+1} = Ax_i + Bu_i + E_{\text{rad}} v_{i,\text{rad}} + E_{\text{amb}} T_{i,\text{amb}} \\ & y_i = Cx_i \\ & |y_i - 21| \leq 2 + \sigma_i \\ & 0 \leq u_i \leq 1 \\ & \sigma_i \geq 0 \end{aligned}$$

where ρ is a very large penalty weight to discourage comfort violations unless necessary.

Introducing a price signal fundamentally changes how the controller behaves. Unlike the energy-optimal controller, which puts off heating until the very last moment, the economic-optimal controller uses the building's thermal mass as a kind of energy battery.

As Figure 1.12 shows, the controller engages in load shifting and pre-heating. Knowing that prices will jump at 09:00, it heats the zone aggressively during the cheap off-peak hours in the early morning, pushing the temperature up towards the upper comfort limit of 23°. The thermal energy stored in the building's fabric then allows the system to dial back or switch off heating entirely during the expensive peak hours (09:00–18:00), keeping costs down whilst still maintaining occupant comfort.

1.8 A First MPC Example in MATLAB

Before diving into the mathematical theory in the chapters that follow, it is instructive to see MPC in action on a simple example. The short MATLAB script below implements a complete MPC controller for a scalar unstable system using the YALMIP toolbox, so that the reader can run it immediately and build intuition for how the algorithm works.

```
% 1. Define the scalar unstable system: x_{k+1} = 1.2*x_k + u_k
A = 1.2;  B = 1;
Q = 1;    R = 0.1;
```

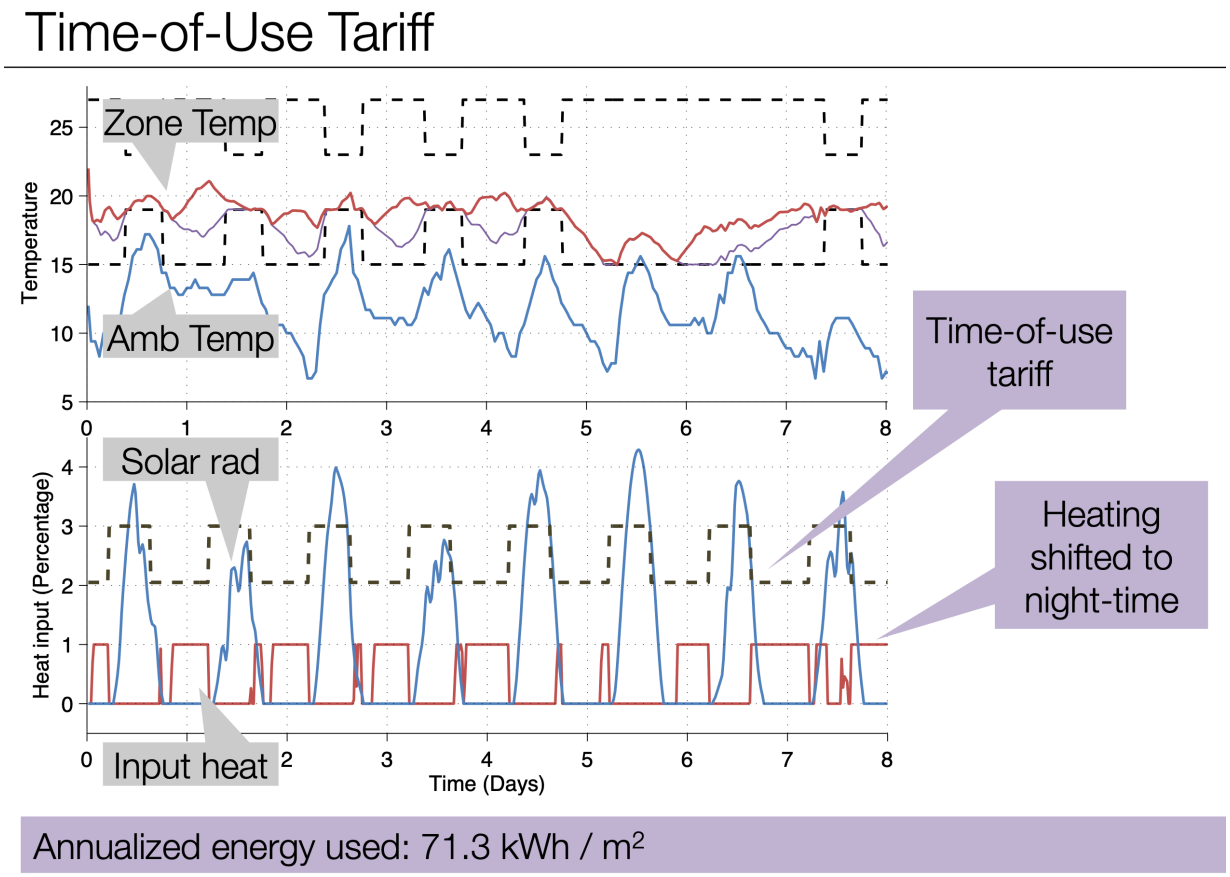


Figure 1.12: Simulation results under Time-of-Use (ToU) tariff. The bottom plot demonstrates the shifting of heating inputs to night-time hours to avoid high daytime electricity prices. (*Taken from lecture slides of ME-425, EPFL with the permission of Prof C. Jones.*)

```
N = 10;   % Prediction horizon

% Terminal cost from the discrete Riccati equation
[P,~,~] = dare(A, B, Q, R);

% 2. YALMIP formulation
u = sdpvar(1, N);
x = sdpvar(1, N+1);
x0 = sdpvar(1, 1);        % Parameter: current state

obj = 0;
cons = [x(:,1) == x0];    % Initial condition constraint

for k = 1:N
  obj  = obj + x(:,k)'*Q*x(:,k) + u(:,k)'*R*u(:,k);
  cons = [cons, x(:,k+1) == A*x(:,k) + B*u(:,k)];
  cons = [cons, -1 <= u(:,k) <= 1];   % Input constraint
end
obj = obj + x(:,N+1)'*P*x(:,N+1);       % Terminal cost

% 3. Build the optimizer object (parametric in x0)
opts = sdpsettings('verbose', 0, 'solver', 'quadprog');
controller = optimizer(cons, obj, opts, x0, u(:,1));

% 4. Closed-loop simulation
T_sim = 30;
x_state = 5;              % Initial condition
x_hist = zeros(1, T_sim+1);  u_hist = zeros(1, T_sim);
x_hist(1) = x_state;

for t = 1:T_sim
  [u_opt, errcode] = controller(x_state);
  if errcode ~= 0, warning('Solver failed!'); end
  u_hist(t) = u_opt;
  x_state = A*x_state + B*u_opt;
  x_hist(t+1) = x_state;
end

% 5. Plot results
figure;
subplot(2,1,1);
stairs(0:T_sim, x_hist, 'b', 'LineWidth', 2); grid on;
ylabel('State (x)'); title('MPC Closed-Loop Response');
subplot(2,1,2);
stairs(0:T_sim-1, u_hist, 'b', 'LineWidth', 2); hold on;
yline( 1, 'r--', 'LineWidth', 1.5);
yline(-1, 'r--', 'LineWidth', 1.5);
ylabel('Input (u)'); xlabel('Time step (k)'); grid on;
```

Listing 1.1: A minimal MPC example for a scalar unstable system.

When you run this script, you should observe that the controller successfully drives the unstable state from $x_0 = 5$ to the origin despite the system being open-loop unstable ($a = 1.2$). Notice that the control input saturates at $u = -1$ during the first several time steps, when the state is large and the controller must apply maximum effort; once the state is sufficiently small, the input comes off the constraint and the controller smoothly regulates the state to zero.

Prerequisites

This script requires MATLAB with the YALMIP toolbox and a QP solver such as `quadprog`. Installation instructions for YALMIP are given in the Foreword.

1.9 Chapter Summary

This chapter introduced Model Predictive Control (MPC) and contrasted it with classical control methods like PID. Unlike classical approaches that rely on pre-designed, fixed control laws, MPC solves a constrained optimisation problem on-line at each time step.

The main points are:

- **Optimisation-Based Control:** MPC separates the *control law* from the *performance specifications*. Instead of tuning gains, the engineer defines a mathematical model of the plant, a cost function (objective), and physical constraints. The controller then solves an optimisation problem at every time step to find the best input sequence.
- **Receding Horizon Principle:** The operational backbone of MPC is the Receding Horizon Control (RHC) strategy. The controller measures the current state, plans a future trajectory over a finite horizon N, executes only the first step (u_0), and then repeats the process. This iterative re-planning provides the necessary feedback to handle uncertainties and disturbances, similar to a driver adjusting steering through a corner.
- **Explicit Constraint Handling:** A distinguishing feature of MPC is its ability to explicitly account for physical and safety limitations (e.g., valve limits, temperature bounds) during the design phase. While classical controllers clip signals when limits are reached, MPC anticipates these limits and steers the system along the constraint boundary to maximise performance safely.
- **Anticipation and Future Knowledge:** MPC is inherently predictive. It can utilise forecasts of known disturbances (such as weather or customer demand) to take pre-emptive action.
- **Scalability:** The methodology is applicable across a vast range of time scales, from nanosecond-level processor control to week-level production planning, provided the optimisation problem can be solved within the sampling interval.

The chapter concluded with a practical application of MPC to a single-zone building, illustrating two distinct behaviours:

1. **Energy Minimisation:** By incorporating weather forecasts, the MPC controller can reject disturbances (solar radiation, ambient temperature) efficiently, maintaining occupant comfort while minimising total energy use.
2. **Economic Optimisation (Time-of-Use):** When electricity prices vary, MPC exploits the time-varying tariff structure. The controller exhibits *load shifting* behaviours—specifically *pre-heating*—where it overheats the building during off-peak hours (using the thermal mass as storage) to reduce expensive consumption during peak hours.

MPC provides a systematic approach to controlling constrained, multivariable systems by formulating the control problem directly as a mathematical optimisation. For a comprehensive treatment of MPC theory and practice, the reader is referred to [17].

Exercises

1. **The Receding Horizon Principle.** The receding horizon strategy is the operational backbone of MPC.

   (a) In your own words, explain what "receding horizon" means. Why does the horizon "recede" at each time step?

   (b) At each time step, MPC computes an optimal sequence of N future control inputs but only *applies the first one* before solving the problem again. A student argues: "This is wasteful—why not just apply the entire sequence and save computation?" Write a short paragraph explaining why this approach would fail in practice. Your answer should mention at least two reasons (e.g., model mismatch, disturbances).

   (c) Consider the flowchart in Figure 1.8. List, in order, the five steps the MPC controller performs at each sampling instant.

2. **MPC versus PID and LQR.** Classical controllers such as PID and LQR are still widely used in industry.

   (a) Give *two* advantages that MPC has over a standard PID controller. For each advantage, briefly describe a real-world scenario where that advantage is critical.

   (b) Give *one* practical disadvantage of MPC compared with PID. Under what circumstances would you recommend keeping a PID controller rather than replacing it with MPC?

   (c) The Linear Quadratic Regulator (LQR) also minimises a quadratic cost function of the form $\sum(x^\top Qx + u^\top Ru)$. Identify *one key difference* between LQR and MPC that explains why MPC is preferred when hard constraints on inputs or states must be satisfied.

   (d) Referring to the "shower analogy" in Section 1.4, describe in plain language how a *decentralised PID* strategy and the *MPC centralised* strategy each attempt to regulate both temperature and pressure. Why does the PID strategy lead to oscillation?

3. **Numerical Exercise: Scalar System with Input Constraints.** Consider the scalar unstable system used in the MATLAB example (Listing 1.1):

$$x_{k+1} = 1.2\, x_k + u_k,$$

   with initial state $x_0 = 5$ and input constraint $-1 \leq u_k \leq 1$.

   (a) Without any controller ($u_k = 0$ for all k), compute the state values x_1, x_2, and x_3 starting from $x_0 = 5$. What does this tell you about the open-loop behaviour of the system?

   (b) Suppose the MPC controller applies the maximum allowed braking effort $u_k = -1$ at every step. Compute x_1, x_2, and x_3 under this saturated input. Is the system being stabilised? (*Hint:* check whether $|x_k|$ is growing or shrinking.)

(c) From the MATLAB description (Listing 1.1 and the paragraph following it), the control input saturates at $u = -1$ during the first several time steps. Explain in one or two sentences *why* the constraint is active at the start and why it eventually becomes inactive as the simulation progresses.

(d) The cost function uses weights $Q = 1$ and $R = 0.1$. What does the choice $R < Q$ imply about the controller's priorities? What would change qualitatively if you set $R = 10$ instead?

4. **Constraint Types: Hard vs. Soft, State vs. Input.** Constraints in MPC are categorised in two ways: by *type* (hard or soft) and by *variable* (state or input).

   (a) *Hard constraints* are constraints that must never be violated, while *soft constraints* are relaxed by introducing a penalty term in the cost function. Give one example of a hard constraint and one example of a soft constraint in the building energy case study (Section 1.6). Explain why treating the comfort band as a soft constraint—rather than a hard one—can be important for ensuring the optimisation problem always has a solution.

   (b) In the building model, the actuator bound $0 \leq u_k \leq 1$ is an *input constraint*, while the comfort requirement $|y_k - 21| \leq 2$ is a *state (output) constraint*. Explain, in plain language, the difference between these two types. Which type is typically easier to guarantee will always be satisfied, and why?

   (c) The Time-of-Use formulation in Section 1.6 introduces a slack variable $\sigma_i \geq 0$ that relaxes the comfort constraint to $|y_i - 21| \leq 2 + \sigma_i$. Describe what happens to the comfort constraint if the penalty weight ρ is set to a very large value versus a very small value.

   (d) Consider the vehicle example in Section 1.5 with state $x = [x_1, x_2]^\top$ (position, velocity) and acceleration input u. Write down, in words rather than matrices, one input constraint and one state constraint that would be physically reasonable for a road vehicle.

5. **Building Energy Application: Forecast Errors and Controller Behaviour.** The building MPC case study relies on weather forecasts for outdoor temperature and solar radiation over the prediction horizon.

   (a) Suppose the weather forecast predicts a mild day, but in reality a sudden cold snap arrives halfway through the day. Describe qualitatively what you would expect to happen to the indoor temperature under MPC. Would the situation be better or worse if the same forecast error occurred under a classical reactive (PID) controller? Justify your answer.

   (b) In the Time-of-Use strategy, the controller pre-heats the building during cheap off-peak hours in anticipation of expensive peak electricity prices (09:00–18:00). If the forecast for solar radiation is *overestimated* (i.e., the controller expects more free solar heat than actually arrives), describe the likely consequence for occupant comfort during peak hours. Would you classify this as a failure of the model, the constraints, or the forecast?

   (c) The case study states that the annualised energy consumption is 81.6 kWh/m^2. Explain, in two or three sentences, how incorporating a Time-of-Use electricity tariff changes the *pattern* of energy consumption even if the *total* energy used over a day remains roughly the same.

(d) MPC for buildings is listed as a “negative abatement cost” measure on the Marginal Abatement Cost Curve in Figure 1.9. In plain terms, what does this mean for a building owner? Why does smart control fall into this category whereas many other carbon-reduction measures do not?

Chapter 2

Optimisation: The Engine of MPC

> *"Since the fabric of the universe is most perfect and the work of a most wise Creator, nothing at all takes place in the universe in which some rule of maximum or minimum does not appear."*
>
> — **Leonhard Euler**

In Chapter 1, we introduced MPC as a controller that looks ahead and adjusts control actions by solving an optimisation problem at every time step. But this raises a fundamental question: how exactly does the computer decide which adjustment is the *correct* one?

A classical PID controller follows a simple, fixed rule: if the error increases, push back harder. It is reactive and rigid. An MPC controller, however, does not follow a pre-written rule. Instead, at every single time step, it faces a choice. It determines the best control action within the given constraints. This process of systematic selection is called *optimisation*.

Optimisation provides the mathematical framework for translating engineering objectives—such as *drive fast but be safe*—into precise numerical instructions that a computer can understand. If the system model is the map that tells the controller where it *can* go, optimisation is the navigator that decides where it *should* go.

In this chapter, we step away from system dynamics for a moment to focus purely on this decision-making engine. We will learn how to formulate engineering goals as objective functions, how to translate physical limits into mathematical constraints, and how to distinguish between problems that are easy for computers to solve and those that are computationally impossible.

2.1 What is Optimisation?

In a general sense, the Oxford Dictionary defines optimisation as:

- "The action of making the **best** or **most effective** use of a situation or resource."

In engineering, we translate this concept into a precise mathematical framework. We are not

just "trying to do our best"; we are systematically searching for a specific numerical value that satisfies a mathematical criterion.

Mathematical Definition of Optimisation

A numerical approach for seeking a **best** decision or action from a **set of alternatives**.

2.2 The Anatomy of an Optimisation Problem

Formally, optimisation equates to the minimisation (or maximisation) of an *objective function* $J(x)$, depending on a variable vector x, subject to constraints. The general mathematical problem is written as:

$$\begin{aligned} \min_{x} \quad & J(x) \\ \text{s.t.} \quad & x \in S \end{aligned} \tag{2.1}$$

To translate a real-world engineering problem into the mathematical form described above, we must identify three fundamental components. Regardless of the complexity, almost every optimisation problem consists of:

1. **Decision Variables (x):** These are the *knobs* the solver can turn to minimise the cost. In the context of MPC, these variables usually represent the sequence of future control inputs (e.g., valve positions, motor currents, steering angles) over the prediction horizon.

2. **Objective Function (J):** A scalar mathematical expression that quantifies *performance*. The solver attempts to find the decision variables that result in the smallest possible value for this function. It typically represents physical quantities such as energy cost, tracking error, fuel consumption, or elapsed time.

3. **Constraints:** The rules that must be obeyed. These define the *feasible set* (denoted as S). Constraints distinguish practical engineering from pure mathematics by strictly enforcing physical limits (e.g., maximum voltage, tank capacity) or safety bounds (e.g., obstacle avoidance).

Mathematically, we often denote the decision variables as z to distinguish them from the system state x used in control theory. The standard optimisation form is written as:

$$\begin{aligned} \min_{x} \quad & J(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \quad \text{(Inequality constraints)} \\ & h_j(x) = 0, \quad j = 1, \dots, q \quad \text{(Equality constraints)} \end{aligned} \tag{2.2}$$

Study note: Constraint Types

- **Inequalities ($g \leq 0$)** describe ranges, like "speed must be less than 100 km/h".
- **Equalities ($h = 0$)** describe strict physics, like $F = ma$. In MPC, the system dynamics ($x_{k+1} = Ax_k + Bu_k$) are treated as equality constraints. Note that here x is the state of the system and not a decision variable.

2.3 Illustrative Examples

To make the abstract definitions of decision variables, objective functions, and constraints concrete, let us consider two distinct optimisation problems.

The process of translating a physical or data-driven problem into a mathematical optimisation format is known as **modelling**. This step is often more challenging than the solution itself. In both examples below, we follow a systematic three-step approach:

1. **Identify the Decision Variables:** What are the *knobs* we can turn?
2. **Formulate the Objective:** What is the single metric that defines *best*?
3. **Define the Constraints:** What are the physical or logical limits we must obey?

The first example (Box Volume) illustrates a geometric design problem where *physical constraints* play a critical role—analogous to actuator limits in control systems. The second example (Linear Regression) introduces the concept of *least-squares minimisation*, which is the mathematical foundation for data science.

Example 2.1: Maximising Box Volume

Problem Statement: Given a square piece of cardboard with dimensions 1 m $\times$ 1 m as shown in Figure 2.1, we wish to construct an open-top box with the maximum possible volume. We achieve this by cutting equal squares of side length x from the four corners and folding up the sides.

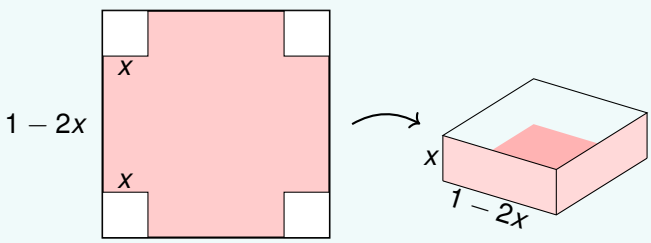


Figure 2.1: Box volume maximisation problem.

Solution:
We map this problem to the three components of optimisation:

1. **Decision Variable (x):** The size of the corner cut.
2. **Constraints:**
   - The cut must be positive: $x \geq 0$.
   - The cut cannot overlap (total width is 1m): $2x \leq 1 \implies x \leq 0.5$.
3. **Objective Function:** Maximise Volume V = length $\times$ width $\times$ height.

$$J(x) = \underbrace{(1-2x)\cdot(1-2x)}_{\textbf{Base Area}} \cdot x = x(1-2x)^2$$

Mathematical Formulation:[a]

$$\begin{aligned} \max_x \quad & x(1-2x)^2 \\ \text{s.t.} \quad & 0 \leq x \leq 0.5 \end{aligned}$$

or equivalently

$$\begin{aligned} \min \quad & -x(1-2x)^2 \\ \text{s.t.} \quad & 0 \leq x \leq 0.5 \end{aligned}$$

[a]**Note on Standard Form:** While this example seeks to *maximise* volume, most standard solvers (including `quadprog` in MATLAB) are designed to *minimise* functions. In engineering practice, we simply flip the sign of the objective function. Maximising $f(x)$ is mathematically equivalent to minimising $-f(x)$.

Example 2.2: Linear Regression (Least Squares)

Problem Statement: Suppose we are given a dataset containing N pairs of experimental measurements: $\{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$ as shown in Figure 2.2.

We want to find the linear model (a straight line) that "best fits" this data.

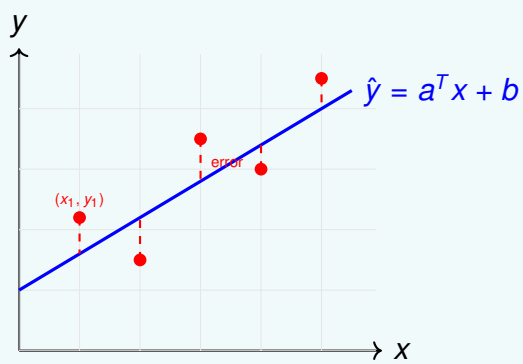


Figure 2.2: Dataset (red), linear interpolation (blue).

Solution:

Note that in this specific problem, the x values are **constants** (data we measured), not decision variables.

1. **Decision Variables:** The parameters of the line: the slope vector $a \in \mathbb{R}^n$ and the intercept $b \in \mathbb{R}$.
   - Note: In the 2D plot above, $n = 1$, so a is a scalar slope. However, the math holds for n-dimensional vectors.
2. **Objective Function:** We define the prediction for the i-th data point as $\hat{y}_i = a^T x_i + b$. We want to minimise the Sum of Squared Errors (SSE):

$$J(a, b) = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{N} \left(y_i - (a^T x_i + b) \right)^2$$

3. **Constraints:** In this case, a and b can be any real numbers. There are no restrictions. Therefore this is an unconstrained optimisation.

Mathematical Formulation:

$$\min_{a,b} \quad \sum_{i=1}^{N} \left(y_i - (a^T x_i + b) \right)^2$$

2.4 Ingredients of Multi-Variable Optimisation

In the previous examples (the box and the line fitting), we dealt with specific cases. Now, let us formalise the general structure of an optimisation problem involving multiple variables. This is the exact structure used in Model Predictive Control, where we must simultaneously optimise a sequence of inputs over time.

2.4.1 Problem Setting: The Decision Vector

In real-world engineering, we rarely make a single scalar decision. Instead, we must determine a set of n values simultaneously. To handle this mathematically, we collect all individual decision

variables $(x_1, x_2, \dots, x_n)$ into a single *decision vector*: $x \in \mathbb{R}^n$:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \tag{2.3}$$

Then the objective function J maps this vector to a scalar real number (a score or cost):

$$J : \mathbb{R}^n \to \mathbb{R}$$

For example, in MPC, if we have a control horizon of 10 steps and 2 inputs, our decision vector x has a dimension of $n = 20$.

2.4.2 Understanding Vector-to-Scalar Mapping: The *cost* function

The mathematical notation $J : \mathbb{R}^n \to \mathbb{R}$ often confuses beginners. It reads: *"The function f takes a vector of n numbers and transforms it into a single real number."*

To understand why this is necessary, let us consider a relatable real-world analogy: The Shopping Basket.

The Analogy

Imagine you are at a supermarket checkout.

1. **The Vector (x):** Your shopping basket contains many different items. You might have 2 loaves of bread, 6 eggs, and 1 litre of milk. We can list these quantities as a vector:

$$x = \begin{bmatrix} 2 \\ 6 \\ 1 \end{bmatrix} \begin{matrix} \leftarrow \text{Bread} \\ \leftarrow \text{Eggs} \\ \leftarrow \text{Milk} \end{matrix}$$

   This vector represents the *state* of your shopping. It is multi-dimensional because it contains different types of information.

2. **The Objective Function (J):** The checkout till scans your items. It applies a specific rule (the price list) to your vector. It multiplies each item by its value and sums them up:

$$J(x) = (2 \times 1.00) + (6 \times 0.20) + (1 \times 0.80)$$

3. **The Scalar (J):** The till produces a single number: £4.00.

This is visually described in Figure 2.3.

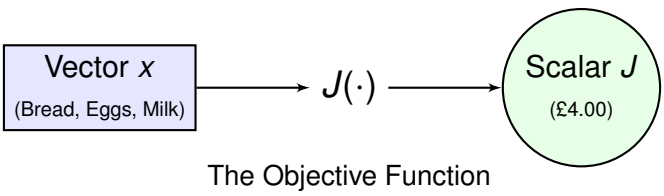


Figure 2.3: Analogy of cost function.

Why is this important in Engineering?

You cannot easily say which of two shopping baskets is *larger* just by looking at the items—one might have more eggs but no milk. However, once you map them to a scalar (the total price), comparison becomes easy. You can instantly say which basket costs less.

In Model Predictive Control, our *basket* (x) contains complex decisions: valve positions, heater settings, and motor speeds over the next 20 minutes. We cannot simply look at two different plans and say *Plan A is better than Plan B,* because Plan A might use less fuel but Plan B might track the temperature better.

The objective function $J(x)$ squashes all these complex, multi-dimensional decisions into a single number (a score).

- If Plan A scores 10.5 and Plan B scores 8.2, and we are minimising cost, we know for a fact that Plan B is the mathematical winner.

Key Takeaway

Optimisation algorithms cannot minimise a list or a vector. They can only minimise a single number. The objective function acts as a *translator*, converting a complex list of engineering decisions into a single score that allows us to rank them.

2.4.3 Constraints and the Feasible Set

We rarely operate in an ideal world. Our decisions are restricted by physical limits. These restrictions are defined by a finite number of equations or inequalities involving the decision variables.

1. **Inequality Constraints (g_i):** These represent bounds, such as "Temperature must be less than 100°C" or "Valve must be open at least 10%".

$$g_i(x) \leq b_i, \quad i = 1, \dots, m$$

2. **Equality Constraints (h_j):** These represent strict physics or model dynamics, such as *Input flow must equal output flow* or $F = ma$.

$$h_j(x) = 0, \quad j = 1, \dots, q$$

3. **Domain Restrictions:** Sometimes variables are not continuous. For instance, a switch is either ON (1) or OFF (0). If p variables are discrete (integers) and the rest are continuous, we denote the domain as $x \in \mathbb{R}^{n-p} \times \mathbb{Z}^p$. Note that dealing with domain restrictions is difficult in optimisation.

Collectively, these constraints define a specific region in the n-dimensional space called the *Feasible Set S*. The optimisation algorithm is only allowed to search for solutions within this set.

$$S = \{x \in \mathbb{R}^n \mid g_i(x) \leq b_i \ \forall i, \ h_j(x) = 0 \ \forall j\} \tag{2.4}$$

If the intersection of these constraints is empty (e.g., $x > 5$ AND $x < 3$), the problem is *infeasible*—there is no solution.

Example 2.3: Visualising the Feasible Set

Consider a 2-dimensional problem ($x \in \mathbb{R}^2$) with two inequality constraints:

1. $g_1(x) = x_1^2 + x_2^2 \leq 4$ (Must be inside a circle of radius 2).
2. $g_2(x) = x_1^3 - x_2 \leq 0 \implies x_2 \geq x_1^3$ (Must be above the cubic curve).

The feasible set S is the intersection of these two regions (the purple area as shown in Figure 2.4).

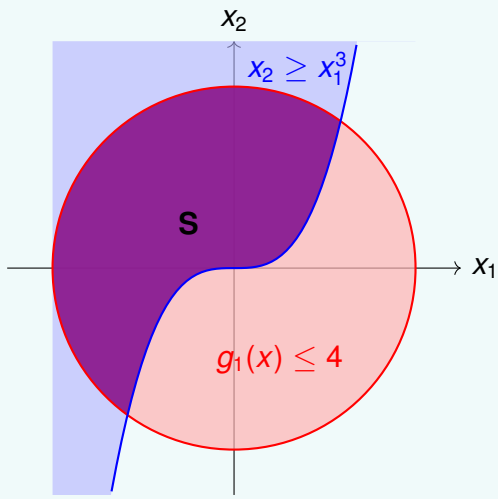


Figure 2.4: Intersection of constraints. Blue region $x_2 \geq x_1^3$, red region $x_1^2 + x_2^2 \leq 4$. S is the intersection and the feasible domain.

2.4.4 Standard Form: Minimisation vs. Maximisation

Standard numerical solvers (like `quadprog` or `fmincon` in MATLAB) are typically written to perform **minimisation**. However, engineering problems are often phrased as maximisation (e.g., maximise profit, maximise volume).

To reconcile this, we rely on a mathematical identity. But first, we must clarify a precise piece of notation often used in optimisation: the difference between min and arg min.

Notation Note: Minimum vs. Argument of the Minimum:

- $\min f(x)$**:** Returns the **value** of the function at the minimum. This is the score (the vertical axis on a graph).

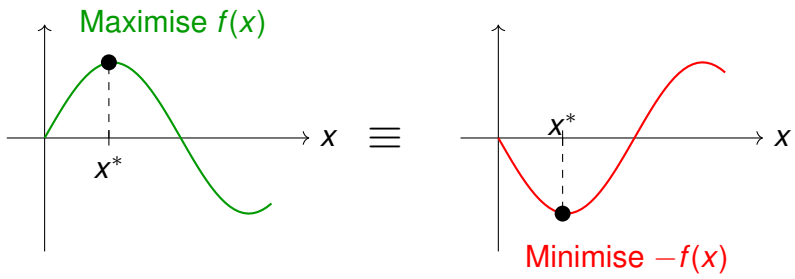


Figure 2.5: Visual representation of the equivalence between maximisation and minimisation. To maximise an objective function $f(x)$ (green curve), one can equivalently minimise its negative, $-f(x)$ (red curve). Note that while the vertical value flips sign, the optimal decision variable x^* (the horizontal position of the black dot) remains identical.

- $\arg\min f(x)$**:** Returns the argument (the variable x) that produces that minimum. This is the decision (the horizontal axis).

In MPC, we are primarily interested in the arg min because we need to know *what input u to apply* to the system, not just what the theoretical cost is.

Converting Maximisation to Minimisation:

With this notation in mind, we can state the equivalence principle. We do not need separate software to solve maximisation problems. We rely on the identity that finding the argument that maximises a function is equivalent to finding the argument that minimises its negative:

$$\underbrace{\arg\max_{x\in S} f(x)}_{\text{Best Decision}} \equiv \underbrace{\arg\min_{x\in S}\{-f(x)\}}_{\text{Equivalent Search}} \tag{2.5}$$

Visually, multiplying the objective function by -1 flips the curve upside down. A peak (maximum) in the original function becomes a valley (minimum) in the negated function. While the vertical values flip signs, the **horizontal location (x)** of the optimal point remains exactly the same. This is pictured in Figure 2.5.

Therefore, in this course, we will consistently formulate our MPC problems as minimisation tasks, usually minimising an energy or error cost function.

2.5 Terminology in Optimisation

With the general structure of an optimisation problem in place, we now refine the vocabulary. Some optimisation problems can be solved in milliseconds on a cheap microcontroller, whilst others might take days on a supercomputer and still fail to find the best solution.

The distinguishing factor is rarely just the size of the problem (the number of variables); rather, it is the geometry of the problem.

The difference between easy and hard problems lies not in the number of variables but in the geometry of the search space. The central concept here is **Convexity**: if the objective is convex and the constraints form a convex set, then any local minimum is guaranteed to be the global minimum. We begin by defining the building blocks of this geometry: sets and functions.

2.5.1 Convex Sets

A fundamental concept in optimisation, particularly in Model Predictive Control (MPC), is that of a **convex set**. Working with convex sets and functions simplifies the optimisation problem significantly, often guaranteeing that any local optimum is also a global optimum.

Convex Set

A set $\mathcal{X}$ is **convex** if and only if for any pair of points x and y in $\mathcal{X}$, any **convex combination** of x and y lies in $\mathcal{X}$.

Mathematically, this definition is stated as:

$$\mathcal{X} \text{ is convex} \quad \Longleftrightarrow \quad \lambda x + (1-\lambda)y \in \mathcal{X}, \quad \forall \lambda \in [0,1], \quad \forall x, y \in \mathcal{X}$$

The term $\lambda x + (1-\lambda)y$ represents a line segment connecting x and y.

Interpretation

All line segments starting and ending in $\mathcal{X}$ stay within $\mathcal{X}$.

To illustrate the concept, consider the following examples of convex and non-convex sets.

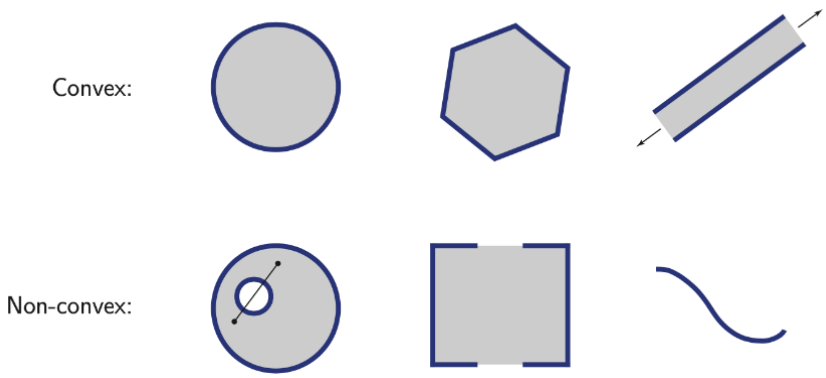


Figure 2.6: Visual examples of convex and non-convex sets. A set is non-convex if a line segment connecting two of its points leaves the set.

In the context of Model Predictive Control (MPC), constraints are often defined by linear inequalities. Geometrically, these constraints represent hyperplanes and half-spaces. Understanding these geometric primitives is essential because they form the building blocks of convex sets.

Hyperplane

A hyperplane is a set of the form:

$$\{x \in \mathbb{R}^n \mid a^\top x = b\} \tag{2.6}$$

where $a \in \mathbb{R}^n$ (with $a \neq 0$) and $b \in \mathbb{R}$. The vector a is the *normal vector* to the hyperplane; it is orthogonal to any vector lying within the hyperplane.

In $\mathbb{R}^2$, a hyperplane defines a line. In $\mathbb{R}^3$, it defines a plane. This is shown on the left of Figure 2.7.

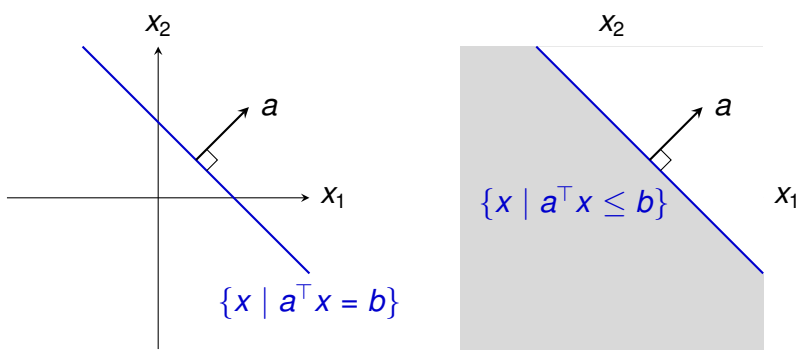
Half space

A hyperplane divides the space $\mathbb{R}^n$ into two halves. A half-space is the set of points on one side of (or on) the hyperplane:

$$\{x \in \mathbb{R}^n \mid a^\top x \leq b\} \tag{2.7}$$

This is known as a *closed* half-space because it includes the boundary (the hyperplane itself). If the inequality is strict ($<$), it is an *open* half-space. This is shown on the right of Figure 2.7.

Hyperplanes and half-spaces are always convex sets. This property ensures that optimisation problems with linear constraints remain convex.



A Hyperplane **A Closed Half-Space**

Figure 2.7: Visual representation of a hyperplane (left) and a closed half-space (right). The vector a is normal to the boundary.

Example 2.4: Linear Constraint in $\mathbb{R}^2$

Consider a linear relation defined by the equation:

$$2x_1 + 1x_2 = 2 \tag{2.8}$$

This represents a line in 2D space. To relate this to the standard hyperplane definition $a^\top x = b$, we define the vectors:

$$a = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad b = 2 \tag{2.9}$$

Thus, the equation becomes:

$$\underbrace{\begin{pmatrix} 2 & 1 \end{pmatrix}}_{a^\top} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \underbrace{2}_{b} \tag{2.10}$$

Connection to the Gradient:

Readers needing a refresher on vectors, matrices, and eigenvalues may consult Appendix A.

We can also view this through the lens of multivariate calculus. Let us define a function $f(x)$ implicitly by the zero-level set:

$$f(x) = 2x_1 + x_2 - 2 = 0 \tag{2.11}$$

The gradient of this function, denoted as ∇f, is a column vector containing the partial derivatives with respect to x_1 and x_2:

$$a = \nabla_x f = \begin{pmatrix} \dfrac{\partial f}{\partial x_1} \\ \dfrac{\partial f}{\partial x_2} \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \tag{2.12}$$

This confirms that the vector $a = (2, 1)^\top$ is the gradient of the linear function defining the hyperplane. Geometrically, the gradient vector is perpendicular to the level sets of the function, meaning a is perpendicular to the line $2x_1 + x_2 = 2$, as shown in Figure 2.8.

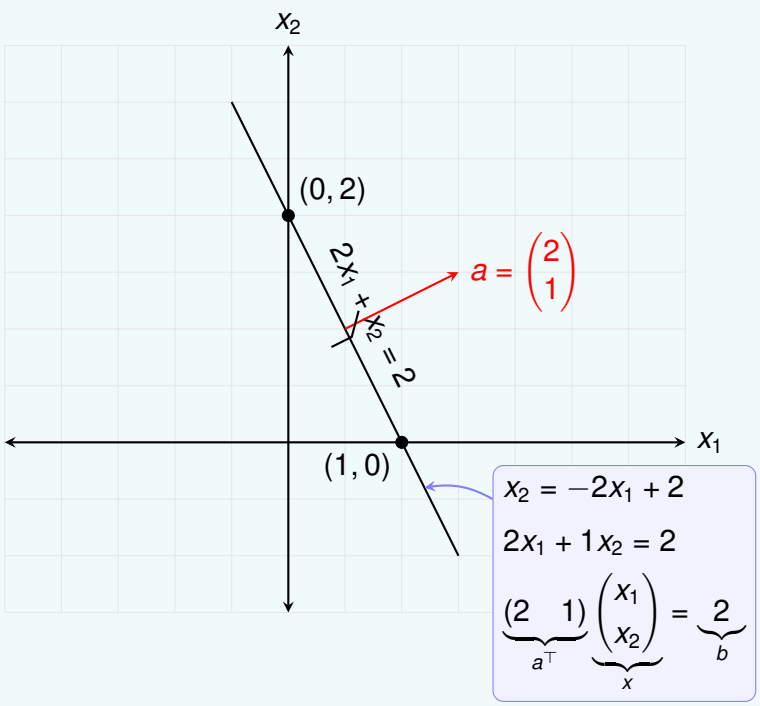


Figure 2.8: Geometric example of the hyperplane $2x_1 + x_2 = 2$. The normal vector $a = (2, 1)^\top$ determines the orientation of the line.

Polyhedron

Let $a_i \in \mathbb{R}^n$ and $b_i \in \mathbb{R}$ for $i = 1, \dots, m$. A *polyhedron* $P \subset \mathbb{R}^n$ is the intersection of a finite number of closed half–spaces,

$$P := \{x \in \mathbb{R}^n \mid a_i^\top x \leq b_i,\ i = 1, \dots, m\}.$$

If we collect the row vectors $a_i^\top$ into a matrix $A := [a_1, \dots, a_m]^\top \in \mathbb{R}^{m \times n}$ and $b := [b_1, \dots, b_m]^\top \in \mathbb{R}^m$, then we may write

$$P := \{x \in \mathbb{R}^n \mid Ax \leq b\},$$

where the inequality is understood row–wise.

Polytope

A *polytope* is a bounded polyhedron, that is, a polyhedron which is contained in a ball of finite radius.

Important observation

Any intersection of convex sets is convex. Each closed half–space $\{x \mid a_i^\top x \leq b_i\}$ is convex, therefore every polyhedron (and hence every polytope) is a convex set. However, union of convex sets are not convex in general.

Example 2.5: Numerical Example: Defining a Polytope

Consider the set $P \subset \mathbb{R}^2$ defined by the inequalities

$$\begin{aligned} x_1 &\geq 0, & x_2 &\geq 0, \\ x_1 &\leq 3, & x_2 &\leq 2, \\ x_1 + x_2 &\leq 4. \end{aligned}$$

We can rewrite these in the standard form $Ax \leq b$ as

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 0 \\ 0 \\ 3 \\ 2 \\ 4 \end{bmatrix}, \qquad P := \{x \in \mathbb{R}^2 \mid Ax \leq b\}.$$

The intersection of these half–spaces is a bounded polygon with vertices

$$\{(0,0),\ (3,0),\ (3,1),\ (2,2),\ (0,2)\},$$

so P is a polytope.

The polytope P can be visualised in the (x_1, x_2)–plane as shown in Figure 2.9:

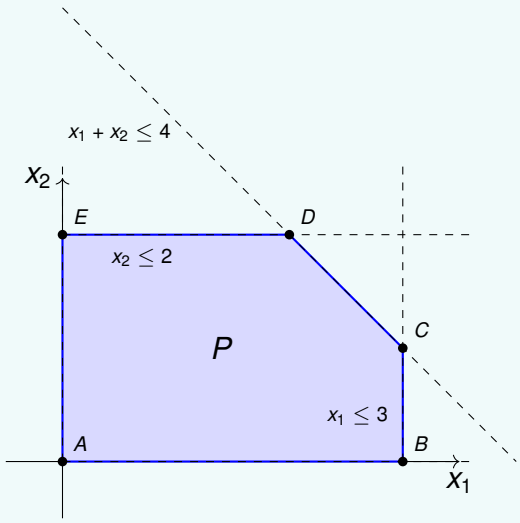


Figure 2.9: Intersection of half spaces is a convex polyhedron.

2.5.2 Convex Functions

Let S be a convex subset of a vector space. We define the properties of functions mapping S to the real numbers $\mathbb{R}$ as follows.

Convex Function

A function $f : S \to \mathbb{R}$ is said to be **convex** if S is convex and the following inequality holds for all $z_1, z_2 \in S$ and for all $\lambda \in [0, 1]$:

$$f(\lambda z_1 + (1 - \lambda)z_2) \leq \lambda f(z_1) + (1 - \lambda)f(z_2) \tag{2.13}$$

Geometrically, this means that the line segment connecting any two points on the graph of the function lies above or on the graph between the two points.

Concave Function

A function $f : S \to \mathbb{R}$ is **concave** if S is convex and $-f$ is a convex function. Equivalently, the inequality in the definition of convexity is reversed.

The concept of convexity is illustrated in Figure 2.10. The curve represents the function f. The red secant line connects two points $(z_1, f(z_1))$ and $(z_2, f(z_2))$. For a convex function, the value of the function evaluated at a weighted average of two points (the black dot on the curve) is always less than or equal to the weighted average of the function values (the black dot on the red line).

2.6 Classification of Optimisation Problems

In the previous sections, we defined the general anatomy of an optimisation problem (Variables, Objectives, Constraints). However, in engineering practice, identifying the specific *class* of a

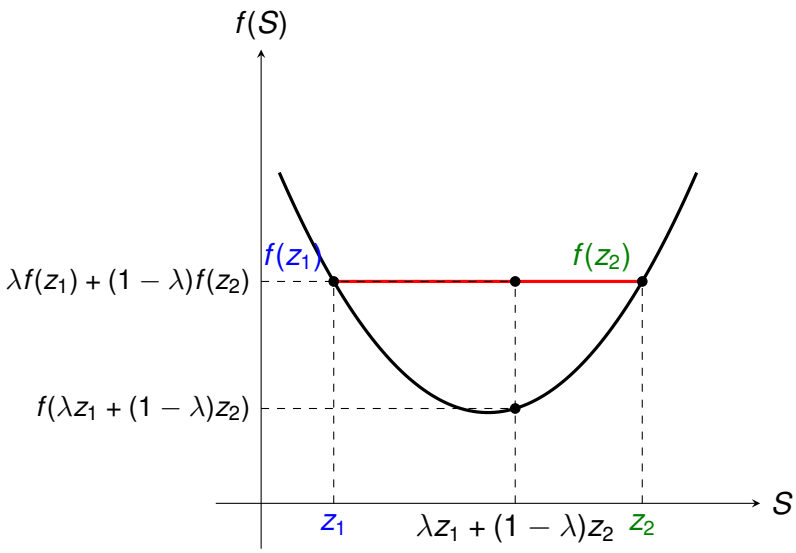


Figure 2.10: Geometric interpretation of a convex function. The line segment connecting any two points on the curve lies above the function values.

problem is critical.

Different classes of problems require different numerical algorithms (solvers). For instance, solving a linear problem is instantaneous, whereas solving a non-convex nonlinear problem might take hours or fail completely. If we can mathematically massage our engineering problem to fit a standard form, we can solve it efficiently using off-the-shelf software.

We typically classify problems in two ways:

1. By the **existence and type of constraints** (Constrained vs. Unconstrained).
2. By the **mathematical structure** of the cost and constraint functions (Linear, Quadratic, Nonlinear).

2.6.1 Classification I: Existence of Constraints

The most fundamental distinction is whether the decision variables are restricted by boundaries.

Unconstrained Optimisation

An unconstrained problem is one where the decision variables can take any value in the Euclidean space $\mathbb{R}^n$. There are no boundaries, fences, or rules limiting x.

$$\min_{x \in \mathbb{R}^n} J(x) \tag{2.14}$$

Visualising Unconstrained Problems: Level Curves When dealing with n variables, the function $J(x)$ describes a surface in $n+1$ dimensions. This is hard to visualise. Instead, we use Level

Curves (or contour lines). A level curve is a hypersurface defined by $J(x) = c$, where c is a constant scalar. This is exactly like looking at a topographical map of a mountain; each line represents a constant height.

Example 2.6: Unconstrained Example: The Bowl

Consider the simple quadratic function:

$$J(x) = x_1^2 + x_2^2$$

Here, $x = [x_1, x_2]^\top$. We want to find the minimum. The level curves are circles where $x_1^2 + x_2^2 = c$.

- If $c = 5$, the curve is a circle of radius $\sqrt{5}$.
- If $c = 0$, the curve is a single point at the origin.

Figure 2.11 shows several level curves $J = 2$, $J = 6$, $J = 12$ and $J = 20$.

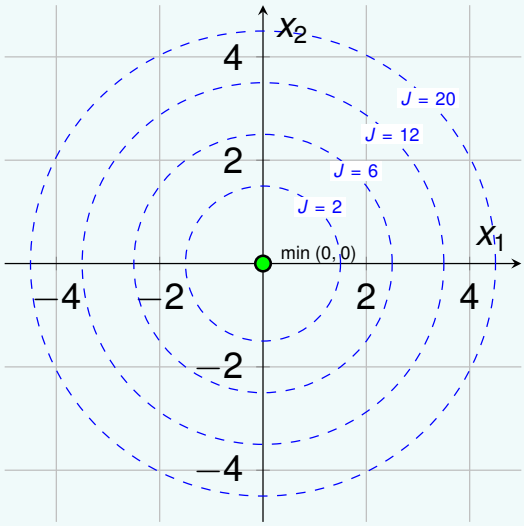


Figure 2.11: Level Curves of $J(x) = x_1^2 + x_2^2$.

The optimiser searches for the level curve with the smallest possible c. Geometrically, this is the centre of the concentric circles.

Constrained Optimisation

In most engineering applications, particularly MPC, we are restricted. Valves cannot open more than 100%, and tanks cannot hold negative volume. This leads to constrained optimisation, where x must belong to a specific feasible set S. We classify these based on how the set S is defined.

Type A: Equality Constraints An equality constrained problem restricts the decision variables to lie strictly on a specific curve or surface ($h(x) = 0$). The solver is not allowed to choose any point off this line. Mathematically, this reduces the dimension of the search space.

Example 2.7: Equality Constraints: The Tangent Point

Problem Statement: Minimise the same quadratic function as before, but subject to a linear equality constraint.

$$\min_x \quad J(x) = x_1^2 + x_2^2$$

subject to:

$$h(x) = x_1 + 2x_2 - 4 = 0$$

Visualisation: This problem is visualised in Figure 2.12.

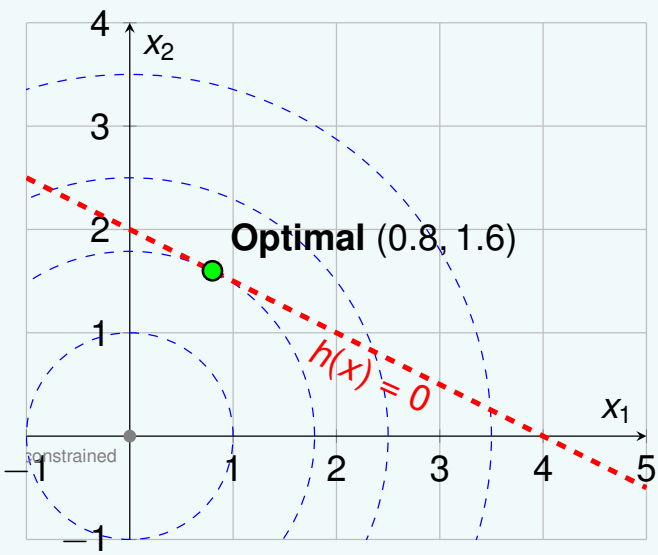


Figure 2.12: Minimising $J(x)$ on the line $h(x) = 0$

.

Solution:

The optimal point is found where the constraint line is exactly tangent to a level curve.

Type B: Inequality Constraints Unlike equality constraints, inequality constraints confine the solution to a *region* (a set of alternatives). The optimal solution may lie on the boundary of this region (active constraint) or strictly inside it (inactive constraint).

Example 2.8: Inequality Constraints: The Feasible Region

Minimise the cost function $J(x) = x_1^2 + x_2^2$, subject to four linear inequalities:

$$\begin{aligned} -x_1 - 2x_2 + 5 &\leq 0 \\ 2x_1 + x_2 - 5 &\leq 0 \\ x_1 \geq 0,\ x_2 &\geq 0 \end{aligned}$$

Solution:

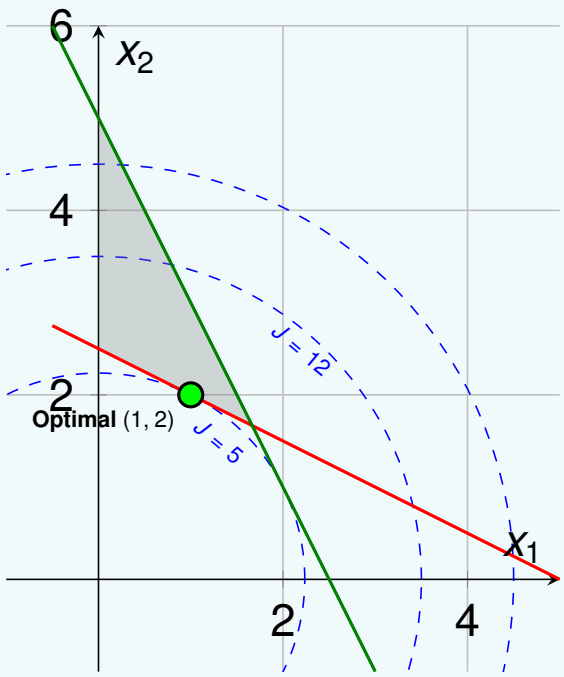
Visualisation is excellent for understanding, but to get numerical answers, we use software. In this course, we utilise **YALMIP**, an open source MATLAB toolbox that allows us to write optimisation problems using syntax that looks almost identical to the mathematics.

The workflow always follows three steps:

Define Optimisation Variables $\rightarrow$ Define Model $\rightarrow$ Solve.

```
% 1. Define Decision Variables
% 'sdpvar' creates a symbolic variable that YALMIP tracks.
x = sdpvar(2,1);

% 2. Define the Objective Function
J = x(1)^2 + x(2)^2;

% 3. Define Constraints
% We create a list of inequalities using standard MATLAB brackets.
Constraints = [ -x(1) - 2*x(2) + 5 <= 0, ...
                2*x(1) +   x(2) - 5 <= 0, ...
                x(1) >= 0, ...
                x(2) >= 0 ];

% 4. Call the Solver
% We ask YALMIP to minimise J subject to Constraints
optimize(Constraints, J);

% 5. Extract Result
% The symbolic variable 'x' now holds the optimal value.
x_star = value(x) % Result: (1, 2)

```

Visualisation: Geometrically, the optimiser "expands" the level curves (blue circles) from the origin until they first touch the feasible region (grey polygon).



Type C: General Constraints In the most general case, a problem contains both equality and inequality constraints. The solver must find a solution that lies inside the inequality region **and** lies exactly on the equality curve.

Geometrically, the feasible set is the intersection of the regions defined by all constraints. This often effectively reduces the dimensionality of the search space.

Example 2.9: General Constraints: Mixing Equalities and Inequalities

Problem Statement: Consider a scenario where we must satisfy both safety limits (inequalities) and strict physical laws (equalities). We wish to minimise the quadratic cost $J(x) = x_1^2 + x_2^2$ subject to:

$$\begin{aligned} \text{s.t.} \quad & -x_1 - 2x_2 + 5 \leq 0 \\ & 2x_1 + x_2 - 5 \leq 0 \\ & x_1 \geq 0, \quad x_2 \geq 0 \\ & x_1 + 2x_2 - 7 = 0 \quad (\textit{Equality}) \end{aligned}$$

Solution:

YALMIP Implementation: When translating this into code, pay close attention to the syntax for the equality constraint. In MATLAB, a single equals sign (=) represents assignment (e.g., setting a variable's value). To represent a logical equality condition ($h(x) = 0$), we must use the double equals sign (==).

```
% 1. Define Variables and Objective
x = sdpvar(2,1);
J = x(1)^2 + x(2)^2;

% 2. Define Inequality Constraints (The Feasible Region)
g1 = [-x(1) - 2*x(2) + 5 <= 0];
g2 = [ 2*x(1) + x(2) - 5 <= 0];
g3 = [x(1) >= 0];
g4 = [x(2) >= 0];

% 3. Define Equality Constraint (The Manifold)
% Note the use of '==' for logical equality
h  = [x(1) + 2*x(2) - 7 == 0];

% 4. Combine and Solve
Constraints = [g1, g2, g3, g4, h];
optimize(Constraints, J);

x_star = value(x) % Result: (1, 3)

```

Visualisation and Analysis: The geometry of this problem is illustrated in Figure 2.13.

- The inequalities form the grey triangular region.
- The equality constraint is represented by the orange dashed line.

A valid solution must satisfy *both* conditions. Therefore, the feasible set is reduced to the specific line segment where the orange line slices through the grey triangle.

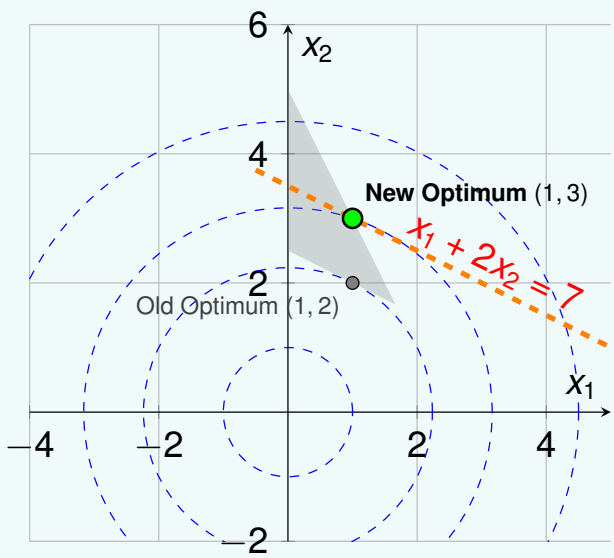


Figure 2.13: Intersection of Equality and Inequalities

The Cost of Constraints: In the previous example (inequalities only), the optimiser could reach the point $(1, 2)$, which yielded a cost of $J = 5$.

However, the new equality constraint forces the solution to lie on the line $x_1 + 2x_2 = 7$. The point $(1, 2)$ is no longer valid because $1 + 2(2) = 5 \neq 7$. The solver is forced to move further away from the origin to satisfy this new rule, settling at $(1, 3)$ with a higher cost of $J = 10$.

This illustrates a fundamental rule of engineering optimisation: Adding constraints generally worsens (increases) the minimum cost, as it restricts the freedom of the solver to find "cheaper" solutions.

2.6.2 Classification II: Mathematical Structure

Optimisation problems vary greatly in difficulty depending on the structure of their objective functions and constraints. In this section we review some of the most commonly encountered problem classes, beginning with those that are considered *easy* from a computational point of view, and progressing to problems that are, in general, far more challenging to solve.

Linear Programmes (LPs)

Linear Programmes constitute one of the most fundamental classes of convex optimisation problems. Both the objective and the constraints are linear:

$$\min_x \; c^\top x$$

subject to:

$$Gx \leq h$$
$$Ax = b$$

The feasible set of an LP is a polyhedron, and the optimum, when it exists, is attained at an extreme point (vertex) of this polyhedron. Linear optimisation on a polytope may be visualised as shifting parallel hyperplanes of constant cost until they touch the feasible region. Figure 2.14 shows a 2D example where the polytope corresponds to the closed area modelled by $Gx \leq h$ and dashed lines shows the cost for different values of $c^\top x$.

Example 2.10: Linear Programme: Minimum-Effort Steady State

The Goal: Consider a discrete-time linear system

$$x_{k+1} = Ax_k + Bu_k$$

We wish to find a steady-state pair (x_{ss}, u_{ss}) (an equilibrium point where the system stays at rest $(x_{k+1} = x_k)$) subject to safety constraints.

Crucially, among all possible valid equilibria, we want to find the one that requires the *least control effort*. In engineering (e.g., satellite thrusters), "effort" is often quantified by fuel consumption, which corresponds to the sum of the absolute values of the inputs.

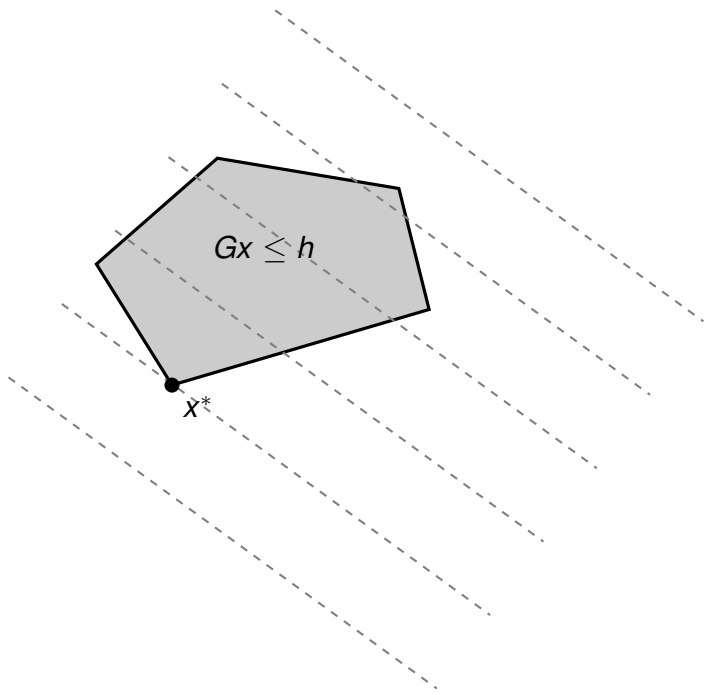


Figure 2.14: Linear optimisation on a polytope.

1. The 1-Norm and Notation: Mathematically, we want to minimise the **1-norm** of the input vector $u \in \mathbb{R}^m$:

$$\|u\|_1 = \sum_{i=1}^{m} |u_i| = |u_1| + |u_2| + \cdots + |u_m| \tag{2.15}$$

However, the absolute value function $|u|$ is non-linear. Standard LP solvers cannot handle it directly. To solve this, we use a standard optimisation trick involving the vector 1.

- Let 1 be a column vector of ones: $1 = [1, 1, \ldots, 1]^\top$.
- Then $1^\top t$ is simply the sum of the elements of vector t: $\sum t_i$.

2. The Epigraph Trick: We introduce an auxiliary decision vector $t \in \mathbb{R}^m$. We constrain t to be an upper bound on the magnitude of u:

$$-t_i \leq u_i \leq t_i \implies |u_i| \leq t_i \tag{2.16}$$

If we minimise the sum of elements in t (i.e., $\min 1^\top t$), the solver will squeeze t_i down until $t_i = |u_i|$. This effectively minimises $|u_i|$ using only linear inequalities.

3. The Formulation: We can now pose the full Linear Programme. The decision variables are the input u, the state x_{ss}, and the auxiliary bound t.

$$\min_{u,\, x_{ss},\, t} \quad 1^\top t \qquad \text{(Minimise total fuel/effort)}$$

subject to:

$$\begin{aligned}
&(I - A)x_{ss} - Bu = 0, && \text{(Equilibrium Physics)}\\
&G\,x_{ss} \leq h, && \text{(Safety Constraints)}\\
&u_{min} \leq u \leq u_{max}, && \text{(Actuator Limits)}\\
&-t \leq u \leq t. && \text{(Norm Linearisation)}
\end{aligned}$$

This formulation allows us to find the most efficient operating point for the system using extremely fast and reliable Linear Programming solvers.

Example 2.11: Stigler's Diet Problem:

The Stigler Diet Problem is a classic linear optimisation problem. The objective is to determine the most cost-effective daily diet that meets specific nutritional requirements using a limited set of foods.

In this simplified 2D version, we consider two food items: Corn and Wheat Bread. We aim to minimise the total daily cost while satisfying constraints for Energy (Calories) and Vitamin A.

We first define the problem using general parameters and units.

Decision Variables:

- x_1: Quantity of Corn [kg]
- x_2: Quantity of Bread [kg]

Parameters:

- c_1, c_2: Price of food [\$/kg]
- k_1, k_2: Energy content [kcal/kg]
- v_1, v_2: Vitamin A content [μg/kg]
- K_{min}, K_{max}: Daily energy limits [kcal]
- V_{min}, V_{max}: Daily Vitamin A limits [μg]

Optimisation Model:

$$\begin{aligned}
\min_{x_1,\, x_2} \quad & J(x) = c_1 x_1 + c_2 x_2\\
\text{Subject to:} \quad & K_{min} \leq k_1 x_1 + k_2 x_2 \leq K_{max}\\
& V_{min} \leq v_1 x_1 + v_2 x_2 \leq V_{max}\\
& x_1 \geq 0, \quad x_2 \geq 0
\end{aligned}$$

Using the specific data provided for the problem:

Data:

- Costs: $c_1 = 2.0$, $c_2 = 0.5$
- Energy: $k_1 = 750$, $k_2 = 650$ (Target: $2000 - 3000$ kcal)
- Vitamin A: $v_1 = 250$, $v_2 = 0$ (Target: $500 - 800$ μg)

Numerically, the problem can be stated as follows:

$$\min_{x_1, x_2} \quad J(x) = 2x_1 + 0.5x_2$$

Subject to:

$$\begin{aligned} &2000 \leq 750x_1 + 650x_2 \leq 3000 && \text{(Energy)} \\ &500 \leq 250x_1 \qquad\quad \leq 800 && \text{(Vitamin A)} \\ &x_1 \geq 0, \quad x_2 \geq 0 && \text{(Nonnegativity)} \end{aligned}$$

Figure 2.15 shows the feasible solution set along with the optimal solution that minimises the cost of nutrition.

```
clear; clc;

x1 = sdpvar(1); % Corn (kg)
x2 = sdpvar(1); % Bread (kg)

Objective = 2*x1 + 0.5*x2;

% 3. Define Constraints
Constraints = [];
Constraints = [Constraints, 2000 <= 750*x1 + 650*x2 <= 3000];
Constraints = [Constraints, 500 <= 250*x1 <= 800];
Constraints = [Constraints, x1 >= 0, x2 >= 0];

% 'linprog' for LP; use 'quadprog' or 'osqp' for QP/SOCP variants.
options = sdpsettings('verbose', 0, 'solver', 'linprog');
sol = optimize(Constraints, Objective, options);

if sol.problem == 0
  fprintf('Optimal Solution:\n');
  fprintf('x1 (Corn) : %.4f kg\n', value(x1));
  fprintf('x2 (Bread): %.4f kg\n', value(x2));
  fprintf('Min Cost  : $%.4f\n', value(Objective));
else
  disp('Problem could not be solved.');
end
```

Listing 2.1: Solving Stigler's Problem with YALMIP

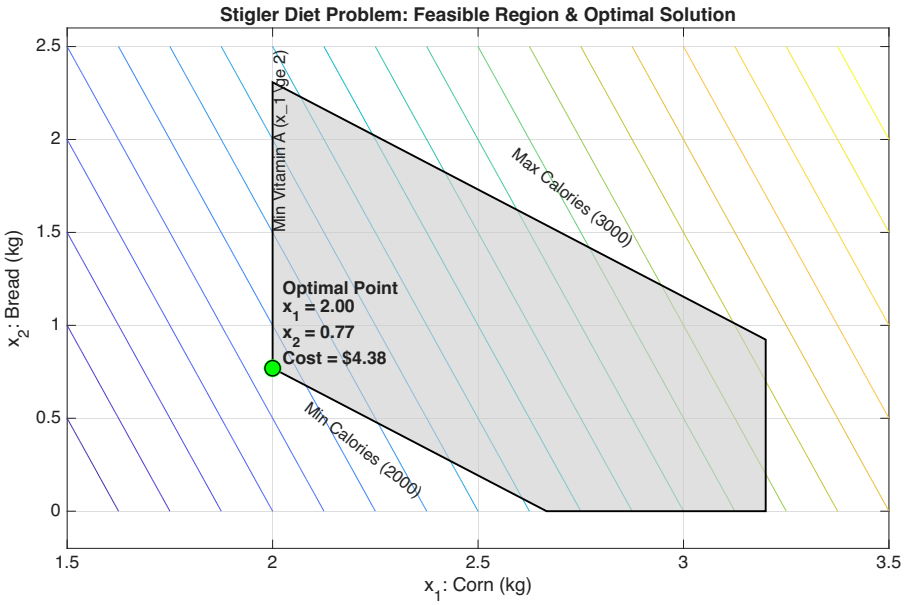


Figure 2.15: Numerical solution of the 2d Stigler problem in MATLAB.

Convex Quadratic Programmes (QPs)

Quadratic Programmes (QPs) extend linear optimisation by allowing a quadratic objective function whilst retaining linear equality and inequality constraints. A QP takes the general form

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^\top P x + q^\top x \\ \text{subject to:} \quad & \\ & Gx \leq h \\ & Ax = b \end{aligned}$$

where, $x \in \mathbb{R}^n$, $P \in \mathbb{R}^{n\times n}$, $q \in \mathbb{R}^n$, $G \in \mathbb{R}^{m\times n}$, and $A \in \mathbb{R}^{p\times n}$. The feasible region defined by the linear constraints is always a polyhedron.

The QP is *convex* if and only if the Hessian matrix P is positive semidefinite:

$$P \succeq 0.$$

This is shown in Figure 2.16.

Tests for Positive Semidefiniteness ($P \succeq 0$)

For a *symmetric* matrix P, the following statements are equivalent and may be used to verify that P is positive semidefinite:

1. **Eigenvalues:** All eigenvalues of P are non-negative:

$$\lambda_i(P) \geq 0, \qquad i = 1, \dots, n.$$

2. **Quadratic Form:** The quadratic form is non-negative for every vector $z \in \mathbb{R}^n$:

$$z^\top P z \geq 0, \qquad \forall z \in \mathbb{R}^n.$$

3. **Principal Minors:** All *principal minors* of P are non-negative. In other words, the determinant of every principal submatrix must satisfy

$$\det(P_{\mathcal{I},\mathcal{I}}) \geq 0$$

for every index set $\mathcal{I} \subseteq \{1, \dots, n\}$.

Important note: For *positive definite* matrices ($P \succ 0$), Sylvester's criterion states that it is enough to check that all *leading principal minors* are strictly positive. However, for *positive semidefinite* matrices ($P \succeq 0$), checking only the leading principal minors is *not* sufficient in general; one must check all principal minors, or use one of the equivalent tests above.

(For a review of eigenvalues, definiteness, and related matrix concepts, see Appendix A.)

Hessian

The Hessian matrix, denoted as $\nabla^2 f(x)$ or $H(x)$, collects all second-order partial derivatives of a scalar-valued function $f : \mathbb{R}^n \to \mathbb{R}$. If $x = [x_1, x_2, \dots, x_n]^\top$, the element in the i-th row and j-th column is given by:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Example 2.12: Example: Hessian

Consider a simple function of two variables:

$$f(x) = 3x_1^2 + 2x_1x_2 + 5x_2^2$$

First, we find the gradient (first derivatives):

$$\frac{\partial f}{\partial x_1} = 6x_1 + 2x_2, \qquad \frac{\partial f}{\partial x_2} = 2x_1 + 10x_2$$

Differentiating again gives the Hessian matrix:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{bmatrix} = \begin{bmatrix} 6 & 2 \\ 2 & 10 \end{bmatrix}$$

Note that for smooth functions, the Hessian is always a **symmetric** matrix ($H_{ij} = H_{ji}$).

In the strictly convex case (where $P \succ 0$), the level sets of the objective function form concentric ellipses. This structure guarantees that the optimisation problem yields a **unique global minimiser**, provided the feasible region is non-empty.

Geometrically, the solver searches for the smallest possible ellipse that still overlaps with the constraints. The optimal solution is found at the precise point where this lowest-cost contour touches (or *grazes*) the feasible polytope. This graphical interpretation is illustrated in Figure 2.16.

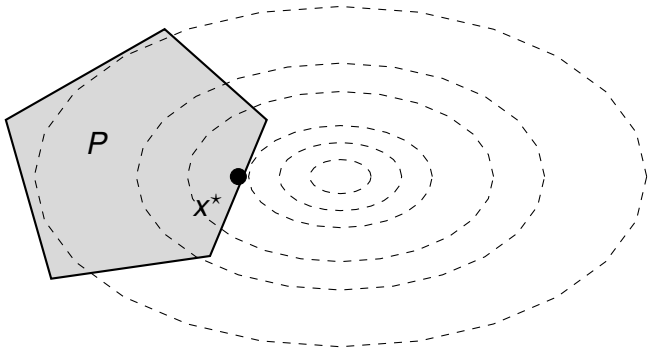


Figure 2.16: Convex quadratic optimisation on a polytope. Contours indicate decreasing objective value towards the optimal point $x^\star$.

Convex QPs benefit from strong theoretical guarantees and can be solved efficiently using interior-point, active-set, or first-order optimisation methods. They are widely used in control, signal processing, finance, estimation, and engineering design.

Example 2.13: Example: Robot Distance to a Restricted Zone

Scenario: Let a robot is located at coordinates $p = (3, 2)$. A restricted geometric zone $\mathcal{X}$ (representing an obstacle or a target landing pad) is defined as a unit box centered at the origin. The robot must compute the shortest distance to this zone to plan its trajectory.

Solution:

Optimisation Problem: Find the point $x \in \mathcal{X}$ closest to p:

$$\min_{x \in \mathbb{R}^2} (x_1 - 3)^2 + (x_2 - 2)^2$$

Constraints (Standard Format): The unit box is defined by bounds $-1 \leq x_1 \leq 1$ and

$-1 \leq x_2 \leq 1$. In standard linear inequality form $Ax \leq b$, these are written as:

$$\begin{aligned} x_1 &\leq 1 \\ -x_1 &\leq 1 \\ x_2 &\leq 1 \\ -x_2 &\leq 1 \end{aligned}$$

Visualising the Solution: By inspecting the geometry (see Figure 2.17), the closest point within the box to $(3, 2)$ is the upper-right corner of the box, $x^* = (1, 1)$.

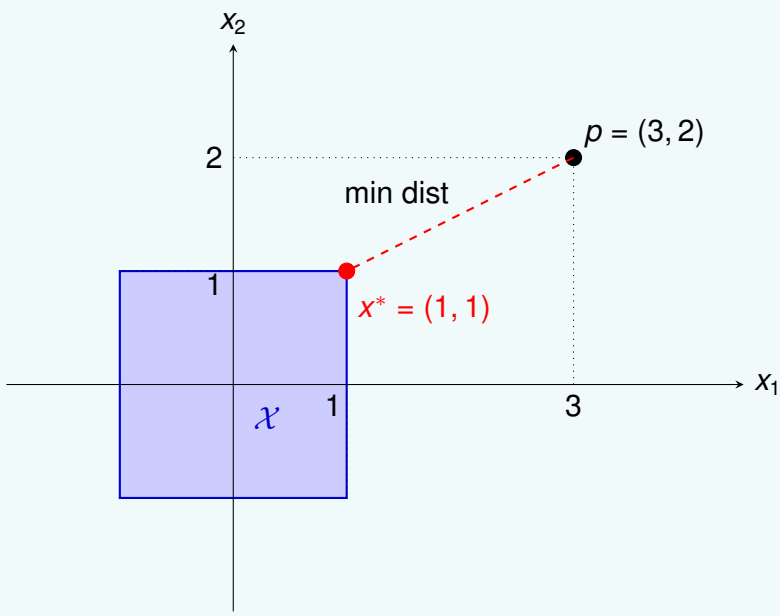


Figure 2.17: Geometry of the optimisation problem.

To solve this problem numerically using a solver (like OSQP, quadprog, or GUROBI), we must map the problem into the standard Quadratic Programming form:

$$\min_x \ \frac{1}{2} x^\top P x + q^\top x \quad \text{subject to} \quad Gx \leq h \tag{2.17}$$

We expand the squared Euclidean distance objective:

$$\begin{aligned} J(x) &= (x_1 - 3)^2 + (x_2 - 2)^2 \\ &= (x_1^2 - 6x_1 + 9) + (x_2^2 - 4x_2 + 4) \\ &= \underbrace{x_1^2 + x_2^2}_{\text{Quadratic part}} \underbrace{-6x_1 - 4x_2}_{\text{Linear part}} + \underbrace{13}_{\text{Constant}} \end{aligned}$$

The constant term (13) does not affect the optimal location x^* and is discarded. To match the standard form $\frac{1}{2}x^\top P x$, the matrix P must be scaled such that the diagonal elements are 2.

$$\frac{1}{2} \underbrace{\begin{bmatrix} x_1 & x_2 \end{bmatrix}}_{x^\top} \underbrace{\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}}_{x} + \underbrace{\begin{bmatrix} -6 & -4 \end{bmatrix}}_{q^\top} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Thus, we define:

$$P = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \quad q = \begin{bmatrix} -6 \\ -4 \end{bmatrix}$$

The unit box constraints are:

$$-1 \leq x_1 \leq 1 \quad \text{and} \quad -1 \leq x_2 \leq 1$$

These must be rewritten as "less than or equal to" inequalities ($Gx \leq h$):

$$\begin{aligned} x_1 &\leq 1 \\ -x_1 &\leq 1 \\ x_2 &\leq 1 \\ -x_2 &\leq 1 \end{aligned}$$

In matrix notation, this becomes:

$$\underbrace{\begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}}_{G} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \leq \underbrace{\begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}}_{h}$$

MATLAB Implementation using YALMIP

Having derived the standard QP matrices manually, we can now solve the problem using MATLAB. The following code demonstrates YALMIP's symbolic framework to automate the process.

```
clear; clc;
ops = sdpsettings('solver', 'quadprog', 'verbose', 0);

p = [3; 2];           % Robot position
x = sdpvar(2, 1);     % Variable

Objective  = (x(1) - p(1))^2 + (x(2) - p(2))^2;

Constraints = [-1 <= x <= 1];

optimize(Constraints , Objective, ops);

x_opt = value(x);
fprintf('Optimal Sol : x1=%.2f, x2=%.2f\n', x_opt(1), x_opt(2));
```

Listing 2.2: Solving the Robot Trajectory QP using YALMIP and quadprog.

Direct Implementation using quadprog

While YALMIP offers a convenient high-level interface, it is essential to understand how to pass the problem matrices directly to a solver. This effectively opens the "black box" of the optimisation tool.

In MATLAB, the native solver `quadprog`[a] solves problems of the form:

$$\min_x \frac{1}{2} x^\top H x + f^\top x \quad \text{subject to} \quad Ax \leq b$$

This syntax requires a direct mapping from our mathematical notation (P, q, G, h) to MATLAB's standard arguments (H, f, A, b).

```
clear; clc;

% 1. Map the Mathematical Notation to MATLAB Syntax
% Objective: 1/2 * x'*P*x + q'*x
H = [2, 0;
     0, 2];          % Our Matrix P
f = [-6; -4];        % Our Vector q

% Constraints: G*x <= h
A = [ 1,  0;
     -1,  0;
      0,  1;
      0, -1];       % Our Matrix G
b = [1; 1; 1; 1];    % Our Vector h

% 2. Set Options (optional, to suppress command window output)
options = optimoptions('quadprog', 'Display', 'off');

% 3. Call the Solver
% Syntax: x = quadprog(H, f, A, b)
x_opt = quadprog(H, f, A, b, [], [], [], [], [], options);

% 4. Display Results
fprintf('Optimal Sol : x1=%.2f, x2=%.2f\n', x_opt(1), x_opt(2));
```

Listing 2.3: Direct solution using MATLAB's built-in quadprog solver.

[a] requires optimization toolbox

Nonconvex Quadratic Programmes

Not all optimisation problems retain convexity. Once convexity is lost, global optimality becomes substantially harder to guarantee. While standard MPC relies on convex optimisation, certain control formulations lead to nonconvex problems. A specific class of these "hard" problems is the Nonconvex Quadratic Programme (QP).

When the matrix P in the quadratic objective is not positive semidefinite ($P \ngeq 0$), the objective

function is nonconvex. This often manifests as saddle-shaped level sets. Consequently, the feasible polytope may contain multiple local minima, making the global optimum difficult to find efficiently.

The formulation is given by:

$$\begin{aligned} \min_x \quad & \frac{1}{2}x^\top Px + q^\top x \\ \text{subject to:} \quad & \\ & Gx \leq h \\ & Ax = b \end{aligned}$$

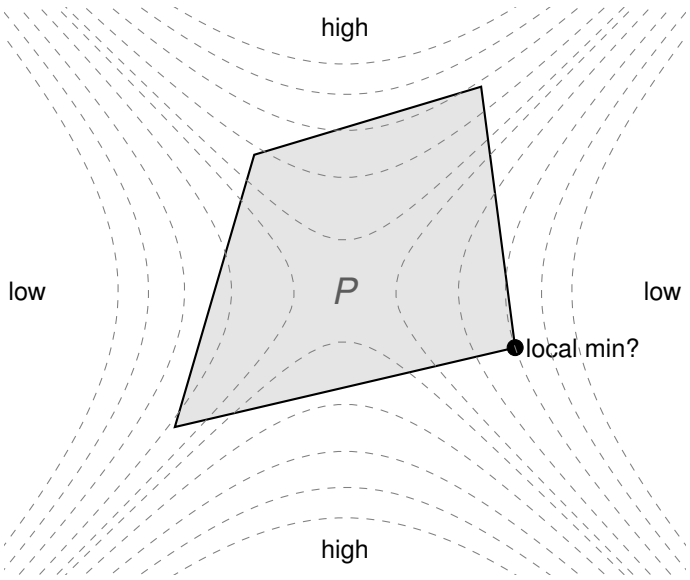


Figure 2.18: Nonconvex quadratic optimisation on a polytope. The dashed lines represent the contours of a saddle-shaped objective function. The global optimum is likely at a vertex, but local minima may exist.

Mixed-Integer Linear Programmes (MILPs)

A Mixed-Integer Linear Programme retains the linear objective and constraints of an LP but imposes integrality restrictions on some or all decision variables:

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{subject to:} \quad & \\ & Gx \leq h \\ & Ax = b \\ & x \in \{0,1\}^n \ \text{ or } \ x \in \mathbb{Z}^n. \end{aligned}$$

The feasible set becomes a set of discrete points embedded within a polyhedron, leading to a combinatorial explosion in complexity. Figure 2.19 shows a sketch of a possible mixed-integer problem.

Example 2.14: MILP: Industrial Energy Management

The General Problem: Consider an industrial site supplied by the electrical grid and equipped with a dispatchable generator and a battery energy storage system. The objective is to minimise the total operating cost over a finite horizon of N discrete time steps of duration Δt, while meeting the site demand and respecting operational constraints.

The generator can be switched on or off and incurs both fuel and start-up costs. The battery is subject to energy dynamics and cannot charge and discharge simultaneously.

For each time step $k = 1, \dots, N$, define:

$p_g(k) \geq 0$	generator power (kW),
$p_{\text{grid}}(k) \geq 0$	grid import power (kW),
$p_{\text{ch}}(k),\ p_{\text{dis}}(k) \geq 0$	battery charge/discharge power (kW),
$E(k)$	battery energy (kWh),
$u(k) \in \{0, 1\}$	generator on/off status,
$s(k) \in \{0, 1\}$	generator start-up indicator,
$z(k) \in \{0, 1\}$	battery charge mode indicator.

Objective function: The total operating cost is given by

$$\min \sum_{k=1}^{N} \Delta t \left(c_{\text{grid}}(k)\, p_{\text{grid}}(k) + c_f\, p_g(k) \right) + \sum_{k=1}^{N} c_{\text{start}}\, s(k), \tag{2.18}$$

where $c_{\text{grid}}(k)$ denotes the time-varying electricity price, c_f the generator fuel cost coefficient, and c_{start} the generator start-up cost.

Constraints:

Power balance:

$$p_{\text{grid}}(k) + p_g(k) + p_{\text{dis}}(k) - p_{\text{ch}}(k) = d(k), \quad \forall k. \tag{2.19}$$

The term $d(k)$ represents the exogenous electrical demand (load) of the system at time step k, assumed to be known over the prediction horizon.

Generator operating limits:

$$P_{g,\min} u(k) \leq p_g(k) \leq P_{g,\max} u(k), \quad \forall k. \tag{2.20}$$

Generator start-up logic:

$$s(1) \geq u(1), \tag{2.21}$$

$$s(k) \geq u(k) - u(k-1), \quad k = 2, \dots, N. \tag{2.22}$$

where $s(k)$ is a binary variable that equals one if the generator is started at time k, i.e. when the on/off status $u(k)$ transitions from 0 to 1.

Battery energy dynamics:

$$E(k+1) = E(k) + \Delta t\, \eta_{\text{ch}}\, p_{\text{ch}}(k) - \Delta t\, \frac{1}{\eta_{\text{dis}}}\, p_{\text{dis}}(k), \tag{2.23}$$

with $E(1) = E_0$. Also, $\eta_{\text{ch}} \in (0, 1]$ and $\eta_{\text{dis}} \in (0, 1]$ denote the charging and discharging efficiencies of the battery, respectively.

Battery and grid limits:

$$E_{\text{min}} \leq E(k) \leq E_{\text{max}}, \tag{2.24}$$
$$0 \leq p_{\text{ch}}(k) \leq P_{\text{ch,max}}, \tag{2.25}$$
$$0 \leq p_{\text{dis}}(k) \leq P_{\text{dis,max}}, \tag{2.26}$$
$$0 \leq p_{\text{grid}}(k) \leq P_{\text{grid,max}}. \tag{2.27}$$

Nonsimultaneous charging and discharging:

$$p_{\text{ch}}(k) \leq P_{\text{ch,max}} z(k), \tag{2.28}$$
$$p_{\text{dis}}(k) \leq P_{\text{dis,max}}[1 - z(k)]. \tag{2.29}$$

Solving this problem: The model above is a Mixed-Integer Linear Programme (MILP) due to the binary variables $u(k), s(k), z(k)$. It requires specialised solvers like Gurobi or CPLEX. In the software section later, we will implement a **simplified** version of this (a continuous LP) to demonstrate the concept in MATLAB.

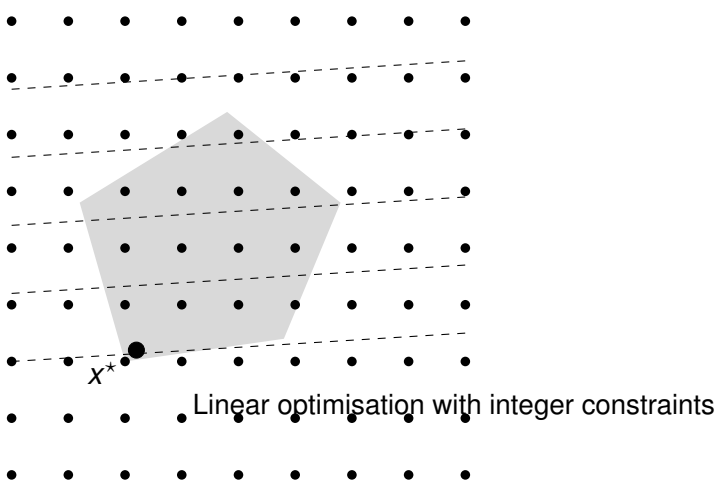


Figure 2.19: MILP feasible set: discrete integer lattice points within a polytope.

2.7 Software Tools for Optimisation

Implementing Model Predictive Control (MPC) requires a robust software stack. Generally, the implementation workflow is divided into two distinct layers:

1. **Parsers (Modeling Languages):** High-level interfaces that allow the user to define the optimisation problem using intuitive algebraic syntax (e.g., `x^2 + y^2 <= 1`). The parser translates this symbolic representation into the standard numerical matrix formats required by the solver.
2. **Solvers:** The low-level numerical engines that perform the mathematical iterations to find the optimal solution (e.g., Active-Set methods or Interior-Point methods).

2.7.1 Modeling Languages (Parsers)

Choosing the right parser depends heavily on the programming environment (MATLAB vs. Python) and the type of problem (Convex vs. Nonlinear).

For **MATLAB** users, **YALMIP** is the de facto standard for prototyping. It supports a vast array of solvers and allows users to switch between them by changing a single line of code. **CVX** is another popular option, specifically designed for Disciplined Convex Programming (DCP), a methodology that requires users to compose their optimisation problems from a fixed library of convex functions and composition rules so that convexity is automatically verified.

For **Python** users, **CVXPY** is the equivalent of CVX, providing a clean object-oriented interface for convex problems.

For **Nonlinear MPC (NMPC)** and high-performance applications, **CasADi** is widely used in both Python and MATLAB. Unlike standard parsers, CasADi implements Algorithmic Differentiation (AD), allowing it to compute exact gradients and Hessians automatically, which is critical for solving nonlinear control problems efficiently.

2.7.2 Numerical Solvers

The choice of solver is dictated by the problem class and the deployment requirements.

Linear and Quadratic Solvers (Standard MPC)

For linear MPC, we typically solve a **Quadratic Programme (QP)**. Solvers like **OSQP** (Operator Splitting Quadratic Programme) and **quadprog** are popular because they are robust, code-generatable, and solve for moderate accuracy very quickly. **qpOASES** is an active-set solver specifically designed for MPC; it uses "warm-starting" to solve the next time-step's problem almost instantly based on the previous solution.

For commercial applications, **Gurobi** and **CPLEX** are the industry leaders, offering unmatched speed for large-scale Mixed-Integer (MIP) and convex problems.

Table 2.1: Comparison of Common Modelling Languages (Parsers)

Name	Platform	License	Primary Purpose & Notes
YALMIP	MATLAB	Free	The *Swiss Army Knife* of MATLAB optimisation. Excellent for prototyping QP, LP, SDP, and MIP.
CVX	MATLAB	Free	Enforces Disciplined Convex Programming (DCP). Guarantees convexity but stricter than YALMIP.
CVXPY	Python	Free (Apache)	The standard for convex optimisation in Python. Intuitive object-oriented syntax.
CasADi	Python/C++	Free (LGPL)	Ideal for Nonlinear MPC (NMPC). Features algorithmic differentiation and C-code generation for embedded targets.
Pyomo	Python	Free (BSD)	General modelling language, widely used in Operations Research (OR) and process systems engineering.
JuMP	Julia	Free (MIT)	Extremely fast modelling language. Gaining significant popularity in the control community.

Conic and Nonlinear Solvers (Robust & Nonlinear MPC)

If the MPC formulation involves ellipsoidal constraints (Robust MPC) or Linear Matrix Inequalities (LMIs), a **Conic Solver** is required. **MOSEK** is the industry leader for these types of problems (SOCP and SDP). For pure MATLAB-based research involving LMIs, **SeDuMi** remains a classic open-source option.

For Nonlinear MPC, **IPOPT** (Interior Point OPTimizer) is the standard open-source engine, often paired with CasADi.

2.8 Solving Optimisation Problems: From Theory to Code

Mathematical formulation is only half the battle. To deploy Model Predictive Control, we must translate these equations into code that a computer can solve. In this section, we explore a complex energy management system as an example.

2.8.1 Economic MPC for Smart Grids

In this example, we explore a sophisticated application of optimisation: **Economic Energy Management**. The goal here is purely economic: minimise the total cost of electricity over a 24-hour horizon.

Table 2.2: List of Common Numerical Solvers

Solver	Type	Method	License	Best For
quadprog	QP	Interior-Point	Proprietary	Standard MATLAB solver (requires Opt. Toolbox). Primary solver used in this course.
OSQP	QP	ADMM[1]	Free (Apache)	Embedded MPC. Very robust, division-free code; widely used in Python/Julia.
qpOASES	QP	Active-Set	Free (LGPL)	Small-to-medium MPC. Excellent warm-starting capabilities for dense problems.
Gurobi	LP, QP, MIP	Simplex/Barrier	Commercial	Gold standard for MIP and convex speed. Free academic licenses available.
MOSEK	SOCP, SDP	Interior-Point	Commercial	The leader for Robust MPC (Conic constraints) and Semidefinite programming.
SeDuMi	SDP, SOCP	Interior-Point	Free (GPL)	Established MATLAB solver for Linear Matrix Inequality(LMI)-based control and stability analysis.
IPOPT	NLP	Interior-Point	Free (EPL)	Large-scale nonlinear optimisation (NMPC). Standard engine for CasADi.
SNOPT	NLP	SQP	Commercial	Highly nonlinear, non-convex problems (e.g., aerospace trajectory planning).
ECOS	SOCP, LP	Interior-Point	Free (GPL)	Lightweight, embedded-friendly Second-Order Cone solver.

Problem Scenario

Although the general problem formulation (the MILP discussed earlier) included complex generator logic, for this MATLAB example we will solve a simplified but highly relevant subset: Battery Energy Arbitrage.

Consider a home with:

- **Load (P_{load}):** The household power consumption (lighting, appliances).
- **Solar Generation (P_{solar}):** Free renewable energy available during the day.
- **Battery Storage:** A unit capable of storing energy (E_{bat}) to bridge the gap between supply and demand.
- **Grid Connection:** Allows buying power when needed or selling excess power.

The Challenge: Electricity prices fluctuate throughout the day (Time-of-Use pricing). Prices are low at night and high in the evening. We must decide the optimal battery charge/discharge schedule to minimise the daily bill.

Mathematical Formulation

We define the discrete time step $\Delta t = 1$ hour and the horizon $N = 24$ hours.

1. Variables:

- $P_{grid}(k)$: Power exchanged with the grid (kW). Positive = Buy, Negative = Sell.
- $P_{bat}(k)$: Battery charging power (kW). Positive = Charge, Negative = Discharge.
- $E_{bat}(k)$: Energy stored in the battery (kWh).

2. The Objective Function: Minimise the cumulative cost of grid usage:

$$\min_{P_{bat}, P_{grid}} \sum_{k=0}^{N-1} \text{Price}(k) \times P_{grid}(k) \cdot \Delta t \tag{2.30}$$

3. Constraints:

$$\text{Power Balance:} \quad P_{grid}(k) + P_{solar}(k) = P_{load}(k) + P_{bat}(k) \tag{2.31}$$

$$\text{Dynamics:} \quad E_{bat}(k+1) = E_{bat}(k) + P_{bat}(k) \cdot \Delta t \tag{2.32}$$

$$\text{Capacity Limits:} \quad E_{min} \leq E_{bat}(k) \leq E_{max} \tag{2.33}$$

$$\text{Power Limits:} \quad P_{min} \leq P_{bat}(k) \leq P_{max} \tag{2.34}$$

Implementation (MATLAB & YALMIP)

We use YALMIP to translate this word problem into a numerical format. Since this problem is purely continuous, we solve it as a Linear Programme (LP) using `quadprog` (with zero quadratic cost) or `linprog`.

```
% --- 1. System Parameters ---
N = 24;                    % Horizon (24 hours)
E_cap = 13.5;              % Battery Capacity (kWh, e.g., Tesla Powerwall)
P_max = 5;                 % Max Charge/Discharge rate (kW)
dt = 1;                    % Time step (1 hour)

% --- 2. Forecast Data (Profiles) ---
% Price: 0.10 (Off-peak), 0.30 (Peak: 17:00-21:00)
Price = 0.10 * ones(N,1);  Price(18:21) = 0.30;

% Solar: Peak at noon (simple sine wave approx)
t = (0:N-1)';
Solar = max(0, 4 * sin(pi * (t - 6) / 12));

% Load: Peaks in morning and evening
```

```
Load = 1 + 2 * exp(-(t-19).^2/10) + 1 * exp(-(t-8).^2/5);

% --- 3. Optimisation Variables ---
P_grid = sdpvar(N, 1);   % Grid Power
P_bat  = sdpvar(N, 1);   % Battery Power
E_bat  = sdpvar(N+1, 1); % Battery State of Charge

% --- 4. Constraints Construction ---
C = [];

% Start with empty battery (Assumption for this specific day)
% In real MPC, E_bat(1) would equal the measured sensor value.
C = [C, E_bat(1) == 0];

for k = 1:N
  % Dynamics: Energy integration
  C = [C, E_bat(k+1) == E_bat(k) + P_bat(k) * dt];

  % Power Balance: Supply (Grid+Solar) = Demand (Load+BatteryCharge)
  C = [C, P_grid(k) + Solar(k) == Load(k) + P_bat(k)];

  % Physical Constraints
  C = [C, 0 <= E_bat(k+1) <= E_cap]; % Capacity limits
  C = [C, -P_max <= P_bat(k) <= P_max]; % Power limits
end

% --- 5. Objective: Minimise Total Cost ---
Objective = Price' * P_grid;

% --- 6. Solve ---
ops = sdpsettings('solver','quadprog','verbose',0);
sol = optimize(C, Objective, ops);
```

Visualising the Result

The result is best understood visually (Figure 2.20).

1. **Pre-Charging:** Notice that between 00:00 and 06:00, the battery charges (blue bars positive) even though the sun isn't shining. It buys cheap electricity to store it.

2. **Peak Shaving:** During the peak price window (17:00–21:00), the battery discharges (blue bars negative). The optimiser "decided" to sell the stored energy to avoid paying the high grid price, drastically reducing the bill.

2.9 Key Takeaways

- **Optimisation is Translation:** MPC translates qualitative engineering goals (e.g., "drive fast but don't crash") into quantitative mathematical statements (e.g., "minimise travel time subject to velocity limits").

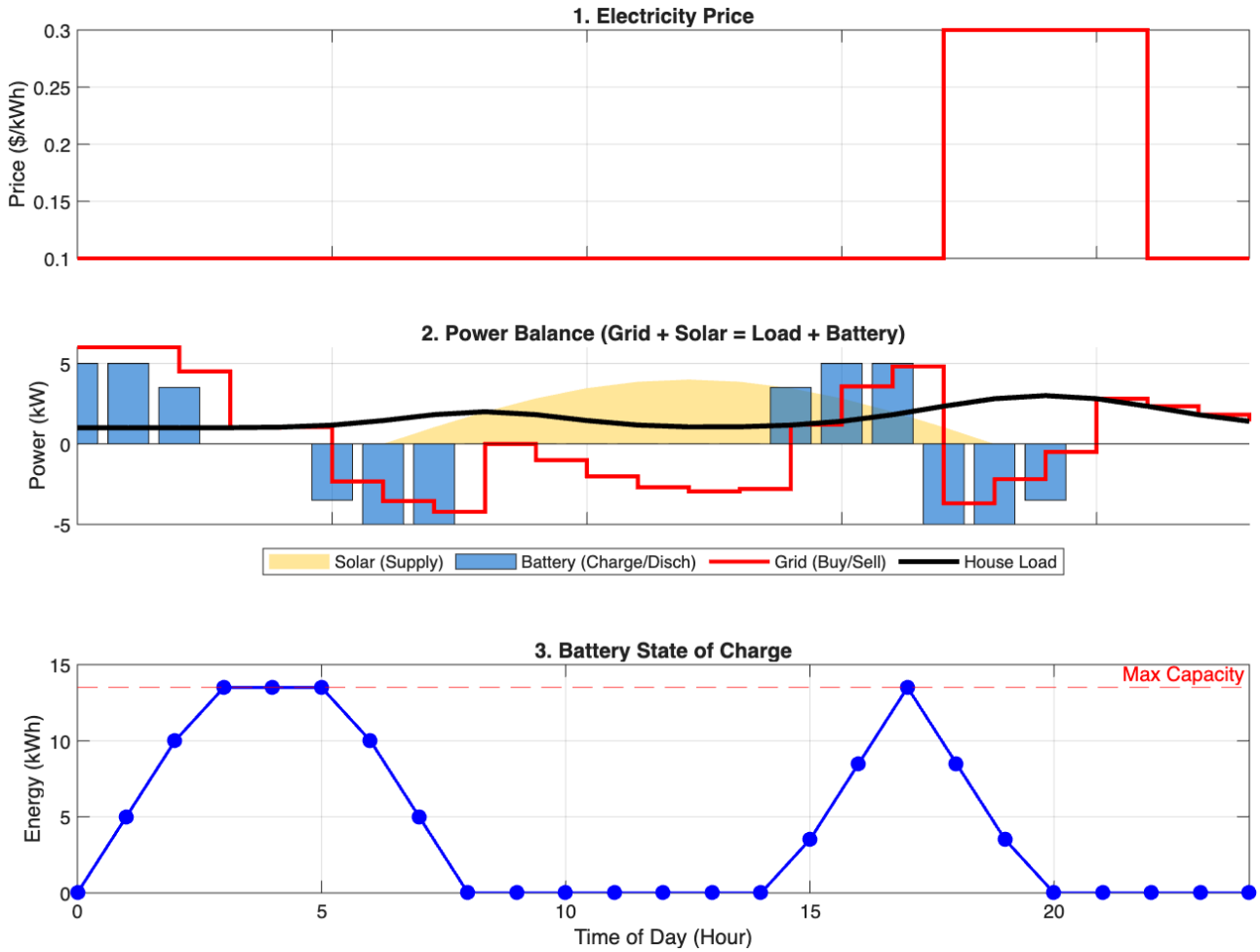


Figure 2.20: **Economic Optimisation Results. Top:** The Time-of-Use price signal with a peak period between hours 17:00 and 21:00. **Middle:** Visual verification of the power balance equation ($P_{grid} + P_{solar} = P_{load} + P_{bat}$). Note the **Grid usage (Red line)**: the system buys power at night to pre-charge the battery (positive red step) and sells power back to the grid during the peak price window (negative red step), effectively performing energy arbitrage. **Bottom:** The state of charge trajectory, showing the battery filling up in anticipation of the price spike and emptying completely to maximise revenue.

- **The Scalar Score:** An algorithm cannot minimise a vector or a list of conflicting goals. The *objective function* $J(x)$ serves to compress complex multi-variable decisions into a single scalar number (a cost), allowing different control strategies to be ranked and compared.
- **Constraints Define Reality:** Unlike classical control, where saturation is often an afterthought, optimisation explicitly handles physical limitations. The *feasible set* is the intersection of all equality and inequality constraints. If this set is empty, the problem is infeasible, and no solution exists.
- **Convexity:** The most critical property of an optimisation problem is convexity. If the objective function is convex (bowl-shaped) and the constraints form a convex set, any local minimum found is guaranteed to be the **global minimum**. This reliability is essential for safety-critical control systems.
- **Standard Forms:**
  - **LP (Linear Programme):** Linear cost and constraints. extremely fast; used for economic dispatch and simple regulation.

  - **QP (Quadratic Programme):** Quadratic cost (energy/error) and linear constraints. This is the standard formulation for linear MPC.
  - **MILP (Mixed-Integer):** Contains discrete decisions (On/Off). Computationally expensive but necessary for logic-based control.

- **Minimisation vs. Maximisation:** Solvers generally seek to minimise cost. To maximise a benefit (like profit), we simply minimise the negative of that function:

$$\arg\max f(x) \equiv \arg\min(-f(x))$$

Exercises

1. **Convex Sets and Functions.** Determine whether each of the following sets is convex. Justify your answer using the definition $\lambda x + (1-\lambda)y \in \mathcal{X}$ for all $\lambda \in [0,1]$ and all $x, y \in \mathcal{X}$.

   (a) $\mathcal{X}_1 = \{x \in \mathbb{R}^2 \mid x_1^2 + x_2^2 \leq 4\}$

   (b) $\mathcal{X}_2 = \{x \in \mathbb{R}^2 \mid x_1^2 + x_2^2 \geq 1\}$

   (c) $\mathcal{X}_3 = \{x \in \mathbb{R}^2 \mid 2x_1 + 3x_2 \leq 6,\ x_1 \geq 0,\ x_2 \geq 0\}$

   Furthermore, determine whether $f(x) = 5x_1^2 - 4x_1x_2 + 3x_2^2$ is convex by computing its Hessian and checking the eigenvalues.

2. **Gradient and Hessian Computation.** Consider the function

$$J(x) = 4x_1^2 + 6x_1x_2 + 8x_2^2 - 10x_1 + 2x_2.$$

   (a) Compute the gradient $\nabla J(x)$ and find the stationary point by solving $\nabla J(x) = 0$.

   (b) Compute the Hessian matrix $H = \nabla^2 J(x)$.

   (c) Determine whether H is positive definite, positive semidefinite, or indefinite. What does this imply about the stationary point found in (a)?

3. **Optimality Conditions for Equality-Constrained Problems.** Consider the problem

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & J(x) = x_1^2 + x_2^2 \\ \text{s.t.} \quad & h(x) = 3x_1 + x_2 - 6 = 0. \end{aligned}$$

   (a) Sketch the level curves of $J(x)$ and the constraint line $h(x) = 0$ in the (x_1, x_2)-plane.

   (b) Write down the Lagrangian $\mathcal{L}(x, \lambda) = J(x) + \lambda\, h(x)$ and derive the first-order (KKT) necessary conditions $\nabla_x \mathcal{L} = 0$ and $h(x) = 0$.

   (c) Solve the resulting system of equations for x_1^*, x_2^*, and the Lagrange multiplier λ^*.

4. **KKT Conditions with Inequality Constraints.** A process engineer wishes to minimise the cost function $J(x) = (x_1 - 4)^2 + (x_2 - 3)^2$ subject to:

$$\begin{aligned} g_1(x) &= x_1 + x_2 - 5 \leq 0 \\ g_2(x) &= -x_1 \leq 0 \\ g_3(x) &= -x_2 \leq 0 \end{aligned}$$

   (a) Sketch the feasible set S and the level curves of $J(x)$.

(b) State the full Karush–Kuhn–Tucker (KKT) conditions for this problem, including stationarity, primal feasibility, dual feasibility, and complementary slackness.

(c) Identify which constraints are active at the optimum and solve for the optimal point x^* and the associated multipliers.

5. **Linear Programme Formulation.** A small factory produces two products, A and B, using two machines. The relevant data is given below:

	Machine 1 (hrs/unit)	Machine 2 (hrs/unit)	Profit (£/unit)
Product A	2	1	8
Product B	1	3	6
Available	10 hrs	12 hrs	

(a) Formulate the problem of maximising total profit as a Linear Programme. Clearly state the decision variables, objective function, and all constraints (including non-negativity).

(b) Convert the maximisation problem to an equivalent minimisation problem in the standard form min $c^\top x$ subject to $Gx \leq h$.

(c) Identify the matrices c, G, and h explicitly.

6. **Quadratic Programme: Standard Form and Convexity.** Consider the following optimisation problem arising from a simplified MPC formulation:

$$\begin{aligned} \min_{u \in \mathbb{R}^2} \quad & J(u) = 3u_1^2 + 2u_1u_2 + 4u_2^2 - 8u_1 - 6u_2 \\ \text{s.t.} \quad & u_1 + u_2 \leq 5 \\ & 0 \leq u_1 \leq 3 \\ & 0 \leq u_2 \leq 4 \end{aligned}$$

(a) Rewrite the objective in the standard QP form $\frac{1}{2}u^\top Pu + q^\top u$ by identifying the matrices P and q.

(b) Show that P is positive definite (and hence the problem is convex) by computing its eigenvalues.

(c) Express all constraints in the standard inequality form $Gu \leq h$ and identify the matrices G and h.

(d) Write a short YALMIP script (or `quadprog` call) that solves this QP in MATLAB.

7. **From Engineering to Optimisation: Satellite Fuel Minimisation.** A satellite has two thrusters providing control inputs u_1 and u_2 (in Newtons). The discrete-time dynamics of its position $x \in \mathbb{R}^2$ are given by:

$$x_{k+1} = Ax_k + Bu_k, \qquad A = \begin{bmatrix} 1 & 0.5 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0.125 \\ 0.5 \end{bmatrix}.$$

The satellite must reach a desired steady-state position x_{ss} satisfying $(I - A)x_{ss} = Bu_{ss}$, while minimising the total fuel consumption $\|u_{ss}\|_1$, subject to $-10 \leq u_{ss} \leq 10$.

(a) Show that the steady-state equilibrium condition leads to an equality constraint that is linear in (x_{ss}, u_{ss}).

(b) Using the epigraph trick described in this chapter, reformulate the 1-norm minimisation as a Linear Programme with decision variables (x_{ss}, u_{ss}, t).

(c) Write down all the constraints explicitly, including the equilibrium condition, actuator bounds, and the auxiliary norm linearisation constraints $-t \leq u_{ss} \leq t$.

Chapter 3

State-Space Representation of Dynamic Systems

> *"The sciences do not try to explain, they hardly even try to interpret, they mainly make models. By a model is meant a mathematical construct which, with the addition of certain verbal interpretations, describes observed phenomena."*
> — **John von Neumann**

The success of any Model Predictive Control strategy hinges on the quality of the controlled system's internal model. To predict the future evolution of a system and optimise control actions, we require a mathematical representation that is both accurate enough to capture the relevant dynamics and computationally efficient enough to be solved in real-time.

While the physical world operates in continuous time and is inherently nonlinear, MPC algorithms typically run on digital processors in discrete time steps. Furthermore, to utilise fast convex solvers (such as Quadratic Programming), we often approximate these dynamics as linear systems. Therefore, the ability to translate physical laws into Linear Time-Invariant (LTI) state-space models and discretise them accurately is a fundamental prerequisite for MPC design.

In this chapter, we establish the mathematical modelling framework used by the predictive controllers developed in later chapters. We will cover:

- A very short review of the **Continuous-Time (CT) LTI state-space** formulation.
- Practical methods for deriving models of **electrical and mechanical systems** from first principles.
- The concept of **equilibrium points** and **local linearisation** to approximate nonlinear dynamics.
- Techniques for **discretisation**, including the analytic solution for linear systems (Zero-Order Hold) and numerical integration methods (Forward Euler, Runge-Kutta) for nonlinear systems.
- An analysis of **stability** in the discrete-time domain.

3.1 What is state-space, and why is it central to MPC?

In control theory, and particularly in Model Predictive Control (MPC), the state-space formulation provides the fundamental language for modelling dynamic systems. Rather than describing behaviour only through input–output relationships, the state-space approach characterises a system through a set of internal variables—called *states*—that capture the system's stored energy, configuration, or memory.

The key idea is that, if the state of a system is known at a given time, together with future inputs, then its future evolution is fully determined. This property makes the state-space framework especially well suited to prediction-based control strategies such as MPC.

General form of a state-space model

A general nonlinear continuous-time state-space model is compactly written as:

$$\dot{x}(t) = f\big(x(t), u(t), t\big), \qquad y(t) = h\big(x(t), u(t), t\big).$$

While this vector notation is elegant, it represents a system of coupled first-order differential equations. Explicitly expanding the vectors reveals the underlying structure:

$$\underbrace{\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \vdots \\ \dot{x}_n(t) \end{bmatrix}}_{\dot{x}(t)} = \underbrace{\begin{bmatrix} f_1\big(x_1(t), \dots, x_n(t),\ u_1(t), \dots, u_m(t),\ t\big) \\ f_2\big(x_1(t), \dots, x_n(t),\ u_1(t), \dots, u_m(t),\ t\big) \\ \vdots \\ f_n\big(x_1(t), \dots, x_n(t),\ u_1(t), \dots, u_m(t),\ t\big) \end{bmatrix}}_{f(x(t),u(t),t)} \tag{3.1}$$

Similarly, the output equation relates the internal states to the measurable quantities:

$$\underbrace{\begin{bmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_p(t) \end{bmatrix}}_{y(t)} = \underbrace{\begin{bmatrix} h_1\big(x_1(t), \dots, x_n(t),\ u_1(t), \dots, u_m(t),\ t\big) \\ h_2\big(x_1(t), \dots, x_n(t),\ u_1(t), \dots, u_m(t),\ t\big) \\ \vdots \\ h_p\big(x_1(t), \dots, x_n(t),\ u_1(t), \dots, u_m(t),\ t\big) \end{bmatrix}}_{h(x(t),u(t),t)} \tag{3.2}$$

Here:

- $x_i(t)$ represents the i-th state variable (e.g., position, voltage).
- $u_j(t)$ represents the j-th control input (e.g., force, current).
- $f_i(\cdot)$ describes the dynamic evolution of the i-th state.

Example 3.1: Nonlinear Modelling of a Simple Pendulum

To illustrate the transition from physical laws to state-space equations, consider the classic simple pendulum system shown in Figure 3.1, consisting of a mass m attached to a rigid, massless rod of length ℓ. The pivot is acted upon by an external control torque $u(t)$ and a viscous friction coefficient b.

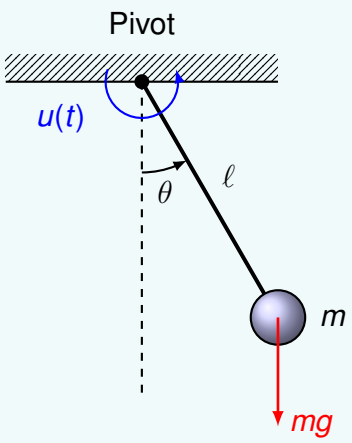


Figure 3.1: Pendulum system.

1. Physical Laws
We apply Newton's second law for rotational motion:

$$\tau_{\text{net}} = I\ddot{\theta}$$

where $I = m\ell^2$ is the moment of inertia and θ is the angular position measured from the vertical downward position. The torques acting on the system are:

- **Control Torque:** $+u(t)$ (applied at the pivot).
- **Gravity:** $-mg\ell \sin\theta$ (restoring torque).
- **Friction:** $-b\dot{\theta}$ (viscous damping opposing motion).

Substituting these into the equation of motion:

$$m\ell^2\ddot{\theta} = u(t) - mg\ell \sin\theta - b\dot{\theta}$$

Solving for the angular acceleration $\ddot{\theta}$:

$$\ddot{\theta} = -\frac{g}{\ell}\sin\theta - \frac{b}{m\ell^2}\dot{\theta} + \frac{1}{m\ell^2}u(t)$$

2. State-Space Formulation
To convert this second-order differential equation into first-order state-space form, we define the state variables:

$$x_1(t) = \theta(t) \quad \text{(Angle)}, \qquad x_2(t) = \dot{\theta}(t) \quad \text{(Angular Velocity)}$$

Taking the time derivative of the state vector:

$$\begin{aligned}\dot{x}_1 &= \dot{\theta} = x_2 \\ \dot{x}_2 &= \ddot{\theta} = -\frac{g}{\ell}\sin x_1 - \frac{b}{m\ell^2}x_2 + \frac{1}{m\ell^2}u\end{aligned}$$

We can now write this explicitly in the vector form $\dot{x} = f(x, u)$ introduced previously:

$$\underbrace{\begin{bmatrix}\dot{x}_1 \\ \dot{x}_2\end{bmatrix}}_{\dot{x}(t)} = \underbrace{\begin{bmatrix} x_2 \\ -\frac{g}{\ell}\sin(x_1) - \frac{b}{m\ell^2}x_2 + \frac{1}{m\ell^2}u \end{bmatrix}}_{f(x,u)} \tag{3.3}$$

If our objective is to measure the angle, the output equation becomes:

$$y(t) = \underbrace{\begin{bmatrix}1 & 0\end{bmatrix}\begin{bmatrix}x_1 \\ x_2\end{bmatrix}}_{h(x)} = x_1$$

This clearly demonstrates the nonlinear nature of the system, arising from the trigonometric term $\sin(x_1)$ inside the vector function f.

3. Linearisation (Small Angle Approximation)
While nonlinear models capture the full physics, they are computationally expensive to use in real-time optimisation. However, for many control applications, the pendulum operates near the vertical downward equilibrium ($\theta = 0$).

For small deviations from this equilibrium ($|\theta| \ll 1$ rad), we can apply the small-angle approximation:

$$\sin(\theta) \approx \theta \implies \sin(x_1) \approx x_1$$

Substituting this into the nonlinear state equation for $\dot{x}_2$, the dynamics become linear:

$$\dot{x}_2 = -\frac{g}{\ell}x_1 - \frac{b}{m\ell^2}x_2 + \frac{1}{m\ell^2}u$$

We can now rewrite the entire system in the standard LTI matrix form $\dot{x} = Ax + Bu$:

$$\underbrace{\begin{bmatrix}\dot{x}_1 \\ \dot{x}_2\end{bmatrix}}_{\dot{x}} = \underbrace{\begin{bmatrix}0 & 1 \\ -\frac{g}{\ell} & -\frac{b}{m\ell^2}\end{bmatrix}}_{A}\underbrace{\begin{bmatrix}x_1 \\ x_2\end{bmatrix}}_{x} + \underbrace{\begin{bmatrix}0 \\ \frac{1}{m\ell^2}\end{bmatrix}}_{B}u \tag{3.4}$$

The output equation remains linear:

$$y = \underbrace{\begin{bmatrix}1 & 0\end{bmatrix}}_{C}x + \underbrace{\begin{bmatrix}0\end{bmatrix}}_{D}u \tag{3.5}$$

This yields the familiar state-space quadruplet (A, B, C, D). Throughout the majority of this course, we will focus on systems that can be described by (or approximated by) such linear models, as they allow for the use of highly efficient convex optimisation solvers (e.g., Quadratic Programming).

Summary

The state vector $x(t)$ contains the minimal set of variables required to describe the system's internal condition. Knowledge of $x(t)$ at the current time allows prediction of future states and outputs, which is precisely the information required by MPC.

For linear time-invariant (LTI) systems, which form the backbone of most MPC formulations, the model reduces to

$$\begin{aligned}\dot{x}(t) &= Ax(t) + Bu(t)\\ y(t) &= Cx(t) + Du(t),\end{aligned}$$

with constant matrices A, B, C, and D formed by physical parameters of the system.

Physical interpretation of states

In practice, states are not abstract quantities: they usually correspond to physically meaningful variables associated with energy storage. Typical examples include:

- **Electrical:** Currents in inductors and voltages across capacitors (magnetic and electric energy).
- **Mechanical:** Positions and velocities of masses (potential and kinetic energy).
- **Process:** Temperatures, pressures, or concentrations (thermal and chemical potential).

This interpretation provides a systematic way to construct models from first principles. Starting from physical laws—such as Kirchhoff's laws in electrical systems or Newton's laws in mechanical systems—we obtain differential equations whose natural variables become the system states.

Caveat: Intentionally incorporated states

It is also common to extend the state vector with non-physical variables to improve control performance. For instance, to ensure the controller eliminates steady-state error (offset-free tracking), we often add an **integrator state** or a **disturbance estimate** to the model.

These are mathematical constructs rather than physical ones, yet they are treated identically within the state-space framework. We will explore these techniques in the upcoming chapters.

Why does MPC rely on state-space models?

The importance of the state-space formulation in MPC can be summarised as follows:

- **Recursive Prediction:** The state-space form ($x_{k+1} = Ax_k + Bu_k$) is inherently recursive. This allows the controller to compute the future evolution of the system over a horizon N simply by iterating matrix multiplications, which is computationally efficient for real-time optimisation.

- **Native MIMO Capability:** Unlike transfer functions, which require complex coupling terms to model interactions, state-space models naturally accommodate Multi-Input Multi-Output (MIMO) systems via matrix dimensions. This is essential for industrial processes where inputs are rarely isolated.

- **Direct Constraint Handling:** MPC is defined by its ability to respect limits. Physical bounds (e.g., valve positions or safety pressures) map directly onto simple linear inequalities involving the state and input vectors ($Cx + Du \leq b$), making them easy for standard solvers to process.

- **Systematic Discretisation:** While physical laws are derived in continuous time (differential equations), digital controllers operate in discrete time (difference equations). State-space theory provides exact methods (such as the Zero-Order Hold) to convert continuous models into discrete forms without losing physical insight.

The state-space representation provides a modelling framework that connects physical intuition, mathematical analysis, and predictive control design. MPC relies on it directly.

In the following examples, we remind readers how to derive state-space models directly from physical principles in order to build intuition and highlight common modelling patterns.

Example 3.2: State–Space Modelling of an RLC Circuit

Consider the electrical circuit shown in Figure 3.2, consisting of a voltage source V_i, resistors R_1 and R_2, an inductor L, and a capacitor C. Write state equations in matrix form.

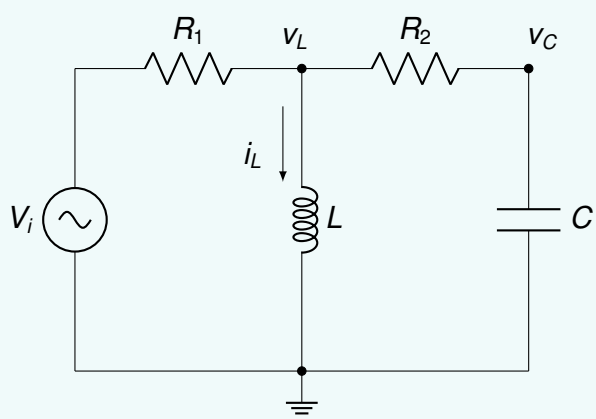


Figure 3.2: An RLC circuit.

Solution:

We select the energy variables as state variables:

$$x_1 = i_L \quad \text{(inductor current)}, \qquad x_2 = v_C \quad \text{(capacitor voltage)},$$

and take the input as

$$u = v_i.$$

We begin with the constitutive relations of the energy–storage elements. Since the state variables are chosen as the inductor current and the capacitor voltage, all other variables are expressed in terms of these states and the input.

$$v_L = L\frac{di_L}{dt}, \tag{3.6}$$

$$i_C = C\frac{dv_C}{dt}. \tag{3.7}$$

The current flowing through resistor R_2 is given by Ohm's law as

$$i_{R_2} = \frac{v_L - v_C}{R_2}.$$

Applying Kirchhoff's Current Law (KCL) at node v_C and noting that the capacitor current equals the current through R_2 yields

$$C\dot{v}_C = \frac{v_L - v_C}{R_2}.$$

Applying KCL at node v_L yields

$$\frac{v_i - v_L}{R_1} = i_L + \frac{v_L - v_C}{R_2}.$$

Solving for v_L:

$$v_L = \frac{R_2}{R_1 + R_2}v_i - \frac{R_1 R_2}{R_1 + R_2}i_L + \frac{R_1}{R_1 + R_2}v_C.$$

State equations. Substituting v_L into the inductor and capacitor equations gives

$$\frac{d}{dt}i_L = -\frac{R_1 R_2}{L(R_1 + R_2)}i_L + \frac{R_1}{L(R_1 + R_2)}v_C + \frac{R_2}{L(R_1 + R_2)}v_i, \tag{3.8}$$

$$\frac{d}{dt}v_C = -\frac{R_1}{C(R_1 + R_2)}i_L - \frac{1}{C(R_1 + R_2)}v_C + \frac{1}{C(R_1 + R_2)}v_i. \tag{3.9}$$

State–space representation. Defining the state vector $x = [\, i_L \;\; v_C \,]^\top$, the system can be written compactly as

$$\dot{x} = Ax + Bu,$$

where

$$A = \begin{bmatrix} -\dfrac{R_1 R_2}{L(R_1 + R_2)} & \dfrac{R_1}{L(R_1 + R_2)} \\ -\dfrac{R_1}{C(R_1 + R_2)} & -\dfrac{1}{C(R_1 + R_2)} \end{bmatrix}, \qquad B = \begin{bmatrix} \dfrac{R_2}{L(R_1 + R_2)} \\ \dfrac{1}{C(R_1 + R_2)} \end{bmatrix}.$$

Example 3.3: Mass–Spring–Damper with External Forcing (State–Space Modelling)

Question. A mass–spring–damper system shown in Figure 3.3 is subjected to an external force input $F(t)$. The displacement of the mass from equilibrium is denoted by $\zeta(t)$ (positive to the right). The system parameters are mass m, damping coefficient c, and spring stiffness k.

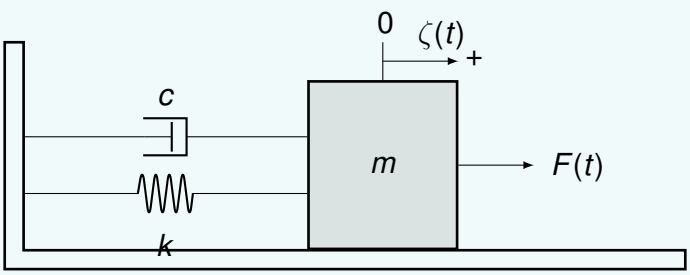


Figure 3.3: Mass-damper-spring system.

1. Write the equation of motion for the system in terms of $\zeta(t)$, $\dot{\zeta}(t)$, $\ddot{\zeta}(t)$, and the input force $F(t)$.
2. Choose a suitable state vector and write the continuous-time state-space model
$$\dot{x}(t) = Ax(t) + Bu(t),$$
clearly defining $x(t)$ and $u(t)$.
3. If the output is the displacement, i.e. $y(t) = \zeta(t)$, write the output equation
$$y(t) = Cx(t) + Du(t).$$

Solution:

1) Equation of motion: Let $\zeta(t)$ denote the displacement of the mass from equilibrium (positive to the right), and let $F(t)$ be the applied external force. By Newton's second law, the sum of forces on the mass equals $m\ddot{\zeta}(t)$. The spring force is $-k\zeta(t)$ and the damper force is $-c\dot{\zeta}(t)$. Hence,
$$m\ddot{\zeta}(t) = F(t) - c\dot{\zeta}(t) - k\zeta(t),$$
or equivalently,
$$m\ddot{\zeta}(t) + c\dot{\zeta}(t) + k\zeta(t) = F(t).$$

2) Choice of states and state equations: Choose the standard second-order state variables
$$x_1(t) = \zeta(t), \qquad x_2(t) = \dot{\zeta}(t), \qquad u(t) = F(t).$$

Then

$$\dot{x}_1(t) = x_2(t),$$

and, from the equation of motion,

$$\dot{x}_2(t) = \ddot{\zeta}(t) = -\frac{k}{m}x_1(t) - \frac{c}{m}x_2(t) + \frac{1}{m}u(t).$$

3) State-space form: Defining $x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}$, the system can be written as

$$\dot{x}(t) = Ax(t) + Bu(t),$$

with

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}.$$

4) Output equation. If the measured output is the displacement $y(t) = \zeta(t) = x_1(t)$, then

$$y(t) = Cx(t) + Du(t),$$

where

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad D = \begin{bmatrix} 0 \end{bmatrix}.$$

3.2 Nonlinear Systems and Local Linearisation

Many real-world systems are inherently nonlinear. Nonlinearities may arise from physical laws, geometric effects, saturation, friction, or actuator limits. While linear state-space models play a central role in MPC, they are often obtained as local approximations of an underlying nonlinear system.

In this section, we introduce nonlinear state-space models, define equilibrium points, and show how nonlinear dynamics can be approximated locally using Taylor expansion. This leads naturally to the concept of Jacobian linearisation.

3.2.1 Nonlinear state-space models

A general nonlinear continuous-time system is described by

$$\begin{aligned} \dot{x}(t) &= f(x(t), u(t)) \\ y(t) &= h(x(t), u(t)), \end{aligned}$$

where the functions $f(\cdot)$ and $h(\cdot)$ are nonlinear mappings.

Unlike linear systems, nonlinear systems:

- may have multiple equilibrium points,
- exhibit behaviour that depends strongly on the operating point,
- cannot, in general, be analysed using superposition.

For MPC design, it is common to operate the system near a desired steady state. Linearisation provides a systematic way to approximate the nonlinear dynamics locally around such an operating point.

3.2.2 Equilibrium points

An equilibrium point represents a state of operation where the system's dynamic evolution ceases. If a system is initialised at an equilibrium state and the external inputs remain constant, the state will not change over time.

We distinguish between the definitions for continuous-time and discrete-time systems below.

Definition: Equilibrium Points

1. Continuous-Time Case:
For a continuous-time system $\dot{x}(t) = f(x(t), u(t))$, an equilibrium point is a pair (x_e, u_e) such that the time derivative vanishes:

$$f(x_e, u_e) = 0 \tag{3.10}$$

2. Discrete-Time Case:
For a discrete-time system $x_{k+1} = f(x_k, u_k)$, an equilibrium point is a pair (x_e, u_e) such that the next state is identical to the current state (a fixed point):

$$x_e = f(x_e, u_e) \tag{3.11}$$

Determination of Equilibria

To find the equilibrium points for a given constant input u_e, one must solve a set of algebraic equations:

- **Continuous:** Solve the system of equations $f(x, u_e) = 0$ for x.
- **Discrete:** Solve the fixed-point equation $x - f(x, u_e) = 0$ for x.

Note that nonlinear systems may have multiple equilibrium points, a single unique equilibrium, or no equilibrium points at all for a specific input.

Physical Interpretation

In engineering contexts, equilibrium points are often referred to as *operating points*, *trim points*, or *steady states*. Physically, they correspond to:

- **Mechanical Systems:** A position of rest (velocity and acceleration are zero) or motion at a constant velocity (acceleration is zero).
- **Thermal/Chemical Systems:** A state where inflows equal outflows, resulting in constant temperature, pressure, or concentration.
- **Electrical Systems:** A state where capacitor voltages and inductor currents have settled to constant DC values.

Ideally, a control system is designed to steer the dynamic system from an arbitrary initial state towards a desired equilibrium point (x_{ref}, u_{ref}).

3.2.3 Local approximation via Taylor expansion

To analyse system behaviour near an equilibrium, we approximate the nonlinear dynamics using a Taylor series expansion.

Scalar case

Consider a scalar nonlinear system

$$\dot{x} = f(x, u),$$

and let (x_e, u_e) be an equilibrium point.

The first-order Taylor expansion of $f(x, u)$ about (x_e, u_e) is

$$f(x, u) \approx f(x_e, u_e) + \frac{\partial f}{\partial x}\Big|_{(x_e,u_e)}(x - x_e) + \frac{\partial f}{\partial u}\Big|_{(x_e,u_e)}(u - u_e).$$

Since $f(x_e, u_e) = 0$ at equilibrium, this simplifies to

$$\dot{x} \approx \frac{\partial f}{\partial x}\Big|_{(x_e,u_e)}(x - x_e) + \frac{\partial f}{\partial u}\Big|_{(x_e,u_e)}(u - u_e).$$

This expression describes a *linear approximation* of the nonlinear system near the equilibrium.

Multivariable case

Now consider the nonlinear vector-valued system

$$\dot{x} = f(x, u), \qquad x \in \mathbb{R}^n,\ u \in \mathbb{R}^m.$$

Expanding $f(x, u)$ about the equilibrium (x_e, u_e) yields

$$f(x, u) \approx f(x_e, u_e) + \frac{\partial f}{\partial x}\Big|_{(x_e,u_e)}(x - x_e) + \frac{\partial f}{\partial u}\Big|_{(x_e,u_e)}(u - u_e).$$

Again using $f(x_e, u_e) = 0$, we obtain

$$\dot{x} \approx A(x - x_e) + B(u - u_e),$$

where the matrix

$$A = \frac{\partial f}{\partial x}\Big|_{(x_e,u_e)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}_{(x_e,u_e)}.$$

Note that each entry A_{ij} represents the sensitivity of the ith state equation to the jth state variable, evaluated at the equilibrium point.

Similarly,

$$B = \frac{\partial f}{\partial u}\Big|_{(x_e,u_e)} = \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \frac{\partial f_1}{\partial u_2} & \cdots & \frac{\partial f_1}{\partial u_m} \\ \frac{\partial f_2}{\partial u_1} & \frac{\partial f_2}{\partial u_2} & \cdots & \frac{\partial f_2}{\partial u_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial u_1} & \frac{\partial f_n}{\partial u_2} & \cdots & \frac{\partial f_n}{\partial u_m} \end{bmatrix}_{(x_e,u_e)}.$$

Each entry B_{ij} quantifies how the jth input influences the ith state equation in the neighbourhood of the equilibrium.

With these matrices, the linearised dynamics in deviation variables

$$\tilde{x} = x - x_e, \qquad \tilde{u} = u - u_e$$

are given by

$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}.$$

If the output equation is nonlinear,

$$y = h(x, u),$$

a similar expansion gives

$$\tilde{y} = C\tilde{x} + D\tilde{u},$$

with

$$C = \frac{\partial h}{\partial x}\Big|_{(x_e,u_e)}, \qquad D = \frac{\partial h}{\partial u}\Big|_{(x_e,u_e)}.$$

Example 3.4: Nonlinear Planar Duorotor

Question: The nonlinear dynamics of a planar duorotor (shown in Figure 3.4) with mass m, inertia J, and arm length ℓ are given by:

$$\begin{aligned} m\ddot{p}_x &= -(u_1 + u_2)\sin(\theta) \\ m\ddot{p}_y &= (u_1 + u_2)\cos(\theta) - mg \\ J\ddot{\theta} &= \frac{\ell}{2}(u_2 - u_1) \end{aligned}$$

where p_x is the horizontal position and p_y is the vertical position. Linearise these dynamics around the hover equilibrium ($\theta = 0$, $u_1 = u_2 = mg/2$) to obtain an LTI state-space model, $\dot{x} = Ax + Bu$.

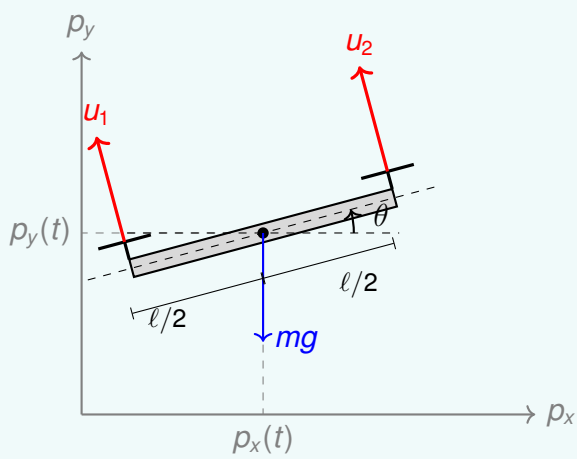


Figure 3.4: Planar duorotor model showing states (p_x, p_y, θ) and inputs (u_1, u_2). The coordinate system is inertial, with θ measured counter-clockwise from the vertical (upward).

Solution:

1) Define states and inputs: We select the state vector such that the generalized coordinates appear first, followed by their velocities:

$$x(t) = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 \end{bmatrix}^\top = \begin{bmatrix} p_x & p_y & \theta & \dot{p}_x & \dot{p}_y & \dot{\theta} \end{bmatrix}^\top.$$

The input vector is $u = [\, u_1 \;\; u_2 \,]^\top$.

The nonlinear state-space equations $\dot{x} = f(x, u)$ are:

$$\begin{aligned}
\dot{x}_1 &= x_4, \\
\dot{x}_2 &= x_5, \\
\dot{x}_3 &= x_6, \\
\dot{x}_4 &= -\frac{1}{m}(u_1 + u_2)\sin(x_3), \\
\dot{x}_5 &= \frac{1}{m}(u_1 + u_2)\cos(x_3) - g, \\
\dot{x}_6 &= \frac{\ell}{2J}(u_2 - u_1).
\end{aligned}$$

2) Determine the equilibrium point: At hover, the drone is stationary at the origin with zero angle:

$$x_e = [0 \;\; 0 \;\; 0 \;\; 0 \;\; 0 \;\; 0]^\top.$$

Setting accelerations $(\dot{x}_4, \dot{x}_5, \dot{x}_6)$ to zero:

- From $\dot{x}_5 = 0$: $\frac{1}{m}(u_{1,e} + u_{2,e})\cos(0) - g = 0 \implies u_{1,e} + u_{2,e} = mg$.
- From $\dot{x}_6 = 0$: $\frac{\ell}{2J}(u_{2,e} - u_{1,e}) = 0 \implies u_{1,e} = u_{2,e}$.

Thus, the equilibrium inputs are $u_{1,e} = u_{2,e} = \frac{mg}{2}$.

3) Jacobian Linearisation: We compute the Jacobian matrices $A = \frac{\partial f}{\partial x}$ and $B = \frac{\partial f}{\partial u}$ evaluated at (x_e, u_e).

For matrix A: Rows 1–3 are purely kinematic ($\dot{p} = v$), resulting in an identity matrix in the top-right block. For rows 4–6, we check the derivatives with respect to the angle θ (state x_3):

$$\begin{aligned}
\left.\frac{\partial \dot{x}_4}{\partial x_3}\right|_{eq} &= -\frac{u_{1,e} + u_{2,e}}{m}\cos(x_{3,e}) = -\frac{mg}{m}(1) = -g, \\
\left.\frac{\partial \dot{x}_5}{\partial x_3}\right|_{eq} &= -\frac{u_{1,e} + u_{2,e}}{m}\sin(x_{3,e}) = -\frac{mg}{m}(0) = 0.
\end{aligned}$$

For matrix B: We compute derivatives of the accelerations with respect to inputs u_1 and u_2:

$$\begin{aligned}
\left.\frac{\partial \dot{x}_4}{\partial u}\right|_{eq} &= -\frac{1}{m}\sin(x_{3,e}) = 0, \\
\left.\frac{\partial \dot{x}_5}{\partial u}\right|_{eq} &= \frac{1}{m}\cos(x_{3,e}) = \frac{1}{m}.
\end{aligned}$$

The angular acceleration $\dot{x}_6$ remains linear in u, with coefficients $\mp\frac{\ell}{2J}$.

4) Final State-Space Model: Substituting these values, we obtain the linearized model $\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}$:

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & -g & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ \frac{1}{m} & \frac{1}{m} \\ -\frac{\ell}{2J} & \frac{\ell}{2J} \end{bmatrix}.$$

where $\tilde{x} = x - x_e$ and $\tilde{u} = u - u_e$.

3.3 Obtaining DT Realisation of CT LTI State-space systems

For a homogeneous LTI system $\dot{x} = Ax$, the state at a future time $t + \Delta t$ is given by:

$$x(t + \Delta t) = e^{A\Delta t}x(t) \tag{3.12}$$

The term $e^{A\Delta t}$ is the matrix exponential. It is not computed element-wise; rather, it is defined by the infinite Taylor series expansion:

$$e^M = I + M + \frac{1}{2!}M^2 + \frac{1}{3!}M^3 + \cdots$$

In MATLAB, this is computed efficiently using the `expm()` function.

3.3.1 Exact Discretisation of LTI Systems

This analytical solution gives us a way to *exactly* discretise a linear system. If we choose a fixed time step $h = \Delta t$, the discrete-time dynamics are:

$$x_{k+1} = A_d x_k \quad \text{where} \quad A_d = e^{Ah} \tag{3.13}$$

This is fundamentally different from numerical integrators like Euler or RK4, which are approximations. For LTI systems, the matrix exponential provides the exact transition matrix A_d.

Linear Systems	**Nonlinear Systems**
$\dot{x} = Ax$ Can be solved exactly in closed form. Discretised exactly using e^{Ah}.	$\dot{x} = f(x)$ Almost never have closed-form solutions. Discretised approximately using numerical integrators (Euler, RK4, ODE45, etc.).

Table 3.1: Comparison of solving linear vs. nonlinear ODEs.

3.4 Discretisation of Forced Linear Systems

How do we discretise a forced system $\dot{x} = Ax + Bu$? The matrix exponential only applies to homogeneous systems (of the form $\dot{z} = Tz$). We can use a clever trick: augment the state to absorb the input and then apply the matrix exponential.

First, we must assume something about how the control input $u(t)$ behaves *between* discrete time steps. The most common assumption is a *Zero-Order Hold (ZOH)*, which means the control input is held constant over each time step:

$$u(t) = u_k \quad \text{for} \quad t \in [t_k, t_{k+1})$$

This is equivalent to stating that the derivative of the input is zero during the interval: $\dot{u} = 0$.

With the ZOH assumption, we can create an augmented state vector $z = [x^T, u^T]^T$. The dynamics of this new state are:

$$\dot{z} = \begin{bmatrix} \dot{x} \\ \dot{u} \end{bmatrix} = \begin{bmatrix} Ax + Bu \\ 0 \end{bmatrix} = \underbrace{\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix}}_{T} \begin{bmatrix} x \\ u \end{bmatrix} = Tz$$

We now have a larger, but homogeneous, linear system. We can apply the matrix exponential to find its exact discrete-time solution:

$$z_{k+1} = e^{Th} z_k \implies \begin{bmatrix} x_{k+1} \\ u_{k+1} \end{bmatrix} = e^{\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} h} \begin{bmatrix} x_k \\ u_k \end{bmatrix}$$

The matrix exponential of this block matrix has a special structure that gives us our final discrete-time system:

$$e^{Th} = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix}$$

Expanding the equation for x_{k+1} gives the standard discrete-time form:

$$x_{k+1} = A_d x_k + B_d u_k \tag{3.14}$$

```
% Define a continuous-time LTI system
A = [0, 1; -5, -2]; % Example A matrix
B = [0; 1];         % Example B matrix
h = 0.1;            % Time step

n = size(A, 1); % Number of states
m = size(B, 2); % Number of inputs

% Form the augmented matrix T
T = [A, B;
     zeros(m, n), zeros(m, m)];

% Compute the matrix exponential of T*h
M = expm(T * h);

% Extract the discrete-time matrices Ad and Bd
Ad = M(1:n, 1:n);
Bd = M(1:n, n+1:end);

disp('Ad = '); disp(Ad);
disp('Bd = '); disp(Bd);
```

Listing 3.1: MATLAB code to perform ZOH discretization of a forced LTI system.

3.5 Numerical Discretisation of Nonlinear Systems

In the previous sections, we established that Linear Time-Invariant (LTI) systems can be discretised *exactly* using the matrix exponential (e^{Ah}). However, the majority of physical systems—such as robotic arms, chemical reactors, and aerial vehicles—are inherently nonlinear:

$$\dot{x}(t) = f(x(t), u(t)).$$

For these systems, a closed-form solution for $x(t)$ rarely exists. To implement MPC, or to simulate these systems on a computer, we must approximate the continuous-time evolution using numerical integration methods.

In digital control, we typically assume a Zero-Order Hold (ZOH) on the input, meaning the control input is held constant over the sampling interval h:

$$u(t) = u_k \quad \text{for} \quad t \in [t_k, t_{k+1}).$$

3.5.1 Forward Euler Method

The simplest numerical approach is the Forward Euler method, which approximates the derivative as a constant slope evaluated at the beginning of the interval. It is derived from the first-order truncation of the Taylor series:

$$x_{k+1} = x_k + h \cdot f(x_k, u_k). \tag{3.15}$$

While Euler's method is algebraically simple and easy to differentiate (which is useful for optimization gradients), it suffers from significant drawbacks.

Warning: Avoid Euler's Method

Forward Euler is a first-order method, meaning the global error scales linearly with the step size ($\mathcal{O}(h)$). It is notoriously inaccurate and numerically unstable for *stiff* systems (systems with fast dynamics).

Using Euler's method can introduce artificial energy into the simulation, causing stable physical systems to appear unstable (blow up) unless the time step h is chosen to be infinitesimally small. For this reason, Forward Euler should generally be avoided for accurate simulation of physical plants.

3.5.2 Runge–Kutta 4 (RK4) Method

The standard workhorse for engineering simulation is the 4th-order Runge–Kutta (RK4) method. Rather than relying on a single slope calculation, RK4 takes a weighted average of four slopes calculated at different points within the time step.

Given the current state x_k, input u_k, and time step h, the update is calculated as follows:

$$\begin{aligned} k_1 &= f(x_k, u_k) \\ k_2 &= f\left(x_k + \frac{h}{2}k_1, u_k\right) \\ k_3 &= f\left(x_k + \frac{h}{2}k_2, u_k\right) \\ k_4 &= f(x_k + hk_3, u_k) \\ x_{k+1} &= x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

The RK4 method has a global error of $\mathcal{O}(h^4)$, providing a significantly higher degree of accuracy and stability than Euler's method for the same time step.

Example 3.5: MATLAB Implementation: RK4 Integrator

A robust implementation of a single RK4 step in MATLAB for a generic nonlinear system:

```
function x_next = rk4_step(f_dynamics, x_curr, u_curr, h)
  % f_dynamics: Function handle @(x,u) ...
  % x_curr:     Current state vector [n x 1]
  % u_curr:     Current input vector [m x 1]
  % h:          Time step (dt)

  k1 = f_dynamics(x_curr, u_curr);
  k2 = f_dynamics(x_curr + 0.5*h*k1, u_curr);
  k3 = f_dynamics(x_curr + 0.5*h*k2, u_curr);
  k4 = f_dynamics(x_curr + h*k3, u_curr);

  x_next = x_curr + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
end
```

Example 3.6: Comparison of Numerical Integrators: The Harmonic Oscillator

System Description

Consider a simple undamped mass-spring system (harmonic oscillator) with mass $m = 1$ and spring constant $k = 1$. The equation of motion is:

$$\ddot{p}(t) + p(t) = 0$$

We define the state vector $x = [x_1, x_2]^\top = [p, \dot{p}]^\top$. The continuous-time dynamics $\dot{x} = f(x)$ are:

$$\frac{d}{dt}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -x_1 \end{bmatrix}.$$

The exact analytical solution for an initial condition $x_0 = [1, 0]^\top$ is a perfect sinusoid, $x_1(t) = \cos(t)$, which represents a circle in the phase plane (energy is conserved).

1. Forward Euler Discretisation

Applying the Euler update law $x_{k+1} = x_k + hf(x_k)$ yields:

$$\begin{aligned} x_{1,k+1} &= x_{1,k} + h \cdot x_{2,k} \\ x_{2,k+1} &= x_{2,k} - h \cdot x_{1,k} \end{aligned}$$

Notice that the new state depends linearly on the current slope.

2. Runge–Kutta 4 (RK4) Discretisation
Applying the RK4 method requires four evaluations of the dynamics function $f(x)$ per step:

$$\begin{aligned} k_1 &= f(x_k) \\ k_2 &= f(x_k + 0.5hk_1) \\ k_3 &= f(x_k + 0.5hk_2) \\ k_4 &= f(x_k + hk_3) \end{aligned}$$

$$x_{k+1} = x_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4).$$

Simulation Results
Figure 3.5 compares these two methods against the exact solution across different sampling times h.

It can be observed that the **Forward Euler** method is numerically unstable for every selection of sampling time h; it introduces artificial energy into the system, causing the amplitude to grow unbounded (spiralling out) as time progresses. In contrast, the **RK4** method maintains high accuracy and preserves stability even at larger time steps.

3.6 Stability of Discrete-Time Systems

The dynamics of a discrete-time system can be viewed as an iterated map:

$$x_{k+1} = f_d(x_k).$$

An equilibrium point x^* satisfies $f_d(x^*) = x^*$. To analyze the local stability of x^*, we linearize the dynamics around this point:

$$\delta x_{k+1} \approx A_d \delta x_k, \quad \text{where} \quad A_d = \left.\frac{\partial f_d}{\partial x}\right|_{x=x^*}.$$

Discrete-Time Stability Condition

The equilibrium point x^* is (locally) asymptotically stable if and only if all eigenvalues of A_d lie strictly inside the unit circle in the complex plane:

$$|\lambda_i(A_d)| < 1 \quad \forall i.$$

For linear systems $x_{k+1} = A_d x_k$, this condition is both necessary and sufficient.

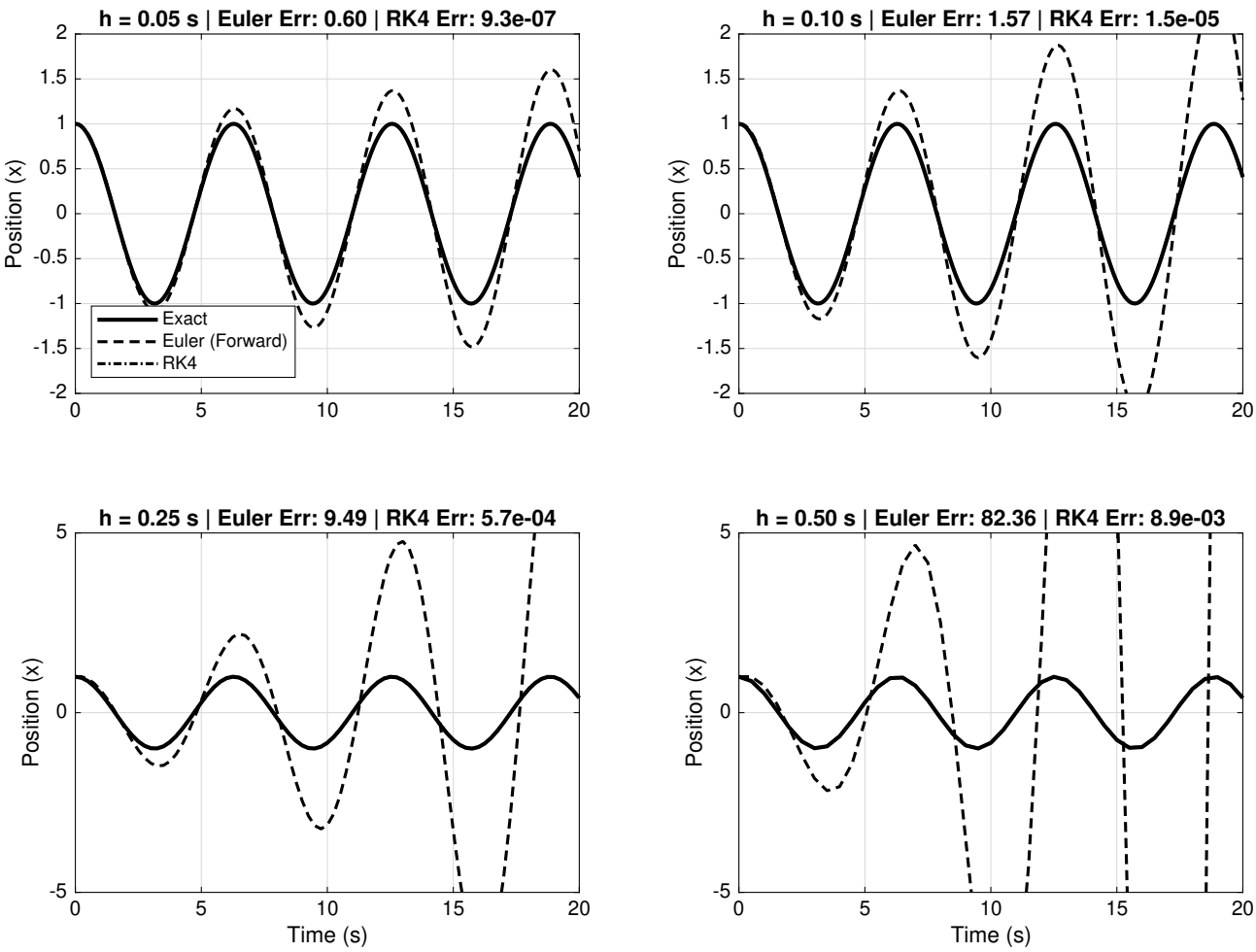


Figure 3.5: Comparison of numerical integration accuracy for an undamped harmonic oscillator ($\ddot{p} + p = 0$) across increasing time steps $h \in \{0.05, 0.10, 0.25, 0.50\}$ seconds. For this system (purely imaginary eigenvalues), the **Forward Euler** method (red dashed) is numerically unstable; it introduces artificial energy into the simulation, causing the amplitude to grow unbounded as time progresses. This error scales linearly with h. In contrast, the **Runge–Kutta 4 (RK4)** method (blue dash-dot) maintains high fidelity to the **Exact** solution (solid gray), preserving stability even at the coarsest time step of $h = 0.50$ s.

3.6.1 Geometric Interpretation: The Unit Circle

Unlike continuous-time systems, where stability is defined by the Left Half Plane (LHP), discrete-time stability is determined by the **Unit Circle**. The mapping between the two domains is governed by the relation $z = e^{sh}$, where h is the sampling time. Figure 3.6 shows an unstable system having 4 eigenvalues.

- **Stable:** Eigenvalues lie strictly *inside* the unit circle ($|\lambda| < 1$). The state vector decays to zero.
- **Unstable:** At least one eigenvalue lies *outside* the unit circle ($|\lambda| > 1$). The state vector grows unbounded.
- **Marginally Stable:** Eigenvalues lie *on* the unit circle ($|\lambda| = 1$). The system may oscillate or remain constant, but does not decay.

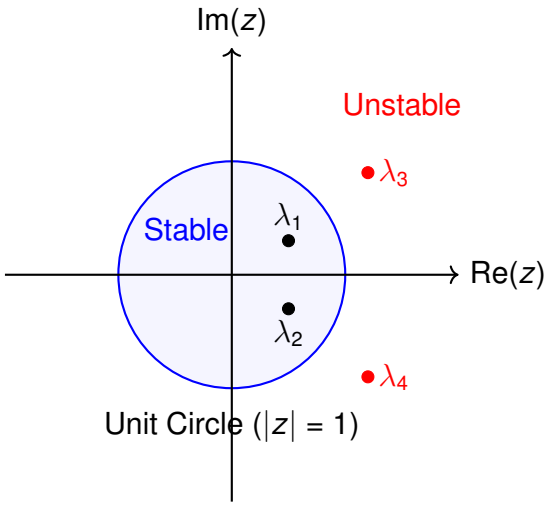


Figure 3.6: Stability in the Z-plane. For the system to be stable, all eigenvalues (poles) must reside within the shaded blue region. Blue eigenvalues are stable whereas red ones are unstable. Therefore the system having these eigenvalues is unstable.

3.6.2 Lyapunov Stability and the Stein Equation

While eigenvalue analysis is useful for linear analysis, a more general approach—fundamental to optimal control—is the **Lyapunov method**. Instead of looking at roots of a polynomial, we look at the system's "energy".

For a linear discrete-time system $x_{k+1} = Ax_k$, let us define a quadratic energy candidate function:

$$V(x_k) = x_k^\top P x_k \tag{3.16}$$

where P is a symmetric, positive definite matrix ($P \succ 0$). For the system to be stable, this energy must decrease at every time step ($V(x_{k+1}) < V(x_k)$).

Substituting the dynamics into the energy difference yields:

$$\begin{aligned} V(x_{k+1}) - V(x_k) &= x_{k+1}^\top P x_{k+1} - x_k^\top P x_k \\ &= (Ax_k)^\top P(Ax_k) - x_k^\top P x_k \\ &= x_k^\top (A^\top PA - P) x_k \end{aligned}$$

For the energy to decrease strictly, the term in the brackets must be negative definite. This leads to the **Discrete Lyapunov Equation**, also known as the *Stein Equation*.

Lyapunov Stability Theorem

The discrete-time linear system $x_{k+1} = Ax_k$ is asymptotically stable if and only if, for any given positive definite matrix $Q \succ 0$, there exists a unique positive definite matrix $P \succ 0$ satisfying:

$$A^\top PA - P = -Q$$

Connection to LQR

This equation is not just a stability check; it is the foundation of the Linear Quadratic Regulator. In LQR, we essentially look for a control law that shapes the matrix $A_{cl} = (A - BK)$ such that this energy-decrease condition is satisfied optimally.

Exercises

1. **Transfer Function to State-Space Conversion.** A system is described by the transfer function

$$G(s) = \frac{s+3}{s^3 + 4s^2 + 6s + 4}.$$

   (a) Convert $G(s)$ into a continuous-time state-space representation $\dot{x} = Ax + Bu$, $y = Cx + Du$ using the controllable canonical form. Clearly state the matrices A, B, C, and D.

   (b) Verify that $D = 0$ and explain, in terms of the degrees of numerator and denominator, why this is the case.

2. **Matrix Exponential Computation.** Consider the continuous-time system $\dot{x} = Ax$ with

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

   (a) Using the Taylor series definition $e^M = I + M + \frac{1}{2!}M^2 + \frac{1}{3!}M^3 + \cdots$, compute e^{Ah} analytically by first evaluating A^2, A^3, etc.

   (b) For the initial condition $x(0) = [1 \;\; 2]^\top$, write the closed-form expression for $x(t)$.

   (c) Now let $A = \begin{bmatrix} -1 & 0 \\ 0 & -3 \end{bmatrix}$. Compute e^{Ah} and comment on how the diagonal structure simplifies the computation.

3. **Zero-Order Hold Discretisation.** A mass–spring–damper system with mass $m = 1\,\text{kg}$, damping coefficient $c = 2\,\text{Ns/m}$, and spring stiffness $k = 5\,\text{N/m}$ has the continuous-time state-space model

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -5 & -2 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u, \qquad y = \begin{bmatrix} 1 & 0 \end{bmatrix} x.$$

   (a) Form the augmented matrix $T = \begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix}$ and explain why computing e^{Th} yields the exact discrete-time matrices A_d and B_d under a ZOH assumption.

   (b) Using MATLAB (or by hand for bonus practice), compute A_d and B_d for a sampling time $h = 0.1\,\text{s}$.

   (c) Write the resulting discrete-time state-space model $x_{k+1} = A_d x_k + B_d u_k$ and the output equation $y_k = C_d x_k$.

4. **Deviation Variables and Linearisation.** Consider the nonlinear system describing a continuously stirred tank reactor (CSTR):

$$\dot{x}_1 = -x_1 + D_a(1-x_1)\exp\left(\frac{x_2}{1+x_2/\gamma}\right), \qquad \dot{x}_2 = -(1+\beta)x_2 + D_a B(1-x_1)\exp\left(\frac{x_2}{1+x_2/\gamma}\right) + \beta\, u,$$

   where x_1 is the dimensionless concentration, x_2 is the dimensionless temperature, u is the coolant temperature, and D_a, B, β, γ are positive parameters.

   (a) Suppose an equilibrium $(x_{1,e},\, x_{2,e},\, u_e)$ has been found numerically. Write the general expressions for the Jacobian matrices $A = \left.\frac{\partial f}{\partial x}\right|_{(x_e,u_e)}$ and $B = \left.\frac{\partial f}{\partial u}\right|_{(x_e,u_e)}$.

   (b) Define the deviation variables $\tilde{x} = x - x_e$ and $\tilde{u} = u - u_e$, and write the linearised model $\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}$.

   (c) Explain why the linearised model is only valid in a neighbourhood of (x_e, u_e) and what happens to its accuracy as $\|\tilde{x}\|$ and $\|\tilde{u}\|$ increase.

5. **Eigenvalue Analysis for Stability.** Consider the discrete-time system $x_{k+1} = A_d x_k$ with

$$A_d = \begin{bmatrix} 0.5 & 0.3 \\ -0.2 & 0.4 \end{bmatrix}.$$

   (a) Compute the eigenvalues of A_d by solving $\det(A_d - \lambda I) = 0$.

   (b) Determine whether the system is asymptotically stable by checking the discrete-time stability condition $|\lambda_i(A_d)| < 1$ for all i.

   (c) Now consider the modified system matrix $A_d' = \begin{bmatrix} 0.8 & 0.6 \\ -0.6 & 0.8 \end{bmatrix}$. Compute the eigenvalues of A_d' and determine whether the system is stable, unstable, or marginally stable. Sketch the location of the eigenvalues relative to the unit circle in the z-plane.

6. **From Physics to Discrete-Time Model (Combined Exercise).** An electrical circuit consists of a single resistor R in series with an inductor L, driven by a voltage source $v_i(t)$. The current through the inductor is $i_L(t)$.

   (a) Using Kirchhoff's voltage law, derive the first-order differential equation governing $i_L(t)$.

   (b) Write the continuous-time state-space model with $x = i_L$ and $u = v_i$, identifying the scalars A, B, C, and D (assume the output is the inductor current).

(c) Compute the exact ZOH discrete-time model $x_{k+1} = A_d x_k + B_d u_k$ analytically. *Hint:* For a scalar system, $A_d = e^{Ah}$ and $B_d = \int_0^h e^{A\tau}\, d\tau \cdot B$.

(d) Compute the eigenvalue of A_d and determine the condition on R, L, and h for which the discrete-time system is stable.

Chapter 4

Controllability & Observability

Now that we have established the state-space modelling framework (Chapter 3), we turn to two structural questions that determine whether a control or estimation design is even possible.

In previous chapters, we focused on the machinery of optimisation—how to select the *best* input to minimise a cost function. However, before asking "what is the optimal trajectory?", we must answer a more fundamental question: "does a valid trajectory exist?"

This chapter introduces the twin structural properties of **controllability** and **observability**. These concepts define the physical limitations of what a control system can achieve, regardless of the sophistication of the algorithm used.

- **Controllability** answers the question of *actuation*: Is it possible to steer the system from its current state to any desired target state using the available inputs? If a system is unreachable, no amount of mathematical optimisation can force it to the target.
- **Observability** answers the question of *sensing*: Is it possible to reconstruct the full internal state of the system solely by monitoring the inputs and outputs? Since MPC and LQR typically require full state feedback (x_k), observability is the prerequisite for designing the estimators (such as Kalman Filters) required to implement these controllers in reality.

For modern control strategies like Model Predictive Control, these are not merely theoretical curiosities; they are the necessary existence conditions. If a system lacks these properties, the optimisation problem may be ill-posed, leading to controllers that are either unstable or physically incapable of meeting their objectives.

4.1 Controllability

The fundamental question of controllability is: given the dynamics $x_{k+1} = Ax_k + Bu_k$, do the actuators (represented by matrix B) possess sufficient authority over the system states (governed by matrix A) to drive the system to a specific target?

Consider a trivial example where the matrix B is all zeros. In such a case, the control input u_k has no effect on the state evolution, rendering us powerless to change the trajectory. While this is an

extreme case, a system can be uncontrollable even with non-zero actuators if their influence is geometrically restricted to a specific subspace of the state space.

Definition: Controllability

A system is *controllable* if, for any initial state x_0 and any target state x_N, there exists a finite sequence of control inputs that drives the system from x_0 to x_N.

4.1.1 Derivation of the Rank Condition

For Linear Time-Invariant (LTI) systems where the state vector is $x_k \in \mathbb{R}^n$, we can derive a simple algebraic test for controllability. We begin by expanding the discrete-time dynamics to express the state x_N as a function of the initial state x_0 and the control history.

Starting from $k = 0$ and substituting recursively:

$$\begin{aligned}
x_1 &= Ax_0 + Bu_0 \\
x_2 &= Ax_1 + Bu_1 = A(Ax_0 + Bu_0) + Bu_1 = A^2x_0 + ABu_0 + Bu_1 \\
&\vdots \\
x_N &= A^N x_0 + \sum_{i=0}^{N-1} A^{N-1-i} Bu_i
\end{aligned}$$

We can rewrite this summation in a compact matrix-vector form. By collecting all control inputs into a stacked vector $\mathcal{U}$, we obtain:

$$x_N - A^N x_0 = \underbrace{\begin{bmatrix} B & AB & A^2B & \cdots & A^{N-1}B \end{bmatrix}}_{\mathcal{C}_N} \underbrace{\begin{bmatrix} u_{N-1} \\ u_{N-2} \\ \vdots \\ u_0 \end{bmatrix}}_{\mathcal{U}}$$

This equation is a linear system of the form $b = \mathcal{C}_N \mathcal{U}$. The controllability problem reduces to a linear algebra problem:

Question

Does a solution $\mathcal{U}$ exist for any desired vector $b = x_N - A^N x_0$?

For a solution to exist for *any* target state in the n-dimensional state space, the matrix $\mathcal{C}_N$ must span $\mathbb{R}^n$. This requires $\mathcal{C}_N$ to have full row rank.

While this matrix grows with the horizon N, the **Cayley-Hamilton Theorem** tells us that any matrix power A^k for $k \geq n$ can be written as a linear combination of lower powers $(I, \dots, A^{n-1})$. Consequently, adding columns beyond $A^{n-1}B$ provides no new linearly independent directions. The Cayley-Hamilton Theorem states that every square matrix satisfies its own characteristic equation; as a consequence, A^n (and all higher powers) can be expressed as a linear combination of $I, A, \dots, A^{n-1}$. We can therefore define the standard **Controllability Matrix** $\mathcal{C}$ using only n blocks.

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}$$

Theorem: The Kalman Rank Condition

Consider an LTI system with state dimension n. The system pair (A, B) is controllable if and only if the **controllability matrix** $\mathcal{C}$ has full rank:

$$\text{rank}(\mathcal{C}) = n$$

where the matrix is constructed from the system matrices as:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix}$$

Interpretation: If this condition holds, the columns of $\mathcal{C}$ span the entire state space $\mathbb{R}^n$. This guarantees that there is no "hidden" subspace where the state can get stuck; the input u has the authority to steer the state vector anywhere in the n-dimensional space.

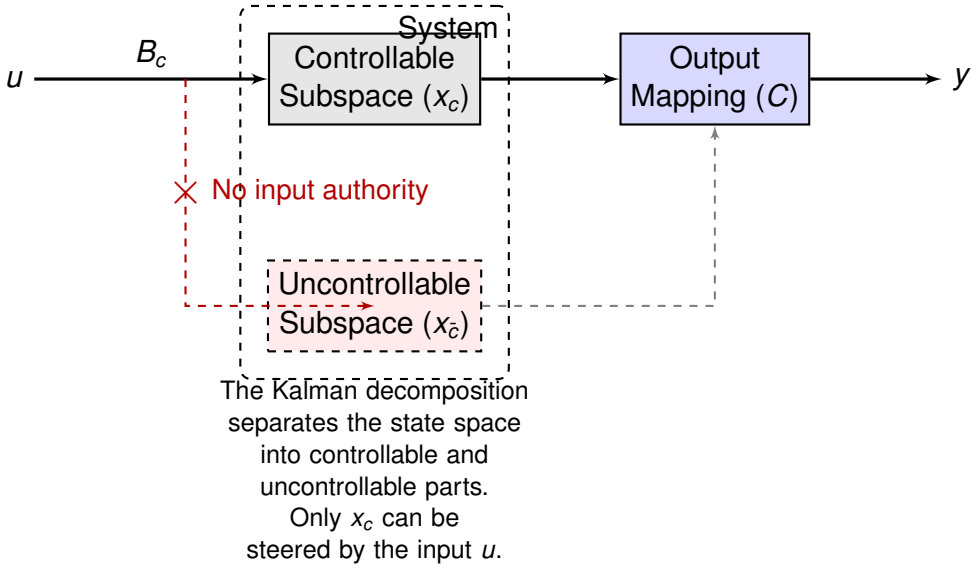


Figure 4.1: Controllability decomposition of a linear system. The input u drives only the controllable subspace x_c. The uncontrollable subspace $x_{\bar{c}}$ evolves independently of u and cannot be steered by any control action. Both subspaces may contribute to the output y.

4.1.2 Controllability in MATLAB

Let us verify this condition for a discrete-time double integrator system. While we could construct the matrix manually, the best practice is to use the MATLAB Control System Toolbox function `ctrb`. This ensures the method generalizes correctly to higher-dimensional systems.

```
% 1. Define the discrete-time double integrator
h = 0.1;                    % Sampling time
A = [1, h;
     0, 1];

B = [0.5*h^2;
     h];
```

```
n = size(A, 1);

% 2. Compute Controllability Matrix
Co = ctrb(A, B);

% 3. Check Rank Condition
rank_Co = rank(Co);

fprintf('State Dimension n = %d\n', n);
fprintf('Rank of Co       = %d\n', rank_Co);
```

Listing 4.1: MATLAB script to check controllability using the `ctrb` command.

Results

Since the rank of the controllability matrix (2) equals the number of states (2), the system is controllable. This mathematically guarantees that an input sequence exists to drive the robot from any initial position and velocity to any target state (assuming no actuator saturation), satisfying the necessary condition for LQR design which will be covered in the next chapter.

4.2 Observability

While controllability deals with the active problem of steering the state x using the input u, **observability** addresses the passive inverse problem: can we reconstruct the internal state $x(t)$ solely by monitoring the system's input $u(t)$ and output $y(t)$?

This concept is fundamental for practical MPC implementation. Most advanced control strategies (including MPC and LQR) are *state-feedback* controllers, meaning they calculate the optimal input $u = -Kx$ based on the full state vector. However, in reality, we rarely measure every state variable directly due to sensor costs or physical impossibility. Instead, we measure a subset $y = Cx$ and use a mathematical algorithm known as a *State Observer* (e.g., a Kalman Filter) to estimate the missing variables. Observability is the mathematical guarantee that this estimation is possible.

Observability

A linear system is defined as **observable** if the initial state $x(t_0)$ can be uniquely determined from the knowledge of the system input $u(t)$ and output $y(t)$ over a finite time interval $t_0 \leq t \leq t_1$.

If a system is **unobservable**, there exists at least one "hidden" mode inside the system that does not affect the output $y(t)$. Consequently, the controller acts "blindly" with respect to this mode, which can lead to undetected instability.

4.2.1 The Observability Matrix

For an LTI system defined by matrices A (dynamics) and C (measurement), observability is a structural property of the pair (A, C). It determines whether the information from the internal states propagates to the output.

To test for this property, we construct the **Observability Matrix**, denoted by $\mathcal{O}$:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

Note that $\mathcal{O}$ is a matrix of dimension $(n \cdot p) \times n$, where n is the number of states and p is the number of outputs.

Kalman Rank Condition for Observability

An LTI system is fully observable if and only if the observability matrix has **full column rank**:

$$\text{rank}(\mathcal{O}) = n$$

If the rank is less than n, the system is unobservable, and the *unobservable states* correspond to the null space of $\mathcal{O}$.

Example 4.1: Unobservable Systems

Consider a simple 2-state system where the dynamics are decoupled, and we only measure the first state:

$$\dot{x} = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix} x + \begin{bmatrix} 1 \\ 1 \end{bmatrix} u$$
$$y = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{C} x$$

Here, x_1 decays with time constant 1, and x_2 decays with time constant 0.5. The sensor y measures x_1 perfectly.

Let us check the Observability Matrix $\mathcal{O}$:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ (1)(-1) + (0)(0) & (1)(0) + (0)(-2) \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix}$$

The second column is entirely zero. The rank of this matrix is 1 (which is less than $n = 2$).

Interpretation: The sensor output y is mathematically disconnected from state x_2. No matter how long we record the data, we can never know the value of x_2. If x_2 were unstable (e.g., a positive eigenvalue), the system would explode, and the controller would be unaware until the machine physically failed.

Example 4.2: Controllability of a Three-State System

Consider a three-state system with the following sparse matrices:

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

This system represents a *triple integrator* (e.g., a frictionless mass where the input is a jerk signal — the rate of change of acceleration). The state vector is $x = \begin{bmatrix} \text{position} & \text{velocity} & \text{acceleration} \end{bmatrix}^\top$.

Since $n = 3$, the controllability matrix requires three blocks: $\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix}$.

Step 1: Compute AB:

$$AB = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Step 2: Compute A^2B:

$$A^2B = A(AB) = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Step 3: Assemble the controllability matrix:

$$\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Step 4: Determine the rank. The matrix $\mathcal{C}$ is a permutation matrix (each row and each column contains exactly one non-zero entry). Its determinant is:

$$\det(\mathcal{C}) = 0 \cdot (0 \cdot 0 - 0 \cdot 0) - 0 \cdot (0 \cdot 0 - 1 \cdot 0) + 1 \cdot (0 \cdot 0 - 1 \cdot 1) = -1 \neq 0$$

Therefore $\text{rank}(\mathcal{C}) = 3 = n$.

Interpretation: The system is fully controllable. Despite the input u entering only the third state equation directly, its effect propagates through the chain of integrators: after one time step, the input influences x_3; after two steps, it reaches x_2; and after three steps, it reaches x_1. The single actuator has full authority over all three states, which is a characteristic property of systems in *controllable canonical form*.

Example 4.3: Observable System: Adding a Second Sensor

Consider the same 2-state system from the previous unobservable example, but suppose we replace the sensor with one that measures a *combination* of both states:

$$\dot{x} = \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix} x + \begin{bmatrix} 1 \\ 1 \end{bmatrix} u$$
$$y = \underbrace{\begin{bmatrix} 1 & 1 \end{bmatrix}}_{C} x$$

The A matrix is identical to the unobservable case. The only change is that $C = \begin{bmatrix} 1 & 1 \end{bmatrix}$ instead of $\begin{bmatrix} 1 & 0 \end{bmatrix}$. Physically, this sensor now measures $y = x_1 + x_2$, coupling both states into the output.

Let us construct the observability matrix $\mathcal{O}$:

Step 1: Compute CA:

$$CA = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} -1 & 0 \\ 0 & -2 \end{bmatrix} = \begin{bmatrix} -1 & -2 \end{bmatrix}$$

Step 2: Assemble the observability matrix:

$$\mathcal{O} = \begin{bmatrix} C \\ CA \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ -1 & -2 \end{bmatrix}$$

Step 3: Check the rank. The determinant is:

$$\det(\mathcal{O}) = (1)(-2) - (1)(-1) = -2 + 1 = -1 \neq 0$$

Therefore $\text{rank}(\mathcal{O}) = 2 = n$.

Interpretation: The system is now fully observable. By measuring a linear combination of both states rather than a single state in isolation, the sensor output $y = x_1 + x_2$ contains information about both modes. Because the two modes decay at different rates ($\lambda_1 = -1$ and $\lambda_2 = -2$), the observer can separate their individual contributions from the mixed signal over time. This example demonstrates that observability is a property of the *pair* (A, C): the same dynamics can be observable or unobservable depending entirely on the sensor configuration.

4.2.2 Derivation of the Rank Condition

We can derive the observability condition intuitively by considering a Discrete-Time LTI system. Our objective is simple: given a sequence of measurements $y_0, y_1, \ldots$, can we mathematically calculate the initial state vector x_0?

Step 1: Isolating the Unforced Response

The general output equation is:

$$y_k = CA^k x_0 + \sum_{j=0}^{k-1} CA^{k-1-j} Bu_j + Du_k$$

Based on the Principle of Superposition, the output consists of two parts: the *free response* (dependent only on initial conditions) and the *forced response* (dependent on inputs). Since the inputs u_k are known, we can calculate the forced response and subtract it from the measurement. Thus, without loss of generality, we treat the problem as solving for x_0 in the unforced system:

$$\begin{aligned} x_{k+1} &= Ax_k \\ y_k &= Cx_k \end{aligned}$$

Step 2: Stacking the Measurements

Let us observe the system for n time steps (where n is the dimension of the state vector) and write the output at each step in terms of the initial state x_0:

$$\begin{aligned} k = 0: &\quad y_0 = Cx_0 \\ k = 1: &\quad y_1 = Cx_1 = C(Ax_0) = CAx_0 \\ k = 2: &\quad y_2 = Cx_2 = C(A^2x_0) = CA^2x_0 \\ &\quad \vdots \\ k = n-1: &\quad y_{n-1} = CA^{n-1}x_0 \end{aligned}$$

Step 3: The Linear System of Equations

We can stack these individual equations into a single large matrix equation $Y = \mathcal{O}x_0$:

$$\underbrace{\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{bmatrix}}_{Y} = \underbrace{\begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}}_{\mathcal{O}} x_0$$

Here:

- Y is the vector of collected measurements (known).
- $\mathcal{O}$ is the **Observability Matrix** (known).
- x_0 is the state vector we wish to find (unknown).

Step 4: The Invertibility Condition

This is a standard linear algebra problem of the form $Ax = b$. We have n unknowns (the elements of x_0). To find a **unique** solution for x_0, the matrix $\mathcal{O}$ must provide enough independent equations to constrain the solution fully.

Mathematically, this requires that the mapping be invertible (specifically, left-invertible). This is only possible if the columns of $\mathcal{O}$ are linearly independent.

The Rank Test

A system is observable if and only if the Observability Matrix has full rank:

$$\text{rank}(\mathcal{O}) = n$$

If this holds, we can uniquely solve for the state via the left pseudo-inverse:

$$x_0 = (\mathcal{O}^\top \mathcal{O})^{-1} \mathcal{O}^\top Y$$

Note: One might ask, "Why stop at $n - 1$?" According to the **Cayley-Hamilton Theorem**, any matrix power A^k for $k \geq n$ can be written as a linear combination of lower powers $(I, A, \dots, A^{n-1})$. Therefore, taking more measurements beyond $k = n - 1$ adds rows to $\mathcal{O}$ that are linearly dependent on previous rows, providing no new information for determining the rank.

4.2.3 Observability in MATLAB

To demonstrate the physical meaning of observability, we analyse the double integrator model (position and velocity) under two different sensor configurations:

1. **Position Sensor ($y = x_1$):** We measure position. Can we infer velocity? Intuitively yes, by observing the change in position over time.
2. **Velocity Sensor ($y = x_2$):** We measure velocity. Can we infer position? Intuitively no, because we do not know the starting integration constant.

```
h = 0.1;
A = [1, h;
     0, 1];
n = size(A, 1);

C1 = [1, 0]; % We measure the first state

Ob1 = obsv(A, C1);
rank_Ob1 = rank(Ob1);

C2 = [0, 1]; % We measure the second state

Ob2 = obsv(A, C2);
rank_Ob2 = rank(Ob2);
```

Listing 4.2: MATLAB script comparing two sensor configurations.

Results:

In Case 1, the matrix $\mathcal{O}$ has rank 2, meaning the rows are linearly independent; information about both states is present in the output history. In Case 2, the matrix has rank 1 (identical rows). The "position" mode is unobservable. If we only measure speed, the robot's absolute position is mathematically impossible to reconstruct.

Stabilisability and Detectability

In practice, MPC does not always require full controllability or observability. The weaker notions of *stabilisability* (all uncontrollable modes are stable) and *detectability* (all unobservable modes are stable) are often sufficient. These conditions appear in the convergence analysis of the Kalman filter (Chapter 6) and the existence of the DARE solution (Chapter 5).

Exercises

1. Consider the discrete-time system $(A,\ B)$ with

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

   (a) Construct the controllability matrix $\mathcal{C} = \begin{bmatrix} B & AB & A^2B \end{bmatrix}$.

   (b) Compute rank($\mathcal{C}$) and determine whether the system is controllable.

   (c) Repeat the analysis for $B' = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ and comment on how the input coupling affects controllability.

2. A discrete-time system is described by

$$A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

   (a) Construct the observability matrix $\mathcal{O}$ and verify that rank($\mathcal{O}$) = 2.

   (b) Now change the output matrix to $C' = \begin{bmatrix} 0 & 0 \end{bmatrix}$ (i.e. no sensor). What is the rank of the resulting observability matrix, and what does this imply physically?

   (c) Suppose instead $C'' = \begin{bmatrix} 1 & 1 \end{bmatrix}$. Determine whether the system $(A,\ C'')$ is observable.

3. The **Popov–Belevitch–Hautus (PBH) test** provides an eigenvalue-based criterion for controllability: the pair $(A,\ B)$ is controllable if and only if

$$\operatorname{rank}\begin{bmatrix} \lambda I - A & B \end{bmatrix} = n \quad \text{for every eigenvalue } \lambda \text{ of } A.$$

   Consider the system

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -3 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

(a) Find the eigenvalues of A.

(b) Apply the PBH test at each eigenvalue to determine which modes are controllable and which are uncontrollable.

(c) Verify your conclusion by computing the controllability matrix $\mathcal{C}$ and checking its rank.

4. A continuous-time system has the state-space representation

$$A = \begin{bmatrix} -2 & 0 \\ 0 & 0.5 \end{bmatrix}, \qquad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

(a) Show that the pair $(A,\ B)$ is *not* controllable by computing rank($\mathcal{C}$).

(b) Identify the uncontrollable mode. Is this mode stable or unstable?

(c) Based on your answer, determine whether the system is *stabilisable*. Justify your reasoning using the definition given in this chapter: a system is stabilisable if all uncontrollable modes are stable.

(d) Now suppose the system matrix is changed to $A' = \begin{bmatrix} -2 & 0 \\ 0 & -0.5 \end{bmatrix}$ while B remains the same. Is this modified system stabilisable? Why or why not?

5. Consider the continuous-time system

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

A state observer (Luenberger observer) reconstructs the state via

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}),$$

where $L \in \mathbb{R}^{2\times 1}$ is the observer gain. The estimation error $e = x - \hat{x}$ satisfies $\dot{e} = (A - LC)\, e$.

(a) Verify that the pair $(A,\ C)$ is observable.

(b) Suppose we wish the observer error dynamics to have eigenvalues at $\{-5,\ -6\}$. Determine the gain vector $L = \begin{bmatrix} l_1 \\ l_2 \end{bmatrix}$ such that the characteristic polynomial of $(A - LC)$ matches $(\lambda + 5)(\lambda + 6) = \lambda^2 + 11\lambda + 30$.

(c) Comment on how the choice of observer eigenvalues (relative to the open-loop eigenvalues of A) affects the speed of state estimation convergence.

Part II

Optimal Control and Optimal Estimation

Chapter 5

LQR Design via Quadratic Programming

> *"The linear variance-minimisation problem... constitutes the most complete and elegant part of optimal control theory."*
>
> — **Huibert Kwakernaak & Raphael Sivan**, *Linear Optimal Control Systems* (1972)

In the preceding chapters, we focused on modelling dynamical systems and analysing their fundamental properties such as stability, controllability, and observability. We now transition from *analysis* to *synthesis*. How do we design a controller that not only stabilises a system but does so in the best possible way?

The Linear Quadratic Regulator (LQR) is a foundational result in optimal control theory. It provides a systematic method to trade off regulation performance against control effort. Traditionally, this problem is introduced via the algebraic Riccati equation—a recursive solution ideal for infinite-horizon unconstrained problems. However, to fully grasp Model Predictive Control (MPC), we must view LQR through a different lens.

In this chapter, we adopt a numerical optimisation perspective. Instead of solving the control problem recursively (time-step by time-step), we will formulate the entire trajectory optimisation as a single, large static task—specifically, a **Quadratic Programme (QP)**.

By "stacking" the predicted states and inputs into large vectors, we transform the dynamic control problem into a standard linear algebra problem. The required linear algebra background is reviewed in Appendix A. This "batch" approach not only provides a one-shot, exact solution but also establishes the essential mathematical framework required to incorporate physical constraints in the chapters that follow.

5.1 Problem Formulation

The discrete-time LQR problem aims to determine the optimal sequence of control inputs u and the corresponding state trajectory x that minimise a quadratic cost function. The optimisation is constrained by the linear evolution of the system, anchored by a known, fixed initial state x_0.

Mathematically, we define the decision variables as the sequence of inputs $\{u_0, \dots, u_{N-1}\}$ and the

sequence of future states $\{x_1, \dots, x_N\}$. The finite-horizon optimisation problem is formulated as follows:

$$\min_{x_{1:N}, u_{0:N-1}} J = \underbrace{\frac{1}{2} x_N^\top Q_N x_N}_{\text{Terminal Cost}} + \sum_{k=0}^{N-1} \left(\underbrace{\frac{1}{2} x_k^\top Q_k x_k}_{\text{State Penalty}} + \underbrace{\frac{1}{2} u_k^\top R_k u_k}_{\text{Control Effort}} \right)$$

This minimisation is subject to two types of constraints:

1. **Initial Condition:** The trajectory must start from the current measured state:

$$x_{k=0} = x(0)$$

2. **System Dynamics:** The states must evolve according to the linear physical laws:

$$x_{k+1} = A_k x_k + B_k u_k, \quad \text{for } k = 0, \dots, N-1$$

Convexity Conditions

To ensure the problem is *well-posed* (i.e., it has a unique, global minimum and is numerically stable), we impose strict requirements on the weighting matrices:

- The state weighting matrices (Q_k, Q_N) must be **Positive Semi-Definite** ($Q \succeq 0$). This ensures the cost function is convex (bowl-shaped), punishing deviations from the origin.
- The control weighting matrices (R_k) must be **Positive Definite** ($R \succ 0$). This ensures strict convexity with respect to the input, preventing the solver from requesting infinite control energy.

5.2 Sparse Quadratic Programming Formulation

To cast the LQR problem into the standard QP form

$$\min_{\zeta} \quad \frac{1}{2} \zeta^\top H \zeta$$

subject to:

$$\mathcal{C}\zeta = d$$

we must stack all decision variables into a single vector z and represent the cost and constraints using large, sparse block matrices. We adopt a "simultaneous" or "sparse" approach, where both inputs and states are treated as optimisation variables.

The Decision Vector (ζ)

We stack the control inputs and the resulting states chronologically. Note that the initial state x_0 is **not** a decision variable (it is a measured constant), so the vector starts with u_0:

$$\zeta = \begin{bmatrix} u_0^\top, & x_1^\top, & u_1^\top, & x_2^\top, & \cdots, & u_{N-1}^\top, & x_N^\top \end{bmatrix}^\top$$

This interleaved ordering $(u, x, u, x, \dots)$ is chosen deliberately to keep the resulting matrices banded (sparse), which significantly speeds up numerical solving.

The Cost Hessian (H)

The objective function is a sum of quadratic terms. Since x_0 is constant, the term $x_0^\top Q_0 x_0$ does not affect the location of the minimum and is omitted. The Hessian matrix H is a block-diagonal matrix containing the weights for each variable in ζ:

$$H = \text{blkdiag}(R_0,\ Q_1,\ R_1,\ \dots,\ R_{N-1},\ Q_N) = \begin{bmatrix} R_0 & & & & \\ & Q_1 & & & \\ & & R_1 & & \\ & & & \ddots & \\ & & & & Q_N \end{bmatrix}$$

Since $Q \succeq 0$ and $R \succ 0$, this large matrix H is Positive Definite (or Semi-Definite), ensuring the problem is convex.

The Equality Constraints ($\mathcal{C}\zeta = d$)

The system dynamics impose a linear equality constraint at every time step. We rearrange the update equation $x_{k+1} = A_k x_k + B_k u_k$ to bring all decision variables to the left-hand side:

$$-A_k x_k - B_k u_k + x_{k+1} = 0$$

For the special case of the first step ($k = 0$), since x_0 is known, we move it to the right-hand side:

$$-B_0 u_0 + x_1 = A_0 x_0$$

Stacking these equations for $k = 0 \dots N-1$ yields the structured system $\mathcal{C}\zeta = d$:

$$\underbrace{\begin{bmatrix} B_0 & -I & 0 & 0 & \cdots & 0 \\ 0 & A_1 & B_1 & -I & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & A_{N-1} & B_{N-1} & -I \end{bmatrix}}_{\mathcal{C}} \zeta = \underbrace{\begin{bmatrix} -A_0 x_0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}}_{d}$$

Notice the *staircase* structure of the matrix $\mathcal{C}$. This sparsity allows specialised solvers to compute the solution very efficiently, even for long horizons (N).

5.3 Solution via KKT Conditions

With the LQR problem cast as an equality-constrained Quadratic Programme, we can determine the exact solution by applying the **Karush-Kuhn-Tucker (KKT) conditions**.

For students unfamiliar with this term, the KKT conditions are simply the generalisation of the method of Lagrange Multipliers to problems with multiple constraints. Since our specific formulation contains no inequality constraints (e.g., no limits on u or x yet), the KKT conditions reduce to a large system of linear equations.

5.3.1 The KKT System

We form the Lagrangian function $\mathcal{L}$ by adjoining the constraints to the cost function using a vector of Lagrange multipliers, λ (often called *costates* in control theory):

$$\mathcal{L}(\zeta, \lambda) = \frac{1}{2}\zeta^\top H\zeta + \lambda^\top(\mathcal{C}\zeta - d) \tag{5.1}$$

The necessary conditions for optimality are found by setting the derivatives with respect to all variables to zero:

1. **Stationarity (Gradient w.r.t variables is zero):**

$$\nabla_\zeta \mathcal{L} = H\zeta + \mathcal{C}^\top\lambda = 0$$

2. **Primal Feasibility (Constraints are satisfied):**

$$\nabla_\lambda \mathcal{L} = \mathcal{C}\zeta - d = 0$$

Combining these yields the **KKT System**, a linear matrix equation:

$$\begin{bmatrix} H & \mathcal{C}^\top \\ \mathcal{C} & 0 \end{bmatrix} \begin{bmatrix} \zeta \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ d \end{bmatrix}$$

Explicit Structure of the KKT Matrix

To see the structure clearly, let us expand this system for a short horizon $N = 4$. For clarity, we assume an LTI system (constant matrices A, B, Q, R), though the method applies equally to time-varying systems.

The matrix below reveals the *sparse* nature of the problem. The top-left block is the Hessian H, the bottom-left is the constraint matrix $\mathcal{C}$, and the top-right is $\mathcal{C}^\top$.

$$
\left[\begin{array}{cccccccc|cccccc}
R & & & & & & & & B^\top & & & & & \\
 & Q & & & & & & & -I & A^\top & & & & \\
 & & R & & & & & & & B^\top & & & & \\
 & & & Q & & & & & & -I & A^\top & & & \\
 & & & & R & & & & & & B^\top & & & \\
 & & & & & Q & & & & & -I & A^\top & & \\
 & & & & & & R & & & & & B^\top & & \\
 & & & & & & & Q_N & & & & -I & & \\
\hline
B & -I & & & & & & & & & & & & \\
 & A & B & -I & & & & & & & & & & \\
 & & & A & B & -I & & & & & & & & \\
 & & & & & A & B & -I & & & & & &
\end{array}\right]
\left[\begin{array}{c}
u_0 \\ x_1 \\ u_1 \\ x_2 \\ u_2 \\ x_3 \\ u_3 \\ x_4 \\ \hline \lambda_1 \\ \lambda_2 \\ \lambda_3 \\ \lambda_4
\end{array}\right]
=
\left[\begin{array}{c}
0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \hline -Ax_0 \\ 0 \\ 0 \\ 0
\end{array}\right]
$$

5.3.2 Solving the KKT System (The Riccati Recursion)

While the large matrix above can be inverted directly using a computer (which is how MPC solvers typically work), we can derive the fundamental equations of optimal control by solving it analytically via **block back-substitution**.

We start from the very last rows (the end of the horizon) and work backwards to the present.

1. Terminal Condition ($k = N$):
Recalling that we chose a prediction horizon of $N = 4$ for this specific example, we begin our analysis at the very end of the matrix. Look at the row corresponding to the derivative with respect to the final state x_4 (the last block-column of H):

$$Q_N x_4 - I\lambda_4 = 0 \implies \lambda_4 = Q_N x_4$$

This equation reveals that the final Lagrange multiplier (costate) is strictly determined by the final state penalty. For the sake of recursion, we define a matrix P_N such that $P_N = Q_N$.

2. Optimising the Control ($k = N - 1$):
Now look at the row for u_3:

$$Ru_3 + B^\top \lambda_4 = 0$$

Substituting $\lambda_4 = P_4 x_4$:

$$Ru_3 + B^\top P_4 x_4 = 0$$

From the dynamics equation (last row of constraints), we know $x_4 = Ax_3 + Bu_3$. Substituting this in:

$$Ru_3 + B^\top P_4 (Ax_3 + Bu_3) = 0$$

Grouping the terms related to u_3:

$$(R + B^\top P_4 B)u_3 = -B^\top P_4 A x_3$$

This allows us to solve for the optimal input u_3 as a linear feedback of the state x_3:

$$u_3 = -\underbrace{(R + B^\top P_4 B)^{-1} B^\top P_4 A}_{K_3} x_3$$

3. Recursive Costate Update:
Finally, look at the row for x_3:

$$Qx_3 - \lambda_3 + A^\top \lambda_4 = 0 \implies \lambda_3 = Qx_3 + A^\top \lambda_4$$

Substitute λ_4 and our new formula for u_3:

$$\lambda_3 = (Q + A^\top P_4(A - BK_3))x_3$$

This implies $\lambda_3 = P_3 x_3$, where P_3 is the term in the brackets.

Conclusion: This pattern repeats for every step backwards, giving us the famous **Riccati Equation**.

Derivation of the Riccati Recursion

By solving the KKT system via back-substitution, we prove that the optimal control is a linear state feedback law $u_k = -K_k x_k$, where the matrices are computed recursively:

1. **Initialise:** Set $P_N = Q_N$.
2. **Iterate Backwards** (for $k = N - 1$ down to 0):
   - Calculate the feedback gain:
   $$K_k = (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A$$
   - Update the cost-to-go matrix (P):
   $$P_k = Q + A^\top P_{k+1}(A - BK_k)$$

Once the sequence of gains $\{K_0, \dots, K_{N-1}\}$ is calculated offline, the optimal trajectory is generated by simply simulating the system forward from x_0.

Why the Batch Formulation?

The attentive reader may ask: *"If the Riccati recursion is efficient, why did we formulate the Batch QP?"*

While recursion is ideal for unconstrained LQR, it fails when we introduce inequality constraints (e.g., $u_{min} \leq u_k \leq u_{max}$). The Batch QP formulation ($\min \zeta^\top H \zeta$ s.t. $\mathcal{C}\zeta = d, \mathcal{G}\zeta \leq h$) is the standard form used by solvers when we move to Model Predictive Control in the next chapter.

Example 5.1: Double Integrator LQR

We apply this method to a double integrator system (mass $m = 1$, force input u). The robust two-pass Riccati algorithm is implemented below.

```
clear; clc; close all;

```

```
%% Problem Setup
h = 0.1;                  % Time step
A = [1, h; 0, 1];         % State matrix
B = [0.5*h^2; h];         % Input matrix
Tfinal = 10.0;
N = floor(Tfinal / h); % Horizon steps

x0 = [1.0; 0];            % Initial state
Q = eye(2);               % State cost
R = 0.1;                  % Control cost
Qn = eye(2);              % Terminal cost

%% Backward Pass (Riccati)
P = cell(1, N + 1);
K = cell(1, N);

P{N + 1} = Qn; % Initialize terminal cost

for k = N:-1:1
  Pk_next = P{k + 1};
  % Compute Gain
  K{k} = (R + B' * Pk_next * B) \ (B' * Pk_next * A);
  % Update Cost-to-Go
  P{k} = Q + A' * Pk_next * (A - B * K{k});
end

%% Forward Pass (Simulation)
x_hist = zeros(2, N + 1);
u_hist = zeros(1, N);
x_hist(:, 1) = x0;

for k = 1:N
  u_hist(:, k) = -K{k} * x_hist(:, k);
  x_hist(:, k + 1) = A * x_hist(:, k) + B * u_hist(:, k);
end

% Plotting results... (omitted)

```

Listing 5.1: MATLAB implementation of the LQR solution via Riccati recursion.

The simulation results given in Figure 5.1 confirm that the finite-horizon optimal control problem drives the system state to the origin efficiently. It is important to note that the quadratic programme and the Riccati recursion are not two different controllers, but two equivalent ways of solving the same finite-horizon LQR problem. In both cases, the optimisation yields an optimal input sequence $(u_0, \dots, u_{N-1})$. The Riccati recursion, however, also makes explicit the underlying time-varying state-feedback law

$$u_k = -K_k x_k,$$

which is computationally more efficient and more natural for closed-loop implementation.

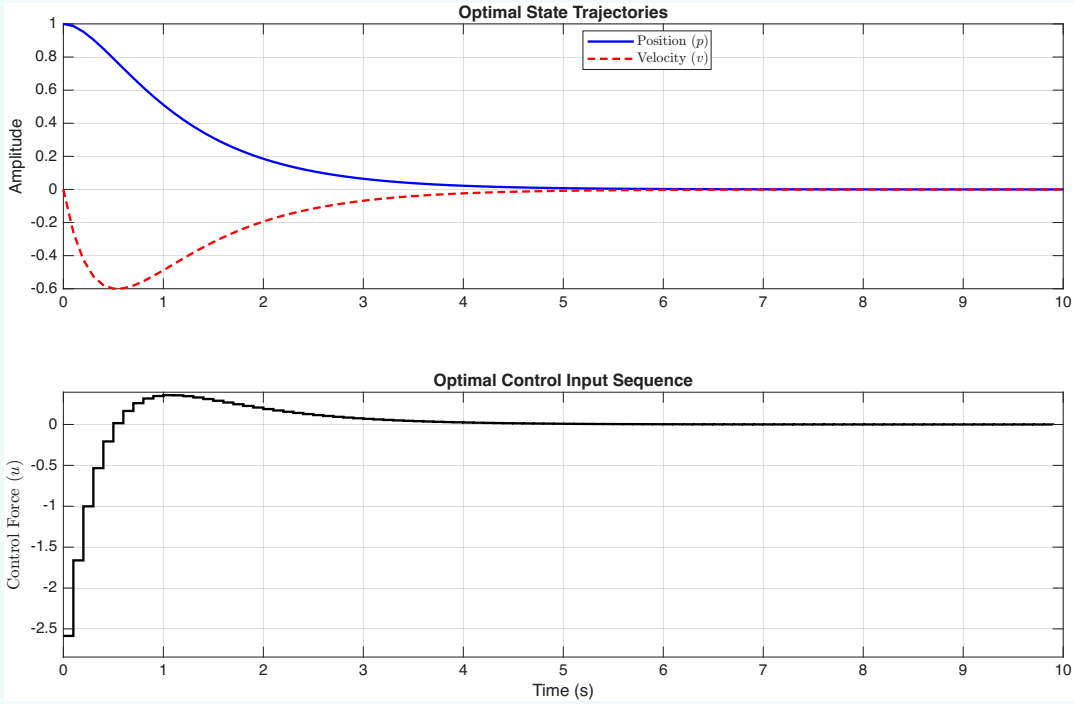


Figure 5.1: Optimal state and control trajectories for the double integrator. The states converge smoothly to the origin using the computed feedback gains.

5.4 Infinite-Horizon LQR for Stabilisation

In the previous section, we derived the LQR solution for a specific horizon N. The resulting controller, $u_k = -K_k x_k$, is **time-varying**; the feedback gain K_k changes at every single time step as the system approaches the final horizon.

However, in the vast majority of engineering applications, we are not interested in controlling a system for a fixed duration (e.g., "stabilise the robot for exactly 10 seconds"). Instead, we aim to maintain the system at a desired setpoint indefinitely—whether it is keeping a drone hovering, maintaining a room temperature, or regulating the voltage in a power grid. In these scenarios, the control horizon is effectively infinite ($N \to \infty$).

5.4.1 Convergence of the Riccati Recursion

As shown in Figure 5.2, for a system with a 2×2 matrix P, a key empirical observation provides the motivation for the infinite-horizon solution. When the finite-horizon Riccati recursion is solved backward in time for a time-invariant system (where matrices A, B, Q, and R are constant), the cost-to-go matrix P_k and the feedback gain matrix K_k converge rapidly to steady-state values.

As the backward recursion progresses from $k = N - 1$ towards $k = 0$ (moving away from the terminal time), the influence of the specific terminal cost $P_N = Q_N$ diminishes. After just a few steps, the matrices typically settle to constant values, P_∞ and K_∞. This suggests that for a

sufficiently long horizon, a single, constant feedback gain is effectively optimal for the majority of the trajectory.

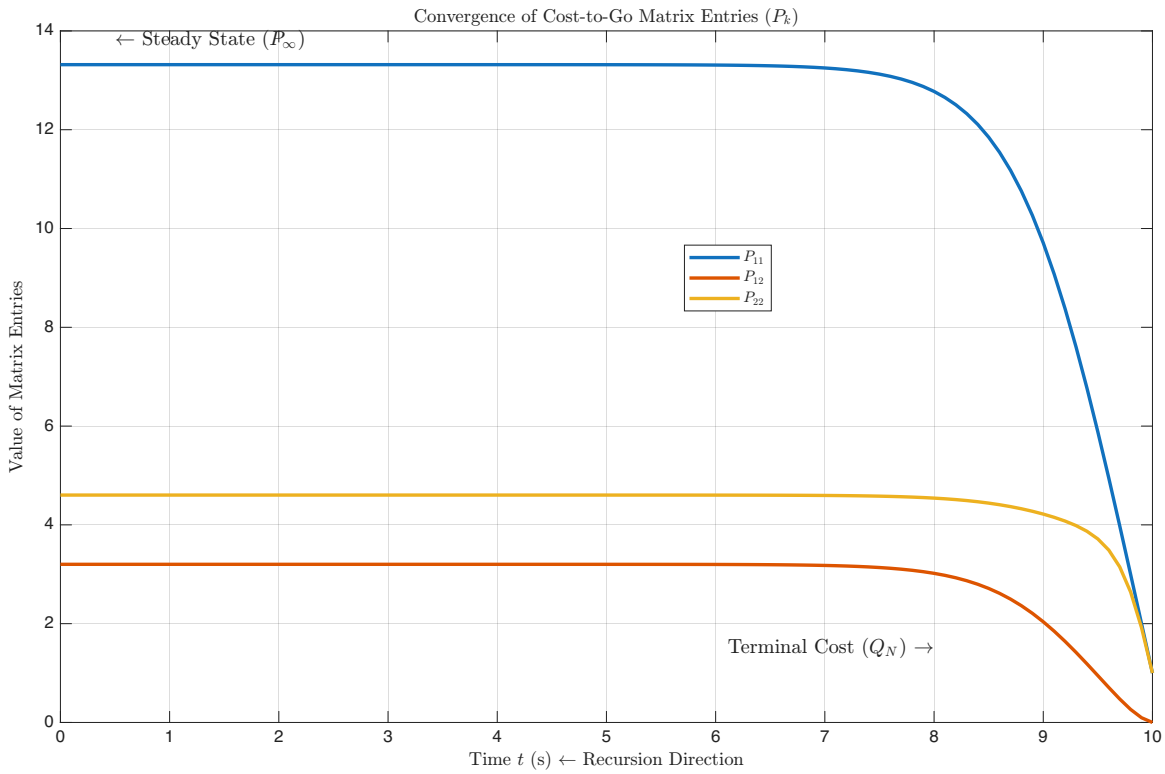


Figure 5.2: Numerical convergence of the Riccati recursion. The plot tracks the unique entries of the symmetric cost-to-go matrix P_k (P_{11}, P_{12}, P_{22}) as the recursion proceeds backward from the terminal time. Starting from the terminal condition $P_N = Q_N$, the values rapidly converge to constant steady-state values as the effective horizon increases, corresponding to the infinite-horizon solution P_∞.

5.5 The Discrete Algebraic Riccati Equation (DARE) and Infinite Horizon LQR

Instead of iterating the recursion many times to find the steady-state solution, we can solve for it directly. The convergence of P_k implies that in the infinite-horizon limit, the matrix is no longer changing from one time step to the next. We can therefore assume a fixed-point condition:

$$P_k = P_{k+1} = P \tag{5.2}$$

where P is the steady-state, positive semi-definite solution. By substituting this condition into the recursive equation for P_k, we transform the recursion into an algebraic equation.

Let us start with the two equations from the recursion:

$$\begin{aligned} K_k &= (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A \\ P_k &= Q + A^\top P_{k+1}(A - BK_k) \end{aligned}$$

Substituting the expression for K_k into the equation for P_k:

$$P_k = Q + A^\top P_{k+1} \left(A - B(R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A\right)$$

Now, applying the steady-state condition $P_k = P_{k+1} = P$, we arrive at the **Discrete Algebraic Riccati Equation (DARE)**:

$$P = Q + A^\top PA - A^\top PB(R + B^\top PB)^{-1}B^\top PA$$

This is a nonlinear matrix equation for the unknown matrix P. It is no longer a recursion but a fixed-point problem that must be solved for the unique positive semi-definite matrix P that satisfies it. (For a review of eigenvalues, positive definiteness, and matrix equations, see Appendix A.)

5.5.1 The Optimal Controller and Guaranteed Stability

Once the DARE is solved for P, the constant, optimal feedback gain K for the infinite-horizon problem is computed as:

$$K = (R + B^\top PB)^{-1}B^\top PA$$

The resulting control law is a simple static state feedback:

$$u_k = -Kx_k$$

When this feedback law is applied to the system, the dynamics of the controlled, or *closed-loop*, system become:

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k \\ &= Ax_k - BKx_k \\ &= (A - BK)x_k \end{aligned}$$

We define the closed-loop dynamics matrix as $A_{cl} = A - BK$. A cornerstone result of LQR theory is that, under standard assumptions of stabilisability and detectability, the resulting closed-loop system is guaranteed to be stable. For a discrete-time system, this means that all eigenvalues of the closed-loop matrix A_{cl} have a magnitude strictly less than one. This built-in stability guarantee is one of the most powerful features of the LQR framework, eliminating the need for manual gain tuning and stability analysis via classical methods like Bode or root locus plots.

In practice, one does not solve the DARE by hand. Specialised and highly efficient numerical solvers are used, such as the `dare` function available in standard control toolboxes for MATLAB, Python, and Julia.

The standard syntax for solving the discrete-time algebraic Riccati equation in MATLAB is:

$$[P, L, K] = \texttt{dare}(A, B, Q, R)$$

The function takes the system matrices (A, B) and the weight matrices (Q, R) as inputs. It returns:

- P: The unique, symmetric, positive-definite solution to the algebraic Riccati equation.
- L: The vector of closed-loop eigenvalues (all of which will have magnitude < 1 if the system is stabilisable).
- K: The optimal steady-state feedback gain matrix.

Example 5.2: Numerical Example: Infinite-Horizon Stabilisation

To demonstrate the power and simplicity of the infinite-horizon LQR controller, we revisit the **Double Integrator** problem. This system represents a simple mass controlled by a force, discretized with a sampling time of $h = 0.1$ seconds.

1. System Definition
The discrete-time state-space matrices (A, B) and the chosen weighting matrices (Q, R) are defined as:

$$A = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5\,h^2 \\ h \end{bmatrix}, \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R = 0.1$$

2. The Objective
Instead of computing a time-varying sequence of gains K_k for a specific duration, our goal is to find a single, constant feedback gain matrix K_∞ that will stabilise the system from any initial condition.

To do this, we solve the **Discrete Algebraic Riccati Equation (DARE)**. In MATLAB, this is automated by the command `dare`, which returns the steady-state solution P_∞ and the optimal gain K_∞.

3. Implementation The following MATLAB code calculates this gain and simulates the closed-loop response starting from $x_0 = [1; 0]$.

```
clear; clc; close all;

h = 0.1;                        % Sampling time
A = [1, h; 0, 1];              % System Dynamics
B = [0.5*h^2; h];          % Input Matrix
Q = eye(2);                   % State Penalty
R = 0.1;                        % Input Penalty

[P_inf, L_eig, K_inf] = dare(A, B, Q, R);

Tfinal = 10.0;
N = floor(Tfinal / h);
x0 = [1 ; 0];
x_hist = zeros(2, N + 1);
u_hist = zeros(1, N);
x_hist(:, 1) = x0;

A_cl = A - B * K_inf;

for k = 1:N
    u_hist(:, k) = -K_inf * x_hist(:, k);
    x_hist(:, k + 1) = A_cl * x_hist(:, k);
end

t_vec = 0:h:Tfinal;

subplot(2,1,1);
plot(t_vec, x_hist(1,:), 'b-', 'LineWidth', 2); hold on;
plot(t_vec, x_hist(2,:), 'r--', 'LineWidth', 2);
grid on;
```

```
    title('State Trajectories');
    xlabel('Time (s)'); ylabel('State');
    legend({'Position', 'Velocity'});

    subplot(2,1,2);
    plot(t_vec(1:end-1), u_hist, 'k-', 'LineWidth', 2);
    grid on;
    title('Control Input');
    xlabel('Time (s)'); ylabel('Control (u)');

```

Listing 5.2: MATLAB code for infinite-horizon LQR control using the `dare` command.

Discussion: The code makes a single call to `dare` to find the optimal policy. The simulation loop then applies the constant feedback law $u_k = -K_\infty x_k$.

The resulting trajectories (produced by the plotting section) would demonstrate that this constant gain successfully drives the state to the origin. Unlike the finite-horizon approach, which requires storing a look-up table of gains, this static K_∞ is extremely memory-efficient, making it the standard choice for regulation tasks on embedded hardware.

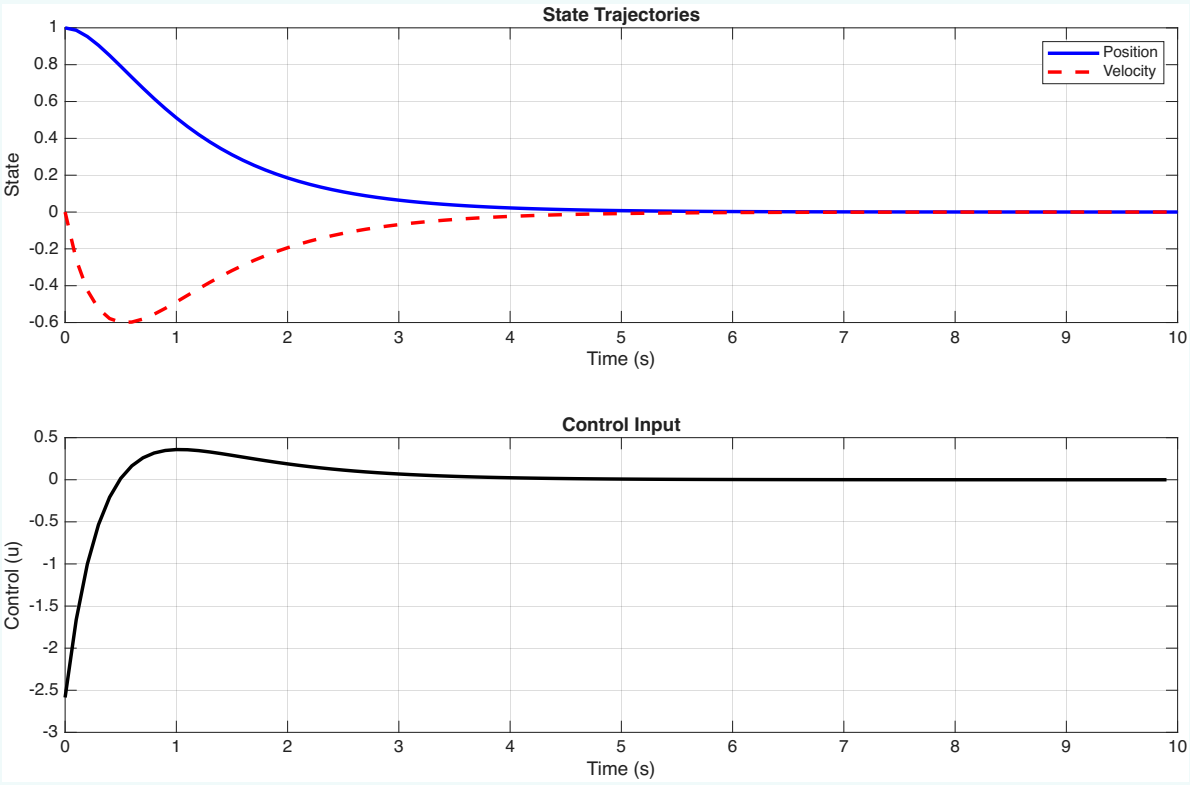


Figure 5.3: Optimal state and control trajectories for the double integrator. The states converge smoothly to the origin using the computed feedback gains.

5.6 LQR for Non-Zero Setpoint Tracking

Up to this point, we have assumed the control objective is to drive the state to the origin (regulation). However, in most practical applications, we wish to drive the system to a specific, non-zero operating point or *setpoint*.

For a target state x_g and input u_g to be a valid setpoint, the system must be able to remain there indefinitely. Mathematically, this pair must satisfy the system's equilibrium condition:

$$x_g = Ax_g + Bu_g \tag{5.3}$$

This condition implies that if the system reaches state x_g and the constant control u_g is applied, the state will not change ($x_{k+1} = x_k$).

Example 5.3: Intuition: The Hovering Drone

Consider a quadrotor drone. The state vector x consists of vertical position z and vertical velocity v.

If our goal is to hover at a height of 5 metres, the target state is $x_g = [5, 0]^\top$. However, we cannot simply turn the motors off ($u = 0$) once we reach 5 metres, or the drone will fall due to gravity.

To satisfy the equilibrium condition ($\dot{v} = 0$), the motors must generate a constant upward thrust exactly equal to the drone's weight (mg). Therefore, the equilibrium input is $u_g = mg$. Without this non-zero **feedforward** term, the controller would constantly fight against gravity, resulting in a steady-state error.

5.6.1 Error Dynamics and the Deviation System

To solve this tracking problem using standard LQR tools, we employ a change of variables. We define "deviation" or "error" variables (denoted by the tilde $\tilde{\cdot}$) representing the distance of the current system from the goal:

$$\begin{aligned} \tilde{x}_k &\triangleq x_k - x_g \\ \tilde{u}_k &\triangleq u_k - u_g \end{aligned} \tag{5.4}$$

Our objective transforms from "reaching x_g" to "driving $\tilde{x}$ to zero". To do this, we derive the dynamics of the error system by subtracting the equilibrium equation from the system dynamics:

$$\begin{aligned} \tilde{x}_{k+1} &= x_{k+1} - x_g \\ &= (Ax_k + Bu_k) - (Ax_g + Bu_g) \\ &= A(x_k - x_g) + B(u_k - u_g) \end{aligned}$$

Substituting the definitions from (5.4), we arrive at a crucial result:

$$\tilde{x}_{k+1} = A\tilde{x}_k + B\tilde{u}_k$$

This reveals that the **error dynamics are identical to the original system dynamics**. Consequently, we can design a standard LQR gain K for the error system to drive $\tilde{x} \to 0$.

5.6.2 The Affine Control Law

The optimal control for the error system is the standard regulator $\tilde{u}_k = -K\tilde{x}_k$. To implement this on the real plant, we substitute the definitions back:

$$\begin{aligned} u_k - u_g &= -K(x_k - x_g) \\ u_k &= \underbrace{u_g}_{\text{Feedforward}} \underbrace{- K(x_k - x_g)}_{\text{Feedback}} \end{aligned}$$

This structure is significant. The control law consists of:

1. A **Feedforward** term (u_g): The baseline energy required to maintain the equilibrium.
2. A **Feedback** term ($-K\tilde{x}$): The correction effort based on how far we are from the target.

Calculation of Targets from Reference Output

In practice, we are often given a desired output reference y_{ref} (e.g., "Temperature = 20°C") rather than a full state vector x_g. To find the required x_g and u_g, we must solve the system's steady-state equations simultaneously:

$$\begin{aligned} x_g &= Ax_g + Bu_g \quad \Longrightarrow \quad (I - A)x_g - Bu_g = 0 \\ y_{ref} &= Cx_g \end{aligned}$$

This can be solved as a linear system:

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_g \\ u_g \end{bmatrix} = \begin{bmatrix} 0 \\ y_{ref} \end{bmatrix} \tag{5.5}$$

Provided the matrix on the left is invertible (which requires the number of inputs to equal the number of tracked outputs), this yields the precise target state and input needed to achieve y_{ref}.

Example 5.4: Case Study: Magnetic Levitation Control

In this example, we apply the setpoint tracking strategy to a nonlinear Magnetic Levitation (MagLev) system. The objective is to control the vertical position of a steel ball by adjusting the current flowing through an electromagnet.

1. Nonlinear Mathematical Model

Let $y(t)$ be the vertical gap between the magnet and the ball, and $i(t)$ be the coil current. The equation of motion is derived from Newton's Second Law:

$$m\ddot{y} = mg - F_{\text{mag}}(y, i)$$

Assuming a simplified magnetic force model $F_{\text{mag}} = C(i/y)^2$, and defining state variables $x_1 = y$ (position) and $x_2 = \dot{y}$ (velocity), the nonlinear state-space model is:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ g - \frac{C}{m}\left(\frac{u}{x_1}\right)^2 \end{bmatrix} \tag{5.6}$$

2. Equilibrium and Linearisation

We linearise the system around a nominal operating gap y_{eq}. At equilibrium ($\dot{x}_2 = 0$), the magnetic force must balance the gravitational force ($F_{\text{mag}} = F_g$):

$$C\left(\frac{u_{eq}}{y_{eq}}\right)^2 = mg \implies u_{eq} = y_{eq}\sqrt{\frac{mg}{C}}$$

Jacobian linearisation yields the continuous-time matrices A_c and B_c:

$$A_c = \begin{bmatrix} 0 & 1 \\ \frac{2g}{y_{eq}} & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ -\frac{2g}{u_{eq}} \end{bmatrix}$$

Note that the $(2,1)$ element of A_c is positive, indicating that the open-loop system is unstable (like an inverted pendulum).

3. MATLAB Implementation

The following code designs an LQR controller based on the *linear* model. However, to prove robustness, we simulate the control law against the full *nonlinear* plant using a high-fidelity Runge-Kutta 4 (RK4) integrator.

```
clear; clc; close all;

% --- System Parameters ---
m = 0.05;       % Mass of ball (kg)
g = 9.81;       % Gravity (m/s^2)
C = 0.001;      % Magnetic constant (N m^2/A^2)

% Nominal Equilibrium (Linearisation Point)
y0 = 0.01;      % 10 mm gap
u0 = y0 * sqrt(m*g/C); % Required current to hover at y0

% --- Linearisation ---
Ac = [0, 1; (2*g)/y0, 0];
Bc = [0; -(2*g)/u0];
Cc = [1, 0];    % We measure position

% Discretisation
Ts = 0.005;     % 5 ms sampling time
sys_c = ss(Ac, Bc, Cc, 0);
sys_d = c2d(sys_c, Ts, 'zoh');
[A, B, C_out, ~] = ssdata(sys_d);

% --- LQR Design ---
Q = diag([5000, 10]);
R = 1;
[K, S, e] = dlqr(A, B, Q, R);


y_target = 0.012;
u_target = y_target * sqrt(m*g/C);
x_target = [y_target; 0];

% Initial Conditions
```

```
    T_final = 1.0;
    N = floor(T_final/Ts);
    x = [y0; 0];

    % Data logging
    x_hist = zeros(2, N);
    u_hist = zeros(1, N);

    f_dyn = @(x, u) [x(2); g - (C/m) * (u / x(1))^2];

    % --- 5. Simulation Loop ---
    for k = 1:N
        u_val = u_target - K * (x - x_target);

        % Store data
        x_hist(:, k) = x;
        u_hist(k) = u_val;

        k1 = f_dyn(x, u_val);
        k2 = f_dyn(x + 0.5*Ts*k1, u_val);
        k3 = f_dyn(x + 0.5*Ts*k2, u_val);
        k4 = f_dyn(x + Ts*k3, u_val);

        x = x + (Ts/6) * (k1 + 2*k2 + 2*k3 + k4);
        if x(1) <= 0,
          x(1) = 1e-4;
          x(2) = 0;
        end
    end

```

Listing 5.3: LQR Setpoint Tracking for a MagLev System with RK4 Simulation.

Result Analysis: The simulation shown in Figure 5.5 confirms that the linear LQR controller successfully drives the *nonlinear* plant to the new setpoint. The final control current settles at u_{target} rather than zero, validating the necessity of the feedforward term u_g to maintain equilibrium against gravity.

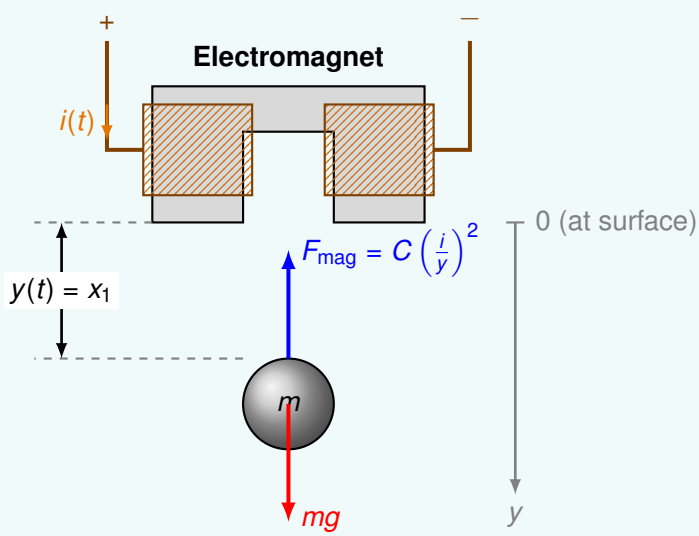


Figure 5.4: Schematic of the Magnetic Levitation system. The coordinate $y(t)$ is defined as the gap between the magnet face and the ball. Gravity acts in the positive y direction (downwards), while the magnetic force acts in the negative direction (upwards).

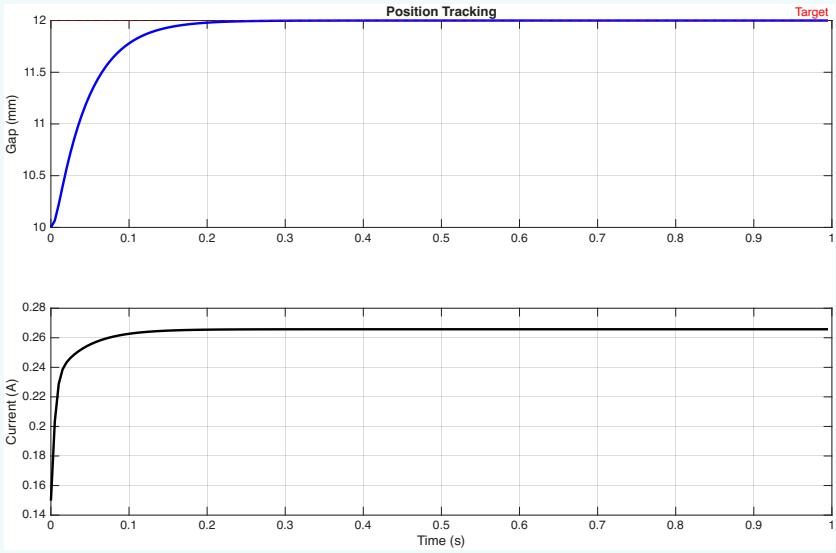


Figure 5.5: Simulation results for the Magnetic Levitation setpoint tracking problem. **Top:** The vertical gap (blue solid line) transitions smoothly from the initial equilibrium of 10 mm to the new target setpoint of 12 mm (red dashed line) with no overshoot. **Bottom:** The control current settles at a new non-zero steady-state value ($\approx$ 0.26 A). This illustrates the necessity of the feedforward term u_g: the current must remain non-zero to generate sufficient magnetic force to balance the weight of the ball at the new distance.

5.7 Chapter Summary

This chapter has established the theoretical and numerical foundations of Optimal Control, marking the transition from system analysis to control synthesis. We introduced the LQR as a specific instance of a broader optimisation framework.

By re-framing the classical recursive control problem as a batch QP, we laid the necessary groundwork for MPC. The key concepts covered in this chapter include:

- **LQR as an Optimisation Problem:** We formulated the regulator problem as the minimisation of a quadratic cost function (penalising state deviations and control effort) subject to linear equality constraints imposed by the system dynamics.
- **The Batch Formulation:** We demonstrated how to stack decision variables into large, sparse vectors to cast the dynamic control problem into a standard static QP format (min $\zeta^\top H\zeta$ s.t. $\mathcal{C}\zeta = d$). This perspective is critical for modern numerical solvers.
- **Equivalence of KKT and Riccati:** We proved that solving the KKT system of this QP via block back-substitution yields the classical Riccati recursion. This derivation confirmed that

the optimal control law is a linear state feedback, $u_k = -K_k x_k$.

- **Infinite-Horizon Stability:** We showed that as the prediction horizon extends to infinity, the time-varying feedback gains converge to a constant steady-state value. This value can be computed directly via the Discrete Algebraic Riccati Equation (DARE), providing a static control law with guaranteed closed-loop stability.
- **Setpoint Tracking:** We extended the LQR framework from simple regulation (driving states to zero) to tracking non-zero setpoints. This requires an *affine* control law combining a feedforward term (to maintain equilibrium) and a feedback term (to correct errors).

Exercises

1. Consider the discrete-time LTI system with

$$A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0.005 \\ 0.1 \end{bmatrix},$$

and weighting matrices $Q = I_2$, $R = 1$.

   (a) Using the Riccati recursion with terminal cost $P_N = Q$, compute the feedback gain K_{N-1} and the cost-to-go matrix P_{N-1} by hand for a single backward step.

   (b) Write down the resulting optimal control law $u_{N-1} = -K_{N-1}\, x_{N-1}$ and verify that the matrix $(R + B^\top P_N B)$ is invertible.

2. For the same system as in Exercise 1, investigate the influence of the weighting matrices on the closed-loop behaviour.

   (a) Solve the DARE (using MATLAB's `dare` or an equivalent tool) for the following three design choices:
   - $Q = I_2$, $R = 0.01$ (cheap control),
   - $Q = I_2$, $R = 1$ (baseline),
   - $Q = I_2$, $R = 100$ (expensive control).

   (b) For each case, compute the steady-state gain K and the eigenvalues of the closed-loop matrix $A_{cl} = A - BK$. Comment on how the magnitude of these eigenvalues changes as R increases.

   (c) Repeat the analysis by fixing $R = 1$ and varying $Q = \alpha I_2$ for $\alpha \in \{0.1,\ 1,\ 100\}$. Discuss the trade-off between fast state convergence and control effort.

3. Consider the finite-horizon LQR cost function

$$J = \frac{1}{2}\, x_N^\top P_N\, x_N + \sum_{k=0}^{N-1} \left(\frac{1}{2}\, x_k^\top Q\, x_k + \frac{1}{2}\, u_k^\top R\, u_k \right).$$

Define the *cost-to-go* at time step k as

$$J_k^*(x_k) = \frac{1}{2}\, x_k^\top P_k\, x_k.$$

(a) Starting from the terminal condition $P_N = Q_N$, show by induction that $J_k^*(x_k) = \frac{1}{2}\, x_k^\top P_k\, x_k$ where P_k satisfies the Riccati recursion

$$P_k = Q + A^\top P_{k+1}(A - BK_k), \quad K_k = (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A.$$

(b) Explain why $P_k \succeq 0$ for all k if $Q \succeq 0$, $R \succ 0$, and $P_N \succeq 0$.

4. Let P be the positive semi-definite solution of the DARE

$$P = Q + A^\top PA - A^\top PB\,(R + B^\top PB)^{-1}\, B^\top PA.$$

(a) Verify numerically (for the system in Exercise 1 with $Q = I_2$, $R = 1$) that P is symmetric and positive definite.

(b) Substitute the computed P back into the right-hand side of the DARE and confirm that both sides are equal (up to numerical precision).

(c) Define $K = (R + B^\top PB)^{-1} B^\top PA$ and compute the eigenvalues of $A_{cl} = A - BK$. Verify that all eigenvalues satisfy $|\lambda_i| < 1$, confirming closed-loop stability.

5. Consider the open-loop unstable system

$$A = \begin{bmatrix} 1.2 & 1 \\ 0 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

(a) Using classical pole placement, design a state-feedback gain K_{pp} that places the closed-loop eigenvalues of $(A - BK_{\text{pp}})$ at $\lambda_1 = 0.3$ and $\lambda_2 = 0.5$.

(b) Design an infinite-horizon LQR controller for the same system with $Q = I_2$ and $R = 1$. Report the resulting gain K_{lqr} and the closed-loop eigenvalues.

(c) Compare the two designs: Which produces a larger control effort for the initial condition $x_0 = [1;\ 0]$? Simulate both closed-loop systems for 30 time steps and discuss the differences in transient response and control magnitude. What advantages does the LQR design offer over ad-hoc pole placement?

6. A student proposes to use the weighting matrices

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad R = 0.5$$

in an LQR design.

(a) Explain why this choice of Q violates the convexity conditions required for a well-posed LQR problem.

(b) Provide a concrete example showing that the cost function $J = \sum_{k=0}^{N-1} \left(\frac{1}{2}\, x_k^\top Q\, x_k + \frac{1}{2}\, u_k^\top R\, u_k\right)$ can be made arbitrarily negative with this Q, and hence no finite minimum exists.

(c) Suggest a corrected Q matrix that penalises only the first state variable (position) while leaving the second (velocity) unpenalised, and verify that it is positive semi-definite.

Chapter 6

State Estimation and the Kalman Filter

In the previous chapter on LQR design, we assumed that the full state vector x_k was available for feedback. In practice, this is rarely the case. We typically have access only to a limited set of noisy measurements y_k. To implement modern control strategies like LQR or MPC, we must therefore reconstruct the internal state of the system from these imperfect observations.

While the concept of *Observability* tells us *if* the state can be reconstructed mathematically, it does not tell us *how* to do so robustly in the presence of uncertainty. This chapter derives the optimal solution to the state estimation problem: the **Kalman Filter** [7].

This chapter is structured as follows:

- We begin with a brief review of *probability and stochastic systems*, defining the mean and covariance matrices necessary to describe uncertainty.
- We formulate the estimation problem and derive the *recursive Kalman Filter algorithm*, showing how it optimally balances the system model against sensor data.
- We explore the mathematical *duality between estimation and control* (LQR), revealing the deep symmetry in linear systems theory.
- We discuss the *Steady-State Kalman Filter* and its relation to the Algebraic Riccati Equation, which simplifies implementation for time-invariant systems.
- Finally, we provide practical guidelines for *tuning* the filter—specifically, how to select the noise covariance matrices W and V—and demonstrate the complete design process with a MATLAB case study.

6.1 Background: Stochastic Systems and Probability

In previous chapters, we assumed that our system models were perfect and our sensors provided exact measurements. In reality, engineering systems are subject to uncertainty: actuators have jitter, sensors have noise, and mathematical models are merely approximations of complex physical reality.

To deal with this, we must transition from *deterministic* variables to *random variables*. In this framework, we do not know the exact state x_k; instead, we track the *probability* of the state being at a certain location. The Kalman Filter is essentially an algorithm that propagates the mean and covariance of these probability distributions over time.

Before deriving the filter, we briefly review the essential concepts of probability theory required for linear estimation. A detailed review of the probability concepts used in this chapter is provided in Appendix B.

6.1.1 Random Variables and Expectation

In many practical control scenarios, we cannot know the state of a system with absolute certainty due to sensor noise, model mismatch, or unknown disturbances. Consequently, we model the state not as a deterministic scalar or vector, but as a **random vector** $x \in \mathbb{R}^n$. This means x does not hold a single fixed value, but is described by a probability distribution function (PDF), denoted as $p(x)$.

The most fundamental statistical property of a random variable is the **Mean** (or Expected Value). Intuitively, this represents the "centre of gravity" of the probability distribution, or the average value one would observe if the experiment were repeated an infinite number of times. Mathematically, it is the first moment of the distribution:

$$\mu = \mathbb{E}[x] = \int_{-\infty}^{\infty} x\, p(x)\, dx$$

Note that for a vector x, this integral is applied element-wise to each component.

A crucial property of the expectation operator $\mathbb{E}[\cdot]$ is that it is **linear**. If we apply a deterministic linear transformation to a random variable—such as multiplying it by a constant matrix A or shifting it by a constant vector b—the mean transforms in exactly the same manner:

$$\mathbb{E}[Ax + b] = A\mathbb{E}[x] + b$$

This property is of extreme importance in linear system theory. It implies that for a linear dynamical system (e.g., $x_{k+1} = Ax_k + Bu_k$), the *average* behaviour of the system can be predicted simply by propagating the mean state through the deterministic system matrices, separating the deterministic dynamics from the stochastic noise.

6.1.2 The Covariance Matrix

While the mean tells us where the state is *likely* to be, it tells us nothing about how confident we are in that estimate. This uncertainty is quantified by the **Covariance Matrix**, typically denoted as P (or Σ).

For a random vector x with mean μ, the covariance is defined as the expected value of the squared error:

$$P = \mathrm{Cov}(x) = \mathbb{E}\left[(x - \mu)(x - \mu)^\top\right]$$

If x has dimension $n \times 1$, then P is an $n \times n$ matrix with specific properties:

- **Symmetric:** $P = P^\top$.

- **Positive Semi-Definite:** $P \succeq 0$.

- **Diagonal Elements (P_{ii}):** Represent the *variance* of the i-th state (σ_i^2). A large value indicates high uncertainty.

- **Off-Diagonal Elements (P_{ij}):** Represent the *covariance* between state i and state j.

6.1.3 The Gaussian Distribution

In linear estimation theory, we predominantly assume that noise and states follow a **Gaussian** (or Normal) distribution (see Appendix B for a detailed treatment of probability distributions). A Gaussian random vector is completely defined by its mean μ and covariance P. We write this as:

$$x \sim \mathcal{N}(\mu, P)$$

The Gaussian assumption is powerful because linear transformations of Gaussian variables remain Gaussian. This property allows the Kalman Filter to define the state entirely using just two parameters: the estimate vector $\hat{x}$ (the mean) and the error covariance matrix P.

6.1.4 White Noise

In system modelling, we often represent disturbances as **White Noise**. A discrete-time stochastic process w_k is called white noise if it is uncorrelated in time:

$$\mathbb{E}[w_k] = 0$$
$$\mathbb{E}[w_k w_j^\top] = \begin{cases} W_k & \text{if } k = j \\ 0 & \text{if } k \neq j \end{cases}$$

Here, W_k is the **Process Noise Covariance**. Similarly, for sensors, we define measurement noise v_k with **Measurement Noise Covariance** V_k.

Key Property: Linear Propagation of Uncertainty

The foundation of the Kalman Filter *Predict* step relies on how Gaussian distributions transform through linear systems.

Consider a random state vector x with mean $\hat{x}$ and covariance P. If this state evolves through linear dynamics with additive *zero-mean* noise w (where w has covariance W):

$$x_{new} = Ax + w$$

The statistics of the new state are given by:

- **New Mean:**
$$\mathbb{E}[x_{new}] = A\hat{x}$$
- **New Covariance:**
$$\text{Cov}(x_{new}) = APA^\top + W$$

The term $APA^\top$ represents how the system dynamics stretch and rotate the uncertainty ellipse, while W represents the additional uncertainty injected by the process noise.

6.2 Problem Formulation

We consider the problem of estimating the internal state of a discrete-time linear system subject to stochastic disturbances. Unlike deterministic control, where the state is assumed known, here we must infer it from noisy data.

6.2.1 The Stochastic System Model

The evolution of the system and the observation process are governed by the following linear difference equations:

$$\begin{aligned} x_{k+1} &= Ax_k + Bu_k + w_k \\ y_k &= Cx_k + v_k \end{aligned}$$

Here, $x_k \in \mathbb{R}^n$ is the state vector, $u_k \in \mathbb{R}^m$ is the known control input, and $y_k \in \mathbb{R}^p$ is the measured output.

6.2.2 Statistical Assumptions

To derive the optimal filter, we must rigorously define the statistical properties of the noise and the initial conditions. We assume the following:

1. **Process Noise (w_k):** Represents model mismatches and external disturbances. It is assumed to be zero-mean, white Gaussian noise with covariance matrix W:

$$w_k \sim \mathcal{N}(0, W), \quad \mathbb{E}[w_k w_j^\top] = W\delta_{kj}$$

   where δ_{kj} is the Kronecker delta (1 if $k = j$, 0 otherwise). This term ensures that the noise is uncorrelated in time (i.e., the disturbance at time k is independent of the disturbance at any other time j).

2. **Measurement Noise (v_k):** Represents sensor inaccuracies. It is assumed to be zero-mean, white Gaussian noise with covariance matrix V:

$$v_k \sim \mathcal{N}(0, V), \quad \mathbb{E}[v_k v_j^\top] = V\delta_{kj}$$

3. **Initial Condition (x_0):** The starting state is not a fixed unknown, but a random variable with a known mean and covariance:

$$x_0 \sim \mathcal{N}(\bar{x}_0, P_0)$$

4. **Independence:** The process noise w_k, measurement noise v_k, and initial state x_0 are mutually uncorrelated.

6.2.3 The Estimation Objective

Our goal is to construct an estimate of the state, denoted as $\hat{x}_k$, using the history of measurements available up to time k.

We define the **estimation error** as:

$$e_k \triangleq x_k - \hat{x}_k$$

The problem is to find the recursive algorithm that computes $\hat{x}_k$ such that the expected value of the squared error norm (the **Mean Squared Error**) is minimised at every time step:

$$\min_{\hat{x}_k} J_k = \mathbb{E}\left[\|e_k\|^2\right] = \mathbb{E}\left[(x_k - \hat{x}_k)^\top (x_k - \hat{x}_k)\right]$$

An estimator that satisfies this condition is known as the **Minimum Variance Unbiased Estimator (MVUE)**. Under the Gaussian assumptions listed above, the Kalman Filter is the unique solution to this problem.

6.3 Derivation of the Kalman Filter

To derive the optimal filter, we postulate a recursive linear estimator structure that operates in two distinct phases at each time step: a **Prediction** phase (based on the system model) and a **Correction** phase (based on the new measurement).

6.3.1 The Estimator Structure

Let $\hat{x}_k^-$ denote the *a priori* state estimate (the prediction before processing the measurement at time k), and let $\hat{x}_k$ denote the *a posteriori* estimate (after incorporating the measurement).

We assume the process noise w_k and measurement noise v_k are zero-mean, white, mutually uncorrelated, and independent of the initial estimation error, with covariances:

$$\mathbb{E}[w_k w_k^\top] = W, \qquad \mathbb{E}[v_k v_k^\top] = V.$$

The observer dynamics are defined as follows:

1. **Prediction (Time Update):** The previous estimate is propagated forward using the deterministic system dynamics:
$$\hat{x}_k^- = A\hat{x}_{k-1} + Bu_{k-1}. \tag{6.1}$$
2. **Correction (Measurement Update):** The prediction is corrected using the difference between the actual measurement and the predicted measurement. This difference is called the **Innovation** ($\tilde{y}_k$):
$$\hat{x}_k = \hat{x}_k^- + L_k \underbrace{(y_k - C\hat{x}_k^-)}_{\tilde{y}_k}. \tag{6.2}$$

Here, L_k is the **Kalman gain**. Our objective is to determine L_k such that the mean squared estimation error is minimised.

Step 1: Prediction Error Statistics Define the *a priori* estimation error

$$e_k^- \triangleq x_k - \hat{x}_k^-.$$

Substituting the system dynamics $x_k = Ax_{k-1} + Bu_{k-1} + w_{k-1}$ and the prediction equation (6.1) yields:

$$\begin{aligned} e_k^- &= A(x_{k-1} - \hat{x}_{k-1}) + w_{k-1} \\ &= Ae_{k-1} + w_{k-1}. \end{aligned}$$

The *a priori* error covariance is defined as $P_k^- \triangleq \mathbb{E}[e_k^-(e_k^-)^\top]$. By substituting the error dynamics into the definition of the covariance, we obtain:

$$P_k^- = \mathbb{E}\left[(Ae_{k-1} + w_{k-1})(Ae_{k-1} + w_{k-1})^\top\right].$$

Expanding the quadratic term yields four components:

$$\begin{aligned} P_k^- &= \mathbb{E}\left[Ae_{k-1}e_{k-1}^\top A^\top + Ae_{k-1}w_{k-1}^\top + w_{k-1}e_{k-1}^\top A^\top + w_{k-1}w_{k-1}^\top\right] \\ &= A\underbrace{\mathbb{E}[e_{k-1}e_{k-1}^\top]}_{P_{k-1}}A^\top + A\underbrace{\mathbb{E}[e_{k-1}w_{k-1}^\top]}_{0} + \underbrace{\mathbb{E}[w_{k-1}e_{k-1}^\top]}_{0}A^\top + \underbrace{\mathbb{E}[w_{k-1}w_{k-1}^\top]}_{W}. \end{aligned}$$

The cross-terms $\mathbb{E}[e_{k-1}w_{k-1}^\top]$ vanish because the white process noise w_{k-1} is uncorrelated with the error accumulated in previous time steps. Thus, the equation simplifies to:

$$P_k^- = AP_{k-1}A^\top + W. \tag{6.3}$$

Step 2: Correction Error Statistics The *a posteriori* estimation error is defined as $e_k \triangleq x_k - \hat{x}_k$. Substituting the correction equation (6.2) and the measurement model $y_k = Cx_k + v_k$ gives:

$$e_k = (I - L_k C)e_k^- - L_k v_k.$$

To derive the covariance update, we substitute the expression for e_k into the definition $P_k = \mathbb{E}[e_k e_k^\top]$:

$$P_k = \mathbb{E}\left[\left((I - L_k C)e_k^- - L_k v_k\right)\left((I - L_k C)e_k^- - L_k v_k\right)^\top\right].$$

Distributing the transpose operator $(X - Y)^\top = X^\top - Y^\top$ inside the second bracket gives:

$$P_k = \mathbb{E}\left[\left((I - L_k C)e_k^- - L_k v_k\right)\left((e_k^-)^\top (I - L_k C)^\top - v_k^\top L_k^\top\right)\right].$$

Expanding the product yields four terms:

$$P_k = \mathbb{E}\Big[\underbrace{(I - L_k C)e_k^-(e_k^-)^\top(I - L_k C)^\top}_{\text{Term 1}} - \underbrace{(I - L_k C)e_k^- v_k^\top L_k^\top}_{\text{Term 2}} - \underbrace{L_k v_k (e_k^-)^\top (I - L_k C)^\top}_{\text{Term 3}} + \underbrace{L_k v_k v_k^\top L_k^\top}_{\text{Term 4}}\Big].$$

We now analyse the expectation of each term:

- **Term 1:** Since $\mathbb{E}[e_k^-(e_k^-)^\top] = P_k^-$, this becomes $(I - L_k C)P_k^-(I - L_k C)^\top$.
- **Terms 2 & 3:** The measurement noise v_k is uncorrelated with the *a priori* estimation error e_k^- (which depends only on past history). Therefore, $\mathbb{E}[e_k^- v_k^\top] = 0$, and these cross-terms vanish.
- **Term 4:** Since $\mathbb{E}[v_k v_k^\top] = V$, this becomes $L_k V L_k^\top$.

Combining these results yields the **Joseph form** covariance update:

$$P_k = (I - L_k C)P_k^-(I - L_k C)^\top + L_k V L_k^\top. \tag{6.4}$$

Step 3: Optimisation of the Gain

To derive the optimal gain, we define a cost function J based on the total mean squared error of the states. This is equivalent to minimising the trace (sum of diagonal elements) of the *a posteriori* covariance matrix:

$$J(L_k) = \mathrm{Tr}(P_k)$$

a) Expanding the Covariance Matrix: First, we substitute the full expression for P_k from Eq. (6.4) and expand the terms:

$$P_k = (P_k^- - L_k C P_k^-)(I - C^\top L_k^\top) + L_k V L_k^\top$$
$$P_k = P_k^- - P_k^- C^\top L_k^\top - L_k C P_k^- + L_k C P_k^- C^\top L_k^\top + L_k V L_k^\top$$

We can group the quadratic terms involving L_k:

$$P_k = P_k^- - P_k^- C^\top L_k^\top - L_k C P_k^- + L_k(C P_k^- C^\top + V)L_k^\top \tag{6.5}$$

b) Matrix Calculus Rules: To minimise J with respect to the matrix L_k, we use the following standard matrix derivative identities:

- $\frac{\partial}{\partial X}\text{Tr}(XA) = A^\top$
- $\frac{\partial}{\partial X}\text{Tr}(AX^\top) = A$
- $\frac{\partial}{\partial X}\text{Tr}(XAX^\top) = 2XA$ (if A is symmetric)

Note that the trace is invariant under transpose, so $\text{Tr}(P_k^- C^\top L_k^\top) = \text{Tr}((P_k^- C^\top L_k^\top)^\top) = \text{Tr}(L_k C P_k^-)$. Thus, the two linear terms in (6.5) contribute identically to the derivative.

c) Taking the Derivative: We differentiate $J = \text{Tr}(P_k)$ with respect to L_k and set the result to zero:

$$\frac{\partial J}{\partial L_k} = \underbrace{0}_{\text{Const}} - \underbrace{2P_k^- C^\top}_{\text{Linear Terms}} + \underbrace{2L_k(CP_k^- C^\top + V)}_{\text{Quadratic Terms}} = 0$$

d) Solving for the Optimal Gain: Rearranging the equation yields:

$$L_k(CP_k^- C^\top + V) = P_k^- C^\top$$

The term in the parentheses represents the uncertainty of the measurement prediction. We define this as the **Innovation Covariance**, S_k:

$$S_k \triangleq CP_k^- C^\top + V$$

Multiplying from the right by S_k^{-1} gives the explicit solution for the optimal Kalman gain:

$$\boxed{L_k = P_k^- C^\top S_k^{-1}} \tag{6.6}$$

e) Simplifying the Covariance Update: The full update equation (6.5) is computationally expensive. We can simplify it by substituting the optimality condition back into the equation. From (6.6), we know that $L_k S_k = P_k^- C^\top$. Substituting this into the quadratic term of Eq. (6.5):

$$\begin{aligned} L_k \underbrace{(CP_k^- C^\top + V)}_{S_k} L_k^\top &= (L_k S_k) L_k^\top \\ &= (P_k^- C^\top) L_k^\top \end{aligned}$$

We substitute this back into the expanded covariance equation:

$$\begin{aligned} P_k &= P_k^- \cancel{- P_k^- C^\top L_k^\top} - L_k C P_k^- + \cancel{P_k^- C^\top L_k^\top} \\ &= P_k^- - L_k C P_k^- \end{aligned}$$

Factoring out P_k^- from the right yields the canonical covariance update equation:

$$\boxed{P_k = (I - L_k C) P_k^-}$$

Note: While this simple form is mathematically correct for the optimal gain, numerical implementations often use the longer *Joseph form* (Eq. 6.4) because it guarantees the resulting matrix remains symmetric and positive definite even in the presence of floating-point rounding errors.

The Discrete-Time Kalman Filter Algorithm

Predict:

$$\begin{aligned}\hat{x}_k^- &= A\hat{x}_{k-1} + Bu_{k-1}\\ P_k^- &= AP_{k-1}A^\top + W\end{aligned}$$

Compute Gain:

$$\begin{aligned}S_k &= CP_k^- C^\top + V\\ L_k &= P_k^- C^\top S_k^{-1}\end{aligned}$$

Correct:

$$\begin{aligned}\hat{x}_k &= \hat{x}_k^- + L_k(y_k - C\hat{x}_k^-)\\ P_k &= (I - L_k C)P_k^-\end{aligned}$$

6.4 The Steady-State Kalman Filter

In the recursive derivation above, the estimation error covariance P_k and the Kalman gain L_k are re-calculated at every time step. This is computationally expensive.

However, for **Linear Time-Invariant (LTI)** systems—where A, B, C, W, V are constant—the covariance matrix P_k^- converges to a constant steady-state matrix P_∞ as $k \to \infty$, provided certain structural properties are met.

6.4.1 Convergence Conditions

Convergence is guaranteed if the system satisfies two conditions related to observability and controllability:

1. The pair (A, C) must be **Detectable**. This ensures that any unstable mode in the system is visible at the output y, allowing the filter to correct it.

2. The pair $(A, W^{1/2})$ must be **Stabilisable**. Here, $W^{1/2}$ is the matrix square root of the process noise covariance ($W = W^{1/2}(W^{1/2})^\top$). Intuitively, this condition requires that the process noise "excites" all unstable modes of the system. If an unstable mode is not excited by noise and cannot be seen by sensors, its estimation error variance will grow unbounded.

6.4.2 The Discrete Algebraic Riccati Equation (DARE)

When these conditions are met, the *a priori* covariance P_∞ satisfies the **Discrete Algebraic Riccati Equation**:

$$P_\infty = AP_\infty A^\top + W - AP_\infty C^\top (CP_\infty C^\top + V)^{-1} CP_\infty A^\top.$$

Consequently, the Kalman gain also converges to a constant matrix:

$$L_\infty = P_\infty C^\top (CP_\infty C^\top + V)^{-1}.$$

6.4.3 The Constant Gain Implementation

Using the steady-state gain L_∞ simplifies the algorithm significantly. We no longer need to update covariance matrices online. The filter reduces to a standard linear state-space system.

We can implement this in two ways:

1. The Two-Step Form (Standard): This preserves the predict-correct structure but uses a fixed gain.

$$\begin{aligned} \textbf{Correct:} \quad & \hat{x}_k = \hat{x}_k^- + L_\infty (y_k - C\hat{x}_k^-) \\ \textbf{Predict:} \quad & \hat{x}_{k+1}^- = A\hat{x}_k + Bu_k \end{aligned}$$

2. The One-Step Form (Luenberger): We can substitute the correction step into the prediction step to create a single update equation:

$$\hat{x}_{k+1}^- = A\hat{x}_k^- + Bu_k + \underbrace{AL_\infty}_{L_{pred}}(y_k - C\hat{x}_k^-)$$

Implementation Warning

Note the term AL_∞ in the one-step form. This is because standard Kalman gains correct the *current* estimate, while the one-step form updates the *prediction* for the next step directly. When implementing this on a microcontroller, ensure you apply the gain at the correct point in the loop.

6.5 Duality between Estimation and Control

A remarkable result in linear systems theory is the mathematical symmetry, or **duality**, between the optimal control problem (LQR) and the optimal estimation problem (Kalman Filter).

In the LQR problem, we propagate a cost matrix P *backwards* in time (from the future horizon to the present) to minimise a weighted sum of states and inputs. In the Kalman Filter, we propagate

an error covariance matrix P *forwards* in time (from the initial condition to the present) to minimise the mean squared estimation error.

Despite the difference in the direction of time, the recursive equations governing these two matrices are identical under a specific substitution of variables. This means that the **same numerical algorithms** (such as the Riccati solver) used to design a controller can be used to design an observer.

Table 6.1: The Duality Mappings. To solve for the Kalman Gain using an LQR solver, substitute the LQR parameters (left) with the corresponding KF parameters (right).

Property	**LQR (Control)**	**Kalman Filter (Estimation)**
Direction	Backward ($t \leftarrow T$)	Forward ($0 \rightarrow t$)
Dynamics	A	$A^\top$
Actuation/Sensing	B	$C^\top$
State Penalty / Noise	Q	W (Process Noise)
Input Penalty / Noise	R	V (Measurement Noise)
Resulting Gain	K (Feedback)	$L^\top$ (Observer)

6.5.1 Implications for Implementation

This duality allows us to compute the steady-state Kalman gain L_∞ using standard LQR commands in software like MATLAB.

Recall that the MATLAB command for LQR is:

```
[K, P, E] = dlqr(A, B, Q, R);
```

By applying the substitutions from Table 6.1, we can compute the optimal estimator gain using the `dare` solver to obtain the steady-state error covariance P_∞, and then form the gain directly:

```
% Solve the dual DARE for the steady-state error covariance
[P_inf, ~, ~] = dare(A', C', W, V);

% Compute the steady-state Kalman gain: L = P*C'*(C*P*C'+V)^{-1}
L = P_inf * C' / (C * P_inf * C' + V);
```

This indicates that if the pair (A, C) is detectable and $(A, W^{1/2})$ is stabilisable (see Section 6.4.1), the Kalman filter estimates will converge, just as an LQR controller stabilises a controllable system. Observability and controllability are sufficient but stronger conditions.

6.6 Practical Tuning: Selecting W and V

While the system matrices (A, B, C) obey physical laws, the noise covariance matrices often serve as tuning parameters that dictate the filter's **behaviour**.

6.6.1 Measurement Noise (V)

This matrix represents sensor precision and is straightforward to determine. It is typically diagonal (assuming uncorrelated sensors) and obtained via:

- **Datasheets:** Using the manufacturer's specified noise variance (σ^2).
- **Experimentation:** Logging data from a stationary sensor and computing the statistical variance of the signal.

6.6.2 Process Noise (W)

The matrix W is more abstract. It lumps together physical disturbances (e.g., wind gusts) and **modelling errors** (e.g., linearisation inaccuracies). Since these are difficult to quantify, W is treated as a tuning handle to adjust the trade-off between the model and the measurements:

- **High W (Trust the Sensor):** The filter assumes the model is poor. The Kalman gain L increases. Result: Fast transient response, but the estimate will be noisy.
- **Low W (Trust the Model):** The filter assumes the model is accurate. The Kalman gain L decreases. Result: Smooth estimates (noise rejection), but the filter will lag during sudden changes.

Practical Rule: Fix V based on physical sensor data, then tune W until the desired balance between smoothness and responsiveness is achieved.

Example 6.1: MATLAB: Kalman Filter Implementation (LQG Control)

The Problem: Consider the double integrator system (representing a simple mass). Our objective is to stabilise the system using full-state feedback ($u_k = -Kx_k$). However, we face a practical constraint: we can only measure the **position**, and that measurement is corrupted by noise. We cannot measure the velocity directly.

The Solution: We design a **Linear Quadratic Gaussian (LQG)** controller. This consists of two parts:

1. An **LQR Controller** (K) designed for the ideal system to regulate state to zero.
2. A **Steady-State Kalman Filter** (L) to reconstruct the velocity from the noisy position data.

The Code: The script below computes the optimal gains using the duality principle ($A \to A^\top$, $B \to C^\top$) and simulates the closed-loop system.

```
clear; clc; close all;

dt = 0.1;
```

```
    A = [1, dt; 0, 1];          % States: [Position; Velocity]
    B = [0.5*dt^2; dt];      % Input: Force
    C = [1, 0];                  % Output: Position only

    Q_lqr = eye(2);
    R_lqr = 0.1;

    % B. Estimator Covariances (Kalman Filter)
    % W: Process Noise Covariance (Trust in Model)
    % V: Measurement Noise Covariance (Trust in Sensor)
    W = 1e-4 * eye(2);
    V = 1e-2;

    %% Compute Gains (Offline)
    % A. Controller Gain (K)
    [K, ~, ~] = dlqr(A, B, Q_lqr, R_lqr);

    % B. Estimator Gain (L)
    % Solve the dual DARE for the steady-state error covariance P_ss
    [P_ss, ~, ~] = dare(A', C', W, V);

    % Compute steady-state Kalman gain: L = P*C'*(C*P*C'+V)^{-1}
    L = P_ss * C' / (C * P_ss * C' + V);

    % Calculate theoretical 3-sigma error bounds for plotting
    sigma_pos = sqrt(P_ss(1,1));
    sigma_vel = sqrt(P_ss(2,2));

    %% Simulation Setup
    T_final = 10;
    N = floor(T_final/dt);
    t_vec = 0:dt:T_final-dt;

    x_true = [0.5; -0.5];    % Physical truth (starts with error)
    x_est  = [0; 0];         % Estimator initial guess (starts at 0)

    log_x_true = zeros(2, N);
    log_x_est  = zeros(2, N);
    log_y_meas = zeros(1, N);


    for k = 1:N

        w_k = sqrt(1e-4) * randn(2,1); % Process noise
        v_k = sqrt(1e-2) * randn;      % Measurement noise

        y_k = C * x_true + v_k;

        y_tilde = y_k - C * x_est;
        x_est = x_est + L * y_tilde;

        u_k = -K * x_est;

        x_est_next = A * x_est + B * u_k;
        x_true_next = A * x_true + B * u_k + w_k;

        log_x_true(:,k) = x_true;
```

```
        log_x_est(:,k)   = x_est;
        log_y_meas(k)    = y_k;

        x_true = x_true_next;
        x_est  = x_est_next;
    end

```

Listing 6.1: Closed-loop simulation of LQR control using Kalman Filter estimates.

Result: Simulation results (Figure 6.1) confirm the filter's performance. Even though the velocity is never measured directly, the Kalman Filter converges to the true velocity within a few seconds, allowing the LQR controller to stabilise the system effectively.

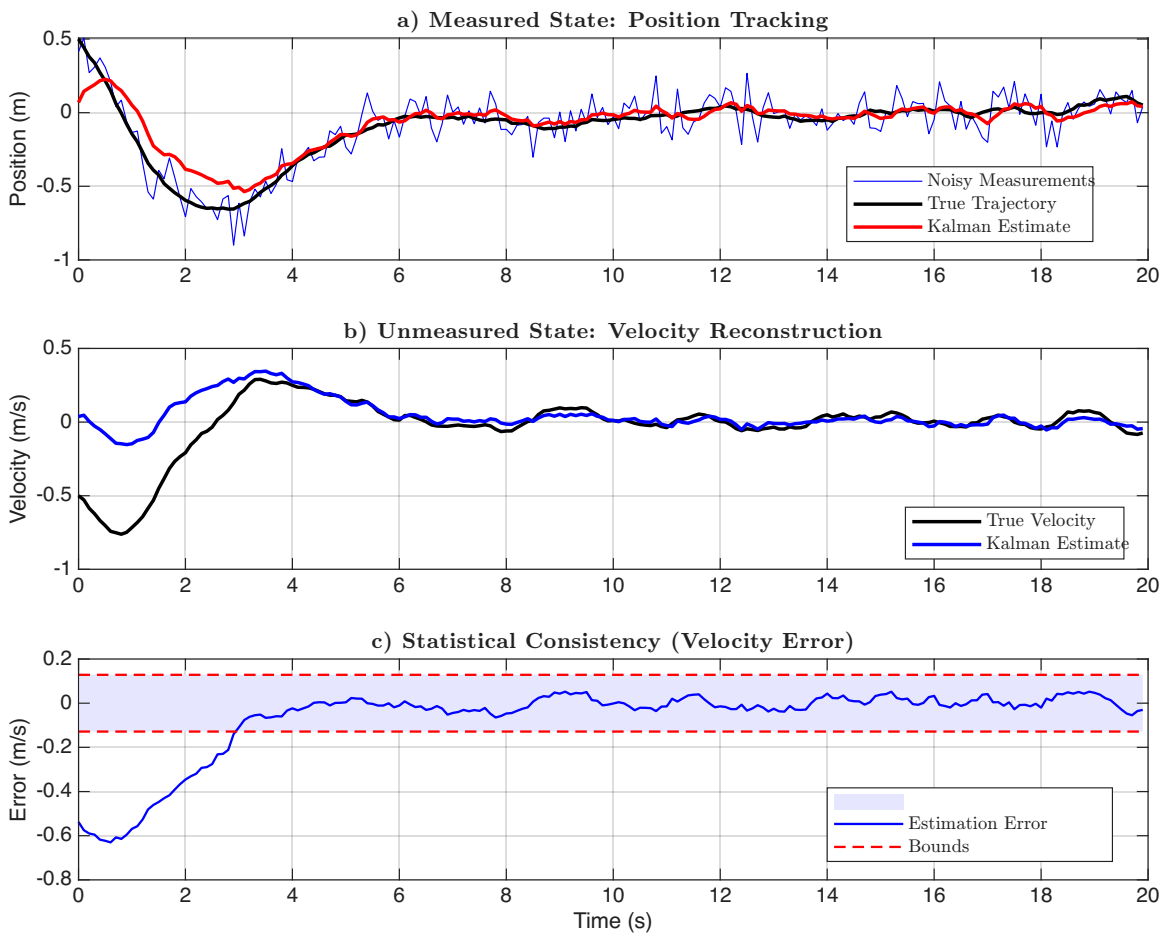


Figure 6.1: Simulation results for the LQG control example. **(a)** Measured State: The Kalman Filter estimate (red) effectively filters the noisy position measurements (blue) to track the true trajectory (black). **(b)** Unmeasured State: Although velocity is not measured directly, the filter successfully reconstructs it using the system model and position data. Note the convergence of the estimate (blue) to the truth (black) after the initial transient. **(c)** Statistical Consistency: The velocity estimation error (blue) remains bounded within the $\pm 3\sigma$ confidence intervals (red dashed lines) derived from the steady-state covariance P_∞. The large error at $t < 4$s is due to the incorrect initial guess ($x_{est} = 0$ vs $x_{true} = 0.5$), which the filter quickly corrects.

Exercises

1. **Scalar Kalman Filter by Hand.** Consider a scalar system with $A = 1$, $B = 0$, $C = 1$, process noise covariance $W = 0.5$, and measurement noise covariance $V = 2$. The initial conditions are $\hat{x}_0 = 0$ and $P_0 = 1$.

   You receive the following measurements: $y_1 = 1.2$, $y_2 = 0.9$, $y_3 = 1.5$.

   For each time step $k = 1, 2, 3$, carry out the full Kalman filter recursion by hand:

   (a) Compute the predicted estimate $\hat{x}_k^-$ and predicted error variance P_k^-.

   (b) Compute the innovation covariance $S_k = CP_k^-C + V$ and the Kalman gain $L_k = P_k^-CS_k^{-1}$.

   (c) Compute the corrected estimate $\hat{x}_k = \hat{x}_k^- + L_k(y_k - C\hat{x}_k^-)$ and the updated error variance $P_k = (1 - L_kC)P_k^-$.

   Comment on how the Kalman gain and error variance evolve over the three steps.

2. **Effect of W and V on the Kalman Gain.** Consider the same scalar system as in Exercise 1 ($A = 1$, $C = 1$), but now treat the noise covariances as parameters.

   (a) For the steady-state case, show that the scalar Kalman gain satisfies

   $$L_\infty = \frac{P_\infty}{P_\infty + V},$$

   where P_∞ is the steady-state predicted error variance.

   (b) Analyse the two limiting cases:

   - $W \gg V$ (process noise dominates): What happens to L_∞? What does this mean physically?
   - $W \ll V$ (measurement noise dominates): What happens to L_∞? What does this mean physically?

   (c) Fix $V = 1$ and compute the steady-state Kalman gain for $W \in \{0.01, 0.1, 1, 10\}$. Discuss how the filter transitions from "trusting the model" to "trusting the sensor" as W increases.

3. **Steady-State Kalman Gain via the DARE.** Consider the double integrator system from the MATLAB example in this chapter:

   $$A = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} \frac{1}{2}\Delta t^2 \\ \Delta t \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix},$$

   with $\Delta t = 0.1$, $W = 10^{-4}I_2$, and $V = 10^{-2}$.

   (a) Write out the Discrete Algebraic Riccati Equation (DARE) that determines the steady-state predicted error covariance P_∞.

   (b) Using the duality between estimation and control (Table 6.1), explain how to compute P_∞ and L_∞ using a standard LQR solver by substituting $A \to A^\top$, $B \to C^\top$, $Q \to W$, $R \to V$.

   (c) Verify numerically (e.g. in MATLAB using `dare(A', C', W, V)`) that the steady-state gain L_∞ is consistent with the values used in the chapter's simulation.

4. **Augmented State Estimation: Constant Disturbance.** Consider a scalar plant

$$x_{k+1} = a\,x_k + b\,u_k + d_k + w_k, \qquad y_k = c\,x_k + v_k,$$

where d_k is an unknown but *constant* disturbance ($d_{k+1} = d_k$). The noise w_k and v_k have covariances W and V, respectively.

(a) Define the augmented state vector $\bar{x}_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}$ and derive the augmented system matrices $\bar{A}$, $\bar{B}$, and $\bar{C}$ such that

$$\bar{x}_{k+1} = \bar{A}\,\bar{x}_k + \bar{B}\,u_k + \bar{w}_k, \qquad y_k = \bar{C}\,\bar{x}_k + v_k.$$

State the augmented process noise covariance $\bar{W}$, assuming that the disturbance drift has a small variance W_d.

(b) Discuss what conditions on the pair $(\bar{A}, \bar{C})$ are needed for the augmented Kalman filter to converge (i.e. detectability of the augmented system).

(c) For $a = 0.9$, $b = 1$, $c = 1$, $W = 0.01$, $V = 0.1$, and $W_d = 10^{-4}$, set up the DARE for the augmented system and explain how the resulting steady-state gain $\bar{L}_\infty$ simultaneously estimates x_k and d_k.

5. **Comparing Prediction and Update Steps.** Consider a system with

$$A = \begin{bmatrix} 0.8 & 0.1 \\ 0 & 0.95 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad W = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.05 \end{bmatrix}, \quad V = 0.5.$$

At time step k, suppose the corrected estimate and covariance are

$$\hat{x}_{k-1} = \begin{bmatrix} 2.0 \\ 1.0 \end{bmatrix}, \qquad P_{k-1} = \begin{bmatrix} 0.3 & 0.05 \\ 0.05 & 0.2 \end{bmatrix}.$$

(a) Compute the predicted estimate $\hat{x}_k^- = A\hat{x}_{k-1}$ (assuming $u_{k-1} = 0$) and the predicted covariance $P_k^- = AP_{k-1}A^\top + W$.

(b) Compare $\mathrm{Tr}(P_{k-1})$ with $\mathrm{Tr}(P_k^-)$. Explain why the trace increases during prediction.

(c) Suppose the measurement at time k is $y_k = 1.85$. Compute the innovation $\tilde{y}_k = y_k - C\hat{x}_k^-$, the innovation covariance S_k, the Kalman gain L_k, the corrected estimate $\hat{x}_k$, and the corrected covariance $P_k = (I - L_kC)P_k^-$.

(d) Compare $\mathrm{Tr}(P_k^-)$ with $\mathrm{Tr}(P_k)$. Explain why the trace decreases during the correction step.

6. **Observability and Filter Convergence.** Consider a two-state system with

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad C_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad C_2 = \begin{bmatrix} 0 & 0 \end{bmatrix}.$$

(a) Compute the observability matrix $\mathcal{O} = \begin{bmatrix} C \\ CA \end{bmatrix}$ for both C_1 and C_2. Which system is observable?

(b) For the observable case (C_1), explain why the Kalman filter covariance P_k is guaranteed to converge to a finite steady-state value P_∞ (assuming $W > 0$).

(c) For the unobservable case (C_2), explain why P_k grows without bound regardless of the choice of W and V. What does this imply about the practical feasibility of state estimation?

Part III

Model Predictive Control

Chapter 7

Linear Model Predictive Control

> *"MPC is like playing chess: you predict several moves ahead, pick the best move, execute it, and then – because the opponent [uncertainty] might play differently than expected – you re-evaluate the board and plan again."*
> — paraphrasing the receding horizon principle

This chapter extends unconstrained optimal control (LQR) to handle physical limits. We define the Linear MPC problem, showing how actuator saturation and safety constraints are incorporated directly into the optimisation. We then cover the **Receding Horizon** principle, introduce the **YALMIP** toolbox for implementation, and analyse how the prediction horizon N and the terminal cost P affect closed-loop stability and performance.

7.1 Motivation: The Need for Constraints

In the previous chapters, we derived the Linear Quadratic Regulator (LQR). This method provides a closed-form solution to the optimal control problem, yielding a static feedback gain matrix K. However, LQR relies on a critical, often unrealistic assumption: **the system operates in an unconstrained linear world.**

In physical reality, every engineering system has limits:

- **Actuator Saturation:** A motor has a maximum voltage; a valve cannot open beyond 100%.
- **Safety Constraints:** A chemical reactor must not exceed a critical pressure; a robotic arm must stay within a defined workspace.
- **Performance Bounds:** A passenger vehicle must limit acceleration to ensure comfort.

Why *Clipping* in LQR Fails

A common naive approach is to design an LQR controller and simply *clip* or saturate the control signal if it exceeds the limit:

$$u = \text{sat}(-Kx) = \begin{cases} u_{\max} & -Kx \geq u_{\max} \\ u_{\min} & -Kx \leq u_{\min} \\ -Kx & \text{otherwise} \end{cases}$$

While this works for small disturbances, it breaks the mathematical guarantee of optimality and stability. Saturating the input effectively turns a linear system into a non-linear one. If the LQR controller requests a strong braking force to stabilise a fast-moving vehicle, but the actuator saturates, the vehicle may overshoot or become unstable because the "planned" energy dissipation did not occur.

From Offline Gains to Online Optimisation

Model Predictive Control (MPC) fundamentally alters this approach. Instead of calculating a control law *offline* (like the LQR gain K), MPC solves a constrained optimisation problem *online* at every time step.

By incorporating the constraints directly into the prediction model, MPC "knows" about the limits in advance. It does not blindly request impossible inputs; instead, it plans a trajectory that rides the very edge of the feasible region without crossing it. This capability—to operate safely and optimally against constraints—is what makes MPC the standard in advanced industrial control.

In this chapter, we will formulate the Linear MPC problem and demonstrate how to implement it efficiently using the YALMIP toolbox.

7.2 Problem Formulation

The fundamental goal of MPC is to compute a sequence of future control inputs that drives the system state to the origin (or a target setpoint) whilst satisfying all physical constraints.

At every sampling instant t, we measure the current state of the system, denoted as $x(t)$. We then define an optimal control problem over a finite *prediction horizon* of N steps. To distinguish the prediction from the actual system evolution, we use the index k to denote the steps into the future (where $k = 0$ represents the current time t).

We solve the following constrained optimisation problem:

$$\min_{u,x} J = \underbrace{x_N^\top Q_N x_N}_{\text{Terminal Cost}} + \sum_{k=0}^{N-1} \left(\underbrace{x_k^\top Q x_k}_{\text{State Penalty}} + \underbrace{u_k^\top R u_k}_{\text{Control Effort}} \right) \tag{7.1}$$

Subject to:

$$
\begin{aligned}
\textbf{Initialisation:} \quad & x_0 = x(t) && \text{(Current measured state)} \\
\textbf{Model Dynamics:} \quad & x_{k+1} = Ax_k + Bu_k, && k = 0, \dots, N-1 \\
\textbf{Actuator Limits:} \quad & u_{\min} \leq u_k \leq u_{\max}, && k = 0, \dots, N-1 \\
\textbf{Safety Limits:} \quad & x_{\min} \leq x_k \leq x_{\max}, && k = 1, \dots, N
\end{aligned}
$$

Where:

- $u = \{u_0, u_1, \dots, u_{N-1}\}$ is the sequence of future control inputs to be optimised.
- $x = \{x_1, x_2, \dots, x_N\}$ is the sequence of predicted states.
- $Q \succeq 0$ and $R \succ 0$ are the stage cost weighting matrices.
- $Q_N \succeq 0$ is the terminal weight, often chosen to approximate the infinite-horizon cost (i.e., the solution to the Riccati equation, P).

7.3 The Receding Horizon Principle

The defining feature of MPC is its **Receding Horizon** strategy. Rather than computing a single fixed trajectory from start to finish, the controller operates in a loop:

1. **Measure:** Obtain the current state $x(t)$.
2. **Solve:** Compute the optimal sequence $\{u_0, u_1, \dots, u_{N-1}\}$ by solving the constrained optimisation problem (7.1).
3. **Apply:** Inject **only the first input** u_0 into the system. The rest of the sequence is discarded.
4. **Recede:** Wait for the next sampling instant $t+1$, measure the new state $x(t+1)$, and repeat the process.

By re-solving the problem at every time step based on fresh measurements, the MPC converts an open-loop optimal plan into a **closed-loop feedback strategy**, capable of rejecting disturbances and correcting model mismatches.

Example 7.1: Hand-Worked MPC: Scalar Integrator with Constraints

We now solve the MPC problem **entirely by hand** to make the mechanics concrete before moving to MATLAB.

System:

$$x_{k+1} = x_k + u_k \qquad \text{(pure integrator)}$$

Parameters: $Q = 1$, $R = 1$, $Q_N = 1$, prediction horizon $N = 2$, input constraint $|u_k| \leq 0.5$.

Initial condition: $x_0 = 1$.

Step 1 — Write the predicted states in terms of the decisions:

$$x_1 = x_0 + u_0 = 1 + u_0$$
$$x_2 = x_1 + u_1 = 1 + u_0 + u_1$$

Step 2 — Substitute into the cost function:

$$J = \underbrace{x_0^2}_{=1} + u_0^2 + \underbrace{x_1^2}_{(1+u_0)^2} + u_1^2 + \underbrace{x_2^2}_{(1+u_0+u_1)^2}$$

Expanding and collecting:

$$J = 3 + 4u_0 + 2u_1 + 3u_0^2 + 2u_0u_1 + 2u_1^2$$

This is a quadratic in $u = [u_0 \;\; u_1]^\top$:

$$J = u^\top \underbrace{\begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}}_{H} u + \underbrace{\begin{bmatrix} 4 \\ 2 \end{bmatrix}^\top}_{f^\top} u + 3$$

Step 3 — Solve the unconstrained QP:

Setting $\nabla_u J = 2Hu + f = 0$:

$$u^\star = -\tfrac{1}{2}H^{-1}f = -\frac{1}{2}\cdot\frac{1}{5}\begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix}\begin{bmatrix} 4 \\ 2 \end{bmatrix} = -\frac{1}{10}\begin{bmatrix} 6 \\ 2 \end{bmatrix} = \begin{bmatrix} -0.6 \\ -0.2 \end{bmatrix}$$

Step 4 — Check constraints:

Since $|u_0^\star| = 0.6 > 0.5$, the constraint is violated. We set $u_0 = -0.5$ (at its bound) and re-optimise over u_1 alone:

$$x_1 = 1 - 0.5 = 0.5, \qquad x_2 = 0.5 + u_1$$
$$J(u_1) = u_1^2 + (0.5 + u_1)^2 + \text{const.} = 2u_1^2 + u_1 + \text{const.}$$
$$\frac{dJ}{du_1} = 4u_1 + 1 = 0 \quad \Longrightarrow \quad u_1^\star = -0.25 \quad \text{(feasible)}$$

Step 5 — Apply the Receding Horizon:

We apply **only** $u_0^\star = -0.5$ and **discard** $u_1^\star = -0.25$.

The plant advances: $x_1 = 0.5$. We now re-measure and re-solve with $x_0 = 0.5$.

Since the problem is linear in x_0, the new unconstrained solution scales: $u^\star = 0.5 \times [-0.6, \; -0.2]^\top = [-0.3, \; -0.1]^\top$. Both inputs satisfy $|u_k| \leq 0.5$, so the constraints are inactive. We apply $u_0 = -0.3$ and the state moves to $x_1 = 0.2$.

Summary of the closed-loop trajectory:

Step t	$x(t)$	$u_0^\star$	Constraint active?
0	1.0	-0.50	Yes
1	0.50	-0.30	No
2	0.20	-0.12	No
3	0.08	$\cdots$	No

Key observations:

1. The constraint is active only at the first step, when the state is large. As x shrinks, the problem becomes unconstrained and the MPC law approaches the LQR law.
2. Only the first input is ever applied — re-planning at each step provides feedback.
3. The QP matrices H and f are fixed; only f changes with x_0. This is why the `optimizer` object in the next section treats x_0 as a parameter.

7.4 Efficient Implementation via YALMIP

To implement MPC, we utilise **YALMIP** [10], a MATLAB toolbox that allows us to define optimisation problems using high-level symbolic code.

A naive implementation would define and solve the problem from scratch inside the simulation loop. This is computationally prohibitive because YALMIP would need to re-parse the mathematical structure every few milliseconds. Instead, we use a technique known as **Parametric Programming**.

We construct the problem **once** offline, treating the initial state x_0 as a symbolic **parameter**. The YALMIP `optimizer` object then maps this parameter to the optimal decision variables, allowing for extremely fast execution during the simulation.

7.5 Stability and the Terminal Cost

Before writing the code that solves standard MPC, we must ensure stability [11]. For a finite horizon N, MPC is not automatically guaranteed to stabilise the system. To address this, we include a **Terminal Cost** weighted by a matrix Q_N:

$$J_{term} = x_N^\top Q_N x_N$$

Selection of the terminal weight Q_N

Choosing the terminal weight as

$$Q_N = P,$$

where P is the positive semidefinite solution of the discrete algebraic Riccati equation

$$P = A^\top PA - A^\top PB(R + B^\top PB)^{-1}B^\top PA + Q,$$

is a natural and important choice because it matches the infinite-horizon LQR cost-to-go. In the unconstrained case, this choice makes the finite-horizon MPC law consistent with the LQR solution near the origin.

In the constrained case, however, the terminal cost P alone does not in general guarantee closed-loop stability for arbitrary horizon lengths. A rigorous stability guarantee typically also requires a suitable terminal set and a terminal control law such that, inside the terminal set, the constraints remain satisfied and the terminal cost decreases under the terminal controller. Therefore, $Q_N = P$ should be viewed as an essential stabilising ingredient, but not as a complete stability guarantee by itself unless the unconstrained LQR tail is known to be feasible.

7.5.1 The YALMIP Workflow

The following script demonstrates the complete setup. Note that MATLAB uses 1-based indexing, so the symbolic variable `x{k}` corresponds to the mathematical state x_{k-1}.

```
% 1. Define Model and Parameters
A = [1.1 1; 0 1]; B = [0; 1]; % Example System
nx = 2; nu = 1;
N = 10;                       % Prediction Horizon

% 2. Weights and Constraints
Q = eye(nx); R = 0.1;
[P, ~, ~] = dare(A, B, Q, R); % Terminal Weight (Stability)
u_max = 1; x_max = 5;
u_min = -1; x_min = -5;

% 3. Define Symbolic Variables
% u(:,k) represents input u_{k-1}
% x(:,k) represents state x_{k-1}
u = sdpvar(nu, N);
x = sdpvar(nx, N+1);

% 4. Build Objective and Constraints Loop
constraints = [];
objective = 0;

for k = 1:N
  % Stage Cost
  objective = objective + x(:,k)'*Q*x(:,k) + u(:,k)'*R*u(:,k);

  % Dynamics Constraint: x_{k} = A*x_{k-1} + B*u_{k-1}
  constraints = [constraints, x(:,k+1) == A*x(:,k) + B*u(:,k)];

  % Physical Constraints
```

```
    constraints = [constraints, u_min <= u(:,k) <= u_max];
    constraints = [constraints, x_min <= x(:,k) <= x_max];
  end

  % Terminal Cost (The "Anchor" for stability)
  objective = objective + x(:,N+1)'*P*x(:,N+1);

  % 5. Create the Optimizer Object
  % We treat the 'current state' (x(:,1)) as a PARAMETER.
  % The solver will return the 'first input' (u(:,1)).
  parameters_in = x(:,1);
  solutions_out = u(:,1);

  controller = optimizer(constraints, objective, [], parameters_in,
   solutions_out);

  % --- SIMULATION USAGE ---
  % Inside the simulation loop, we simply call:
  % u_optimal = controller(x_current);
```

Listing 7.1: Efficient MPC setup using the YALMIP `optimizer` object.

Indexing Alert

Pay close attention to indexing. In mathematics, we start at $k = 0$ (current state). In MATLAB, indices start at 1. Therefore:

- Math $x_0 \to$ Code `x(:,1)` (The Parameter)
- Math $u_0 \to$ Code `u(:,1)` (The Output to apply)
- Math $x_N \to$ Code `x(:,N+1)` (The Terminal State)

Example 7.2: MATLAB: Full MPC Simulation with YALMIP

System Definition: We consider a discrete-time linear system with unstable and oscillatory dynamics. The state-space matrices are:

$$A = \begin{bmatrix} 1.5 & 0 \\ 1 & -1.5 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Note that the eigenvalue $\lambda = -1.5$ introduces a discrete-time oscillation (sign flipping at every step) combined with unstable growth, making this a challenging control target.

The MPC Optimisation Problem: At each sampling instant, the controller solves the following quadratic programme (i.e. an optimisation problem with a quadratic cost function and linear constraints) with a prediction horizon of $N = 5$:

$$\min_{u_0,\ldots,u_{N-1}} J = x_N^\top P x_N + \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k \right)$$

where P is the LQR solution of the discrete-time Riccati equation.

Subject to:

$$
\begin{aligned}
x_{k+1} &= Ax_k + Bu_k && \text{(Dynamics)} \\
-5 \leq u_k &\leq 5 && \text{(Input Saturation)} \\
\begin{bmatrix} -10 \\ -10 \end{bmatrix} \leq x_k &\leq \begin{bmatrix} 10 \\ 10 \end{bmatrix} && \text{(State Safety)} \\
x_0 &= x(t) && \text{(Measurement)}
\end{aligned}
$$

Parameters:

- **Weights:** $Q = I_2$, $R = 20$. The high input cost R penalises aggressive control actions.
- **Terminal Cost:** P_N is the unique positive-definite solution to the unconstrained LQR Algebraic Riccati Equation.

Figure 7.1 shows the simulation result obtained after using the code provided below.

```
% 1. System Matrices & MPC Weights
A = [1.5 0; 1 -1.5];  B = [1; 0];
nx = 2; nu = 1;
Q = eye(2); R = 20;

[P,~,~] = dare(A,B,Q,R);

u_min = -5; u_max = 5;
x_min = [-10; -10]; x_max = [10; 10];

N = 5;

% 2. YALMIP Formulation
u = sdpvar(nu, N);
x = sdpvar(nx, N+1);
x_init = sdpvar(nx, 1);

obj = 0;
cons = [x(:,1) == x_init];

for k = 1:N
    obj = obj + x(:,k)'*Q*x(:,k) + u(:,k)'*R*u(:,k);
    cons = [cons, x(:,k+1) == A*x(:,k) + B*u(:,k)];
    cons = [cons, u_min <= u(:,k) <= u_max];
    cons = [cons, x_min <= x(:,k) <= x_max];
end

obj = obj + x(:,N+1)'*P*x(:,N+1);

% 3. Optimizer
opts = sdpsettings('verbose', 0, 'solver', 'quadprog');
mpc_controller = optimizer(cons, obj, opts, x_init, u(:,1));

% 4. Simulation Loop
```

```
    T_sim = 30;
    x_state = [1; -2]; % Initial Condition

    % Initialize History arrays
    x_hist = zeros(nx, T_sim+1);
    u_hist = zeros(nu, T_sim);

    % Save the initial state at index 1 (Time 0)
    x_hist(:,1) = x_state;

    for t = 1:T_sim
        % A. Solve MPC
        [u_opt, errorcode] = mpc_controller(x_state);

        if errorcode ~= 0,
          warning('Solver failed!');
        end

        % B. Log Control (at time k)
        u_hist(:,t) = u_opt;

        % C. Apply Control / Plant Physics
        x_state = A*x_state + B*u_opt;

        % D. Log Next State (at time k+1)
        x_hist(:,t+1) = x_state;
    end

    % 5. Plotting
    t_vec_x = 0:T_sim;    % State time vector (0 to 30)
    t_vec_u = 0:T_sim-1;  % Control time vector (0 to 29)

    figure;
    subplot(2,1,1);
    plot(t_vec_x, x_hist(1,:), 'LineWidth', 2); hold on;
    plot(t_vec_x, x_hist(2,:), 'LineWidth', 2);
    grid on;
    ylabel('Position'); title('State Regulation');
    legend('x_1', 'x_2');
    xlim([0 T_sim]);

    subplot(2,1,2);
    stairs(t_vec_u, u_hist(1,:), 'LineWidth', 2); hold on;
    yline(u_max, 'r--', 'Max Limit', 'LineWidth', 2);
    yline(u_min, 'r--', 'Min Limit', 'LineWidth', 2);
    ylabel('Control Input (u)'); grid on;
    xlabel('Time Steps');
    xlim([0 T_sim]);

```

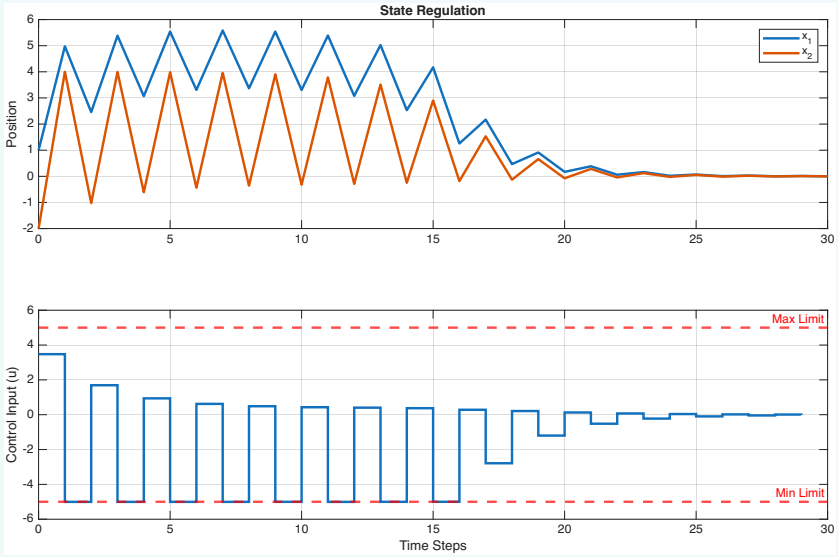


Figure 7.1: **MPC Regulation of an Unstable Oscillatory System.** The top plot displays the state trajectories starting from $x_0 = [1; -2]^\top$. The characteristic "zig-zag" behaviour is caused by the negative eigenvalue $\lambda = -1.5$, while the instability drives the amplitude growth. The bottom plot shows the control effort. The MPC controller effectively "clamps" the input to the constraint boundary ($u_{min} = -5$) for the first 16 steps to counteract the unstable dynamics. Once the system energy is sufficiently reduced (around $t = 18$), the control signal enters the linear region and smoothly drives the states to the origin.

7.6 The Effect of Horizon (N) and Terminal Cost (P)

A common pitfall in MPC design is choosing parameters that make the controller "myopic" (short-sighted). If the horizon N is too short, the controller may not see the approaching constraints or the equilibrium point in time to stop, leading to instability.

We can solve this problem in two ways:

1. **Increase N:** This extends the controller's vision, but increases computational complexity linearly (or polynomially, depending on the solver). Typically it would be reasonable to choose

$$N \cdot T_s \approx T_{\text{settling}}$$

This selection is dependent on T_{sampling}. In other words, we aim to have approximately 10 to 20 samples during the rising phase of the step response:

$$T_{\text{sampling}} \approx \frac{T_r}{10} \quad \text{to} \quad \frac{T_r}{20} \tag{7.2}$$

where T_r is the rise time of the system that is controlled. Hence,

$$N \approx 20\text{–}50 \text{ samples}$$

2. **Add Terminal Cost P:** This adds an "artificial" cost to the end of the horizon. By setting P equal to the infinite-horizon LQR cost, we trick the controller into thinking it plans for eternity, even if N is small.

While short horizons can lead to instability, a more common issue in practice is **Performance Degradation**. If the control authority is "expensive" (high R) and the horizon is short, the controller becomes "lazy."

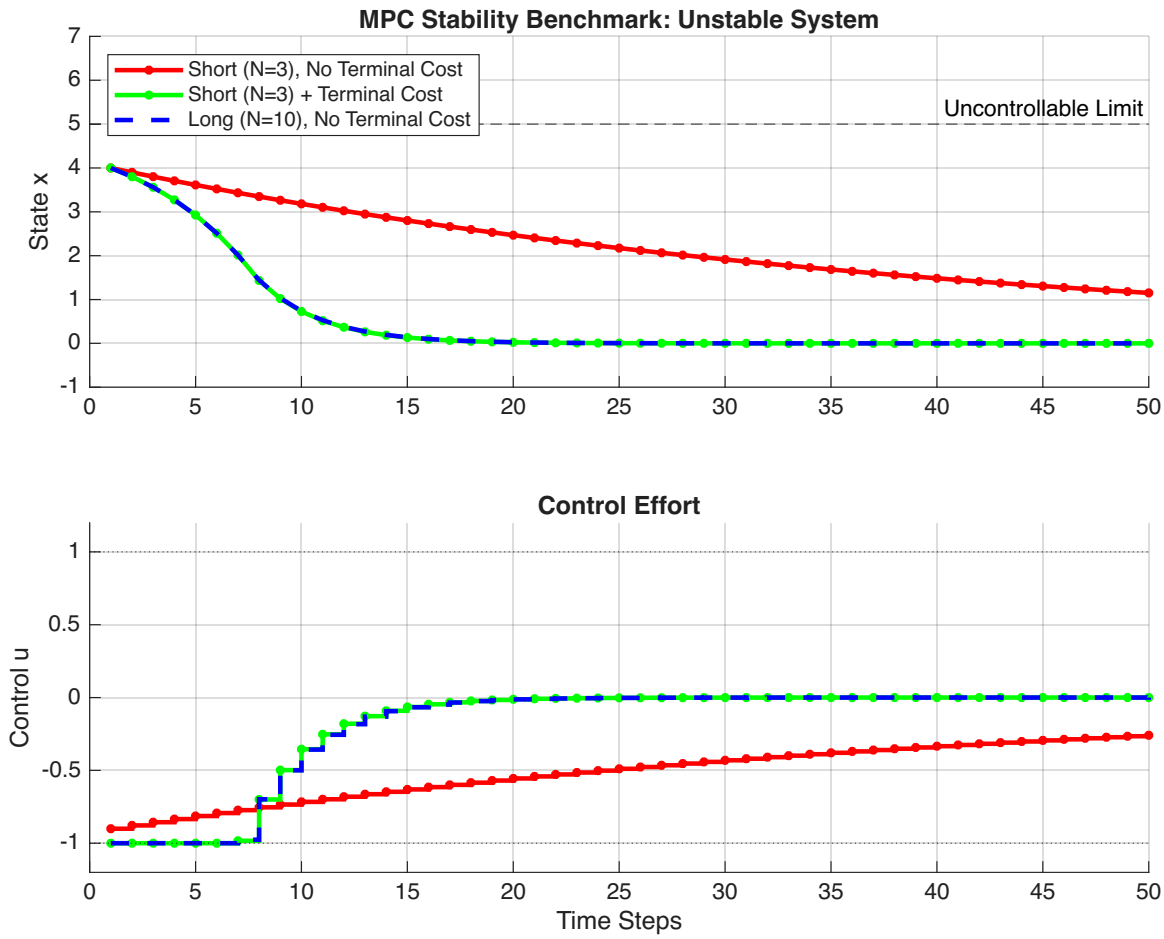


Figure 7.2: **Performance Degradation with Short Horizons.** The system starts at $x_0 = 4$ with expensive control ($R = 10$). The Short Horizon controller (Red) tries to save control effort in the short term as it cannot see long enough, leading to extremely slow convergence. The Terminal Cost controller (Green) recognises the long-term value of aggressive action, matching the performance of the Long Horizon controller (Blue).

Example 7.3: MATLAB Benchmark: Stability Analysis

To rigorously test the stability properties of the MPC controller, we utilise an *open-loop unstable scalar system*.

1. The Experiment: We define an unstable system subject to strict input constraints:

$$x_{k+1} = 1.2x_k + u_k, \qquad -1 \leq u_k \leq 1$$

The initial condition $x_0 = 4$ is chosen close to the controllability limit. We compare three controller configurations:

1. **Short Horizon ($N = 3$):** A very myopic controller.
2. **Long Horizon ($N = 10$):** A far-sighted controller.
3. **Short Horizon with P ($N = 3, P = P_{LQR}$):** A myopic controller compensated with a terminal cost.

2. The Results (see Figure 7.2):

- **The Lazy Controller (Red Line):** With $N = 3$ and no terminal cost, the controller minimises the cost over a very short window. It calculates that applying maximum braking ($u = -1$) is *too expensive* ($R = 10$) relative to the short-term reduction in state error. It chooses a "soft" braking strategy ($u \approx -0.9$). While this prevents immediate explosion, convergence is very slow. The controller fails to realise that dragging out the error over a long time will result in a much higher *total* cost.
- **The Optimal Controller (Green/Blue Lines):** By adding the Terminal Cost (P) or extending the horizon ($N = 10$), the controller accounts for the *infinite tail* of the cost function. It realises that while braking hard ($u = -1$) is expensive *now*, it is the only way to minimise the sum of squared errors over the long run. Consequently, it acts aggressively, driving the state to zero in minimal time.

Conclusion: The Terminal Cost P is not just a stability anchor; it is a crucial tuning parameter that restores **long-term optimality** to short-horizon controllers.

7.7 Recursive Feasibility and Formal Stability Guarantees$^{(*)}$

The previous sections showed experimentally that adding the terminal cost P improves MPC performance and can prevent instability. We now explain *why* this works, and what additional ingredient is needed to *guarantee* stability rigorously.

7.7.1 The Gap in the Argument

Consider the MPC cost function:

$$J = \sum_{k=0}^{N-1} (x_k^\top Q\, x_k + u_k^\top R\, u_k) + x_N^\top P\, x_N$$

The terminal cost $x_N^\top P\, x_N$ is supposed to represent the cost of controlling the system from x_N to the origin *after* the horizon ends. This representation is exact when the LQR controller $u = -Kx$ is applied from step N onward **and no constraints are violated** during that infinite tail.

But nothing in the formulation above prevents x_N from landing in a region where the LQR input

$-Kx_N$ would violate the constraint $|u| \leq u_{\max}$. If that happens, the terminal cost P is wrong — it promises a future cost that cannot actually be achieved without breaking the constraints.

This is not only a theoretical concern. When x_N is large and the constraints are tight, the MPC may plan a trajectory that looks optimal over the N-step window but leads to constraint violations (or solver infeasibility) at the very next time step.

To close this gap, we need two things:

1. A **terminal set** $\mathcal{X}_f$ — a region of the state space where the LQR controller can operate indefinitely without violating any constraint.
2. A **terminal constraint** $x_N \in \mathcal{X}_f$ added to the MPC optimisation, which forces the predicted terminal state to land inside this safe region.

7.7.2 Positive Invariant Sets

The terminal set $\mathcal{X}_f$ must have a very specific property: once the state enters $\mathcal{X}_f$, the LQR controller must keep it there forever, while satisfying all constraints at every future step. A set with this property is called *positively invariant*.

Definition: Positive Invariant Set

A set $\mathcal{X}_f \subseteq \mathbb{R}^n$ is a **positive invariant set** for the closed-loop system $x_{k+1} = (A - BK)\,x_k$ under the terminal control law $u = -Kx$, subject to constraints $u \in \mathcal{U}$, $x \in \mathcal{X}$, if for every state $x \in \mathcal{X}_f$ the following three conditions hold:

1. The input $u = -Kx$ satisfies the input constraints: $-Kx \in \mathcal{U}$.
2. The next state satisfies the state constraints: $(A - BK)x \in \mathcal{X}$.
3. The next state stays in the set: $(A - BK)x \in \mathcal{X}_f$.

Think of $\mathcal{X}_f$ as a *safe harbour*. The MPC controller must steer the state into $\mathcal{X}_f$ by the end of the prediction horizon. Once the state is inside, the LQR controller takes over and handles everything from that point on — no constraint violations, no instability, guaranteed convergence to the origin.

7.7.3 Computing $\mathcal{X}_f$: A Scalar Example

We compute $\mathcal{X}_f$ step by step for the scalar system used throughout this chapter.

System: $x_{k+1} = 1.2\,x_k + u_k$, with input constraint $|u_k| \leq 1$.

Step 1 — Find the LQR gain. Solving the DARE with $Q = 1$, $R = 1$ gives $P \approx 1.95$ and:

$$K = \frac{aPb}{R + b^2 P} = \frac{1.2 \times 1.95}{1 + 1.95} \approx 0.79$$

The terminal controller is $u = -Kx \approx -0.79\,x$, and the closed-loop eigenvalue is:

$$a_{cl} = a - bK = 1.2 - 0.79 = 0.41$$

Since $|0.41| < 1$, the closed-loop system under LQR is stable.

Step 2 — Find the largest interval where the input constraint holds. The LQR input is $u = -Kx$. The constraint $|u| \leq 1$ becomes $|Kx| \leq 1$, which gives:

$$|x| \leq \frac{1}{K} = \frac{1}{0.79} \approx 1.27$$

So the candidate terminal set is $\mathcal{X}_f = \{x : |x| \leq 1.27\}$.

Step 3 — Verify positive invariance. Take any x inside $\mathcal{X}_f$, so $|x| \leq 1.27$. Under the LQR controller, the next state is:

$$|x^+| = |a_{cl} \cdot x| = 0.41 \times |x| \leq 0.41 \times 1.27 = 0.52$$

Since $0.52 < 1.27$, the next state is well inside $\mathcal{X}_f$. The set is positively invariant.

In fact, the state *contracts* by a factor of 0.41 at each step inside $\mathcal{X}_f$. After entering the safe harbour, the system converges to zero quickly under the LQR controller.

Why does this work?

The key is that the closed-loop eigenvalue satisfies $|a_{cl}| < 1$. This means $|x^+| < |x|$ for any non-zero x, so the state always shrinks. And because the boundary of $\mathcal{X}_f$ is defined by the constraint $|Kx| = 1$, the shrinking state can never reach the boundary again once it moves inward. The constraints remain satisfied at every future step.

If the system also had state constraints $|x| \leq x_{\max}$, we would need $\mathcal{X}_f \subseteq \{x : |x| \leq x_{\max}\}$. For the example above, $1.27 < x_{\max}$ is required.

7.7.4 The Complete MPC Problem with Terminal Constraint

We now modify the MPC formulation from Section 7.2 by adding the terminal constraint. The optimisation becomes:

$$\begin{aligned}
\min_{u} \quad & J = x_N^\top P\, x_N + \sum_{k=0}^{N-1} (x_k^\top Q\, x_k + u_k^\top R\, u_k) \\
\text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k, \quad k = 0, \dots, N-1 \\
& u_{\min} \leq u_k \leq u_{\max}, \quad k = 0, \dots, N-1 \\
& x_{\min} \leq x_k \leq x_{\max}, \quad k = 1, \dots, N \\
& \boxed{x_N \in \mathcal{X}_f}
\end{aligned} \tag{7.3}$$

The only difference from before is the last line. This extra constraint restricts where the prediction horizon is allowed to end. The QP becomes slightly more constrained, but in return it provides a formal stability guarantee.

For the scalar case, $x_N \in \mathcal{X}_f$ simply means $|x_N| \leq 1/K$. For multi-state systems, $\mathcal{X}_f$ is typically a polytope (a region defined by linear inequalities), and the constraint $x_N \in \mathcal{X}_f$ adds a set of linear inequalities to the QP — so it remains a QP.

7.7.5 Recursive Feasibility

A controller is useless if it can solve the optimisation at time t but not at time $t+1$. We need the guarantee that once the MPC problem is feasible, it *stays* feasible forever. This property is called **recursive feasibility**.

Definition: Recursive Feasibility

The MPC problem is **recursively feasible** if, whenever a feasible solution exists at time t, a feasible solution also exists at time $t+1$ after applying the optimal input $u_0^\star$ to the plant.

The proof uses a simple but powerful idea called the **shifted sequence**.

Setup. Suppose at time t, the MPC solver returns the optimal input sequence:

$$u^\star = \{u_0^\star,\ u_1^\star,\ \dots,\ u_{N-1}^\star\}$$

with predicted states $\{x_0, x_1, \dots, x_N\}$, where $x_N \in \mathcal{X}_f$ (the terminal constraint is satisfied).

We apply $u_0^\star$ to the plant. The system moves to x_1.

Constructing a feasible candidate at time $t+1$. At the next time step, we need *some* feasible input sequence starting from x_1. We do not need the optimal one — any feasible sequence proves that the problem has a solution. The optimiser will then find an even better one.

Here is our candidate:

$$\tilde{u} = \{u_1^\star,\ u_2^\star,\ \dots,\ u_{N-1}^\star,\ \underbrace{-Kx_N}_{\text{LQR input at } x_N}\}$$

We take the old optimal sequence, drop $u_0^\star$ (already used), shift everything one step to the left, and append the LQR input $-Kx_N$ at the end.

Why is this candidate feasible?

- **Inputs $u_1^\star, \dots, u_{N-1}^\star$:** These satisfied the constraints in the original problem, so they still do.
- **Predicted states $x_2, x_3, \dots, x_N$:** These are the same states from the original prediction (the dynamics are deterministic). They satisfied the state constraints before, so they still do.
- **The appended input $-Kx_N$:** Since $x_N \in \mathcal{X}_f$ and $\mathcal{X}_f$ is positively invariant, the LQR input $-Kx_N$ satisfies the input constraints (by condition 1 of the definition).
- **The new terminal state $\tilde{x}_N = (A - BK)x_N$:** Since $x_N \in \mathcal{X}_f$ and the set is positively invariant, $(A - BK)x_N \in \mathcal{X}_f$ (by condition 3). So the terminal constraint is satisfied.

Every constraint is met. The candidate $\tilde{u}$ is feasible. The solver is guaranteed to find a solution (possibly a better one), so the MPC problem will never become infeasible.

The role of each ingredient

Remove any one ingredient and the argument breaks down:

- **Without $\mathcal{X}_f$:** We cannot guarantee that $-Kx_N$ satisfies the input constraints.
- **Without the terminal constraint $x_N \in \mathcal{X}_f$:** We cannot guarantee that the appended input is feasible or that the new terminal state stays in $\mathcal{X}_f$.
- **Without positive invariance:** The new terminal state $(A - BK)x_N$ might leave $\mathcal{X}_f$, breaking the terminal constraint.

7.7.6 The Stability Proof

We now show that the closed-loop system is **asymptotically stable**: the state converges to the origin. The argument uses the optimal cost $J^\star(x_t)$ as a **Lyapunov function**.

A Lyapunov function is any function $V(x)$ that satisfies three conditions:

1. $V(x) > 0$ for all $x \neq 0$, and $V(0) = 0$. (Positive definite.)
2. $V(x_{t+1}) < V(x_t)$ whenever $x_t \neq 0$. (Strictly decreasing.)
3. $V(x) \to \infty$ as $\|x\| \to \infty$. (Radially unbounded.)

If such a function exists, the state must converge to zero. The reasoning is straightforward: V is positive and keeps decreasing, so it must converge to some limit. Since V can only be zero at $x = 0$, the state must converge there too.

We claim that $V(x_t) = J^\star(x_t)$ — the optimal MPC cost — satisfies all three conditions.

Condition 1: Positivity. Since $Q \succ 0$, any non-zero initial state incurs a strictly positive first stage cost: $J^\star(x) \geq x^\top Q\, x > 0$ for $x \neq 0$. And $J^\star(0) = 0$ because the zero sequence $u = 0$ keeps the state at zero with zero cost.

Condition 3: Radial unboundedness. $J^\star(x) \geq x^\top Q\, x$, so as $\|x\| \to \infty$, $J^\star \to \infty$.

Condition 2: Strict decrease. This is the main step. We must show that $J^\star(x_{t+1}) < J^\star(x_t)$.

Write $\ell(x, u) = x^\top Q\, x + u^\top R\, u$ for the stage cost, and $V_f(x) = x^\top P\, x$ for the terminal cost. At time t, the optimal cost is:

$$J^\star(x_t) = \ell(x_t, u_0^\star) + \sum_{k=1}^{N-1} \ell(x_k, u_k^\star) + V_f(x_N)$$

At time $t+1$, the state is $x_{t+1} = x_1$. We do not know the new optimal cost, but we can bound it from above using the shifted candidate $\tilde{u}$, since the optimal cost is at most the cost of any feasible sequence:

$$J^\star(x_{t+1}) \;\leq\; \underbrace{\sum_{k=1}^{N-1} \ell(x_k, u_k^\star)}_{\text{old stage costs (shifted)}} + \underbrace{\ell(x_N, -Kx_N) + V_f((A - BK)x_N)}_{\text{cost of the appended LQR step}}$$

Now subtract:

$$J^\star(x_t) - J^\star(x_{t+1}) \;\geq\; \ell(x_t, u_0^\star) + V_f(x_N) - \ell(x_N, -Kx_N) - V_f((A - BK)x_N)$$

Here is where the choice of P matters. Because P is the DARE solution, it satisfies:

$$V_f(x) = \ell(x, -Kx) + V_f((A - BK)x) \quad \text{for all } x$$

This is exactly what the Riccati equation says in cost-to-go language: the LQR cost at state x equals the current stage cost under the LQR law plus the LQR cost at the next state. Substituting $x = x_N$:

$$V_f(x_N) - \ell(x_N, -Kx_N) - V_f((A - BK)x_N) = 0$$

So the middle terms cancel, leaving:

$$J^\star(x_t) - J^\star(x_{t+1}) \;\geq\; \ell(x_t, u_0^\star) \;=\; x_t^\top Q\, x_t + u_0^{\star\top} R\, u_0^\star \;\geq\; x_t^\top Q\, x_t \;>\; 0$$

The optimal cost decreases by at least $x_t^\top Q\, x_t$ at every step. Since $Q \succ 0$, this decrease is strictly positive whenever $x_t \neq 0$.

Theorem: Stability of MPC with Terminal Ingredients

Consider the linear system $x_{k+1} = Ax_k + Bu_k$ with constraints $u_k \in \mathcal{U}$, $x_k \in \mathcal{X}$, stage cost weights $Q \succ 0$, $R \succ 0$, and suppose:

1. The terminal cost is $V_f(x) = x^\top P\, x$, where P solves the DARE.
2. The terminal controller is $\kappa_f(x) = -Kx$ (the LQR gain from the same DARE).
3. A positive invariant set $\mathcal{X}_f$ exists for the closed-loop $(A - BK)$ under the constraints.
4. The terminal constraint $x_N \in \mathcal{X}_f$ is included in the MPC formulation.

Then:

- **Recursive feasibility:** If the MPC is feasible at $t = 0$, it is feasible for all $t \geq 0$.
- **Asymptotic stability:** The closed-loop state converges to the origin: $x_t \to 0$ as $t \to \infty$.
- **Lyapunov function:** The optimal cost $J^\star(x_t)$ is non-increasing and serves as a Lyapunov function for the closed-loop system.

7.7.7 What This Means in Practice

The stability theorem gives a clean recipe: choose P and K from the DARE, compute the positive invariant set $\mathcal{X}_f$, and add the constraint $x_N \in \mathcal{X}_f$ to the QP. However, there are practical trade-offs.

- **The terminal constraint shrinks the feasible region.** Only initial states from which the MPC can drive x_N into $\mathcal{X}_f$ in N steps are feasible. If N is small or $\mathcal{X}_f$ is small, the set of feasible initial states may be too restrictive.

- **Increasing N enlarges the feasible region.** With more steps, the controller has more room to manoeuvre the terminal state into $\mathcal{X}_f$. In the limit $N \to \infty$, the set of feasible initial states approaches the maximal controllable set.
- **Computing $\mathcal{X}_f$ for multi-state systems is harder.** For a scalar system, $\mathcal{X}_f$ is just an interval. For higher-dimensional systems, it is a polytope (a region defined by many linear inequalities). Algorithms exist to compute it iteratively, but the number of inequalities can grow quickly.
- **Many practical implementations omit the terminal constraint.** Engineers often rely on choosing N large enough that x_N naturally ends up near the origin, where the LQR law does not violate constraints. This is not rigorous, but it works well when the horizon covers the system's settling time ($N \cdot T_s \approx T_{\text{settling}}$, as discussed in Section 7.6). The terminal cost P alone provides adequate stability in most applications, even without the formal terminal set constraint.

The following table summarises the three levels of MPC design, from simplest to most rigorous:

Design	Terminal cost	Terminal set	Guarantee
Basic MPC	$Q_N = Q$	None	No formal guarantee
MPC with P	$Q_N = P$ (DARE)	None	Works in practice; no proof
Rigorous MPC	$Q_N = P$ (DARE)	$x_N \in \mathcal{X}_f$	Recursive feasibility + stability

For an MSc-level course and most industrial applications, using $Q_N = P$ with a sufficiently long horizon is the standard approach. The rigorous formulation with $\mathcal{X}_f$ is used when formal certification is required (e.g., aerospace, medical devices) or when the horizon N must be kept very short for computational reasons.

7.7.8 Why Terminal Set Size Matters

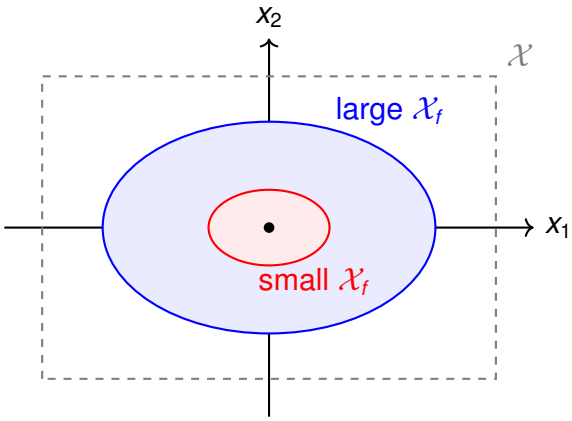
The size of $\mathcal{X}_f$ directly controls the set of initial states from which MPC can find a feasible solution. Understanding this trade-off is important for controller design.

A small terminal set means $\mathcal{X}_f$ is a tight region around the origin. The MPC must drive x_N into this small target within N steps, which is harder. The consequence: fewer initial states lead to a feasible problem, and a longer horizon N may be needed to compensate.

A large terminal set means $\mathcal{X}_f$ covers more of the state space. The MPC has an easier target for x_N, so feasibility is maintained for a wider range of initial conditions, even with a short horizon.

However, there is an upper limit: $\mathcal{X}_f$ cannot exceed the *maximal positive invariant set* for the closed-loop system $(A - BK)$ under the given constraints. Any state outside this maximal set will eventually be driven to a constraint violation by the LQR controller, so it cannot belong to $\mathcal{X}_f$.

The ideal design therefore computes the largest possible $\mathcal{X}_f$ (the maximal positive invariant set), which maximises the region of attraction.



In the figure above, the dashed box is the state constraint set $\mathcal{X}$. With a small $\mathcal{X}_f$ (red), the MPC must work harder to hit the target. With a large $\mathcal{X}_f$ (blue), more initial states lead to feasible problems.

7.7.9 Computing $\mathcal{X}_f$ for Multi-State Systems Using MPT3

For scalar systems, we computed $\mathcal{X}_f$ by hand in the earlier example. For systems with two or more states, the terminal set is a polytope (a region bounded by many linear inequalities), and computing it by hand becomes impractical. The standard approach uses an iterative algorithm:

1. Start with the constraint set: $\Omega_0 = \{x \in \mathcal{X} \mid Kx \in \mathcal{U}\}$.
2. At each iteration, intersect with the pre-image: $\Omega_{i+1} = \Omega_i \cap \{x \mid (A - BK)x \in \Omega_i\}$.
3. Repeat until $\Omega_{i+1} = \Omega_i$ (the set stops shrinking). This fixed point is the maximal positive invariant set $\mathcal{X}_f$.

The iteration is guaranteed to converge in a finite number of steps for stable closed-loop systems with polytopic constraints.

In MATLAB, the **Multi-Parametric Toolbox 3 (MPT3)** [5] automates this computation. MPT3 is a free toolbox available at `https://www.mpt3.org`. The following code computes $\mathcal{X}_f$ for a two-state system:

```
% System matrices
A = [1.2 1; 0 1];
B = [0.5; 1];

% Constraints
umax = 1;
xmax = [5; 5];  % state bounds: -xmax <= x <= xmax

% Step 1: Compute LQR gain and DARE solution
Q = eye(2);
```

```
R = 1;
[K_lqr, P_dare] = dlqr(A, B, Q, R);

% Closed-loop matrix under LQR
Acl = A - B*K_lqr;

% Step 2: Define constraint polytopes using MPT3
X = Polyhedron('lb', -xmax, 'ub', xmax);   % state constraints
U = Polyhedron('lb', -umax, 'ub', umax);   % input constraints

% Step 3: Build the closed-loop LTI system
sys = LTISystem('A', Acl);
sys.x.min = -xmax;
sys.x.max = xmax;

% Map input constraints through K: |K*x| <= umax
KPoly = Polyhedron('A', [K_lqr; -K_lqr], 'b', [umax; umax]);

% Step 4: Compute the maximal positive invariant set
Xf = sys.invariantSet();

% Intersect with input constraint mapped through K
Xf = Xf & KPoly;

% Step 5: Visualise
figure;
plot(X, 'color', [0.9 0.9 0.9], 'alpha', 0.3); hold on;
plot(Xf, 'color', 'blue', 'alpha', 0.5);
xlabel('x_1'); ylabel('x_2');
legend('State constraints X', 'Terminal set X_f');
title('Maximal Positive Invariant Set');
```

Listing 7.2: Computing $\mathcal{X}_f$ using MPT3

The key function is `sys.invariantSet()`, which performs the iterative algorithm described above automatically. For the example above, the result is a polytope with several faces (linear inequalities) that can be directly used as the terminal constraint $x_N \in \mathcal{X}_f$ in the MPC formulation.

MPT3 and YALMIP

MPT3 uses YALMIP internally, so both toolboxes must be installed. Once $\mathcal{X}_f$ is computed as a `Polyhedron` object, its half-space representation $H_f x \leq h_f$ can be extracted with `Xf.A` and `Xf.b`, and added directly to the MPC optimisation as the linear constraint $H_f x_N \leq h_f$.

Combining MPC with State Estimation

This chapter assumed that the full state x_k is available for measurement. In practice, only noisy partial measurements $y_k = Cx_k + v_k$ are available, and the state must be estimated using a Kalman filter (Chapter 6). Chapter 8 develops the output-feedback MPC architecture that combines the estimator, a target calculator, and the MPC regulator into a complete control system.

Exercises

1. Consider the scalar system
$$x_{k+1} = 1.5\,x_k + u_k$$
with weighting matrices $Q = 1$, $R = 2$, and a prediction horizon $N = 2$.

   (a) Write out the predicted states x_1 and x_2 as functions of x_0 and the decision variables u_0, u_1.

   (b) Using these expressions, eliminate the states from the cost function
$$J = x_2^\top Q_N\, x_2 + \sum_{k=0}^{1} (x_k^\top Q\, x_k + u_k^\top R\, u_k)$$
(set $Q_N = Q$ for simplicity) and express J in the standard quadratic form
$$J = u^\top H\, u + 2\, x_0^\top f^\top u + \text{const.}$$
where $u = [u_0 \;\; u_1]^\top$. State the matrices H and f explicitly.

2. For the system
$$x_{k+1} = 1.2\,x_k + u_k, \qquad Q = 1, \quad R = 0.5, \quad N = 1, \quad Q_N = Q,$$
with no constraints and an initial condition $x_0 = 3$:

   (a) Show that the MPC optimisation at $k = 0$ reduces to a single-variable unconstrained quadratic minimisation in u_0.

   (b) Find the optimal input $u_0^\star$ and the resulting next state x_1.

   (c) Explain why only $u_0^\star$ is applied to the plant and what happens at the next sampling instant under the receding horizon principle.

3. Consider the same system as in Exercise 2, but now add the input constraint $-1 \le u_k \le 1$.

   (a) For $x_0 = 3$, determine whether the unconstrained optimum $u_0^\star$ from Exercise 2 violates the constraint. If so, find the constrained optimum.

   (b) Repeat for $x_0 = 10$. Simulate the closed-loop response for several steps by applying the receding horizon strategy with $N = 1$. Does the state converge to the origin?

   (c) Determine (analytically or by trial) the largest initial condition $|x_0|$ for which the MPC problem with $N = 1$ and $-1 \le u_k \le 1$ remains feasible. What happens if x_0 exceeds this value?

4. Consider a double integrator in discrete time:
$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad B = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix},$$
with $Q = I_2$, $R = 1$, $Q_N = Q$, input constraint $-1 \le u_k \le 1$, and initial condition $x_0 = [4 \;\; 0]^\top$.

   (a) Using YALMIP (or by hand for $N = 2$), solve the MPC problem for horizons $N = 2, 5, 10, 20$ and record the first optimal input $u_0^\star$ in each case.

   (b) Simulate the closed-loop response for each horizon length over 30 time steps. Plot the state trajectories and input sequences.

(c) Discuss how N affects the speed of convergence, constraint activity, and computational cost. What is the minimum N for which the closed-loop system appears stable?

5. Return to the scalar system

$$x_{k+1} = 1.2\, x_k + u_k, \qquad -1 \leq u_k \leq 1,$$

with $Q = 1$, $R = 10$, and $x_0 = 4$.

(a) Solve the discrete algebraic Riccati equation to obtain the terminal cost weight P.

(b) Simulate the closed-loop system under MPC with $N = 3$ and $Q_N = Q$ (no terminal cost correction). Repeat with $Q_N = P$. Compare the convergence rates.

(c) Explain, using the concept of the infinite-horizon cost-to-go, why adding the terminal cost P improves performance even though the horizon N remains short.

(d) For the case $Q_N = P$, under what conditions is the terminal cost a valid approximation of the true cost-to-go beyond the horizon? State the role of the terminal set $\mathcal{X}_f$ and the terminal control law $u = -Kx$ in a rigorous stability proof.

6. A control engineer designs an MPC controller for a chemical reactor modelled as

$$x_{k+1} = Ax_k + Bu_k, \qquad x_k \in \mathbb{R}^3, \quad u_k \in \mathbb{R}^2,$$

with state constraints $x_{\min} \leq x_k \leq x_{\max}$ representing safety limits and input constraints $u_{\min} \leq u_k \leq u_{\max}$ representing valve positions.

(a) The engineer observes that the MPC solver returns an infeasibility warning at certain operating points. List three possible causes of infeasibility and, for each, suggest a practical remedy.

(b) The engineer proposes softening the state constraints by introducing slack variables $\epsilon_k \geq 0$ and replacing $x_{\min} \leq x_k \leq x_{\max}$ with $x_{\min} - \epsilon_k \leq x_k \leq x_{\max} + \epsilon_k$, adding a penalty $\rho\|\epsilon_k\|^2$ to the cost. Explain the trade-off involved in choosing ρ and why input constraints should generally *not* be softened.

(c) The engineer is considering two design choices: (i) a short horizon $N = 5$ with the DARE terminal cost $Q_N = P$, and (ii) a long horizon $N = 50$ with $Q_N = Q$. Discuss the advantages and disadvantages of each choice in terms of closed-loop stability, computational load, and robustness to model uncertainty.

Chapter 8

Output Feedback and Offset-Free MPC

8.1 Introduction

In the previous chapter, we derived Model Predictive Control under idealised assumptions: full state measurement, a perfect model, and no disturbances. Under these pure conditions, a simple regulator is sufficient to drive the system to the origin.

However, real-world control engineering is defined by its imperfections. Sensors are noisy and limited in number, meaning that the full state is rarely measurable. Models are merely approximations, failing to capture complex phenomena such as friction, aerodynamic drag or slowly varying biases. Furthermore, disturbances are rarely *zero-mean*; a drone is subject to a constant gravitational force, and a building experiences continuous solar heat gains.

If we apply a standard MPC regulator to such a system, it will generally fail to track the reference exactly. The mismatch between the model's prediction and the plant's actual behaviour results in a permanent *steady-state error* (or offset).

This chapter addresses these limitations. We introduce the *output-feedback* framework, replacing the assumption of full-state access with a *Kalman filter* that estimates the state from noisy measurements. We then develop *offset-free MPC* [14]: by augmenting the system model with a *disturbance state* and employing a *target calculator*, the MPC gains an integral-action-like capability. Under the usual observability, feasibility and modelling assumptions, this enables zero steady-state tracking error for constant or slowly varying disturbances and references.

Connection to Classical Control

In PID control, steady-state error is eliminated by adding an *integrator* term $\frac{1}{s}$. In MPC, the same effect is achieved by *estimating a disturbance* d. In the language of the *internal model principle*, augmenting the controller with a constant-disturbance model is closely related to embedding integral action.

8.2 The Disturbance Model

Standard LQR/MPC typically models plant uncertainty through zero-mean stochastic terms such as process noise w_k. However, many practically important effects behave instead like persistent biases or slowly varying loads, such as gravity acting on a drone or solar gains acting on a building.

To handle this, we augment the system model with a *disturbance state* d_k, assumed constant or slowly varying:

$$x_{k+1} = Ax_k + Bu_k + B_w d_k + w_k \tag{8.1}$$

$$d_{k+1} = d_k + \xi_k \tag{8.2}$$

$$y_k = Cx_k + C_d d_k + v_k \tag{8.3}$$

where:

- $d_k \in \mathbb{R}^{n_d}$ is the integrating disturbance state.
- B_w determines how the disturbance enters the state equation. A common choice is the *input-disturbance model*, in which $B_w = B$, meaning that the disturbance acts through the same channel as the control input.
- C_d models disturbances that affect the output directly, for example a measurement bias.
- ξ_k is a *fictitious* process-noise term. Physically, d_k may be constant, but we allow ξ_k in the estimator design so that the Kalman filter remains responsive if the real disturbance changes slowly over time.

8.3 State and Disturbance Estimation

Since the disturbance d_k is not ordinarily measurable, it must be estimated. The key idea is to treat the unknown disturbance as an additional state variable.

We therefore define the *augmented state vector*

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix},$$

and construct the corresponding augmented system:

$$A_{\text{aug}} = \begin{bmatrix} A & B_w \\ 0 & I \end{bmatrix}, \qquad B_{\text{aug}} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \qquad C_{\text{aug}} = \begin{bmatrix} C & C_d \end{bmatrix}.$$

The identity matrix in the lower-right block of A_{aug} expresses the modelling assumption that the disturbance remains constant from one step to the next unless the estimator sees evidence to the contrary.

A standard Kalman filter may now be designed for the augmented system $(A_{\text{aug}}, C_{\text{aug}})$.

Observability Condition

For the disturbance to be estimable, the augmented pair $(A_{\text{aug}}, C_{\text{aug}})$ must be observable, or at least detectable. Intuitively, the available measurements must contain enough information to distinguish a change in the physical state x_k from a change in the disturbance d_k.

A useful practical rule of thumb is not to introduce more independent integrating-disturbance components than can be supported by the measured outputs. Otherwise, the augmented model is likely to become unobservable.

8.4 The Target Calculator

To achieve offset-free tracking, we cannot simply drive the input towards zero. If a disturbance $\hat{d}_k$ is pushing the system, the actuators must generally maintain a non-zero steady-state input u_{ss} in order to counteract it.

We therefore compute steady-state targets by solving the equilibrium equations. At steady state we require:

1. The state to remain stationary,

$$x_{ss} = Ax_{ss} + Bu_{ss} + B_w\hat{d}_k,$$

2. The output to match the reference,

$$r_k = Cx_{ss} + C_d\hat{d}_k.$$

8.4.1 Method 1: Linear Solution (Unconstrained)

In the simplest case, where the target is reachable and no steady-state constraints are enforced, the equations above reduce to the linear system

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_{ss} \\ u_{ss} \end{bmatrix} = \begin{bmatrix} B_w\hat{d}_k \\ r_k - C_d\hat{d}_k \end{bmatrix}. \tag{8.4}$$

If the coefficient matrix is nonsingular, this yields a computationally cheap steady-state target. However, it ignores actuator and state/output constraints, and therefore does not guarantee that the resulting target is physically reachable.

8.4.2 Method 2: Constrained Optimisation (Robust)

The Reachability Problem

If the linear solution produces, for example, $u_{ss} > u_{\max}$, then the target is infeasible. Feeding an unreachable target to the MPC regulator can compromise recursive feasibility and may lead to poor performance or failure.

To handle this, we reformulate the target calculation as a constrained **steady-state optimisation** (SSO) problem. Instead of insisting on exact equality $y_{ss} = r_k$, we minimise the tracking error whilst enforcing the equilibrium equations and any relevant constraints.

At each time step, after estimating $\hat{d}_k$, we solve

$$\begin{aligned} \min_{x_{ss},u_{ss}} \quad & \|Cx_{ss} + C_d\hat{d}_k - r_k\|_Q^2 + \|u_{ss}\|_R^2 \\ \text{subject to:} \quad & (I - A)x_{ss} - Bu_{ss} = B_w\hat{d}_k, \\ & u_{\min} \leq u_{ss} \leq u_{\max}, \\ & x_{\min} \leq x_{ss} \leq x_{\max}, \end{aligned} \tag{8.5}$$

or, where more appropriate, output constraints may be imposed in place of state constraints.

Advantages:

- The MPC is always asked to track a target that exists within the admissible set.
- If the reference r_k is unreachable, the SSO automatically returns the closest feasible operating point.

8.5 The MPC Formulation

Having established the mechanisms for estimating the augmented state and calculating the steady-state targets, we now assemble the complete control architecture. The topology is illustrated in Figure 8.1, with detailed mathematical definitions shown in Figure 8.2.

8.5.1 Control Topology

The architecture operates as a hierarchical loop, separating the task of *setpoint determination* from the task of *regulation*. The information flow at time step k is as follows:

1. **Observer Block:** The cycle begins with the measurement y_k. The *augmented observer* fuses this measurement with the previously applied input u_{k-1} to produce an estimate of the physical state $\hat{x}_k$ and the raw disturbance estimate $\hat{d}_{\text{raw},k}$. To prevent high-frequency sensor noise from propagating into the controller, the disturbance estimate may be smoothed by a *disturbance filter*, yielding $\hat{d}_k$.

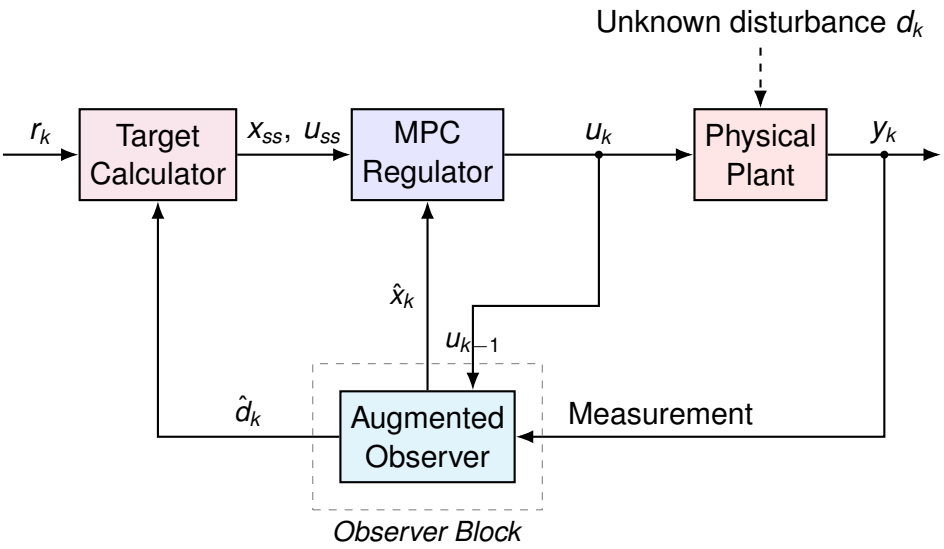


Figure 8.1: Offset-free control architecture. The target calculator uses the disturbance estimate $\hat{d}_k$ to compute the steady-state targets (x_{ss}, u_{ss}). The MPC regulator steers the system towards these targets using the estimated state $\hat{x}_k$. The augmented observer uses the measurement y_k and the previously applied control input u_{k-1}. The disturbance filter is optional and is included here to reduce noise in the disturbance estimate.

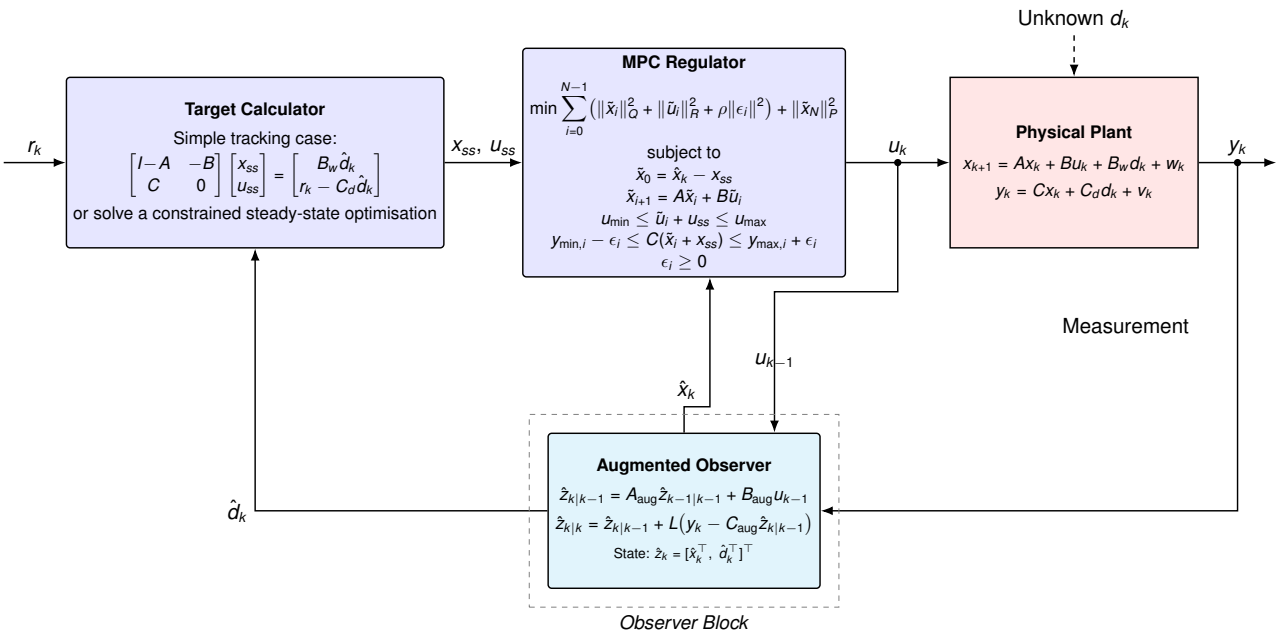


Figure 8.2: Detailed offset-free control architecture. The target calculator may be implemented either as the simple linear equilibrium solve of (8.4) or, more generally, as a constrained steady-state optimisation. The regulator is shown with soft output constraints, matching the examples developed later in the chapter.

2. **Target Calculator:** This block acts as the bridge between the external reference and the controller. It takes the reference r_k and the estimated disturbance $\hat{d}_k$ as inputs. By solving the equilibrium equations, or more generally a constrained steady-state optimisation problem, it computes the target state x_{ss} and target input u_{ss}. These represent an operating point at which the output matches the reference whilst compensating for the estimated disturbance.

3. **MPC Regulator:** Finally, the MPC regulator receives the current state estimate $\hat{x}_k$ together with the calculated targets (x_{ss}, u_{ss}). Its role is to steer the system from $\hat{x}_k$ towards x_{ss} whilst satisfying the imposed constraints.

8.5.2 Optimisation Problem

Once the steady-state targets (x_{ss}, u_{ss}) have been computed, the regulation layer is designed in **deviation variables**:

$$\tilde{x}_i = x_i - x_{ss}, \qquad \tilde{u}_i = u_i - u_{ss}.$$

The full plant prediction model at step i inside the horizon follows from (8.1) with noise terms set to zero (certainty equivalence) and the disturbance replaced by its current estimate $\hat{d}_k$:

$$x_{i+1} = Ax_i + Bu_i + B_w\hat{d}_k.$$

The steady-state pair (x_{ss}, u_{ss}) satisfies the equilibrium condition obtained by setting $x_{k+1} = x_k = x_{ss}$ and $d_k = \hat{d}_k$ in (8.1)–(8.2):

$$x_{ss} = Ax_{ss} + Bu_{ss} + B_w\hat{d}_k, \qquad y_{ss} = Cx_{ss} + C_d\hat{d}_k.$$

Subtracting the equilibrium equation from the prediction model gives

$$x_{i+1} - x_{ss} = A(x_i - x_{ss}) + B(u_i - u_{ss}) + \underbrace{B_w\hat{d}_k - B_w\hat{d}_k}_{=0}.$$

Because the steady-state pair absorbs the disturbance estimate $\hat{d}_k$ through both the state equation and the output equation, all affine terms cancel exactly, leaving the **disturbance-free linear deviation dynamics**:

$$\tilde{x}_{i+1} = A\tilde{x}_i + B\tilde{u}_i.$$

Notation: Real Time vs Prediction Horizon

MPC operates on two distinct time scales:

- k denotes the current *real-time* sampling instant.
- i denotes the *prediction step* within the horizon.

Thus, $\hat{x}_k$ is the current estimated state, whereas $\tilde{x}_i$ is the predicted deviation i steps ahead within the optimisation.

A standard offset-free tracking MPC problem is

$$\begin{aligned}
\min_{\tilde{u}_0,\dots,\tilde{u}_{N-1},\,\epsilon_0,\dots,\epsilon_{N-1}} \quad & \sum_{i=0}^{N-1} \left(\|\tilde{x}_i\|_Q^2 + \|\tilde{u}_i\|_R^2 + \rho\|\epsilon_i\|^2 \right) + \|\tilde{x}_N\|_{P_{\text{term}}}^2 \\
\text{subject to:} \quad & \tilde{x}_0 = \hat{x}_k - x_{ss}, \\
& \tilde{x}_{i+1} = A\tilde{x}_i + B\tilde{u}_i, \qquad i = 0, \dots, N-1, \\
& u_{\min} \le \tilde{u}_i + u_{ss} \le u_{\max}, \\
& y_{\min,i} - \epsilon_i \le C(\tilde{x}_i + x_{ss}) \le y_{\max,i} + \epsilon_i, \\
& \epsilon_i \ge 0, \qquad i = 0, \dots, N-1.
\end{aligned} \tag{8.6}$$

If $C_d \neq 0$, the corresponding term $C_d\hat{d}_k$ should also be included in the predicted output equation. In the examples below, $C_d = 0$, so the simpler form above is sufficient.

Topology Insight

The initialisation constraint

$$\tilde{x}_0 = \hat{x}_k - x_{ss}$$

is the key link between the *observer* and the *target calculator*. The observer supplies $\hat{x}_k$, the target calculator supplies x_{ss}, and the MPC regulator penalises deviations from this target.

Because the cost function penalises $\tilde{x}_i$ and $\tilde{u}_i$ rather than the absolute input u_i, the controller is not penalised for maintaining the non-zero steady-state input u_{ss} required to reject the disturbance.

The control input applied to the plant is

$$u_k = \tilde{u}_0^\star + u_{ss},$$

possibly followed by a final saturation step for numerical safety.

8.5.3 Handling Infeasibility: Hard vs Soft Constraints

In an ideal world, the MPC solver always finds a trajectory satisfying every constraint. In reality, disturbances, model mismatch or estimation errors may push the system into a region from which it is impossible to satisfy all limits simultaneously. If the feasible set becomes empty, the optimisation problem fails.

To improve robustness, it is useful to distinguish between two types of constraints.

Hard Constraints (Inputs)

Input constraints represent immutable physical limits of the actuators, such as the maximum voltage of a motor or the fully open position of a valve. Since the controller cannot command an actuator beyond its physical capability, input constraints should normally be modelled as *hard constraints*.

Soft Constraints (States/Outputs)

State or output constraints often represent safety or performance requirements. If a large disturbance makes one of these temporarily impossible to satisfy, a hard constraint would render the optimisation infeasible exactly when corrective action is most needed. To avoid this, such constraints are often softened by introducing a slack variable.

Mathematical Implementation

We introduce a non-negative *slack variable* $\epsilon_i \geq 0$, which acts as a safety valve for the soft constraints. Ideally, $\epsilon_i = 0$. To encourage this, we add a large penalty weight ρ to the cost function. The modified problem is

$$\begin{aligned}
\min_{\tilde{u},\epsilon} \quad & \sum_{i=0}^{N-1} \left(\|\tilde{x}_i\|_Q^2 + \|\tilde{u}_i\|_R^2 + \rho \|\epsilon_i\|^2 \right) + \|\tilde{x}_N\|_{P_{\text{term}}}^2 \\
\text{subject to:} \quad & \tilde{x}_{i+1} = A\tilde{x}_i + B\tilde{u}_i, \\
& u_{\min} \leq \tilde{u}_i + u_{ss} \leq u_{\max}, \\
& y_{\min,i} - \epsilon_i \leq C(\tilde{x}_i + x_{ss}) \leq y_{\max,i} + \epsilon_i, \\
& \epsilon_i \geq 0.
\end{aligned}$$

Interpretation

Because ρ is chosen to be very large, the optimiser keeps $\epsilon_i = 0$ whenever possible, so that the soft constraint behaves like a hard one. If, however, the strict bound is temporarily impossible to satisfy, the optimiser increases ϵ_i only as much as necessary to maintain feasibility.

Example 8.1: MATLAB: Offset-Free MPC with Disturbance Estimation

Problem Formulation:

We consider a second-order discrete-time linear system subject to measurement noise and an unknown piecewise-constant input disturbance. The objective is to track a piecewise-constant reference r_k whilst respecting hard input constraints and soft output constraints.

System Dynamics:

$$\begin{aligned}
x_{k+1} &= Ax_k + Bu_k + B_w d_k \\
y_k &= Cx_k + v_k
\end{aligned}$$

where the system matrices are

$$A = \begin{bmatrix} 0.9535 & 0.0761 \\ -0.8454 & 0.5478 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0465 \\ 0.8454 \end{bmatrix}, \quad B_w = B, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Constraints:

- Hard input saturation: $-5 \leq u_k \leq 5$,
- Soft output bounds: $-0.5 \leq y_k \leq 0.5$.

Solution Strategy:

A standard MPC controller based on the nominal model would exhibit a steady-state offset in the presence of the unknown disturbance. The offset-free implementation therefore uses three components:

1. **Augmented Kalman Filter:** The state is augmented with a constant disturbance model,
$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \qquad z_{k+1} = \begin{bmatrix} A & B_w \\ 0 & 1 \end{bmatrix} z_k + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k,$$
and a Kalman filter provides estimates $\hat{x}_k$ and $\hat{d}_k$.
2. **Target Calculator:** The estimated disturbance is used to compute the steady-state operating point (x_{ss}, u_{ss}) satisfying
$$(I - A)x_{ss} - Bu_{ss} = B_w \hat{d}_k, \qquad Cx_{ss} = r_k.$$
3. **Deviation-Form MPC:** The regulator then solves an MPC problem in deviation coordinates $\tilde{x}_i = x_i - x_{ss}$, $\tilde{u}_i = u_i - u_{ss}$, with soft output constraints and hard input constraints.

MATLAB Implementation:

```
clear; clc; close all; rng(1);

%% 1. System Definition
A  = [0.9535  0.0761; -0.8454  0.5478];
B  = [0.0465 ; 0.8454];
Bw = B;
C  = [1 0];

nx = 2; nu = 1; ny = 1;

u_min = -5;  u_max =  5;
y_min = -0.5; y_max =  0.5;
```

```
%% 2. Augmented Observer
% Augmented state: z_k = [x_k; d_k],  d_{k+1} = d_k
A_aug = [A, Bw; zeros(1,nx), 1];
B_aug = [B; 0];
C_aug = [C, 0];

% W(3,3) is the single tuning knob for disturbance tracking bandwidth
W = diag([1e-4, 1e-4, 1e-4]);   % Kalman filter tuning weights
V = 1e-3;                        % Measurement noise variance

[P_ss, ~, ~] = dare(A_aug', C_aug', W, V);
L = P_ss * C_aug' / (C_aug * P_ss * C_aug' + V);

%% 3. Target Calculator
% [I-A  -B] [x_ss]   [Bw * d_hat]
% [ C    0] [u_ss] = [   ref    ]
M_target = [eye(nx)-A, -B; C, zeros(ny,nu)];

if rank(M_target) < (nx + nu)
  error('Target calculator matrix is rank deficient.');
end

%% 4. MPC Formulation
N   = 20;
q_y = 10;
R   = 0.1;
rho = 1e5;

Q_state = C' * q_y * C;
[P_N, ~, ~] = dare(A, B, Q_state, R);

du    = sdpvar(nu, N,   'full');
dx    = sdpvar(nx, N+1, 'full');
slack = sdpvar(ny, N,   'full');

x_hat = sdpvar(nx, 1);
x_ss  = sdpvar(nx, 1);
u_ss  = sdpvar(nu, 1);

obj  = 0;
cons = [dx(:,1) == x_hat - x_ss];

for k = 1:N
  cons = [cons, dx(:,k+1) == A*dx(:,k) + B*du(:,k)];

  dy_k = C * dx(:,k);
  obj  = obj + dy_k'*q_y*dy_k + du(:,k)'*R*du(:,k) + rho*slack(:,k)'*slack(:,k);

  u_abs = du(:,k) + u_ss;
  y_abs = C * (dx(:,k) + x_ss);

  cons = [cons, u_min <= u_abs <= u_max];
  cons = [cons, y_min - slack(:,k) <= y_abs <= y_max + slack(:,k)];
  cons = [cons, slack(:,k) >= 0];
```

```
end

obj = obj + dx(:,N+1)' * P_N * dx(:,N+1);

ops = sdpsettings('solver', 'quadprog', 'verbose', 0);
mpc_controller = optimizer(cons, obj, ops, {x_hat, x_ss, u_ss}, {du
  (:,1), slack(:,1)});

%% 5. Simulation Setup
T_sim = 300;
x_true = zeros(nx, 1);
z_hat  = zeros(nx+1, 1);
u_applied = 0;

ref_signal = [repmat( 0.4,1,100), repmat(-0.4,1,100), repmat(
  0.4,1,100)];
w_true     = [repmat( 0.2,1,150), repmat(-0.2,1,150)];

log_y_true = zeros(1,T_sim);  log_y_meas = zeros(1,T_sim);
log_ref    = zeros(1,T_sim);  log_u      = zeros(1,T_sim);
log_u_ss   = zeros(1,T_sim);  log_d_true = zeros(1,T_sim);
log_d_hat  = zeros(1,T_sim);  log_slack  = zeros(1,T_sim);
log_x1     = zeros(1,T_sim);  log_x2     = zeros(1,T_sim);

fprintf('Starting simulation...\n');

%% 6. Simulation Loop
for t = 1:T_sim

  ref = ref_signal(t);

  % A. Plant output
  y_true = C * x_true;
  y_meas = y_true + sqrt(V) * randn;

  % B. Observer
  z_hat_pred = A_aug * z_hat + B_aug * u_applied;
  z_hat      = z_hat_pred + L * (y_meas - C_aug * z_hat_pred);

  x_hat_k = z_hat(1:nx);
  d_hat_k = z_hat(nx+1);

  % C. Target calculator
  targets = M_target \ [Bw * d_hat_k; ref];
  x_ss_k  = targets(1:nx);
  u_ss_k  = targets(nx+1:end);

  % D. MPC
  [sol, diagnostics] = mpc_controller{x_hat_k, x_ss_k, u_ss_k};

  if diagnostics == 0
    du_opt    = full(sol{1});
    slack_now = full(sol{2});
  else
    warning('MPC infeasible at t = %d (code %d). Applying u_ss.', t,
  diagnostics);
    du_opt    = 0;
```

```
      slack_now = NaN;
    end

    u_applied = min(max(du_opt + u_ss_k, u_min), u_max);

    % E. Plant update
    x_true = A * x_true + B * u_applied + Bw * w_true(t);

    % F. Logging
    log_y_true(t) = y_true;  log_y_meas(t) = y_meas;
    log_ref(t)    = ref;     log_u(t)      = u_applied;
    log_u_ss(t)   = u_ss_k;  log_d_true(t) = w_true(t);
    log_d_hat(t)  = d_hat_k; log_slack(t)  = slack_now;
    log_x1(t)     = x_true(1); log_x2(t)   = x_true(2);
  end


  %% Visualisation omitted for brevity.


```

Figure 8.3 shows the closed-loop output, the control signals, the disturbance estimate and the activity of the soft output constraint.

The code follows the algorithm summarised in Algorithm 1.

Note. Algorithm 1 describes the general offset-free MPC structure based on augmented-state disturbance estimation, steady-state target calculation and deviation-form regulation. In the simplest reference-tracking case, the steady-state pair (x_{ss}, u_{ss}) may be obtained directly from the linear equilibrium equations (8.4). In constrained or economic MPC, however, it is generally more appropriate to compute (x_{ss}, u_{ss}) by solving a constrained steady-state optimisation problem. Moreover, if soft output constraints are used, the slack variables must satisfy $\epsilon_i \geq 0$, and the indexing of predicted outputs in the constraints should be chosen consistently with the state-update convention adopted in the MPC formulation.

Example 8.2: MATLAB: Offset-Free MPC for Quadrotor Position Control

1. Problem Formulation: Nonlinear Dynamics

We consider the translational dynamics of a quadrotor. Let $p = [x, y, z]^T$ denote the position and $v = [\dot{x}, \dot{y}, \dot{z}]^T$ the velocity. The control inputs are the roll angle ϕ, pitch angle θ, and total thrust T. Assuming zero yaw $\psi = 0$, the translational equations of motion are

$$\begin{aligned} \ddot{x} &= \frac{T}{m}\cos\phi\sin\theta + w_x \\ \ddot{y} &= -\frac{T}{m}\sin\phi + w_y \\ \ddot{z} &= \frac{T}{m}\cos\phi\cos\theta - g + w_z \end{aligned} \tag{8.7}$$

where m is the mass, g is gravity, and $w = [w_x, w_y, w_z]^T$ represents external wind distur-

Algorithm 1 Offset-Free MPC with Disturbance Estimation

Input: System matrices (A, B, B_w, C), optional output-bias matrix C_d, weights $(Q, R, P_{\text{term}}, \rho)$, input/output constraints, prediction horizon N.
Output: Control input u_k.
procedure OFFLINE INITIALISATION
  Construct the augmented model

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \qquad z_{k+1} = A_{\text{aug}} z_k + B_{\text{aug}} u_k, \qquad y_k = C_{\text{aug}} z_k.$$

  Compute the estimator gain L from the DARE associated with $(A_{\text{aug}}, C_{\text{aug}}, W, V)$.
  Compute the terminal cost matrix P_{term} for the deviation dynamics.
  Define the parametric MPC problem:

$$\begin{aligned} \min_{\tilde{u},\epsilon} \quad & \sum_{i=0}^{N-1} \left(\|\tilde{x}_i\|_Q^2 + \|\tilde{u}_i\|_R^2 + \rho \|\epsilon_i\|^2 \right) + \|\tilde{x}_N\|_{P_{\text{term}}}^2 \\ \text{s.t.} \quad & \tilde{x}_0 = \hat{x}_k - x_{ss}, \\ & \tilde{x}_{i+1} = A\tilde{x}_i + B\tilde{u}_i, \qquad i = 0, \dots, N-1, \\ & u_{\min} \le \tilde{u}_i + u_{ss} \le u_{\max}, \\ & y_{\min,i} - \epsilon_i \le C(\tilde{x}_i + x_{ss}) \le y_{\max,i} + \epsilon_i, \\ & \epsilon_i \ge 0, \qquad i = 0, \dots, N-1. \end{aligned}$$

  Compile the solver object, including any real-time parameters required by the chosen formulation.
end procedure
procedure ONLINE CONTROL LOOP
  Initialise $\hat{z}_{0|0}$, u_{-1}, and optionally $\hat{d}_0$.
  for $k = 0, 1, 2, \dots$ **do**
    Measure the output y_k.
    // 1. Augmented-state estimation
    $\hat{z}_{k|k-1} \leftarrow A_{\text{aug}} \hat{z}_{k-1|k-1} + B_{\text{aug}} u_{k-1}$
    $\hat{z}_{k|k} \leftarrow \hat{z}_{k|k-1} + L(y_k - C_{\text{aug}} \hat{z}_{k|k-1})$
    Extract $\hat{x}_k$ and the raw disturbance estimate $\hat{d}_{,k}$ from $\hat{z}_{k|k}$.
    // 2. Steady-state target calculation
    Compute (x_{ss}, u_{ss}) from the current reference and disturbance estimate, either

$$x_{ss} = Ax_{ss} + Bu_{ss} + B_w \hat{d}_k, \qquad r_k = Cx_{ss} + C_d \hat{d}_k,$$

    or, more generally, by solving a constrained steady-state optimisation problem.
    // 3. MPC regulation
    Solve the MPC problem using the current parameters $(\hat{x}_k, x_{ss}, u_{ss})$ and any future bound sequences if required.
    Obtain the first optimal move $\tilde{u}_0^\star$.
    // 4. Apply input
    $u_k \leftarrow \tilde{u}_0^\star + u_{ss}$
    $u_k \leftarrow \max(u_{\min}, \min(u_{\max}, u_k))$
    Apply u_k to the plant.
  end for
end procedure

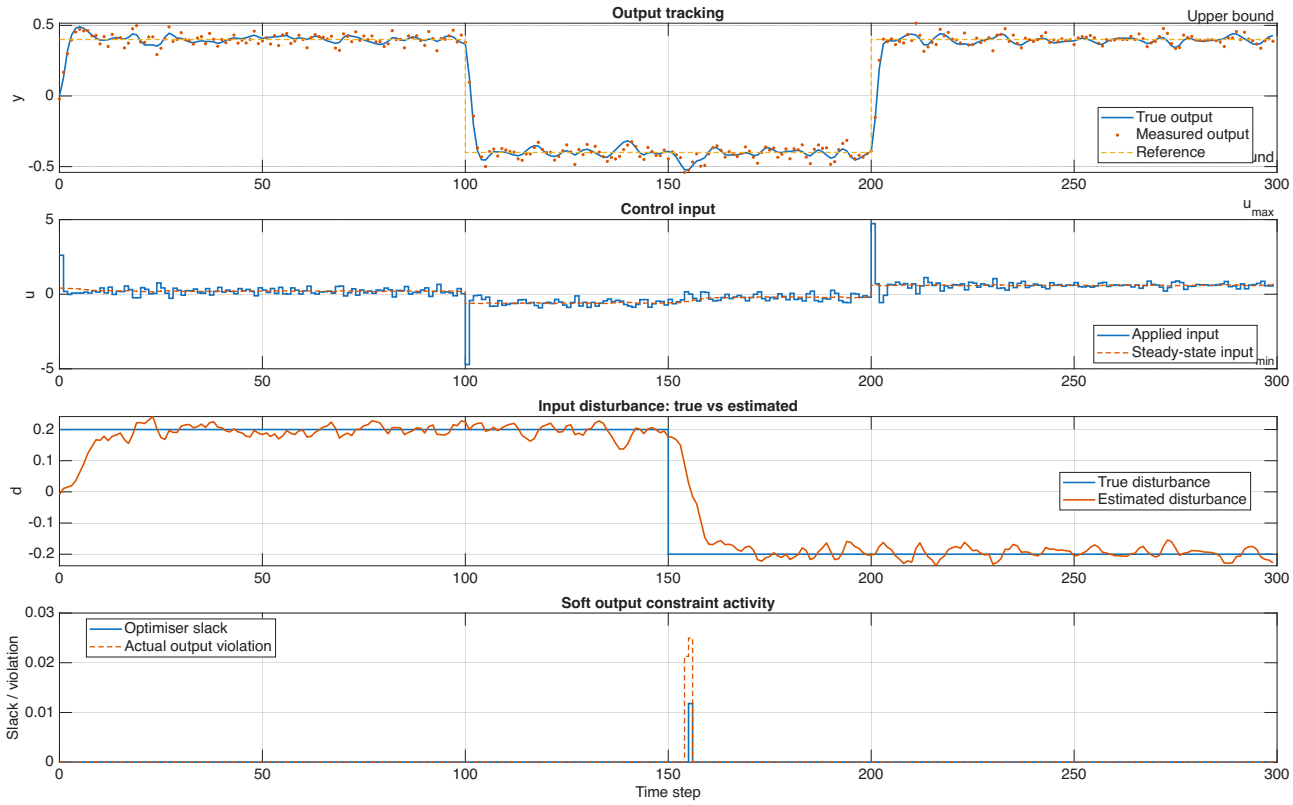


Figure 8.3: Closed-loop performance of the offset-free MPC with soft output constraints under piecewise-constant reference changes and an unknown step disturbance. The top panel shows accurate output tracking of the constrained reference whilst keeping the output close to the prescribed bounds. The second panel shows the applied control input together with the steady-state target input, with brief transient peaks at the reference-switching instants. The third panel shows that the augmented observer estimates the constant input disturbance well, including the disturbance sign change. The bottom panel shows that the soft output constraint is active only briefly during the disturbance transition, indicating that constraint violations are small and short-lived.

bances. These dynamics are nonlinear and coupled. Tilting to generate lateral motion, for example, reduces the vertical lift component and therefore changes the thrust required to maintain altitude.

2. Linearisation and Virtual Inputs

To design a linear MPC, we linearise the dynamics about the hover equilibrium. In hover, the attitude angles are small and the thrust balances gravity. Using the small-angle approximations $\sin\theta \approx \theta$, $\sin\phi \approx \phi$, $\cos\phi \approx \cos\theta \approx 1$, (8.7) becomes

$$\ddot{x} \approx g\theta, \qquad \ddot{y} \approx -g\phi, \qquad \ddot{z} \approx \frac{T}{m} - g.$$

We now define the **virtual control inputs**

$$u_{\text{lin}} = [a_x,\ a_y,\ a_z]^T,$$

which represent commanded translational accelerations. The corresponding mapping to

physical inputs is

$$\begin{aligned} a_x &\triangleq g\theta && \Rightarrow \theta_{\text{cmd}} = \frac{a_x}{g}, \\ a_y &\triangleq -g\phi && \Rightarrow \phi_{\text{cmd}} = -\frac{a_y}{g}, \\ a_z &\triangleq \frac{T}{m} - g && \Rightarrow T_{\text{cmd}} = m(a_z + g). \end{aligned} \tag{8.8}$$

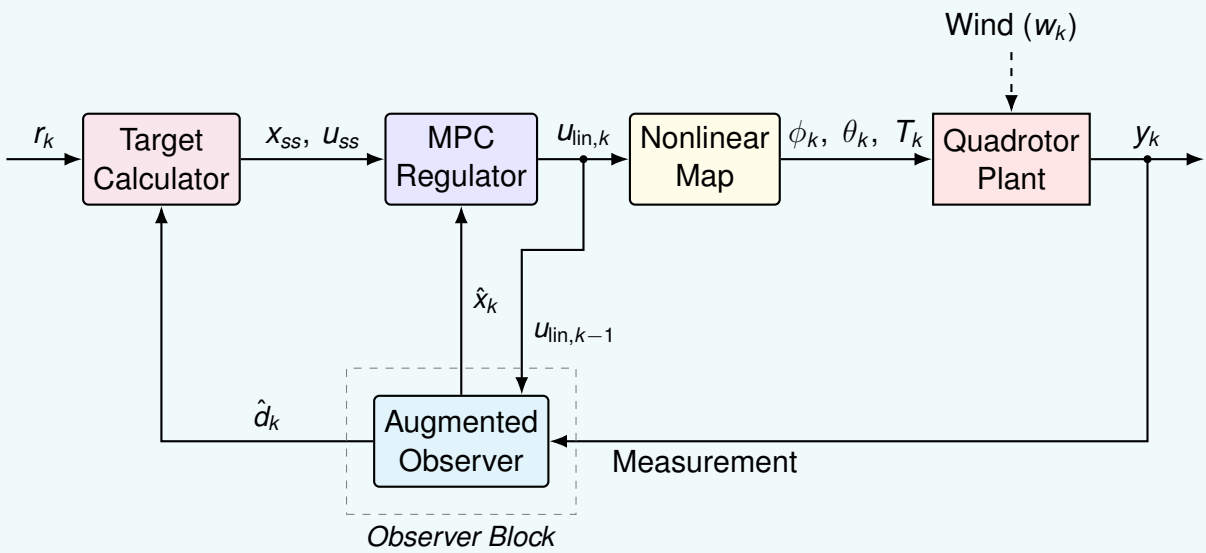


Figure 8.4: Control architecture for the quadrotor example. The target calculator uses the filtered wind estimate $\hat{d}_k$ to compute steady-state targets (x_{ss}, u_{ss}) in the linearised model. The MPC regulator computes the virtual input $u_{\text{lin},k}$, which is then mapped to the physical commands (ϕ_k, θ_k, T_k). The observer uses the measured output and the previously applied virtual input.

Substituting these virtual inputs into the linearised model decouples the motion into three double integrators, so that the discrete-time predictor used by the MPC has the form

$$x_{k+1} = Ax_k + Bu_{\text{lin},k}.$$

3. Offset-Free Control Strategy

The control loop proceeds in three stages:

1. **Estimation:** An augmented Kalman filter estimates the state and a constant disturbance vector representing wind acceleration. The raw disturbance estimate is then low-pass filtered to reduce noise.

2. **Target Calculation:** Given the filtered disturbance estimate $\hat{d}_k$, the steady-state pair (x_{ss}, u_{ss}) is computed from the equilibrium equations

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_{ss} \\ u_{ss} \end{bmatrix} = \begin{bmatrix} B_w \hat{d}_k \\ r_{\text{ref}} \end{bmatrix}.$$

   Here u_{ss} is the steady virtual acceleration required to counteract the estimated wind.

3. **Regulation:** The MPC operates in deviation coordinates and computes the optimal corrective input sequence that steers the current state estimate towards the target.

Finally, the virtual control input is mapped back to the physical commands (ϕ, θ, T) using (8.8).

4. MATLAB Implementation

The code below simulates this strategy. The controller uses a linear prediction model, but the plant is simulated with the full nonlinear dynamics using RK4 integration. A step wind disturbance is applied at $t = 50$ s. Figure 8.5 shows the simulation result for tracking and disturbance observation.

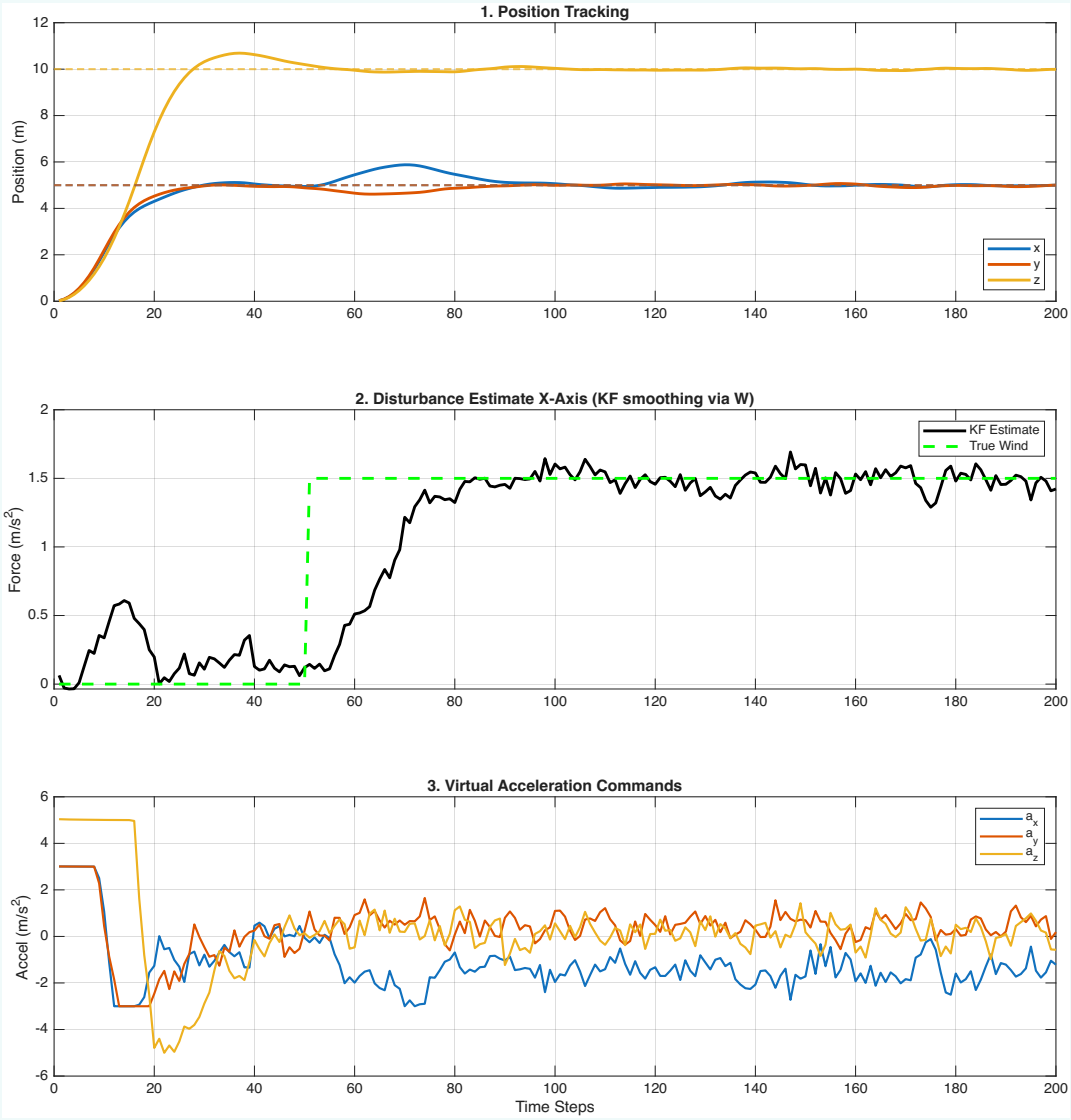


Figure 8.5: **Simulation results: offset-free MPC on a nonlinear quadrotor. Top:** Position tracking. The quadrotor tracks the reference setpoint $[5, 5, 10]$ m. At $t = 50$ s, a step wind disturbance is applied in the horizontal plane; the trajectory deviates transiently and then returns to the reference with negligible steady-state error. **Middle:** Disturbance estimation using Kalman Filter. The estimate converges towards the true wind disturbance. **Bottom:** Virtual control inputs. After the wind disturbance is applied, the controller settles to non-zero steady virtual accelerations that counteract the wind and maintain the reference position.

```
clear; clc; close all;

%% 1. System & Linear Model
g = 9.81; m = 1.0; dt = 0.1;
Ac = [zeros(3,3) eye(3); zeros(3,3) zeros(3,3)];
Bc = [zeros(3,3); eye(3)];
Cc = [eye(3), zeros(3,3)];
sys_d = c2d(ss(Ac, Bc, Cc, 0), dt, 'zoh');
A = sys_d.A; B = sys_d.B; C = sys_d.C;
nx = 6; nu = 3; ny = 3;

%% 2. Augmented Observer
Bw = B;
A_aug = [A, Bw; zeros(3,nx), eye(3)];
B_aug = [B; zeros(3,nu)];
C_aug = [C, zeros(3,3)];

% --- KEY TUNING ---
% W(1:6,1:6)  : process noise on physical states  (keep small)
% W(7:9,7:9)  : process noise on disturbance states
%   LOWER  -> Kalman gain for d is small -> slow update -> implicit
    LPF
%   HIGHER -> fast disturbance tracking  -> noisier estimate
W = diag([1e-3*ones(1,6), 5e-3*ones(1,3)]);
V = 1e-2 * eye(3);

[~,~,L] = dare(A_aug', C_aug', W, V);
L = L';

%% 3. Target Calculator
M_target = [eye(nx)-A, -B; C, zeros(ny,nu)];
P_target = pinv(M_target);

%% 4. MPC Formulation (Deviation Form)
N = 50;
Q = diag([20 20 40, 1 1 1]);
R = diag([2 2 2]);
rho = 1e4;
[P_term,~,~] = dare(A, B, Q, R);

du    = sdpvar(nu, N, 'full');
dx    = sdpvar(nx, N+1, 'full');
slack = sdpvar(nu, N, 'full');
x_hat = sdpvar(nx, 1);
x_ss  = sdpvar(nx, 1);
u_ss  = sdpvar(nu, 1);

```

```
obj  = 0;
cons = [dx(:,1) == x_hat - x_ss];
for k = 1:N
  cons = [cons, dx(:,k+1) == A*dx(:,k) + B*du(:,k)];
  obj  = obj + dx(:,k)'*Q*dx(:,k) + du(:,k)'*R*du(:,k) + rho*norm(
   slack(:,k),2)^2;
  u_abs = du(:,k) + u_ss;
  cons = [cons, -3 - slack(1:2,k) <= u_abs(1:2) <= 3 + slack(1:2,k)];
  cons = [cons, -5 - slack(3,k)   <= u_abs(3)   <= 5 + slack(3,k)];
  cons = [cons, slack(:,k) >= 0];
end
obj = obj + dx(:,N+1)'*P_term*dx(:,N+1);
ops = sdpsettings('solver','quadprog','verbose',0);
controller = optimizer(cons, obj, ops, {x_hat, x_ss, u_ss}, du(:,1));

%% 5. Simulation Loop
T_sim      = 200;
x_true     = zeros(6,1);
u_applied  = zeros(3,1);
z_est      = zeros(9,1);
wind_force = [0;0;0];
ref        = [5; 5; 10];

log_x      = zeros(3, T_sim);
log_u      = zeros(3, T_sim);
log_d      = zeros(3, T_sim);
log_d_true = zeros(3, T_sim);

for t = 1:T_sim
  if t > 50,
    wind_force = [1.5; -0.5; 0];
  end

  % A. Measure & Estimate
  y_meas = C*x_true + sqrt(0.01)*randn(3,1);
  z_pred = A_aug*z_est + B_aug*u_applied;
  y_pred = C_aug*z_pred;
  z_est  = z_pred + L*(y_meas - y_pred);

  x_hat_val = z_est(1:6);
  d_hat     = z_est(7:9);

  % B. Calculate Targets
  rhs      = [Bw * d_hat; ref];
  ss_sol   = P_target * rhs;
  x_target = ss_sol(1:nx);
  u_target = ss_sol(nx+1:end);

  % C. MPC Solve
  [du_opt, err] = controller{x_hat_val, x_target, u_target};
  if err,
    du_opt = zeros(3,1);
  end
  u_mpc = du_opt + u_target;

  % D. Nonlinear Mapping & Saturation
  accel_z_total = u_mpc(3) + g;
```

```
  Thrust     = m * accel_z_total;
  theta_cmd = max(-0.5, min(0.5,  u_mpc(1) / g));
  phi_cmd    = max(-0.5, min(0.5, -u_mpc(2) / g));
  Thrust     = max(0,    min(20,   Thrust));
  u_phys     = [phi_cmd; theta_cmd; Thrust];

  % E. Plant Update (Nonlinear)
  x_true     = rk4_step(@quadrotor_dyn, x_true, u_phys, wind_force, dt,
    m, g);
  u_applied = u_mpc;

  log_x(:,t)       = x_true(1:3);
  log_u(:,t)       = u_mpc;
  log_d(:,t)       = d_hat;
  log_d_true(:,t) = wind_force;
end

%% Plotting
t_vec = 1:T_sim;
figure('Color','w','Position',[100 100 800 800]);

subplot(3,1,1);
h = plot(t_vec, log_x, 'LineWidth', 2); hold on;
clr = get(gca,'ColorOrder');
yline(ref(1),'--','Color',clr(1,:),'LineWidth',1.5);
yline(ref(2),'--','Color',clr(2,:),'LineWidth',1.5);
yline(ref(3),'--','Color',clr(3,:),'LineWidth',1.5);
grid on; ylabel('Position (m)');
title('1. Position Tracking');
legend(h,{'x','y','z'},'Location','SouthEast');

subplot(3,1,2);
plot(t_vec, log_d(1,:),       'k',  'LineWidth', 2); hold on;
plot(t_vec, log_d_true(1,:), 'g--','LineWidth', 2);
grid on; ylabel('Force (m/s^2)');
title('2. Disturbance Estimate X-Axis (KF smoothing via W)');
legend('KF Estimate','True Wind');

subplot(3,1,3);
plot(t_vec, log_u, 'LineWidth', 1.5);
legend('a_x','a_y','a_z');
grid on; title('3. Virtual Acceleration Commands');
ylabel('Accel (m/s^2)'); xlabel('Time Steps');

%% Helper Functions
function x_next = rk4_step(fun, x, u, w, dt, m, g)
  k1 = fun(x,            u, w, m, g);
  k2 = fun(x+0.5*dt*k1, u, w, m, g);
  k3 = fun(x+0.5*dt*k2, u, w, m, g);
  k4 = fun(x+dt*k3,      u, w, m, g);
  x_next = x + (dt/6)*(k1 + 2*k2 + 2*k3 + k4);
end

function dxdt = quadrotor_dyn(x, u, w, m, g)
  phi = u(1); theta = u(2); T = u(3);
  Fx  = T*(cos(phi)*sin(theta));
  Fy  = T*(-sin(phi));
```

```
  Fz   = T*(cos(phi)*cos(theta));
  ax   = (Fx + w(1))/m;
  ay   = (Fy + w(2))/m;
  az   = (Fz + w(3))/m - g;
  dxdt = [x(4:6); ax; ay; az];
end


```

Example 8.3: Single-zone building heating with economic offset-free MPC

We consider a heated office room ($\sim$50 m^2) whose thermal dynamics are described by a three-state physically derived RC model discretised at $T_s = 1800$ s (30 min) via the matrix exponential. The states are

$$x_k = \begin{bmatrix} T_{\text{air},k} \\ T_{w,\text{in},k} \\ T_{w,\text{out},k} \end{bmatrix},$$

representing indoor air temperature, inner wall layer temperature, and outer wall layer temperature (all in °C). The discrete-time model is

$$x_{k+1} = Ax_k + Bu_k + E\,T_{\text{amb},k} + B_{\text{sol}}\,\varphi_{\text{sol},k} + B_w\,d_k, \qquad y_k = Cx_k,$$

where $u_k \in [0, 2000]$ W is the heater power, $T_{\text{amb},k}$ (°C) is the *measured* outdoor temperature, $\varphi_{\text{sol},k}$ (W/m^2) is the *measured* solar irradiance (both treated as known feedforwards), and d_k (°C/step) is an *unmeasured* scaled disturbance representing occupancy and internal heat gains. The output $y_k = 0.56\,T_{\text{air},k} + 0.44\,T_{w,\text{in},k}$ is the operative temperature (ISO 7726), measured with noise standard deviation $\sigma_v = 0.1$ °C.

The pre-computed discrete model matrices are

$$A = \begin{bmatrix} 0.8753 & 0.1102 & 0.0004 \\ 0.0275 & 0.9649 & 0.0073 \\ 0.0001 & 0.0049 & 0.9851 \end{bmatrix},$$

$$B = \begin{bmatrix} 7.91 \times 10^{-4} \\ 1.00 \times 10^{-4} \\ 0 \end{bmatrix}, \quad E = \begin{bmatrix} 0.01404 \\ 0.00025 \\ 0.00993 \end{bmatrix}, \quad B_{\text{sol}} = \begin{bmatrix} 9.04 \times 10^{-6} \\ 1.43 \times 10^{-6} \\ 3.47 \times 10^{-4} \end{bmatrix},$$

$$B_w = \begin{bmatrix} 1.7647 \\ 0.0267 \\ 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0.56 & 0.44 & 0 \end{bmatrix}.$$

Exogenous profiles (48-hour winter scenario):

- **Outdoor temperature** ($T_{\text{amb},k}$):

$$T_{\text{amb},k} = 5 + 6\sin\left(\frac{2\pi(t_k - 14)}{24}\right) + 0.5\,\eta_k,$$

  ranging nominally between -1 °C and 11 °C; treated as a known feedforward.

- **Solar irradiance** ($\varphi_{\text{sol},k}$):

$$\varphi_{\text{sol},k} = \max\Big(0,\ 600\sin\Big(\tfrac{\pi(\text{mod}(t_k,24)-8)}{9}\Big)\Big)\,,$$

  treated as a known feedforward.

- **Occupancy disturbance** (d_k): an effective scaled thermal disturbance,

$$d_k = \begin{cases} 0.191\,^\circ\text{C/step} & 08{:}00 \le \text{time}(t_k) < 18{:}00, \\ 0 & \text{otherwise}, \end{cases}$$

  corresponding to 300 W of internal gains; *unknown* to the controller and estimated by the Kalman filter.

- **Electricity price** (p_k):

$$p_k = \begin{cases} 0.28\ /\text{kWh} & 07{:}00 \le \text{time}(t_k) < 22{:}00, \\ 0.05\ /\text{kWh} & \text{otherwise}. \end{cases}$$

Time-varying comfort bounds:

$$[y_{\min,k},\ y_{\max,k}] = \begin{cases} [20,\ 23]\,^\circ\text{C} & \text{daytime (07:00–22:00)}, \\ [15,\ 26]\,^\circ\text{C} & \text{night-time}. \end{cases}$$

Control architecture (three layers):

1. **Augmented Kalman observer.** The disturbance is appended to the state as a random walk, $z_k = [x_k^\top,\ d_k]^\top$, giving the augmented model

$$z_{k+1} = \begin{bmatrix} A & B_w \\ 0 & 1 \end{bmatrix} z_k + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k + \begin{bmatrix} E \\ 0 \end{bmatrix} T_{\text{amb},k} + \begin{bmatrix} B_{\text{sol}} \\ 0 \end{bmatrix} \varphi_{\text{sol},k},$$

$$y_k = [C,\ 0]\, z_k + v_k.$$

   The process noise covariance is selected as $W = \text{diag}(10^{-3}, 10^{-3}, 10^{-3}, (0.01)^2)$ is directly interpretable: $\sqrt{W_d} = 0.01\,^\circ$C/step represents the expected occupancy change between sampling instants. The steady-state Kalman gain L is obtained from the DARE with measurement noise variance $V = 0.01$.

2. **Economic steady-state target calculator.** At each step, the following QP selects an economically optimal steady-state pair (x_{ss}, u_{ss}):

$$\begin{aligned} \min_{x_{ss},u_{ss},\epsilon_{ss}} \quad & p_k\, u_{ss} + 10^4\, \epsilon_{ss}^2 \\ \text{s.t.} \quad & x_{ss} = Ax_{ss} + Bu_{ss} + ET_{\text{amb},k} + B_{\text{sol}}\varphi_{\text{sol},k} + B_w \hat{d}_k, \\ & y_{\min,k} - \epsilon_{ss} \le Cx_{ss} \le y_{\max,k} + \epsilon_{ss}, \quad 0 \le u_{ss} \le 2000, \quad \epsilon_{ss} \ge 0. \end{aligned}$$

3. **Economic MPC regulator with price preview.** Defining $\tilde{x}_i = x_i - x_{ss}$, $\tilde{u}_i = u_i - u_{ss}$, the regulator solves over a horizon $N = 24$ (12 hours):

$$\begin{aligned}
\min_{\tilde{u},\epsilon} \quad & \sum_{i=0}^{N-1}\Big(\|\tilde{x}_{i+1}\|_Q^2 + \beta\, p_{k+i}\,(\tilde{u}_i + u_{ss})\tfrac{T_s}{3600} + \rho\,\epsilon_i^2\Big) + \|\tilde{x}_N\|_P^2 \\
\text{s.t.} \quad & \tilde{x}_0 = \hat{x}_k - x_{ss}, \\
& \tilde{x}_{i+1} = A\tilde{x}_i + B\tilde{u}_i, \\
& 0 \le \tilde{u}_i + u_{ss} \le 2000, \\
& y_{\min,k+i} - \epsilon_i \le C(\tilde{x}_{i+1} + x_{ss}) \le y_{\max,k+i} + \epsilon_i, \quad \epsilon_i \ge 0,
\end{aligned}$$

with $Q = \mathrm{diag}(5, 2, 1)$, $\beta = 6$, $\rho = 10^7$, and P from the DARE. The term $\beta p_{k+i}(\tilde{u}_i + u_{ss})(T_s/3600)$ is the actual energy expenditure in pounds at future step $k + i$. The penalty hierarchy $\rho \gg \beta\, p_{\max}\, u_{\max}\,(T_s/3600)$, i.e. $10^7 \gg 2800$, ensures comfort violations are never preferred over heating.

MATLAB implementation: The plotting section is omitted below.

```
clear; clc; close all; rng(1);

%% 1. Pre-computed Discrete Model (3R2C, ZOH, Ts = 1800 s)
%     x = [T_air; T_w_in; T_w_out] (degC)
%     u = heater power (W),  u in [0, 2000]

Ts = 1800;
A  = [  0.8753, 0.1102, 0.0004;
        0.0275, 0.9649, 0.0073;
        0.0001, 0.0049, 0.9851];
B      = [7.91e-4; 1.00e-4; 0      ];
E      = [0.01404; 0.00025; 0.00993];
Bsol   = [9.04e-6; 1.43e-6; 3.47e-4];
Bw     = [1.7647;  0.0267;  0      ];
C      = [0.56, 0.44, 0];

nx = 3;  nu = 1;  ny = 1; u_min = 0;  u_max = 2000;

%% 2. Augmented Kalman Observer
%     z = [x; d],  random-walk disturbance model

A_aug = [A, Bw; zeros(1,nx), 1];
B_aug = [B; 0];  E_aug = [E; 0];  Bsol_aug = [Bsol; 0];
C_aug = [C, 0];

W_kf = diag([1e-3, 1e-3, 1e-3, (0.01)^2]);
V_kf = 0.1^2;

[P_kf,~,~] = dare(A_aug', C_aug', W_kf, V_kf);
L_kf = P_kf*C_aug'/(C_aug*P_kf*C_aug' + V_kf);

%% 3. Economic Steady-State Target Calculator
price_in   = sdpvar(1);  d_in      = sdpvar(1);
T_amb_in   = sdpvar(1);  phi_in    = sdpvar(1);
y_min_in   = sdpvar(1);  y_max_in  = sdpvar(1);
xs = sdpvar(nx,1);  us = sdpvar(nu,1);  ss_slack = sdpvar(1);
```

```
ss_cons = [xs == A*xs + B*us + E*T_amb_in + Bsol*phi_in + Bw*d_in, ...
y_min_in - ss_slack <= C*xs <= y_max_in + ss_slack, ...
u_min <= us <= u_max, ss_slack >= 0];
ss_obj  = price_in*us + 1e4*ss_slack^2;

target_calc = optimizer(ss_cons, ss_obj, ...
sdpsettings('solver','quadprog','verbose',0), ...
{price_in, d_in, T_amb_in, phi_in, y_min_in, y_max_in}, [xs; us]);

%% 4. Economic MPC Regulator with Price Preview (N = 24, horizon = 12 h)
%    Stage cost: ||x_tilde||_Q^2 + beta*price*u*(Ts/3600) + rho*slack^2
%    Penalty hierarchy: rho >> beta*p_max*u_max*(Ts/3600)
%    i.e.  1e7 >> 10*0.28*2000*0.5 = 2800
N = 24;  Q = diag([5,2,1]);  rho = 1e7;  beta = 6;
[P_term,~,~] = dare(A, B, Q, 1e-3);

du        = sdpvar(nu,N,'full');   dx    = sdpvar(nx,N+1,'full');
mpc_slack = sdpvar(1,N,'full');
xhat_in   = sdpvar(nx,1);  xss_in = sdpvar(nx,1);  uss_in = sdpvar(nu,1);
Ymin_seq  = sdpvar(1,N,'full');    Ymax_seq  = sdpvar(1,N,'full');
Price_seq = sdpvar(1,N,'full');

mpc_obj  = 0;
mpc_cons = [dx(:,1) == xhat_in - xss_in];
for i = 1:N
  u_abs    = du(:,i) + uss_in;
  x_pred_i = dx(:,i+1) + xss_in;
  mpc_cons = [mpc_cons, ...
  dx(:,i+1) == A*dx(:,i) + B*du(:,i), ...
  u_min <= u_abs <= u_max, ...
  Ymin_seq(i)-mpc_slack(i) <= C*x_pred_i <= Ymax_seq(i)+mpc_slack(i), ...
  mpc_slack(i) >= 0];
  mpc_obj = mpc_obj + dx(:,i)'*Q*dx(:,i) ...
  + beta*Price_seq(i)*u_abs*(Ts/3600) ...
  + rho*mpc_slack(i)^2;
end
mpc_obj = mpc_obj + dx(:,N+1)'*P_term*dx(:,N+1);

mpc_reg = optimizer(mpc_cons, mpc_obj, ...
sdpsettings('solver','quadprog','verbose',0), ...
{xhat_in,xss_in,uss_in,Ymin_seq,Ymax_seq,Price_seq}, du(:,1));

%% 5. Exogenous Profiles (48-hour simulation)
N_sim   = 48*(3600/Ts);
t_hours = (0:N_sim+N)*(Ts/3600);

T_amb_profile = 5 + 6*sin(2*pi*(t_hours-14)/24) + 0.5*randn(size(t_hours));
Solar_profile = max(0, 600*sin(pi*(mod(t_hours,24)-8)/9));
Elec_Price    = 0.05*ones(size(t_hours));
day_mask      = mod(t_hours,24) >= 7 & mod(t_hours,24) < 22;
```

```
Elec_Price(day_mask) = 0.28;
Tmin_profile  = 15*ones(size(t_hours));  Tmin_profile(day_mask) = 20;
Tmax_profile  = 26*ones(size(t_hours));  Tmax_profile(day_mask) = 23;
% True occupancy: 300 W = 0.191 degC/step (unknown to controller)
d_occ_profile = 0.191*double(mod(t_hours,24) >= 8 & mod(t_hours,24) < 18);

%% 6. Closed-Loop Simulation
x_true = [19; 17; 10];  z_est = [19; 17; 10; 0];  u_prev = 0;

log_Top  = zeros(1,N_sim);  log_Tmeas = zeros(1,N_sim);
log_u    = zeros(1,N_sim);  log_d_est = zeros(1,N_sim);
log_dtrue= zeros(1,N_sim);  log_price = zeros(1,N_sim);
log_Tmin = zeros(1,N_sim);  log_Tmax  = zeros(1,N_sim);
log_cost = zeros(1,N_sim);

for k = 1:N_sim
  T_amb_k   = T_amb_profile(k);
  phi_sol_k = Solar_profile(k);
  y_meas    = C*x_true + sqrt(V_kf)*randn;

  % KF predict & correct (current-step feedforwards)
  if k == 1,
    z_pred = z_est;
  else
    z_pred = A_aug*z_est + B_aug*u_prev ...
  + E_aug*T_amb_k + Bsol_aug*phi_sol_k;
  end
  z_est = z_pred + L_kf*(y_meas - C_aug*z_pred);
  x_hat = z_est(1:nx);  d_hat = z_est(end);

  % Economic target
  y_min = Tmin_profile(k);  y_max = Tmax_profile(k);
  [ss_sol, err_ss] = target_calc{Elec_Price(k),d_hat,T_amb_k,phi_sol_k,y_min,y_max};
  if err_ss ~= 0,
    x_ss = x_hat;   u_ss = u_prev;
  else,
    ss_sol=full(ss_sol); x_ss=ss_sol(1:nx); u_ss=ss_sol(nx+1);
  end
  % Economic MPC with N-step price preview
  [du_opt, err_mpc] = mpc_reg{x_hat, x_ss, u_ss, ...
      Tmin_profile(k+1:k+N), Tmax_profile(k+1:k+N), Elec_Price(k+1:k+N)};
  if err_mpc ~= 0,
    du_opt = 0;
  else,
    du_opt = full(du_opt);
  end
  u_prev = min(max(u_ss + du_opt, u_min), u_max);

  % Plant update (true occupancy unknown to controller)
  x_true = A*x_true + B*u_prev + E*T_amb_k ...
  + Bsol*phi_sol_k + Bw*d_occ_profile(k);

```

```
    % Logging
    log_Top(k)   = C*x_true;    log_Tmeas(k) = y_meas;
    log_u(k)     = u_prev;       log_d_est(k) = d_hat;
    log_dtrue(k)= d_occ_profile(k);
    log_Tmin(k)  = y_min;        log_Tmax(k)  = y_max;
    log_price(k)= Elec_Price(k);
    log_cost(k)  = Elec_Price(k)*u_prev*(Ts/3600)/1000;  % GBP per step
  end
  fprintf('Total heating cost: GBP %.3f\n', sum(log_cost));

  % Visualisation is omitted for brevity
```

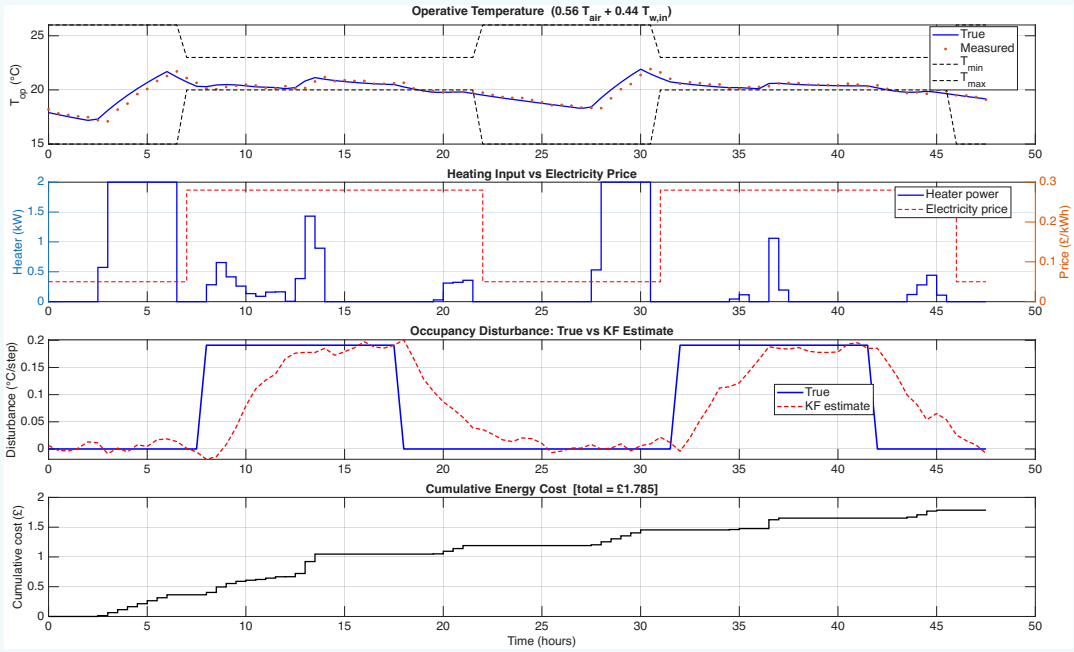


Figure 8.6: Closed-loop performance of the economic offset-free MPC for the single-zone building heating example over a 48-hour simulation. *Top:* Operative temperature tracking the time-varying comfort region; the controller pre-heats during the cheap off-peak window before 07:00 and reduces heating during the expensive daytime tariff. *Second:* Heater power overlaid with the electricity price; the load-shifting behaviour is clearly visible. *Third:* True occupancy disturbance versus the Kalman filter estimate; convergence is fast because $CB_w = 1$. *Bottom:* Cumulative energy cost in GBP over the 48-hour horizon.

Example 8.4: Offset-free MPC of a 3-DOF Articulated Robot Manipulator

Problem Description

Consider a three-degree-of-freedom (3-DOF) *RRR* articulated robot manipulator operating in three-dimensional space. The three revolute joints are arranged as follows: joint 1 rotates about the vertical (z) axis (base yaw), whilst joints 2 and 3 rotate about parallel horizontal (y) axes (shoulder and elbow pitch, respectively). This configuration is representative of the positioning structure found in industrial serial manipulators.

The control objective is to track piecewise-constant joint-angle references $r(t) \in \mathbb{R}^3$ in the presence of persistent disturbances (joint friction and unexpected payload changes) whilst respecting hard torque limits and soft joint-angle limits. An offset-free MPC strategy based on computed-torque feedback linearisation and an augmented Kalman filter (AKF) is employed.

Kinematic Configuration

Figure 8.7 shows the side-view projection of the arm and the definition of the joint angles q_1, q_2, and q_3. The base yaw axis is perpendicular to the plane of the figure.

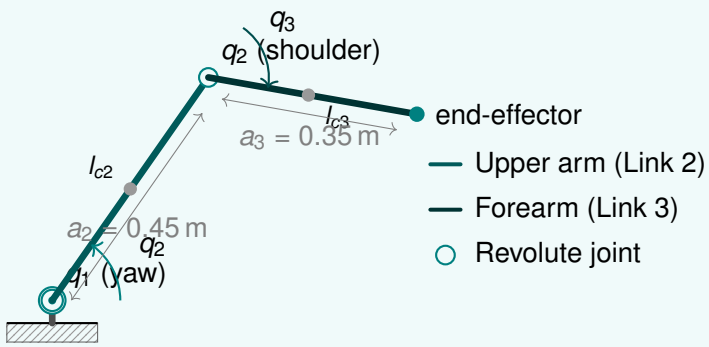


Figure 8.7: Side-view projection of the 3-DOF RRR articulated arm. Joint 1 (base yaw) is perpendicular to the page; joints 2 and 3 (shoulder and elbow pitch) lie in the plane shown.

Physical Parameters

Table 8.1: Parameters used in the simulation.

Symbol	Description	Value	Unit
I_{z1}	Base yaw inertia (joint 1)	1.50	kg m^2
m_2	Mass of upper arm (link 2)	6.00	kg
a_2	Length of upper arm	0.45	m
l_{c2}	COM distance from joint 2	0.22	m
I_2	Pitch inertia of link 2 about COM	0.15	kg m^2
m_3	Mass of forearm (link 3)	3.50	kg
a_3	Length of forearm	0.35	m
l_{c3}	COM distance from joint 3	0.17	m
I_3	Pitch inertia of link 3 about COM	0.08	kg m^2
T_s	Sampling period	0.02	s

Control Technique: Computed-Torque MPC with Disturbance Observer

The full nonlinear robot dynamics are:

$$M(q)\,\ddot{q} + Cor(q, \dot{q})\,\dot{q} + G(q) = \tau + d_\tau, \tag{8.9}$$

where $q \in \mathbb{R}^3$ are the joint angles, $M(q)$ is the symmetric positive-definite inertia matrix, $Cor(q, \dot{q})\dot{q}$ collects Coriolis and centrifugal torques, $G(q)$ is the gravity vector, τ is the applied torque, and $d_\tau \in \mathbb{R}^3$ represents lumped torque-space disturbances (friction, payload) in [Nm].

Computed-torque (CT) feedback linearisation is applied by choosing:

$$\tau = M(q)\, v + Cor(q, \dot{q})\, \dot{q} + G(q), \tag{8.10}$$

which reduces (8.9) to three decoupled double integrators, one per joint:

$$\ddot{q} = v + \underbrace{M(q)^{-1}\, d_\tau}_{d}, \tag{8.11}$$

where $d \triangleq M(q)^{-1}\, d_\tau \in \mathbb{R}^3$ is the *acceleration-space disturbance* [rad/s^2]. Because the MPC and observer operate entirely in the linearised (virtual-acceleration) domain, d is the natural disturbance variable for the remainder of the formulation. If the physical torque-space disturbance is needed, it can be recovered as $\hat{d}_\tau = M(q)\, \hat{d}$.

Defining the state $x = [q^\top,\ \dot{q}^\top]^\top \in \mathbb{R}^6$, the linearised continuous-time plant is:

$$\dot{x} = \underbrace{\begin{bmatrix} 0 & I \\ 0 & 0 \end{bmatrix}}_{A_c} x + \underbrace{\begin{bmatrix} 0 \\ I \end{bmatrix}}_{B_c} v, \qquad y = \underbrace{\begin{bmatrix} I & 0 \end{bmatrix}}_{C} x, \tag{8.12}$$

which is discretised exactly via zero-order hold at $T_s = 0.02\,\mathrm{s}$ to give (A, B, C). Entire control scheme is shown in Figure 8.8.

MPC Formulation

Augmented observer for offset-free control. A random-walk model $d_{k+1} = d_k$ for the acceleration-space disturbance is appended to the plant state, yielding the augmented system $z = [x^\top,\ d^\top]^\top \in \mathbb{R}^9$:

$$\begin{bmatrix} x_{k+1} \\ d_{k+1} \end{bmatrix} = \underbrace{\begin{bmatrix} A & B \\ 0 & I \end{bmatrix}}_{A_{\text{aug}}} \begin{bmatrix} x_k \\ d_k \end{bmatrix} + \underbrace{\begin{bmatrix} B \\ 0 \end{bmatrix}}_{B_{\text{aug}}} v_k, \qquad y_k = \underbrace{\begin{bmatrix} C & 0 \end{bmatrix}}_{C_{\text{aug}}} \begin{bmatrix} x_k \\ d_k \end{bmatrix}. \tag{8.13}$$

Note that d enters the state equation through B (the same input matrix as v), confirming that it represents the disturbance in virtual-acceleration units [rad/s^2], not in torque units [Nm]. The observer gain L is obtained from the discrete algebraic Riccati equation (DARE) with process noise covariance $Q = \mathrm{diag}(10^{-6} I_6,\ 10^{-3} I_3)$ and measurement noise covariance $R = \sigma_{\text{enc}}^2\, I_3$ where $\sigma_{\text{enc}} = 0.05^\circ$ (encoder standard deviation). The controller can only access to the measurements of angular positions, q. Therefore, $C = \begin{bmatrix} I_{3\times3} & 0_{3\times3} \end{bmatrix}$

Target calculator. At steady state, $x_{k+1} = x_k = x_{\text{ss}}$ and $y_{\text{ss}} = r_k$, so the pair $(x_{\text{ss}},\ v_{\text{ss}})$ satisfies:

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_{\text{ss}} \\ v_{\text{ss}} \end{bmatrix} = \begin{bmatrix} B\,\hat{d}_k \\ r_k \end{bmatrix}. \tag{8.14}$$

This ensures that the MPC drives the plant to the reference with zero steady-state offset despite the persistent acceleration-space disturbance estimate $\hat{d}_k$ [rad/s^2].

Deviation-form optimal control problem. Let $\delta x_k = x_k - x_{ss}$ and $\delta v_k = v_k - v_{ss}$. The QP solved at each sampling instant is:

$$\begin{aligned}
\min_{\{\delta v_k\},\{s_k\}} \quad & \sum_{k=0}^{N-1} \left[\delta y_k^\top Q_y\, \delta y_k + \delta v_k^\top R\, \delta v_k + \rho\, s_k^\top s_k\right] + \delta x_N^\top P_N\, \delta x_N \\
\text{subject to} \quad & \delta x_{k+1} = A\,\delta x_k + B\,\delta v_k, \quad \delta x_0 = \hat{x}_k - x_{ss}, \\
& v_{min} \le \delta v_k + v_{ss} \le v_{max}, \\
& y_{min} - s_k \le C(\delta x_k + x_{ss}) \le y_{max} + s_k, \\
& s_k \ge 0, \quad k = 0, \dots, N-1,
\end{aligned} \tag{8.15}$$

where $\delta y_k = C\,\delta x_k$, P_N is the terminal LQR cost matrix (from the DARE for (A, B, Q_{mpc}, R)), and $s_k \in \mathbb{R}^3$ are slack variables that soften the joint-angle constraints.

The MPC tuning parameters are summarised in Table 8.2.

Table 8.2: Robot parameters.

Parameter	Description	Value
N	Prediction horizon	15
Q_y	Output tracking weight	$200\, I_3$
R	Control effort weight	$0.1\, I_3$
ρ	Soft-constraint penalty	10^6
v_{min}	Virtual acceleration lower bound	$-50\,\mathrm{rad/s^2}$
v_{max}	Virtual acceleration upper bound	$+50\,\mathrm{rad/s^2}$
y_{min}	Joint angle lower limits	$[-170°, -90°, -135°]^\top$
y_{max}	Joint angle upper limits	$[+170°, +90°, +135°]^\top$
τ_{max}	Actuator torque limits (absolute)	$[150, 120, 60]$ Nm

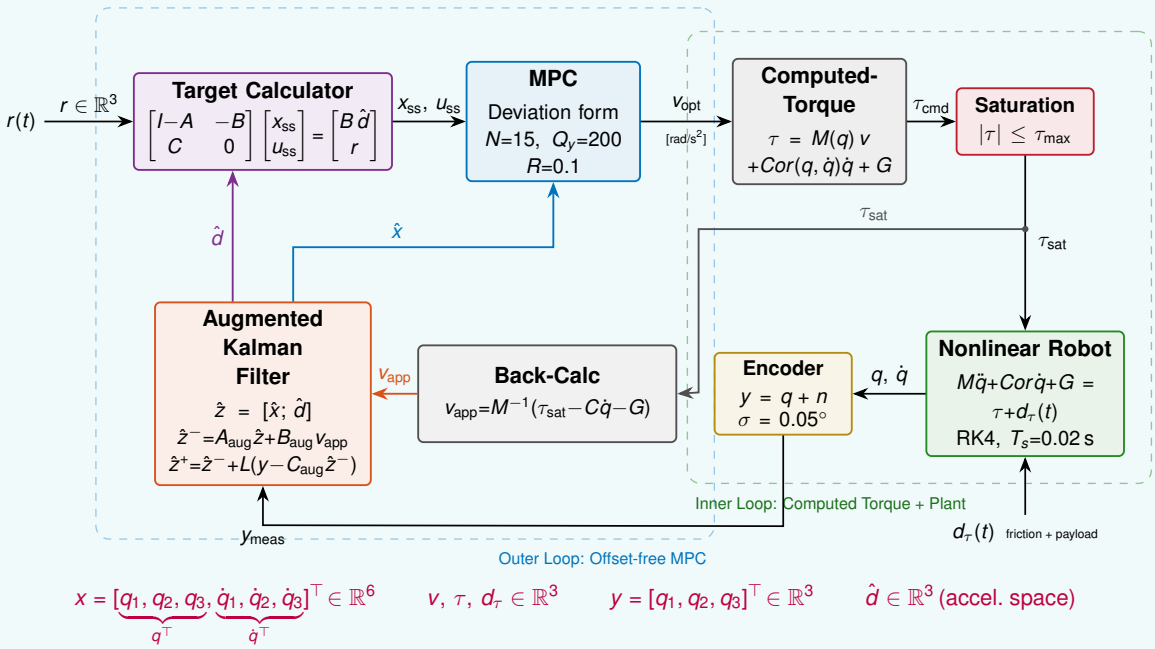


Figure 8.8: Block diagram of the computed-torque based feedback linearisation + MPC scheme. The disturbance d_τ [Nm] acts on the physical plant; after computed-torque cancellation the observer estimates the acceleration-space disturbance $\hat{d} = M^{-1}\hat{d}_\tau$ [rad/s^2].

```
clear; clc; close all; rng(1);

%% ==================================================================
%  Offset-free MPC for a 3-DOF Spatial Articulated Robot Manipulator
% ==================================================================

%% 1. Robot Physical Parameters
% --- Base (Link 1) ----------------------------------------------
Iz1 = 1.50;          % z-axis inertia

% --- Upper arm (Link 2) -----------------------------------------
m2  = 6.00;                 % mass [kg]
a2  = 0.45;                  % length [m]
lc2 = 0.22;                   % COM distance from joint 2
I2  = 0.15;                     % pitch inertia about COM

% --- Forearm (Link 3) -------------------------------------------
m3  = 3.50;       % mass [kg]
a3  = 0.35;        % length [m]  (geometry only)
lc3 = 0.17;         % COM distance from joint 3
I3  = 0.08;         % pitch inertia about COM

g_acc = 9.81;            % gravity
Ts    = 0.02;                % sample time [s]  (50 Hz)

nq = 3;       % degrees of freedom
nx = 2*nq;    % state  [q1 q2 q3  dq1 dq2 dq3]
nu = nq;      % virtual inputs
ny = nq;      % measured outputs (joint angles)
nd = nu;      % one disturbance per joint

% Dynamics function handles
robot_M = @(q) inertia_matrix_3dof (q, m2,m3,a2,lc2,lc3,Iz1,I2,I3);
robot_C = @(q,dq) coriolis_matrix_3dof(q, dq,  m2,m3,a2,lc2,lc3);
robot_G = @(q) gravity_vector_3dof (q, m2,m3,lc2,a2,lc3,g_acc);

%% 2. Linearised Plant
Ac  = [zeros(nq), eye(nq); zeros(nq), zeros(nq)];
Bc  = [zeros(nq); eye(nq)];
Cc  = [eye(nq),   zeros(nq)];   % measure joint angles only

sys_d = c2d(ss(Ac, Bc, Cc, 0), Ts, 'zoh');
A     = sys_d.A;
B     = sys_d.B;
C     = sys_d.C;

% 3. Augmented Observer
%
%  Augmented state:  z = [x; d],   d_{k+1} = d_k
%
```

```
%  A_aug = [A   B;  0   I]   B_aug = [B;  0]   C_aug = [C  0]
%
nz     = nx + nd;
A_aug = [A,             B*eye(nd);
zeros(nd,nx), eye(nd)  ];
B_aug = [B; zeros(nd,nu)];
C_aug = [C, zeros(ny,nd)];

% Kalman filter noise covariances
sigma_enc = deg2rad(0.05);
R_kf      = sigma_enc^2 * eye(ny);
Q_state   = 1e-6 * eye(nx);
Q_dist    = 1e-3 * eye(nd);
Q_kf      = blkdiag(Q_state, Q_dist);

[P_ss, ~, ~] = dare(A_aug', C_aug', Q_kf, R_kf);
L = P_ss * C_aug' / (C_aug * P_ss * C_aug' + R_kf);

fprintf('\nObserver gain L (first 3 rows shown):\n'); disp(L(1:3,:))
  ;

%% 4. Target Calculator
%  Solve  [I-A  -B; C  0]  [x_ss; u_ss] = [B d; ref]
M_target = [eye(nx)-A, -B; C, zeros(ny,nu)];
if rank(M_target) < nx+nu
  error('Target calculator matrix is rank-deficient.');
end

% 5. MPC Formulation  (YALMIP + quadprog, deviation form)
N     = 15;       % prediction horizon
q_y   = 200;      % output tracking weight
R_u   = 0.10;     % control effort weight
rho   = 1e6;      % soft-constraint penalty

Q_mpc = C' * (q_y*eye(ny)) * C;
[P_N, ~, ~] = dare(A, B, Q_mpc, R_u*eye(nu));   % terminal (LQR)
  cost

du    = sdpvar(nu, N,   'full');
dx    = sdpvar(nx, N+1, 'full');
slack = sdpvar(ny, N,   'full');

% Parameters updated each step
x_hat = sdpvar(nx, 1);
x_ss  = sdpvar(nx, 1);
u_ss  = sdpvar(nu, 1);

% ---- Physical bounds ------
% Virtual acceleration bounds [rad/s^2]  (after computed-torque
  cancellation)
v_min = -50*ones(nu,1);    v_max = 50*ones(nu,1);

% Joint angle limits [rad]
y_min = deg2rad([-170; -90; -135]);
y_max = deg2rad([ 170;  90;  135]);

% ---- Optimal Control Problem
```

```
  ----------------------------------------
obj  = 0;
cons = [dx(:,1) == x_hat - x_ss];

for k = 1:N
  cons = [cons, dx(:,k+1) == A*dx(:,k) + B*du(:,k)];

  dy_k = C * dx(:,k);
  obj  = obj + dy_k'*(q_y*eye(ny))*dy_k ...
      + du(:,k)'*(R_u*eye(nu))*du(:,k) ...
      + rho * slack(:,k)' * slack(:,k);

  v_abs = du(:,k) + u_ss;
  y_abs = C * (dx(:,k) + x_ss);

  cons = [cons, v_min <= v_abs <= v_max];
  cons = [cons, y_min - slack(:,k) <= y_abs <= y_max + slack(:,k)];
  cons = [cons, slack(:,k) >= 0];
end
obj = obj + dx(:,N+1)' * P_N * dx(:,N+1);

ops      = sdpsettings('solver','quadprog','verbose',0);
mpc_ctrl = optimizer(cons, obj, ops, {x_hat, x_ss, u_ss}, {du(:,1),
 slack(:,1)});

fprintf('MPC compiled (N=%d, q_y=%.0f, R=%.3f, nq=%d).\n', N, q_y,
 R_u, nq);

%% 6. Simulation Setup
T_sim = 600;
t_vec = (0:T_sim-1) * Ts;

% True initial state  (robot at rest, slightly off-home)
q_true  = [deg2rad(-15); deg2rad(25); deg2rad(-50)];
dq_true = zeros(nq, 1);

% Observer initial state  (matched to true state - avoids transient
 d spike)
z_hat     = [q_true; dq_true; zeros(nd,1)];
v_applied = zeros(nu, 1);

% ---- Reference trajectory  [rad]
 ----------------------------------
%  Six evenly-timed segments; each joint commanded independently.
ref_signal = zeros(ny, T_sim);

% Joint 1 - base yaw (5 segments x 120 steps = 600)
ref_signal(1,:) = [repmat(deg2rad(  0), 1, 120), ...
repmat(deg2rad( 45), 1, 120), ...
repmat(deg2rad(-30), 1, 120), ...
repmat(deg2rad( 60), 1, 120), ...
repmat(deg2rad(  0), 1, 120)];

% Joint 2 shoulder pitch (4 segments)
ref_signal(2,:) = [repmat(deg2rad( 30), 1, 150), ...
repmat(deg2rad( 60), 1, 150), ...
repmat(deg2rad( 20), 1, 150), ...
```

```
repmat(deg2rad( 50), 1, 150)];

% Joint 3  elbow pitch (4 segments)
ref_signal(3,:) = [repmat(deg2rad(-60), 1, 100), ...
repmat(deg2rad(-30), 1, 150), ...
repmat(deg2rad(-90), 1, 150), ...
repmat(deg2rad(-45), 1, 200)];

% ---- True disturbances  [Nm] (friction + sudden payload step) ------
%  Constant friction bias on all joints; extra load added at t = 7 s
d_true             = zeros(nd, T_sim);
d_true(:, 1:end)   = repmat([0.50;  -0.30;  0.20], 1, T_sim);      % friction
d_true(:, 351:end)= d_true(:,351:end) ...
+ repmat([2.00;   1.50;  0.80], 1, T_sim-350);   % payload

% ---- Measurement noise (encoder ~0.05 deg) ----------------------------
sigma_y = deg2rad(0.05) * ones(ny, 1);

% ---- Log arrays -------------------------------------------------------
log_q          = zeros(ny, T_sim);
log_q_ref      = zeros(ny, T_sim);
log_tau        = zeros(nu, T_sim);
log_v          = zeros(nu, T_sim);
log_d_hat      = zeros(nd, T_sim);
log_d_true_acc = zeros(nd, T_sim);   % true disturbance in [rad/s^2] = M^{-1} d_Nm
log_slack      = zeros(ny, T_sim);

fprintf('Simulation: %d steps, Ts=%.3f s, duration=%.1f s\n', ...
T_sim, Ts, T_sim*Ts);
tic;

%% 7. Simulation Loop
for t = 1:T_sim

  ref = ref_signal(:, t);

  % ==== A.  Plant output (angle measurement) ======================
  y_true = q_true;
  y_meas = y_true + sigma_y .* randn(ny, 1);

  % ==== B.  Augmented Kalman Filter ===============================
  z_pred = A_aug * z_hat + B_aug * v_applied;
  z_hat  = z_pred + L * (y_meas - C_aug * z_pred);

  x_hat_k = z_hat(1:nx);
  d_hat_k = z_hat(nx+1:end);

  % ==== C.  Target Calculator =====================================
  rhs     = [B * d_hat_k; ref];
  targets = M_target \ rhs;
  x_ss_k  = targets(1:nx);
  u_ss_k  = targets(nx+1:end);
```

```
    % ==== D.  MPC (deviation-form QP) ==============================
    [sol, diag_flag] = mpc_ctrl{x_hat_k, x_ss_k, u_ss_k};

    if diag_flag == 0
      dv_opt    = full(sol{1});
      slack_now = full(sol{2});
    else
      warning('MPC infea at t=%d (code %d). Holding u_ss.', t,
   diag_flag);
      dv_opt    = zeros(nu, 1);
      slack_now = NaN(ny, 1);
    end

    v_opt = dv_opt + u_ss_k;   % total virtual acceleration command

    % ==== E.  Computed-Torque Law (exact feedback linearisation) ====
    %
    %

    M_mat = robot_M(q_true);
    C_vec = robot_C(q_true, dq_true) * dq_true;
    G_vec = robot_G(q_true);

    tau_cmd = M_mat * v_opt + C_vec + G_vec;

    % Saturate to physical actuator limits [Nm]
    tau_max = [150; 120; 60];
    tau_min = -tau_max;
    tau_cmd = min(max(tau_cmd, tau_min), tau_max);

    % Actual virtual input after saturation (fed back to observer)
    v_applied = M_mat \ (tau_cmd - C_vec - G_vec);

    % ==== F.  True Plant Integration (RK4, full nonlinear model) ====
    x_rob  = [q_true; dq_true];
    x_next = rk4_robot(x_rob, tau_cmd, d_true(:,t), Ts, ...
    robot_M, robot_C, robot_G, nq);
    q_true  = x_next(1:nq);
    dq_true = x_next(nq+1:end);

    % ==== G.  Logging ===============================================
    log_q(:,t)           = y_true;
    log_q_ref(:,t)       = ref;
    log_tau(:,t)         = tau_cmd;
    log_v(:,t)           = v_opt;
    log_d_hat(:,t)       = d_hat_k;
    log_d_true_acc(:,t)  = M_mat \ d_true(:,t);
    log_slack(:,t)       = slack_now;
  end

  elapsed = toc;
  fprintf('Done.  Wall-clock: %.2f s  (%.1f ms/step)\n', ...
  elapsed, elapsed/T_sim*1e3);
```

```
%% ==================================================================
%%  LOCAL FUNCTIONS
%% ==================================================================

% ------------------------------------------------------------------
function M = inertia_matrix_3dof(q, m2, m3, a2, lc2, lc3, Iz1, I2, I3)
  % 3 x 3 symmetric positive-definite inertia matrix of the 3-DOF RRR arm.
  %
  %  Block-diagonal structure (q1 decoupled from q2,q3):
  %    M = diag( M11(q2,q3) , [M22 M23; M23 M33](q3) )
  %
  %  Derived via Lagrangian mechanics (Christoffel-symbol formulation).
  %  The block-diagonal structure follows from the perpendicular

  c2  = cos(q(2));
  c3  = cos(q(3));
  c23 = cos(q(2) + q(3));
  h   = m3 * a2 * lc3;

  R3  = a2*c2 + lc3*c23;                    % horizontal reach of link-3 COM

  M11 = Iz1 + I2 + I3 + m2*(lc2*c2)^2 + m3*R3^2;
  M22 = (I2 + m2*lc2^2) + (I3 + m3*lc3^2) + m3*a2^2 + 2*h*c3;
  M23 = (I3 + m3*lc3^2) + h*c3;
  M33 = I3 + m3*lc3^2;

  M = [M11, 0,   0  ;
       0, M22, M23 ;
       0, M23, M33 ];
end

% ------------------------------------------------------------------
function C_mat = coriolis_matrix_3dof(q, dq, m2, m3, a2, lc2, lc3)

  s2  = sin(q(2));  c2  = cos(q(2));
  s3  = sin(q(3));
  s23 = sin(q(2) + q(3));  c23 = cos(q(2) + q(3));
  h   = m3 * a2 * lc3;

  R3          = a2*c2 + lc3*c23;
  A3          = a2*s2 + lc3*s23;              % = -dR3/dq2
  dM11_dq2    = -2*m2*lc2^2*s2*c2 - 2*m3*R3*A3;
  dM11_dq3    = -2*m3*R3*lc3*s23;

  C_mat = zeros(3, 3);

  % --- Row 1: base yaw coupling terms ---
  C_mat(1,1) = 0.5*dM11_dq2*dq(2)  + 0.5*dM11_dq3*dq(3);
  C_mat(1,2) = 0.5*dM11_dq2*dq(1);
```

```
    C_mat(1,3) = 0.5*dM11_dq3*dq(1);

    % --- Row 2: shoulder pitch ---
    C_mat(2,1) = -0.5*dM11_dq2*dq(1);
    C_mat(2,2) = -h*s3*dq(3);
    C_mat(2,3) = -h*s3*(dq(2) + dq(3));

    % --- Row 3: elbow pitch ---
    C_mat(3,1) = -0.5*dM11_dq3*dq(1);
    C_mat(3,2) =  h*s3*dq(2);
    C_mat(3,3) =  0;
  end

  % ----------------------------------------------------------------------
    %
  function G = gravity_vector_3dof(q, m2, m3, lc2, a2, lc3, g)
    % Gravity torque vector.
    %

    c2  = cos(q(2));
    c23 = cos(q(2) + q(3));

    G1 = 0;
    G2 = m2*g*lc2*c2 + m3*g*(a2*c2 + lc3*c23);
    G3 = m3*g*lc3*c23;

    G = [G1; G2; G3];
  end

  % ----------------------------------------------------------------------
    %
  function x_next = rk4_robot(x, tau, d, Ts, fM, fC, fG, nq)
    % 4th-order Runge-Kutta integration of the full nonlinear robot
   ODE.
    f  = @(xx) robot_ode(xx, tau, d, fM, fC, fG, nq);
    k1 = f(x);
    k2 = f(x + Ts/2 * k1);
    k3 = f(x + Ts/2 * k2);
    k4 = f(x + Ts   * k3);
    x_next = x + (Ts/6)*(k1 + 2*k2 + 2*k3 + k4);
  end

  % ----------------------------------------------------------------------
    %
  function xdot = robot_ode(x, tau, d, fM, fC, fG, nq)
    q    = x(1:nq);
    dq   = x(nq+1:end);
    M    = fM(q);
    Cdq  = fC(q, dq) * dq;
    G    = fG(q);
    ddq  = M \ (tau + d - Cdq - G);
    xdot = [dq; ddq];
  end

```

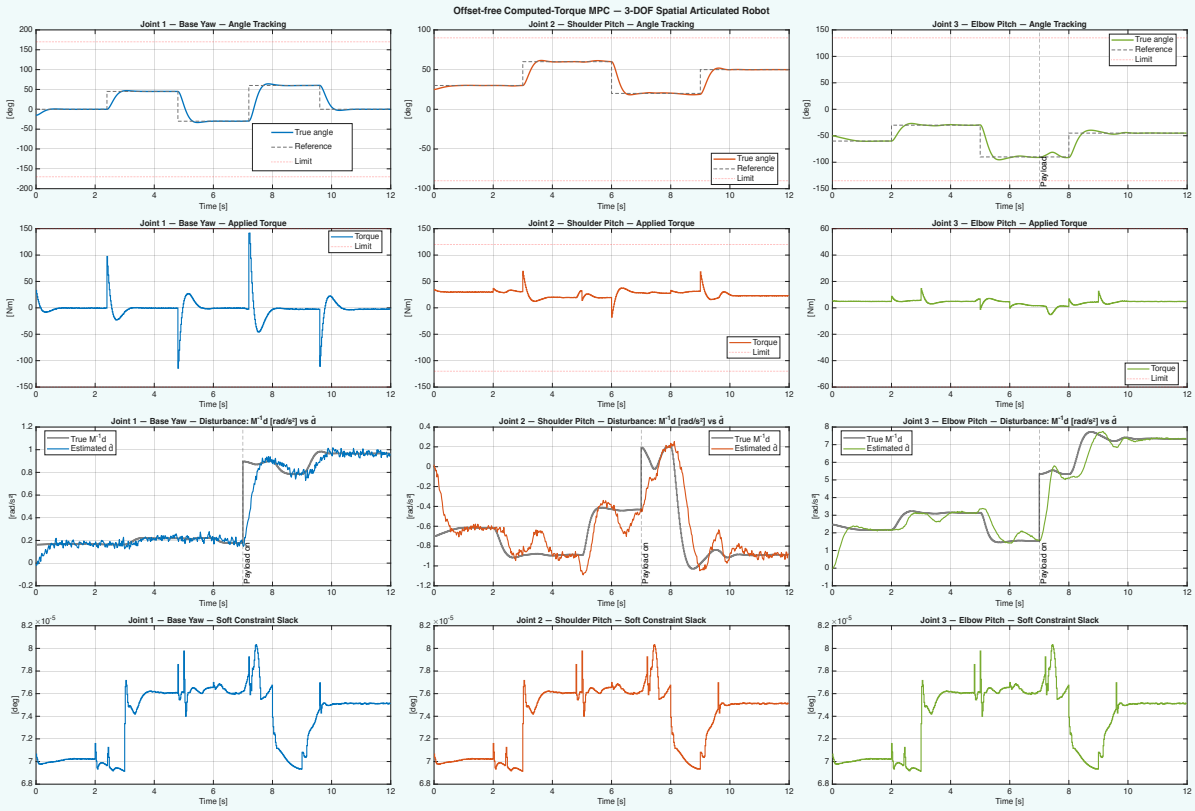


Figure 8.9: Closed-loop performance of the offset-free computed-torque MPC applied to a 3-DOF spatial RRR articulated robot manipulator. The controller combines feedback linearisation (computed-torque law) with a deviation-form quadratic MPC ($N = 15$, $T_s = 20\,\text{ms}$) and an augmented Kalman filter for disturbance estimation. **Row 1**: Joint angle tracking for base yaw (q_1), shoulder pitch (q_2), and elbow pitch (q_3), with piecewise-constant reference trajectories (dashed) and joint limits (dotted red). **Row 2**: Applied actuator torques with saturation limits at ± 150, ± 120, and $\pm 60\,\text{Nm}$, respectively. **Row 3**: Comparison of the true disturbance acceleration $M^{-1}(q)\,d$ (grey) and the Kalman filter estimate $\hat{d}$ (coloured), showing convergence after the payload step at $t = 7\,\text{s}$. **Row 4**: Soft-constraint slack variables, which remain near zero throughout, confirming that joint limits are respected without significant violation. Constant friction biases (0.5, -0.3, $0.2\,\text{Nm}$) act on all joints from $t = 0$, and an additional payload disturbance (2.0, 1.5, $0.8\,\text{Nm}$) is applied at $t = 7\,\text{s}$. The target calculator drives steady-state offsets to zero in all cases, demonstrating robust offset-free tracking despite unmodelled friction, sudden payload changes, and encoder noise ($\sigma = 0.05^\circ$).

Exercises

1. **Augmented State-Space Construction.** Consider a scalar plant

$$x_{k+1} = 0.8\,x_k + u_k + d_k, \qquad y_k = x_k,$$

where d_k is an unknown constant disturbance ($d_{k+1} = d_k$).

(a) Define the augmented state vector $z_k = [x_k,\ d_k]^T$ and write down the augmented system matrices A_{aug}, B_{aug}, and C_{aug}.

(b) Now suppose the disturbance enters only through the output, i.e. $B_w = 0$ and $C_d = 1$. How do the augmented matrices change?

(c) For the two-state system

$$A = \begin{bmatrix} 1 & 0.1 \\ 0 & 0.95 \end{bmatrix}, \quad B = \begin{bmatrix} 0.005 \\ 0.1 \end{bmatrix}, \quad B_w = B, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad C_d = 0,$$

write out the full augmented matrices $A_{\text{aug}} \in \mathbb{R}^{3\times 3}$, $B_{\text{aug}} \in \mathbb{R}^{3\times 1}$, and $C_{\text{aug}} \in \mathbb{R}^{1\times 3}$.

2. **Detectability of the Augmented System.** For the augmented pair $(A_{\text{aug}},\ C_{\text{aug}})$ constructed in Exercise 1(c):

(a) Form the observability matrix

$$\mathcal{O} = \begin{bmatrix} C_{\text{aug}} \\ C_{\text{aug}} A_{\text{aug}} \\ C_{\text{aug}} A_{\text{aug}}^2 \end{bmatrix}$$

and verify that its rank equals the dimension of the augmented state.

(b) Now suppose the system has two outputs $y_k = Cx_k$ with $C = I_{2\times 2}$ and two independent disturbance states $d_k \in \mathbb{R}^2$ with $B_w = B$ and $C_d = 0$. Does the augmented pair remain detectable? Justify your answer by examining the eigenvalues and the rank of the detectability test matrix $\begin{bmatrix} A_{\text{aug}} - \lambda I \\ C_{\text{aug}} \end{bmatrix}$ at the eigenvalue $\lambda = 1$.

(c) Explain the practical rule of thumb relating the number of disturbance states n_d to the number of measured outputs n_y.

3. **Steady-State Target Calculation.** Consider the system from Exercise 1(c) with the reference $r_k = 2$ and a disturbance estimate $\hat{d}_k = 0.5$.

(a) Write out the target-calculator linear system

$$\begin{bmatrix} I - A & -B \\ C & 0 \end{bmatrix} \begin{bmatrix} x_{ss} \\ u_{ss} \end{bmatrix} = \begin{bmatrix} B_w \hat{d}_k \\ r_k - C_d \hat{d}_k \end{bmatrix}$$

with all numerical entries substituted.

(b) Solve for x_{ss} and u_{ss}. Verify that the steady-state output satisfies $Cx_{ss} = r_k$.

(c) Suppose the input is constrained to $-1 \leq u \leq 1$. Is the steady-state target u_{ss} feasible? If not, explain qualitatively how a constrained steady-state optimisation (as in Equation (8.5)) would handle this situation.

4. **Deviation-Form MPC Variables.** Using the targets (x_{ss}, u_{ss}) computed in Exercise 3:

(a) Suppose the current state estimate is $\hat{x}_k = [1.5,\ 0.3]^T$. Compute the initial deviation $\tilde{x}_0 = \hat{x}_k - x_{ss}$.

(b) Write out the deviation-form prediction model $\tilde{x}_{i+1} = A\tilde{x}_i + B\tilde{u}_i$ and explain why the disturbance estimate $\hat{d}_k$ does not appear in this equation.

(c) If the hard input constraints are $-1 \leq u_k \leq 1$, express these as constraints on the deviation input $\tilde{u}_i$. Are the deviation-form constraints symmetric about zero?

(d) Explain the role of the terminal cost matrix P_{term} in the MPC cost function (8.6) and state how it is typically computed.

5. **Effect of Misspecified B_w.** Suppose the true plant has the disturbance entering through $B_w^{\text{true}} = [0;\ 1]$, but the augmented observer and target calculator are designed using the incorrect assumption $B_w = B = [0.005;\ 0.1]$.

   (a) Will the augmented Kalman filter still converge to a constant disturbance estimate $\hat{d}_k$ in steady state? Explain why or why not.

   (b) At steady state, the target calculator enforces $(I - A)x_{ss} = Bu_{ss} + B_w\hat{d}_k$ and $Cx_{ss} = r_k$. Does the controller still achieve zero steady-state output offset despite the mismatch in B_w? Argue from the structure of the equilibrium equations.

   (c) Discuss the transient consequences of the misspecified B_w. In particular, how might the transient response and the value of $\hat{d}_k$ differ from the correctly specified case?

6. **Comparison: MPC With and Without Disturbance Estimation.** Consider a standard (non-augmented) MPC regulator that uses the nominal model $x_{k+1} = Ax_k + Bu_k$ without disturbance estimation, alongside the offset-free architecture of Algorithm 1.

   (a) Suppose a constant disturbance $d_k = d \neq 0$ acts on the plant through B_w. Explain why the standard MPC regulator generally produces a non-zero steady-state tracking error. What is the source of the persistent offset?

   (b) In the offset-free architecture, identify which component provides integral-action-like behaviour. How does the integrating disturbance model $d_{k+1} = d_k$ relate to classical integral action?

   (c) Suppose instead that the disturbance is sinusoidal, $d_k = \sin(\omega k)$. Will the offset-free MPC with a constant disturbance model ($d_{k+1} = d_k$) eliminate the steady-state tracking error in this case? Justify your answer in terms of the internal model principle.

   (d) Briefly outline how the disturbance model could be modified to handle the sinusoidal case.

Chapter 9

Simulink Implementation of Model Predictive Control$^{(*)}$

9.1 General Implementation Guidance

Simulink is widely used for rapid prototyping and hardware-in-the-loop validation of control systems, and Model Predictive Control (MPC) is no exception. Implementing MPC in Simulink without the dedicated MPC Toolbox requires a careful architectural decision: the optimisation solver must be invoked at every simulation time step from within a Simulink block, yet the solver itself — together with the YALMIP modelling layer — relies on dynamic MATLAB features that are incompatible with Simulink's default code-generation pipeline. This section establishes a general framework that resolves this tension. The approach is first described in fully general terms; Section 9.2 then applies it step by step to a concrete building temperature control system.

9.1.1 Why the MPC Toolbox Is Not Required

The MathWorks MPC Toolbox offers convenient pre-built blocks but abstracts away the optimisation layer entirely. This abstraction becomes a limitation whenever the designer requires any of the following:

- **Economic (non-regulatory) objectives** — for instance, minimising energy cost rather than a quadratic deviation from a setpoint;
- **Augmented observers** — such as Kalman filters that jointly estimate the plant state and an unmeasured disturbance;
- **Time-varying constraints** — comfort bands or operational limits that change according to a schedule;
- **Custom terminal costs or terminal sets** — required for rigorous closed-loop stability guarantees.

Using YALMIP together with `quadprog` or OSQP provides all of this flexibility whilst keeping solver call-time well within a typical sampling period.

9.1.2 The Code-Generation Barrier and How to Overcome It

MATLAB Function blocks in Simulink normally compile their contents into C code at simulation start. YALMIP relies on dynamic cell arrays, runtime type inspection, and variable-argument function calls — none of which survive compilation. Attempting to call `sdpvar`, `optimizer`, or any YALMIP routine directly inside a compiled block produces an immediate error.

The standard remedy is the `coder.extrinsic` declaration. Any function declared as extrinsic is *not* compiled; instead, it executes inside the full MATLAB interpreter at run time. The simulation still advances correctly — the block simply runs in interpreted mode for the declared functions.

Rules for `coder.extrinsic`

1. The declaration must appear at the *top* of the MATLAB Function block body, before any use of the function.
2. Every output variable must be *pre-allocated* to a concrete type and size before the extrinsic call, so that Simulink can infer port dimensions at compile time.
3. All YALMIP-dependent logic should live in a separate `.m` helper file; the block itself contains only the declaration and a single forwarding call.

The general pattern inside any MATLAB Function block is:

```
function y = MyBlock(u)
  coder.extrinsic('my_helper');    % declare before use
  y = zeros(size_of_output);       % pre-allocate
  y = my_helper(u);                % forward to helper
end
```

9.1.3 Managing State Across Time Steps: Persistent Variables

A Simulink MATLAB Function block does not retain values in its workspace between consecutive invocations — each call begins with a clean local scope. An MPC controller, however, must preserve several quantities across every time step:

- the observer state estimate $\hat{z}_k$;
- the last applied input u_{k-1} (needed for the observer prediction step);
- any filtered disturbance estimates;
- the compiled YALMIP `optimizer` objects.

MATLAB's `persistent` keyword solves this problem. A persistent variable is initialised once on the first function call and retains its value for the lifetime of the function in memory. Within a Simulink simulation, this means the variable persists across every time step from $k = 1$ to the end of the run.

Persistent Variable Pattern for MPC Helper Functions

Declare all cross-step quantities with `persistent` at the top of the helper `.m` file, then guard initialisation with an `isempty` check:

```
function u = my_mpc_step(y_meas, ...)
  persistent opt_obj observer_state u_prev

  if isempty(observer_state)          % runs on first call only
    opt_obj        = build_optimizers();
    observer_state = x0_guess;
    u_prev         = 0;
  end

  % ... rest of the control pipeline ...
end
```

Important: before starting a new simulation run always execute `clear` *`functionname`* in the MATLAB Command Window. This resets all persistent variables to their initial state; failure to do so causes the controller to inherit the terminal conditions of the previous run.

9.1.4 Building YALMIP Optimiser Objects: One-Time Compilation

YALMIP's `optimizer` function pre-processes the problem structure — symbolic variable registration, sparsity analysis, and conversion to the solver's native QP format — and returns a compiled function handle. This pre-processing is the expensive step (typically 10–30 s for moderate-sized problems) and must not be repeated at every simulation time step.

The correct approach is to create a dedicated initialisation function, call it exactly once inside the persistent-variable guard shown above, and store the resulting objects as persistent variables. All subsequent time steps call the compiled objects directly, which typically completes in milliseconds.

```
function [opt1, opt2] = build_optimizers()
  % Called once. Returns compiled optimizer objects.

  % -- Define symbolic decision variables -----------------------
  x  = sdpvar(nx, 1);
  u  = sdpvar(nu, 1);
  p  = sdpvar(np, 1);    % parametric inputs (e.g. price, bounds)

  % -- Formulate constraints and objective ----------------------
  con = [ ... ];         % linear/quadratic constraints
  obj = ... ;            % quadratic cost

  % -- Compile with chosen solver -------------------------------
  ops  = sdpsettings('solver', 'quadprog', 'verbose', 0);
  opt1 = optimizer(con, obj, ops, {p}, {u});

  % -- Second optimizer (e.g. target calculator) ----------------
  % ...
end
```

Listing 9.1: Skeleton of a one-time YALMIP initialisation function.

9.1.5 Recommended Architecture

The architecture that best reconciles Simulink's execution model with YALMIP's requirements separates the design into two layers, as shown in Figure 9.1.

Layer 1 — the Simulink canvas contains only standard Simulink blocks and thin MATLAB Function wrapper blocks. Each wrapper declares one helper function as extrinsic, pre-allocates its outputs, and forwards its inputs. Signal routing, the feedback connection, noise injection, and input saturation are all handled by built-in Simulink blocks.

Layer 2 — the helper `.m` files contain all computation: observer updates, YALMIP calls, QP solves, and persistent state. Because they execute in the full MATLAB interpreter they impose no restrictions on the code they contain.

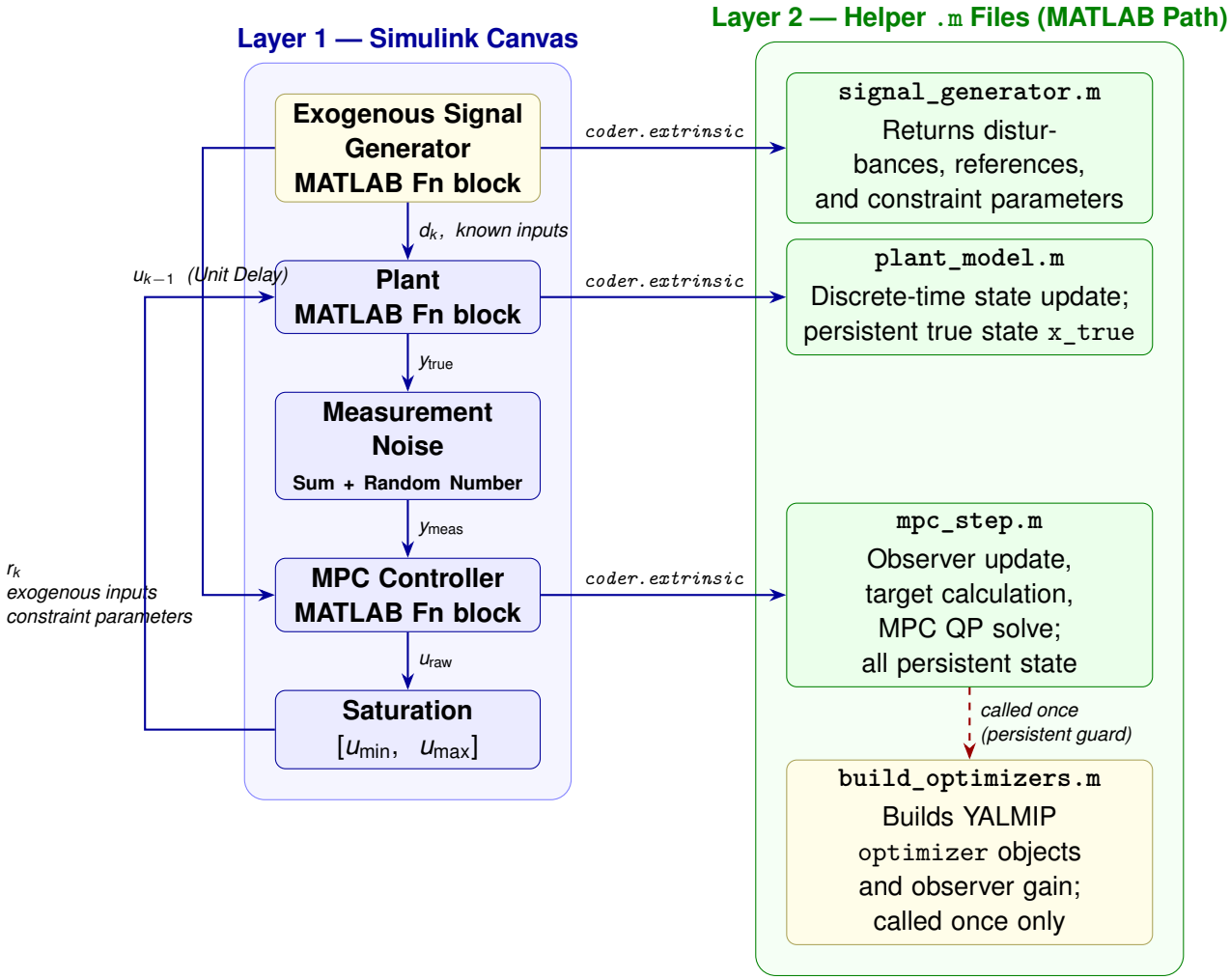


Figure 9.1: General two-layer architecture for Simulink-based MPC without the MPC Toolbox. Layer 1 (Simulink canvas) contains only thin MATLAB Function wrapper blocks; all optimisation logic is delegated to Layer 2 helper files via `coder.extrinsic` calls. Persistent variables in the helper files carry state across time steps and store the compiled `optimizer` objects for the lifetime of the simulation.

9.1.6 Eliminating the Algebraic Loop

In any discrete-time closed-loop Simulink model of the form Plant → Controller → Plant, Simulink detects an *algebraic loop*: computing the plant output y_k requires the current input u_k, which itself requires y_k. The standard remedy is to insert a **Unit Delay** block between the controller output (or the saturation block) and the plant input:

$$\cdots \rightarrow \text{Saturation} \xrightarrow{u_k} \underbrace{\text{Unit Delay}}_{T_s,\ \text{IC}=0} \xrightarrow{u_{k-1}} \text{Plant} \rightarrow \cdots$$

This is not an approximation — in a physically realisable digital controller the input computed at step k cannot reach the actuator until step k+1. The observer in the MPC helper file must therefore use u_{k-1} (stored as a persistent variable) rather than u_k in its prediction equation, which is achieved automatically by the pattern shown in Section 9.1.3.

9.1.7 Solver Recommendations

YALMIP supports a wide range of QP solvers through a common interface. For typical MPC problems — moderate-horizon quadratic programmes with linear constraints — the following two solvers are recommended.

Solver	Availability	Notes
`quadprog`	Ships with MATLAB Optimisation Toolbox	Reliable, no warm-start across calls; sufficient for $N \leq 30$
OSQP	Free, open-source; install via `osqp-matlab`	Supports warm-starting; recommended for long horizons or fast sampling rates

Both are specified in YALMIP identically:

```
sdpsettings('solver','quadprog',...)
```

or

```
sdpsettings('solver','osqp',...).
```

9.1.8 Summary of the Implementation Workflow

Figure 9.2 summarises the complete development workflow. The left column shows the *offline* design steps that are completed once before the Simulink model is built. The right column shows the *runtime* sequence that executes automatically during simulation.

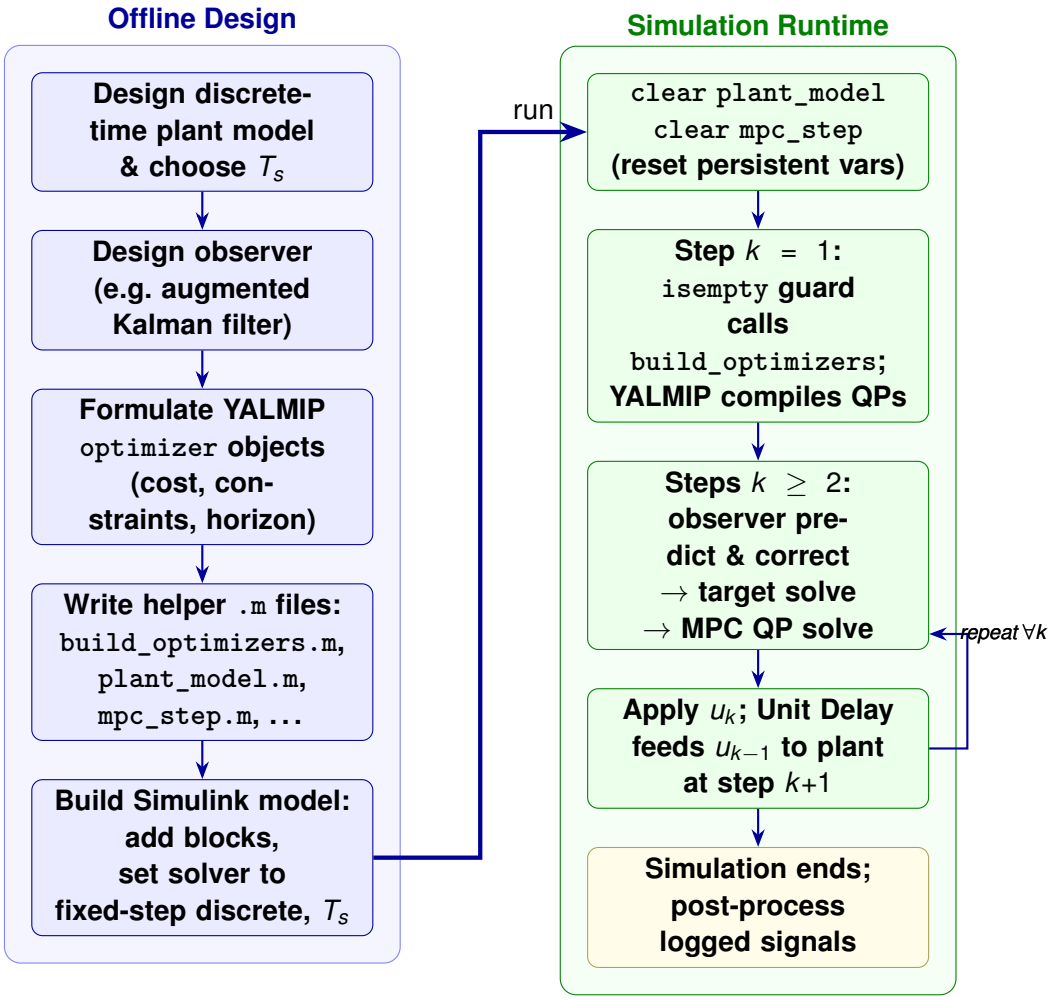


Figure 9.2: Complete Simulink MPC development and execution workflow. The offline design phase (left) produces all `.m` helper files and the Simulink model. On each new simulation run the persistent variables are cleared manually, after which the runtime sequence (right) executes automatically, compiling YALMIP objects once on the first step and solving the QP at every subsequent step.

9.2 Case Study: Single-Zone Building Heating

We revisit the single-zone office heating problem introduced in the previous chapter. The general framework of Section 9.1 is now applied step by step to the three-state physically derived RC model, combining an augmented Kalman filter — which jointly estimates the three thermal states and an unmeasured occupancy disturbance — with a two-layer economic MPC comprising an economic steady-state target calculator and a deviation-form MPC regulator with electricity-price preview. This example exercises every aspect of the general framework: persistent optimiser objects, a multi-output profile generator, an extrinsic plant block, and a feedback loop broken by a Unit Delay.

9.2.1 System Description

The discrete-time model of the single-zone building (sampling period $T_s = 1800\,\text{s}$, i.e. 30 min) is

$$x_{k+1} = Ax_k + Bu_k + E\,T_{\text{amb},k} + B_{\text{sol}}\,\varphi_{\text{sol},k} + B_w\,d_k, \qquad y_k = Cx_k + v_k,$$

where $x_k = [T_{\text{air},k},\ T_{w,\text{in},k},\ T_{w,\text{out},k}]^\top \in \mathbb{R}^3$ contains indoor air temperature and the two wall-layer temperatures, $u_k \in [0,\,2000]\,\text{W}$ is heater power, $T_{\text{amb},k}$ and $\varphi_{\text{sol},k}$ are the *measured* outdoor temperature and solar irradiance (known feedforwards), and d_k (°C/step) is the *unmeasured* occupancy-driven thermal disturbance. The output

$$y_k = 0.56\,T_{\text{air},k} + 0.44\,T_{w,\text{in},k}$$

is the operative temperature (ISO 7726), measured with noise standard deviation $\sigma_v = 0.1\,°\text{C}$.

The pre-computed ZOH-discretised matrices are

$$A = \begin{bmatrix} 0.8753 & 0.1102 & 0.0004 \\ 0.0275 & 0.9649 & 0.0073 \\ 0.0001 & 0.0049 & 0.9851 \end{bmatrix}, \quad B = \begin{bmatrix} 7.91\times10^{-4} \\ 1.00\times10^{-4} \\ 0 \end{bmatrix}, \quad E = \begin{bmatrix} 0.01404 \\ 0.00025 \\ 0.00993 \end{bmatrix},$$

$$B_{\text{sol}} = \begin{bmatrix} 9.04\times10^{-6} \\ 1.43\times10^{-6} \\ 3.47\times10^{-4} \end{bmatrix}, \quad B_w = \begin{bmatrix} 1.7647 \\ 0.0267 \\ 0 \end{bmatrix}, \quad C = \begin{bmatrix} 0.56 & 0.44 & 0 \end{bmatrix}.$$

The control objective is to minimise electricity cost subject to comfort constraints:

$$y_{\min,k} \le y_k \le y_{\max,k}, \qquad 0 \le u_k \le 2000\,\text{W},$$

where the comfort bounds follow a daily occupancy schedule ([20, 23] °C during occupied hours 07:00–22:00, [15, 26] °C overnight).

9.2.2 File Structure

Following the general architecture of Section 9.1.5, the implementation consists of four helper `.m` files and one Simulink model file. Table 9.1 maps each file to the corresponding generic role.

Table 9.1: Helper files for the single-zone building case study and their correspondence to the general architecture.

File	Generic role	Purpose
`build_mpc_optimizers.m`	`build_optimizers.m`	Kalman gain (DARE), economic target `optimizer`, MPC regulator `optimizer` with price preview; called once
`get_profiles.m`	`disturbance_profiles.m`	Ambient temperature, solar irradiance, electricity price, current comfort bounds, and N-step future comfort bound vectors
`plant_step.m`	`plant_model.m`	One-step advance of the discrete RC building model; persistent true state `x_true` and true occupancy disturbance
`mpc_controller_step.m`	`mpc_step.m`	Kalman observer, economic target calculation, economic MPC QP solve with price preview; all state persistent
`EconomicMPC.slx`	Simulink model	Wires all wrapper blocks and standard blocks together

All `.m` files and `EconomicMPC.slx` must reside in the same folder, or all `.m` files must be accessible via `addpath`.

9.2.3 Step 1 — `build_mpc_optimizers.m`

This function fulfils the role of the generic `build_optimizers.m` described in Section 9.1.4. It computes the steady-state Kalman gain via `dare` and pre-compiles two YALMIP `optimizer` objects: one for the economic steady-state target and one for the deviation-form MPC regulator with N-step price preview. The compilation step runs exactly once, since the resulting objects are stored as persistent variables in `mpc_controller_step.m`.

```
function [target_calc, mpc_reg, L] = build_mpc_optimizers()
  % Builds YALMIP optimizer objects and the Kalman gain.
  % Called exactly once via the persistent guard in mpc_controller_step.m.
  fprintf('Building YALMIP optimizers -- please wait...\n');

  Ts = 1800;
  A     = [0.8753, 0.1102, 0.0004;
         0.0275, 0.9649, 0.0073;
         0.0001, 0.0049, 0.9851];
  B     = [7.91e-4; 1.00e-4; 0      ];
  E     = [0.01404; 0.00025; 0.00993];
  Bsol  = [9.04e-6; 1.43e-6; 3.47e-4];
  Bw    = [1.7647;  0.0267;  0      ];
  C     =  [0.56, 0.44, 0];
  nx = 3;  nu = 1;  u_min = 0;  u_max = 2000;

  %% -- Augmented Kalman filter (state + occupancy disturbance) ------
  %    z = [x; d],  random-walk disturbance model
```

```
    A_aug   = [A, Bw; zeros(1,nx), 1];
    B_aug   = [B; 0];
    E_aug   = [E; 0];
    Bsol_aug= [Bsol; 0];
    C_aug   = [C, 0];

    W_kf = diag([1e-3, 1e-3, 1e-3, (0.01)^2]);  % sqrt(W_d)=0.01 degC/step
    V_kf = 0.1^2;
    [P_kf,~,~] = dare(A_aug', C_aug', W_kf, V_kf);
    L = P_kf * C_aug' / (C_aug * P_kf * C_aug' + V_kf);

    %% -- Economic steady-state target optimizer -----------------------
    price_k  = sdpvar(1,1);  d_hat_k  = sdpvar(1,1);
    T_amb_k  = sdpvar(1,1);  phi_k    = sdpvar(1,1);
    y_min_k  = sdpvar(1,1);  y_max_k  = sdpvar(1,1);
    xs = sdpvar(nx,1);  us = sdpvar(nu,1);  ss_slack = sdpvar(1,1);

    ss_cons = [xs == A*xs + B*us + E*T_amb_k + Bsol*phi_k + Bw*d_hat_k, ...
    y_min_k - ss_slack <= C*xs <= y_max_k + ss_slack, ...
    u_min <= us <= u_max,  ss_slack >= 0];
    ss_obj  = price_k * us + 1e4 * ss_slack^2;

    ops_ss      = sdpsettings('solver','quadprog','verbose',0);
    target_calc = optimizer(ss_cons, ss_obj, ops_ss, ...
    {price_k, d_hat_k, T_amb_k, phi_k, y_min_k, y_max_k}, ...
    [xs; us]);

    %% -- Economic MPC regulator with N-step price preview -------------
    %    Stage cost: ||x_tilde||_Q^2 + beta*price*u*(Ts/3600) + rho*slack^2
    %    Penalty hierarchy: rho >> beta*p_max*u_max*(Ts/3600)
    %    i.e.  1e7 >> 6*0.28*2000*0.5 = 1680
    N = 24;  Q = diag([5,2,1]);  beta = 6;  rho = 1e7;
    [P_term,~,~] = dare(A, B, Q, 1e-3);

    du_v     = sdpvar(nu,N,'full');    dx_v  = sdpvar(nx,N+1,'full');
    slk      = sdpvar(1,N,'full');
    xh_v     = sdpvar(nx,1);   xs_v = sdpvar(nx,1);  us_v = sdpvar(nu,1);
    Ymn      = sdpvar(1,N,'full');     Ymx   = sdpvar(1,N,'full');
    Price_v  = sdpvar(1,N,'full');

    obj = 0;
    con = [dx_v(:,1) == xh_v - xs_v];
    for i = 1:N
      u_abs = du_v(:,i) + us_v;
      x_pred_i = dx_v(:,i+1) + xs_v;
      con = [con, ...
      dx_v(:,i+1) == A*dx_v(:,i) + B*du_v(:,i), ...
      u_min <= u_abs <= u_max, ...
      Ymn(i)-slk(i) <= C*x_pred_i <= Ymx(i)+slk(i), slk(i) >= 0];
      obj = obj + dx_v(:,i)'*Q*dx_v(:,i) ...
      + beta*Price_v(i)*u_abs*(Ts/3600) ...
      + rho*slk(i)^2;
    end
    obj = obj + dx_v(:,N+1)' * P_term * dx_v(:,N+1);

    ops_mpc = sdpsettings('solver','quadprog','verbose',0);
    mpc_reg = optimizer(con, obj, ops_mpc, ...
    {xh_v, xs_v, us_v, Ymn, Ymx, Price_v}, du_v(:,1));

    fprintf('Optimizers ready.\n');
```

```
end
```

Listing 9.2: `build_mpc_optimizers.m` — one-time initialisation function for the single-zone building controller.

Horizon and Tuning Parameters

The prediction horizon $N = 24$ steps corresponds to 12 hours at $T_s = 30$ min, allowing the controller to anticipate the morning and evening price transitions. The state penalty $Q = \mathrm{diag}(5, 2, 1)$ weights the air temperature most heavily. The economic weight $\beta = 6$ and the comfort slack penalty $\rho = 10^7$ satisfy the hierarchy $\rho \gg \beta\, p_{\max}\, u_{\max}\, (T_s/3600) = 1680$, guaranteeing that comfort violations are never preferred over heating. The terminal cost P_∞ is the DARE solution, ensuring asymptotic stability of the unconstrained closed-loop.

9.2.4 Step 2 — `get_profiles.m`

This function fulfils the role of the generic `disturbance_profiles.m`. It receives the current simulation time in seconds from the Simulink *Clock* block and returns all time-varying exogenous signals: ambient temperature, solar irradiance, electricity price, current comfort bounds, and the N-step future comfort bound and price vectors required by the MPC optimiser.

```
function [T_amb, Solar, Price, Tmin, Tmax, Tmin_fut, Tmax_fut, Price_fut]
 = ...
get_profiles(t)
  % Input:  t in seconds (Simulink Clock output).
  % Outputs: scalars for current step + 1x24 vectors for MPC horizon.
  N  = 24;  Ts = 1800;  t_h = t / 3600;

  % Outdoor temperature: nominally -1 to 11 degC (winter scenario)
  T_amb = 5 + 6*sin(2*pi*(t_h - 14)/24) + 0.5*randn;

  % Solar irradiance: daytime half-sine, zero at night
  Solar = max(0, 600*sin(pi*(mod(t_h,24) - 8)/9));

  % Electricity price: peak / off-peak tariff
  is_peak = (mod(t_h,24) >= 7) && (mod(t_h,24) < 22);
  if is_peak
    Price = 0.28;  Tmin = 20;  Tmax = 23;
  else
    Price = 0.05;  Tmin = 15;  Tmax = 26;
  end

  % N-step future comfort bounds and price sequence for MPC
  Tmin_fut  = zeros(1,N);  Tmax_fut  = zeros(1,N);
  Price_fut = zeros(1,N);
  for i = 1:N
    t_fut = t_h + i*(Ts/3600);
    if (mod(t_fut,24) >= 7) && (mod(t_fut,24) < 22)
      Tmin_fut(i)  = 20;   Tmax_fut(i)  = 23;
      Price_fut(i) = 0.28;
    else
      Tmin_fut(i)  = 15;   Tmax_fut(i)  = 26;
      Price_fut(i) = 0.05;
    end
```

```
    end
  end
```

Listing 9.3: `get_profiles.m` — provides current and forecast exogenous signals for the MPC horizon (48-hour winter scenario).

9.2.5 Step 3 — `plant_step.m`

This function fulfils the generic `plant_model.m` role. The true state x_{true} is stored as a persistent variable. On each call the function returns the current measured operative temperature $y_{\text{true}} = Cx_{\text{true}}$ and then advances the state by one step, incorporating the heater input, solar irradiance, ambient temperature, and the true occupancy disturbance (unknown to the controller).

```
function y_true = plant_step(u, T_amb, Solar, t)
  % Run  clear plant_step  before each new simulation.
  % t: current time in seconds (used to compute true occupancy).
  persistent x_true
  A    = [0.8753, 0.1102, 0.0004;
         0.0275, 0.9649, 0.0073;
         0.0001, 0.0049, 0.9851];
  B    = [7.91e-4; 1.00e-4; 0      ];
  E    = [0.01404; 0.00025; 0.00993];
  Bsol = [9.04e-6; 1.43e-6; 3.47e-4];
  Bw   = [1.7647;  0.0267;  0      ];
  C    = [0.56, 0.44, 0];

  if isempty(x_true)
    x_true = [19; 17; 10];  % initial temperatures: air, inner wall, outer
  wall
  end

  % True occupancy: 300 W internal gains => 0.191 degC/step
  t_h   = t / 3600;
  d_occ = 0.191 * double(mod(t_h,24) >= 8 && mod(t_h,24) < 18);

  y_true = C * x_true;
  x_true = A*x_true + B*u + E*T_amb + Bsol*Solar + Bw*d_occ;
end
```

Listing 9.4: `plant_step.m` — discrete 3-state RC building model with persistent true state and true occupancy disturbance.

9.2.6 Step 4 — `mpc_controller_step.m`

This is the main control file, fulfilling the role of the generic `mpc_step.m`. It implements the persistent-variable pattern described in Section 9.1.3: on the first call it invokes `build_mpc_optimizers` and initialises the augmented Kalman state; on every subsequent call it executes the three-stage pipeline shown in Figure 9.3.

```
function u = mpc_controller_step(y_meas, T_amb, Solar, Price, ...
Tmin, Tmax, Tmin_fut, Tmax_fut, Price_fut)
  % Full Economic MPC step: observer -> target -> MPC.
  % Run  clear mpc_controller_step  before each new simulation.
```

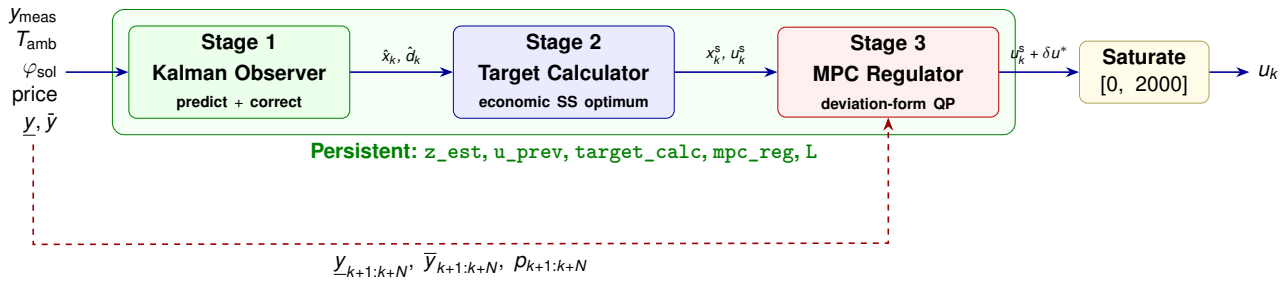


Figure 9.3: Three-stage control pipeline executed at every sampling instant inside `mpc_controller_step.m`. The dashed arrow carries the N-step future comfort bounds *and* electricity-price sequence directly from the input port to the MPC Regulator, bypassing the observer and target stages. All persistent quantities are retained between calls via MATLAB's `persistent` keyword.

```
    persistent target_calc mpc_reg L z_est u_prev

    Ts = 1800;
    A_aug   = [ 0.8753, 0.1102, 0.0004, 1.7647;
                0.0275, 0.9649, 0.0073, 0.0267;
                0.0001, 0.0049, 0.9851, 0.0000;
                0,      0,      0,      1      ];
    B_aug   = [7.91e-4; 1.00e-4; 0; 0];
    E_aug   = [0.01404; 0.00025; 0.00993; 0];
    Bsol_aug= [9.04e-6; 1.43e-6; 3.47e-4; 0];
    C_aug   = [0.56, 0.44, 0, 0];
    nx = 3;  u_min = 0;  u_max = 2000;

    %% -- First call: initialise (Section 9.1.3 pattern) ---------------
    if isempty(z_est)
      [target_calc, mpc_reg, L] = build_mpc_optimizers();
      z_est  = [19; 17; 10; 0];  % [T_air; T_w_in; T_w_out; d_hat]
      u_prev = 0;
    end

    %% -- Stage 1: Kalman observer (predict + correct) -----------------
    z_pred = A_aug*z_est + B_aug*u_prev + E_aug*T_amb + Bsol_aug*Solar;
    z_est  = z_pred + L*(y_meas - C_aug*z_pred);
    x_hat  = z_est(1:nx);
    d_hat  = z_est(end);

    %% -- Stage 2: Economic steady-state target ------------------------
    [ss_sol, flag_ss] = target_calc{Price, d_hat, T_amb, Solar, Tmin, Tmax};
    if flag_ss ~= 0
      warning('Target calculator failed -- applying fallback.');
      x_ss = x_hat;  u_ss = u_prev;
    else
      ss_sol = full(ss_sol);
      x_ss   = ss_sol(1:nx);
      u_ss   = ss_sol(nx+1);
    end

    %% -- Stage 3: Economic MPC regulator (deviation form) -------------
    [du, flag_mpc] = mpc_reg{x_hat, x_ss, u_ss, Tmin_fut, Tmax_fut,
  Price_fut};
    if flag_mpc ~= 0
      warning('MPC regulator failed -- applying steady-state input.');
```

```
      du = 0;
    else
      du = full(du);
    end

    %% -- Saturate and store ----------------------------------------
    u      = min(max(u_ss + du, u_min), u_max);
    u_prev = u;
  end
```

Listing 9.5: `mpc_controller_step.m` — complete economic offset-free MPC pipeline following the general pattern of Section 9.1.3.

9.2.7 Step 5 — Building the Simulink Model

With all four helper files in place, the Simulink model requires only standard block types and three MATLAB Function wrapper blocks. The complete signal flow is shown in Figure 9.4.

Solver Configuration

Open the *Configuration Parameters* dialogue (`Ctrl+E`) and apply the settings in Table 9.2. Selecting the discrete (no continuous states) solver is essential because all dynamics are managed inside the helper files via persistent variables.

Table 9.2: Simulink solver configuration for the single-zone building temperature control model.

Parameter	Value	Reason
Stop time	`172800`	48 h $\times$ 3600 s/h
Solver type	Fixed-step	Discrete controller
Solver	`discrete (no continuous states)`	States in persistent vars
Fixed-step size	`1800`	T_s = 30 min

Block Placement and Wrapper Code

Table 9.3 lists every block in the model. Each of the three MATLAB Function blocks must contain the corresponding wrapper from Listing 9.6.

```
%% ---- ProfileGen block ----------------------------------------
function [T_amb, Solar, Price, Tmin, Tmax, Tmin_fut, Tmax_fut, Price_fut]
  = ...
ProfileGen(t)
  coder.extrinsic('get_profiles');
  T_amb = 0;  Solar = 0;  Price = 0;  Tmin = 0;  Tmax = 0;
  Tmin_fut  = zeros(1,24);  Tmax_fut  = zeros(1,24);
  Price_fut = zeros(1,24);
  [T_amb, Solar, Price, Tmin, Tmax, Tmin_fut, Tmax_fut, Price_fut] = ...
  get_profiles(t);
end

```

Table 9.3: Simulink blocks required for the single-zone building temperature control model.

Label	Library path	Key parameter
`Clock`	Sources > Clock	Decimation: 1
`ProfileGen`	User-Defined Functions > MATLAB Function	Wrapper: Listing 9.6
`Plant`	User-Defined Functions > MATLAB Function	Wrapper: Listing 9.6
`AddNoise`	Math Operations > Sum	Signs: ++
`MeasNoise`	Sources > Random Number	Mean: 0, Var: 0.01, T_s: 1800
`UnitDelay`	Discrete > Unit Delay	T_s: 1800, IC: 0
`Controller`	User-Defined Functions > MATLAB Function	Wrapper: Listing 9.6
`Saturation`	Discontinuities > Saturation	Lower: 0, Upper: 2000
`ToWorkspace`	Sinks > To Workspace	One block per logged signal

```
%% ---- Plant block ------------------------------------------------
function y_true = Plant(u, T_amb, Solar, t)
  coder.extrinsic('plant_step');
  y_true = 0;
  y_true = plant_step(u, T_amb, Solar, t);
end

%% ---- Controller block ----------------------------------------
function u = Controller(y_meas, T_amb, Solar, Price, ...
  Tmin, Tmax, Tmin_fut, Tmax_fut, Price_fut)
  coder.extrinsic('mpc_controller_step');
  u = 0;
  u = mpc_controller_step(y_meas, T_amb, Solar, Price, ...
  Tmin, Tmax, Tmin_fut, Tmax_fut, Price_fut);
end
```

Listing 9.6: Wrapper code for the three MATLAB Function blocks. Each wrapper pre-allocates its outputs, declares the helper as extrinsic, and forwards its inputs — following the pattern of Section 9.1.2.

9.2.8 Step 6 — Running the Simulation and Post-Processing

Pre-Run Procedure

Before every simulation run, reset all persistent variables by executing the following in the MATLAB Command Window:

```
clear plant_step mpc_controller_step
```

Then press `Ctrl+T` in Simulink. On the first time step the Command Window displays:

```
Building YALMIP optimizers -- please wait...
Optimizers ready.
```

The first step takes approximately 10–30 s; all subsequent steps complete in under 50 ms.

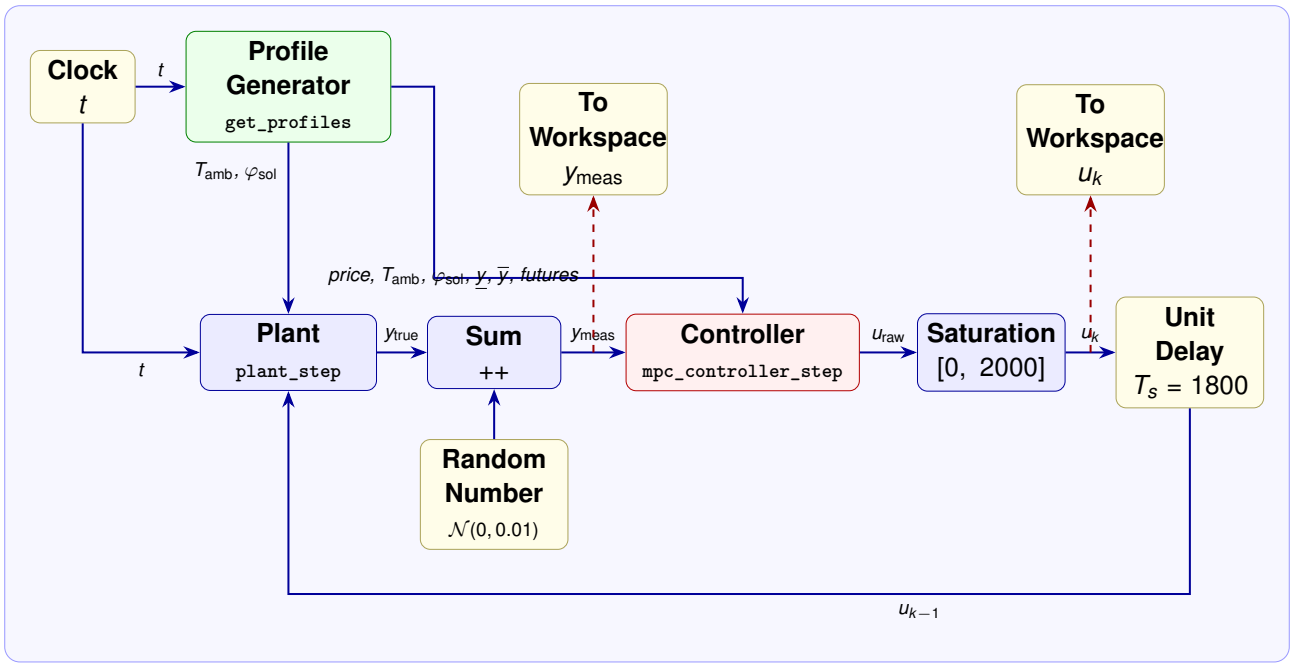


Figure 9.4: Complete conceptual Simulink signal flow for the single-zone building economic MPC. The Clock output t is passed both to the Profile Generator (for exogenous profiles) and to the Plant block (to compute the true occupancy schedule). The Unit Delay on the feedback path breaks the algebraic loop by supplying u_{k-1} to the plant, consistent with the observer prediction step.

Logging and Plotting

Connect *To Workspace* blocks to y_{true}, y_{meas}, u, Price, $\underline{y}$, and $\bar{y}$, saving each as a MATLAB array. After the simulation use the script below to produce a four-panel result plot matching Figure 8.6.

```
Ts    = 1800;
tout  = (0:length(log_y)-1) * Ts / 3600;    % hours

figure('Color','w');
tiledlayout(4,1,'TileSpacing','compact','Padding','compact');

nexttile;
plot(tout, log_y,     'b',  'LineWidth', 1.5); hold on;
plot(tout, log_ymeas,'.', 'Color',[0.85 0.33 0.10]);
plot(tout, log_tmin, 'k--','LineWidth', 1.2);
plot(tout, log_tmax, 'k--','LineWidth', 1.2);
grid on; ylabel('Temperature ($^\circ$C)');
legend('Operative $y_k$','Measured','$\underline{y}$','$\bar{y}$', ...
'Interpreter','latex','Location','best');
title('Operative temperature vs comfort bounds');

nexttile;
yyaxis left;  stairs(tout, log_u, 'LineWidth', 1.5);
ylabel('Heater power (W)');
yyaxis right; stairs(tout, log_price, '--','LineWidth', 1.2);
ylabel('Price (\pounds/kWh)');
grid on; title('Heater power and electricity price');

nexttile;
plot(tout, log_d_true, 'k--','LineWidth', 1.2); hold on;
plot(tout, log_d_est,  'b',  'LineWidth', 1.5);
```

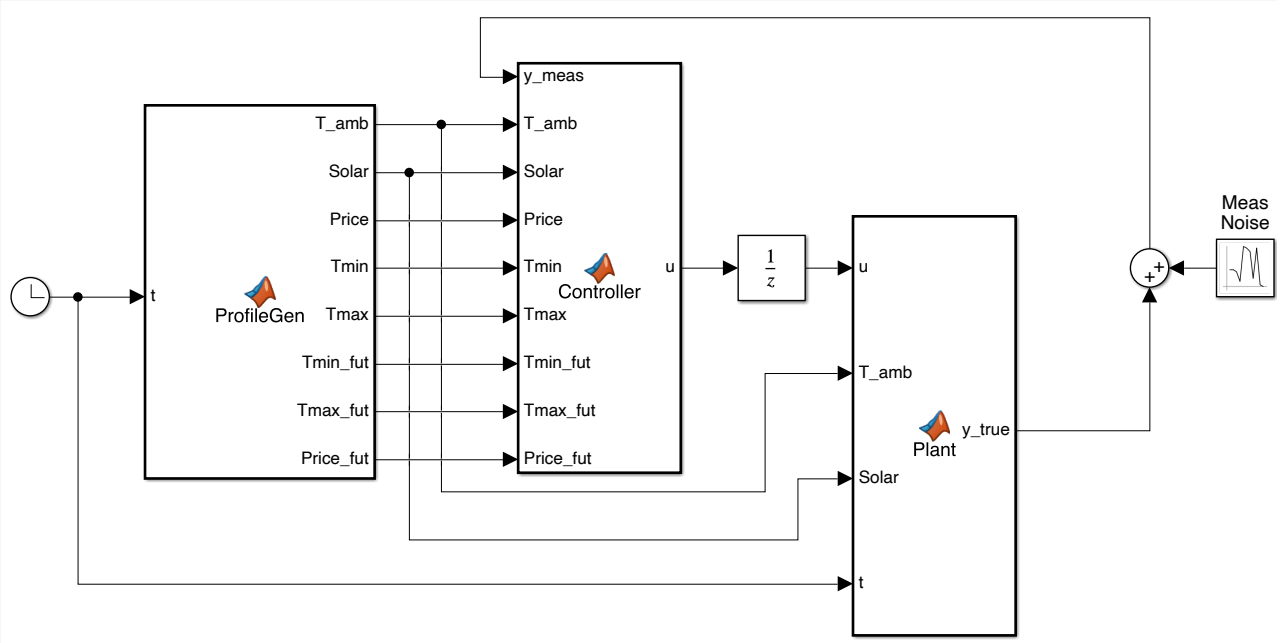


Figure 9.5: HVAC Simulation using SIMULINK.

```
grid on; ylabel('Disturbance ($^\circ$C/step)');
legend('True occupancy $d_k$','KF estimate $\hat{d}_k$', ...
'Interpreter','latex','Location','best');
title('Occupancy disturbance estimation');

nexttile;
stairs(tout, cumsum(log_cost), 'LineWidth', 1.5);
grid on; xlabel('Time (hours)');
ylabel('Cumulative cost (\pounds)');
title(sprintf('Cumulative heating cost  (total: \pounds %.3f)', sum(
 log_cost)));

set(findall(gcf,'-property','FontSize'),'FontSize',13);
```

Listing 9.7: Post-simulation plotting script for the single-zone building case study.

Table 9.4: Common issues and remedies for the Simulink single-zone MPC implementation.

Symptom	Remedy
`Function not found: get_profiles`	Helper files are not on the MATLAB path. Run `addpath(pwd)` or place all `.m` files alongside `EconomicMPC.slx`.
`Undefined function 'sdpvar'`	YALMIP is not installed or not on the path. Run `yalmiptest` before starting Simulink.
Temperature starts from wrong value on second run	Persistent variables were not cleared. Run `clear plant_step mpc_controller_step` before pressing *Run*.
Algebraic loop warning	Unit Delay block (T_s = 1800, IC = 0) is missing on the Saturation → Plant path. See Section 9.1.6.
Port dimension mismatch on `Tmin_fut` or `Price_fut`	Pre-allocation lines `Tmin_fut = zeros(1,24)` and `Price_fut = zeros(1,24)` must appear *before* the extrinsic call in the ProfileGen wrapper.
Controller uses wrong disturbance level at night	Confirm the augmented model includes the correct B_w column and that `d_hat` (not `d_filt`) is passed to the target calculator; the single-zone example does not apply exponential smoothing.
First step takes over 60 s	Normal on low-end hardware; YALMIP is building QP matrices symbolically. Do not interrupt the simulation.

9.2.9 Summary

Simulink MPC Without the MPC Toolbox — Key Steps

1. **Helper files.** Place all YALMIP and solver logic in ordinary `.m` files on the MATLAB path (Layer 2).
2. **Persistent state.** Store the observer state, last applied input, filtered disturbance, and compiled `optimizer` objects as `persistent` variables.
3. **One-time compilation.** Call `build_mpc_optimizers` on the first step only, guarded by `if isempty(...)`.
4. **Wrapper blocks.** Each Simulink MATLAB Function block (Layer 1) declares its helper as `coder.extrinsic`, pre-allocates outputs, and calls the helper.
5. **Break the algebraic loop.** Insert a Unit Delay (T_s = 1800, IC = 0) between Saturation and Plant.
6. **Reset before each run.** Execute `clear plant_step mpc_controller_step` before pressing *Run*.

Exercises

1. **Persistent variables in Simulink MATLAB Function blocks.** In the single-zone building example, the function `mpc_controller_step` declares several `persistent` variables (e.g. the observer state $\hat{x}$, the last applied input u_{k-1}, and the compiled `optimizer` object).

   (a) Explain why ordinary local variables would fail to preserve state across successive

Simulink time steps.

(b) What would happen if the compiled `optimizer` object were *not* stored as a persistent variable? Discuss the implications for both correctness and computational cost.

(c) Describe the role of the `if isempty(...)` guard that protects the one-time initialisation of persistent variables. Why is this pattern preferred over, for example, using a separate Simulink *InitFcn* callback?

2. **The role of `coder.extrinsic`.** Simulink MATLAB Function blocks are processed by MATLAB Coder, which requires statically typed, code-generation–compatible functions.

   (a) Explain why YALMIP functions (such as `sdpvar`, `optimize`, and `optimizer`) cannot be compiled by MATLAB Coder.

   (b) Describe what the `coder.extrinsic` directive does and how it allows YALMIP-based helper functions to be called from within a Simulink MATLAB Function block.

   (c) The chapter recommends pre-allocating every output variable with `zeros` *before* the call to the extrinsic helper. Explain why this pre-allocation step is necessary and what error would occur without it.

3. **Adding a rate-of-change constraint to the building controller.** The MPC formulation in Section 9.2 enforces box constraints $0 \leq u_k \leq 2000\,\mathrm{W}$ on the heater input but does not limit how quickly the input may change between consecutive steps. Suppose you wish to add the constraint $|\Delta u_k| = |u_k - u_{k-1}| \leq \Delta u_{\max}$ with $\Delta u_{\max} = 500\,\mathrm{W}$.

   (a) Describe which file(s) in the two-layer architecture you would need to modify and what YALMIP constraints you would add inside the prediction-horizon loop.

   (b) Explain why the persistent variable `u_prev` (the last applied input) must be passed as a *parameter* to the `optimizer` object so that the rate constraint can reference the transition from u_{k-1} to the first predicted input u_0.

   (c) Discuss qualitatively how adding a tight rate-of-change limit would affect the closed-loop temperature response during morning warm-up, when a large step in heating power would otherwise be applied.

4. **The algebraic-loop problem and the Unit Delay solution.** Section 9.1 explains that a direct connection from the MPC controller output to the plant input, and from the plant output back to the controller, creates an algebraic loop.

   (a) Draw a simplified signal-flow diagram showing the loop and explain, in your own words, why Simulink cannot determine a consistent set of signals at a single time step when the loop is present.

   (b) Explain how inserting a Unit Delay block (with sample time $T_s = 1800\,\mathrm{s}$ and initial condition $u_0 = 0$) breaks the algebraic loop. What is the physical interpretation of applying the control action one sample period later?

   (c) In practice, MPC already computes the control move for the *current* time step. Argue whether the one-step delay introduced by the Unit Delay block materially degrades closed-loop performance for a slowly varying building thermal system with a 30-minute sample time.

5. **Designing a Simulink MPC architecture for a DC motor.** Consider a DC motor whose angular velocity ω is governed by the simplified continuous-time transfer function

$$G(s) = \frac{\omega(s)}{V(s)} = \frac{K}{\tau s + 1},$$

where V is the armature voltage, $K = 10\,\mathrm{rad\,s^{-1}\,V^{-1}}$, and $\tau = 0.1\,\mathrm{s}$. The input is subject to $0 \leq V \leq 24\,\mathrm{V}$, and the objective is to track a reference speed $\omega_{\mathrm{ref}}(t)$.

(a) Discretise the plant with a suitable sample time (justify your choice) and write down the resulting state-space matrices A_d, B_d, C_d, D_d.

(b) Sketch a Simulink block diagram following the two-layer architecture from this chapter. Label each block (*Reference Generator*, *MPC Controller*, *DC Motor Plant*, *Scope / To Workspace*) and indicate where a Unit Delay block should be placed.

(c) List the persistent variables you would declare inside the `mpc_controller_step` function for this system. Identify any differences from the building-heating case study (e.g. is a disturbance observer still needed?).

(d) Briefly discuss how the much shorter time constant of the DC motor ($\tau = 0.1\,\mathrm{s}$) compared with the building ($\tau \approx 10^4\,\mathrm{s}$) affects the choice of prediction horizon and the computational burden of solving the QP at each step.

Chapter 10

Robust MPC and Tube MPC$^{(*)}$

> *"Standard MPC tells you the best thing to do if the world is perfect. Tube MPC tells you what to do when it isn't."*
> — paraphrasing **David Q. Mayne**

Every MPC formulation we have seen so far relies on a perfect model: $x_{k+1} = Ax_k + Bu_k$. The controller solves an optimisation, applies the first input, and re-solves at the next step. This works well in practice — *provided the model is accurate*. In practice, however, systems are always subject to disturbances: sensor noise, wind gusts, friction that the model ignores, or small inaccuracies in the A and B matrices themselves. Even a tiny modelling error, accumulated over many steps, can push the real state across a constraint boundary that the controller thought was safe.

This chapter addresses the question: **how do we guarantee constraint satisfaction when the model is not perfect?** The answer is *robust MPC*, and the most practical and widely-used form is called **Tube MPC**, introduced by Langson et al. [9] and further developed by Mayne et al. [12]. The central idea is straightforward: instead of planning a single trajectory, we plan a "tube" of trajectories. As long as the disturbance is bounded, the real trajectory stays inside the tube, and the tube is designed so that constraints are never violated.

10.1 The Problem: Disturbances and Constraint Violations

Consider the linear system with an additive disturbance:

$$x_{k+1} = Ax_k + Bu_k + w_k \tag{10.1}$$

where $x_k \in \mathbb{R}^n$ is the state, $u_k \in \mathbb{R}^m$ is the control input, and $w_k \in \mathbb{R}^n$ is a disturbance. We assume:

- The disturbance is bounded: $w_k \in \mathcal{W}$ for all $k \geq 0$, where $\mathcal{W}$ is a known convex polytope containing the origin (e.g., $\|w_k\|_\infty \leq \delta$ for some known $\delta > 0$).
- We do *not* know the specific value of w_k in advance. We only know it will lie somewhere in $\mathcal{W}$.

The system is subject to constraints:

$$x_k \in \mathcal{X}, \qquad u_k \in \mathcal{U}, \qquad \text{for all } k \geq 0$$

Why Standard MPC Fails

Standard MPC ignores w_k and plans as if $x_{k+1} = Ax_k + Bu_k$. At each step, it re-measures the state and re-solves, which provides some natural robustness. However, this is not enough to *guarantee* constraint satisfaction.

Suppose the MPC plans a trajectory that passes close to a constraint boundary — say $x_1 \leq 5$. The controller predicts $x_1 = 4.95$ and decides this is safe. But after the input is applied, the disturbance w_0 adds an unexpected +0.1 to the state, producing $x_1 = 5.05$. The constraint is violated.

Standard MPC cannot prevent this because it does not account for the "worst case" during planning. It optimistically assumes the prediction is exact.

The core difficulty

Standard MPC guarantees constraints are satisfied for the *predicted* trajectory. But when disturbances act, the *actual* trajectory deviates from the prediction. No amount of re-solving at the next step can undo a constraint violation that has already occurred.

10.2 Two Approaches to Robust MPC

Two main approaches exist for handling bounded disturbances.

10.2.1 Min–Max MPC

The most direct approach formulates the problem as a game between the controller and an adversary (nature). The controller chooses inputs u to minimise the cost, while nature chooses disturbances $w \in \mathcal{W}$ to maximise it:

$$\min_{u_0,\ldots,u_{N-1}} \max_{w_0,\ldots,w_{N-1}\in\mathcal{W}} J(x, u)$$

subject to the dynamics (10.1) and the constraints $x_k \in \mathcal{X}$, $u_k \in \mathcal{U}$ for all possible disturbance realisations.

This is mathematically rigorous but has serious drawbacks:

- **Computational explosion.** The disturbance branches at every time step. Over N steps with d possible disturbance vertices, the number of scenarios grows as d^N. For even modest problems, this becomes intractable.

- **Conservatism.** The controller assumes nature is adversarial and always picks the worst disturbance. In reality, disturbances are often random, not adversarial. The resulting controller is overly cautious.

Min–max MPC is therefore rarely used in practice. It is included here for completeness and to motivate the more practical alternative.

10.2.2 Tube MPC: The Key Idea

Tube MPC avoids the computational explosion by separating the problem into two parts:

1. A **nominal trajectory** z_k that is planned as if there were no disturbance. This is a standard MPC problem.
2. A **feedback controller** $-K(x_k - z_k)$ that keeps the real state x_k close to the nominal trajectory z_k, despite the disturbance.

The deviation $e_k = x_k - z_k$ between the real and nominal state is bounded because K is stabilising and the disturbance is bounded. The set of all possible deviations forms a "tube" around the nominal trajectory. By tightening the constraints on the nominal trajectory (shrinking them inward by the tube width), we guarantee that the real trajectory satisfies the original constraints.

This is the essence of Tube MPC: **plan the nominal path conservatively, and let a feedback controller handle the disturbances.**

10.3 Set Operations: Minkowski Sum and Pontryagin Difference

Before developing Tube MPC formally, we need two set operations that appear throughout the theory. These are not difficult, but they are essential.

10.3.1 Minkowski Sum

Definition: Minkowski Sum

The **Minkowski sum** of two sets P and Q in $\mathbb{R}^n$ is:

$$P \oplus Q = \{p + q \mid p \in P,\ q \in Q\}$$

In words: take every point in P, add every point in Q, and collect all the results. The resulting set is "P inflated by Q".

Geometric intuition. Place a copy of Q at every point of P. The Minkowski sum is the union of all these copies. If P is a square and Q is a small disk, then $P \oplus Q$ is the square with rounded corners. Figure 10.1 illustrates this operation.

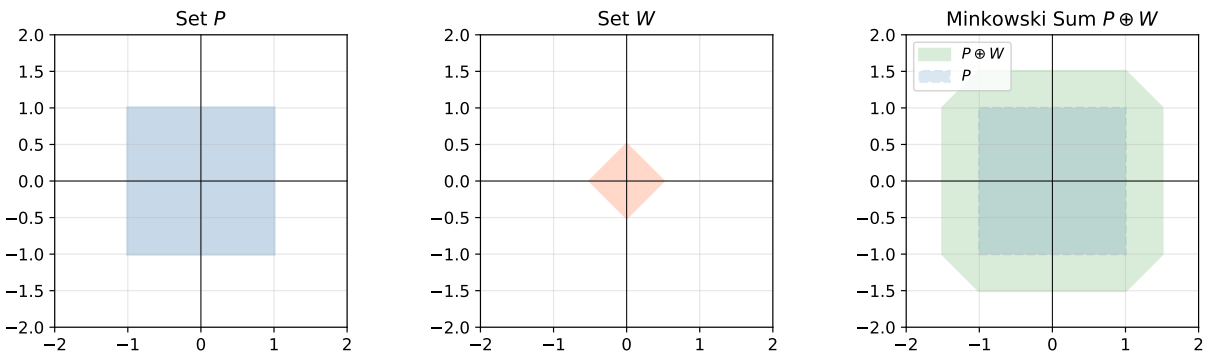


Figure 10.1: Minkowski sum of a square P and a diamond W. The result $P \oplus W$ is larger than P, "inflated" by the shape of W.

Why it matters for Tube MPC. If the nominal state is z_k and the error e_k can be anywhere in a set $\mathcal{E}$, then the actual state $x_k = z_k + e_k$ can be anywhere in $\{z_k\} \oplus \mathcal{E}$. The tube around the nominal trajectory is exactly this: the Minkowski sum of each nominal point with the error set.

10.3.2 Pontryagin Difference

Definition: Pontryagin Difference

The **Pontryagin difference** (also called the *erosion*) of a set $\mathcal{X}$ by a set $\mathcal{E}$ is:

$$\mathcal{X} \ominus \mathcal{E} = \{z \in \mathbb{R}^n \mid z + e \in \mathcal{X} \text{ for all } e \in \mathcal{E}\}$$

In words: $\mathcal{X} \ominus \mathcal{E}$ contains all points z such that *every possible shift by an element of* $\mathcal{E}$ still lands inside $\mathcal{X}$.

Geometric intuition. The Pontryagin difference is the "shrunk" version of $\mathcal{X}$. If you stand at a point $z \in \mathcal{X} \ominus \mathcal{E}$ and someone pushes you by any amount in $\mathcal{E}$, you still remain inside $\mathcal{X}$. It is the opposite of the Minkowski sum: while $\oplus$ inflates a set, $\ominus$ shrinks it. See Figure 10.2 for an illustration.

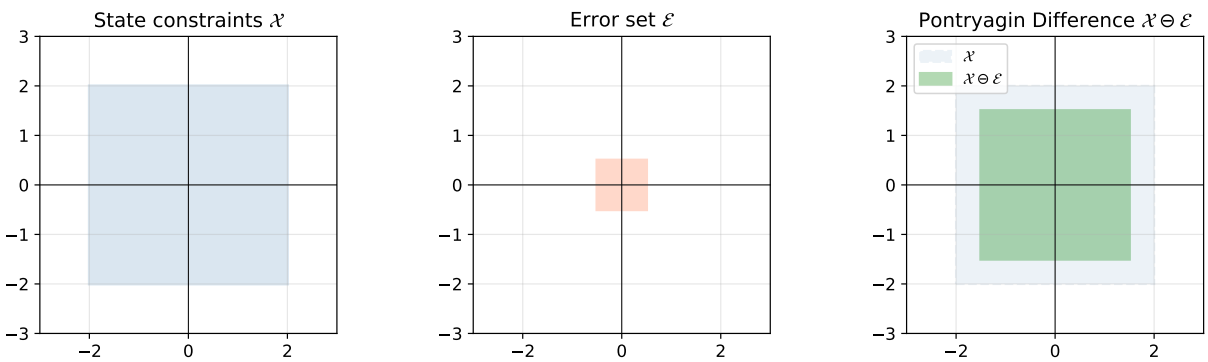


Figure 10.2: Pontryagin difference $\mathcal{X} \ominus \mathcal{E}$. The original constraint set $\mathcal{X}$ (dashed blue) is shrunk inward by the size of $\mathcal{E}$ (red), yielding the tightened set (solid green). Any point in the tightened set remains inside $\mathcal{X}$ even after a worst-case shift from $\mathcal{E}$.

Why it matters for Tube MPC. To guarantee that the real state $x_k = z_k + e_k$ satisfies $x_k \in \mathcal{X}$

for all $e_k \in \mathcal{E}$, we must require the nominal state to satisfy $z_k \in \mathcal{X} \ominus \mathcal{E}$. This is called *constraint tightening*.

Example 10.1: Minkowski Sum and Pontryagin Difference in 1D

Let $\mathcal{X} = [-5,\ 5]$ (the state must stay between -5 and 5) and $\mathcal{E} = [-0.3,\ 0.3]$ (the error is at most 0.3 in magnitude).

Solution:

Minkowski sum: $\mathcal{X} \oplus \mathcal{E} = [-5.3,\ 5.3]$. The set grows by the error width on each side.

Pontryagin difference: $\mathcal{X} \ominus \mathcal{E} = [-4.7,\ 4.7]$. The set shrinks by 0.3 on each side. If the nominal state z stays within $[-4.7, 4.7]$, then $x = z + e$ stays within $[-5, 5]$ for any $|e| \leq 0.3$.

This is constraint tightening: the nominal trajectory must respect the stricter bounds ± 4.7, so that the actual trajectory respects the original bounds ± 5.

10.4 Tube MPC: The Full Development

We now develop Tube MPC step by step.

10.4.1 Step 1: Decompose the System

The uncertain system is $x_{k+1} = Ax_k + Bu_k + w_k$. We introduce a *nominal* system that evolves without disturbance:

$$z_{k+1} = Az_k + Bv_k \tag{10.2}$$

where z_k is the nominal state and v_k is the nominal input. These are the decision variables that the MPC optimiser will compute.

We define the *error* between the real and nominal states:

$$e_k = x_k - z_k$$

The control input is split into two parts:

$$u_k = v_k - K(x_k - z_k) = v_k - Ke_k \tag{10.3}$$

where:

- v_k is the **nominal input**, computed by the MPC optimiser. It steers the nominal trajectory z_k toward the origin.
- K is a **fixed feedback gain** (typically the LQR gain from `dlqr`, so $K > 0$ and $u = -Kx$ stabilises the system). It keeps the real state close to the nominal state by rejecting disturbances.

10.4.2 Step 2: Derive the Error Dynamics

Substituting (10.3) into the real dynamics (10.1) and subtracting the nominal dynamics (10.2):

$$\begin{aligned} x_{k+1} &= Ax_k + B(v_k - Ke_k) + w_k \\ z_{k+1} &= Az_k + Bv_k \end{aligned}$$

Subtracting:

$$\boxed{e_{k+1} = (A - BK)\, e_k + w_k} \tag{10.4}$$

This is the key result. The error dynamics are *autonomous*: they do not depend on v_k at all. The nominal input cancels out. The error evolves purely under the closed-loop matrix $A_{cl} = A - BK$ and the disturbance w_k.

Since K is chosen so that A_{cl} is Schur stable (all eigenvalues inside the unit circle), the error e_k does not grow without bound. But it does not converge to zero either, because the disturbance w_k keeps pushing it around. Instead, e_k settles into a bounded set that we can compute.

10.4.3 Step 3: The Minimal Robust Positively Invariant Set

Definition: Robust Positively Invariant (RPI) Set

A set $\mathcal{E} \subseteq \mathbb{R}^n$ is a **robust positively invariant (RPI) set** for the error dynamics $e_{k+1} = A_{cl}\, e_k + w_k$ with $w_k \in \mathcal{W}$ if:

$$e_k \in \mathcal{E} \implies e_{k+1} \in \mathcal{E} \quad \text{for all } w_k \in \mathcal{W}$$

Once the error enters $\mathcal{E}$, it stays in $\mathcal{E}$ forever, no matter what disturbances occur.

The smallest such set is the **minimal RPI (mRPI) set**, denoted $\mathcal{E}_\infty$ [16]. It can be characterised explicitly:

$$\mathcal{E}_\infty = \bigoplus_{i=0}^{\infty} A_{cl}^i \mathcal{W} \;=\; \mathcal{W} \oplus A_{cl}\mathcal{W} \oplus A_{cl}^2\mathcal{W} \oplus \cdots \tag{10.5}$$

where $\oplus$ denotes the Minkowski sum. In words: take the disturbance set $\mathcal{W}$, apply the closed-loop dynamics to get $A_{cl}\mathcal{W}$, then $A_{cl}^2\mathcal{W}$, and so on. Each set is smaller than the previous one (because A_{cl} is stable and shrinks vectors). The mRPI is the Minkowski sum of all these sets.

Why is this a valid invariant set? If $e_k \in \mathcal{E}_\infty$, then $e_{k+1} = A_{cl}\, e_k + w_k$. Since A_{cl} maps $\mathcal{E}_\infty$ into $\bigoplus_{i=1}^{\infty} A_{cl}^i\, \mathcal{W}$, and adding $w_k \in \mathcal{W}$ gives $\mathcal{W} \oplus \bigoplus_{i=1}^{\infty} A_{cl}^i\, \mathcal{W} = \mathcal{E}_\infty$, the set is invariant.

Practical computation. The infinite sum (10.5) is computed by iterating:

$$\begin{aligned} \Omega_0 &= \mathcal{W} \\ \Omega_{i+1} &= \Omega_i \oplus A_{cl}^{i+1}\mathcal{W} \end{aligned}$$

and stopping when $\Omega_{i+1} \approx \Omega_i$ (the additional term $A_{cl}^{i+1}\mathcal{W}$ becomes negligibly small because A_{cl} is stable). Figure 10.3 illustrates this convergence.

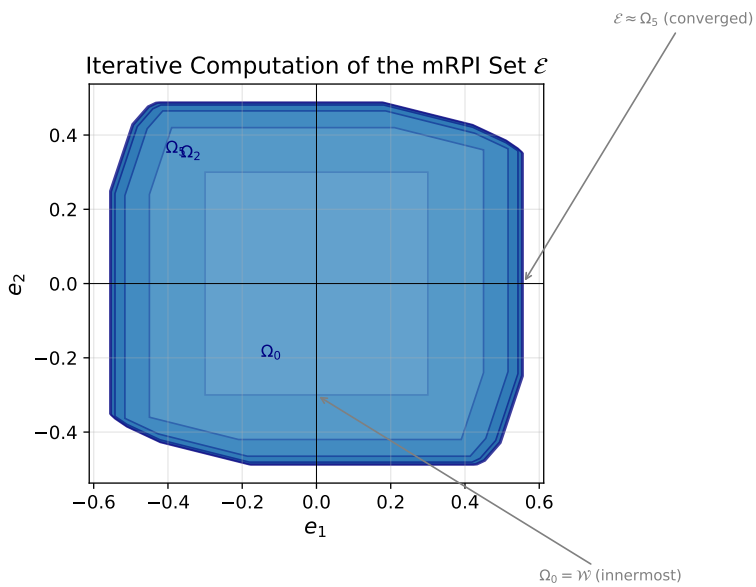


Figure 10.3: Iterative computation of the mRPI set. Starting from $\Omega_0 = \mathcal{W}$ (innermost set), each iteration adds a smaller and smaller set $A_{cl}^i \mathcal{W}$. The sets converge to the mRPI set $\mathcal{E}_\infty$.

The role of K. A faster stabilising gain K (one that places the eigenvalues of A_{cl} closer to zero) produces a smaller mRPI set $\mathcal{E}_\infty$, because $A_{cl}^i \mathcal{W}$ shrinks faster. A smaller tube means less constraint tightening, which means a larger feasible region for the nominal MPC. However, aggressive gains require larger control inputs, which tightens the input constraints more. There is a trade-off in choosing K.

Example 10.2: Computing the mRPI Set for a Scalar System

Consider the scalar system $x_{k+1} = 1.2\,x_k + u_k + w_k$ with $|w_k| \leq 0.1$ and $|u_k| \leq 1$. The stabilising gain is $K = 0.79$ (as returned by `dlqr`, so $u = -Kx$), giving $a_{cl} = 1.2 - 0.79 = 0.41$.

Solution:

The mRPI set is the interval $\mathcal{E}_\infty = [-\bar{e},\ \bar{e}]$, where:

$$\bar{e} = \sum_{i=0}^{\infty} |a_{cl}|^i \cdot 0.1 = \frac{0.1}{1 - |a_{cl}|} = \frac{0.1}{1 - 0.41} = \frac{0.1}{0.59} \approx 0.169$$

So the error $e_k = x_k - z_k$ is guaranteed to stay within $[-0.169,\ 0.169]$, no matter what disturbance sequence occurs (as long as $|w_k| \leq 0.1$).

Constraint tightening. If the original input constraint is $|u_k| \leq 1$, the tightened input constraint for v_k is:

$$|v_k| \leq 1 - |K| \cdot \bar{e} = 1 - 0.79 \times 0.169 \approx 0.867$$

If the state constraint is $|x_k| \leq 5$, the tightened state constraint for z_k is:

$$|z_k| \leq 5 - 0.169 = 4.831$$

The nominal MPC plans with these tighter bounds, and the feedback controller $u_k = v_k - K(x_k - z_k)$ takes care of the rest.

10.5 Step 4: Constraint Tightening

This is the central mechanism that makes Tube MPC robust. Since the real state $x_k = z_k + e_k$ can deviate from the nominal state z_k by any amount in $\mathcal{E}$, we must ensure the constraints hold even in the worst case.

State constraints. We need $x_k = z_k + e_k \in \mathcal{X}$ for all $e_k \in \mathcal{E}$. This is equivalent to requiring:

$$z_k \in \mathcal{X} \ominus \mathcal{E} \ =: \ \mathcal{X}_t \tag{10.6}$$

The tightened state constraint set $\mathcal{X}_t$ is the original constraint set shrunk by the tube width.

Input constraints. The actual input is $u_k = v_k - Ke_k$. We need $u_k \in \mathcal{U}$ for all $e_k \in \mathcal{E}$. This gives:

$$v_k \in \mathcal{U} \ominus K\mathcal{E} \ =: \ \mathcal{V} \tag{10.7}$$

where $K\mathcal{E} = \{Ke \mid e \in \mathcal{E}\}$ is the set of all possible feedback corrections.

Why does this work? By construction:

- If $z_k \in \mathcal{X}_t$ and $e_k \in \mathcal{E}$, then $x_k = z_k + e_k \in \mathcal{X}$. (Definition of $\ominus$.)
- If $v_k \in \mathcal{V}$ and $e_k \in \mathcal{E}$, then $u_k = v_k - Ke_k \in \mathcal{U}$. (Definition of $\ominus$.)

The original hard constraints on (x_k, u_k) are *guaranteed* to be satisfied, regardless of which disturbance $w_k \in \mathcal{W}$ occurs. The price we pay is that the nominal trajectory has less room: the feasible region shrinks by the tube width on all sides.

How much room do we lose?

The amount of constraint tightening depends directly on the size of $\mathcal{E}$, which in turn depends on:

- **The disturbance bound** $\mathcal{W}$: Larger disturbances $\Rightarrow$ larger $\mathcal{E}$ $\Rightarrow$ more tightening.
- **The feedback gain** K: A faster gain shrinks $\mathcal{E}$ (less tightening of state constraints) but enlarges $K\mathcal{E}$ (more tightening of input constraints). A slower gain has the opposite effect.

In the extreme case where $\mathcal{W} = \{0\}$ (no disturbance), we get $\mathcal{E} = \{0\}$, and the tightened constraints equal the original constraints. Tube MPC reduces to standard MPC.

10.6 Step 5: The Tube MPC Optimisation Problem

At each time step k, we measure the current state x_k and solve the following optimisation over the nominal trajectory (z, v):

$$\begin{aligned}
\min_{z_0,\dots,z_N,\ v_0,\dots,v_{N-1}} \quad & z_N^\top P\, z_N + \sum_{i=0}^{N-1} (z_i^\top Q\, z_i + v_i^\top R\, v_i) \\
\text{subject to:} \quad & z_{i+1} = Az_i + Bv_i, \qquad i = 0, \dots, N-1 \\
& z_i \in \mathcal{X}_t = \mathcal{X} \ominus \mathcal{E}, \qquad i = 0, \dots, N-1 \\
& v_i \in \mathcal{V} = \mathcal{U} \ominus K\mathcal{E}, \qquad i = 0, \dots, N-1 \\
& z_N \in \mathcal{X}_f \\
& x_k \in z_0 \oplus \mathcal{E}
\end{aligned} \tag{10.8}$$

The key differences from standard MPC are:

1. **The optimisation is over the nominal system** (z, v), not the real system (x, u). The nominal system has no disturbance, so this is a standard QP — exactly the same type of problem we solved in Chapter 7.

2. **The constraints are tightened.** The state and input constraints are shrunk inward by the tube width to account for the worst-case disturbance.

3. **The initial condition links the nominal and real systems.** The constraint $x_k \in z_0 \oplus \mathcal{E}$ ensures the nominal starting point z_0 is close enough to the measured state x_k that the error $e_0 = x_k - z_0$ lies within $\mathcal{E}$. In practice, this is often implemented by fixing $z_0 = x_k$ (which satisfies the constraint since $0 \in \mathcal{E}$), or by optimising over z_0 (which can improve performance by choosing a better tube centre).

4. **The terminal set** $\mathcal{X}_f$ is a positive invariant set for the nominal system under the tightened constraints, analogous to the terminal set in Chapter 7. It must satisfy $\mathcal{X}_f \subseteq \mathcal{X}_t$.

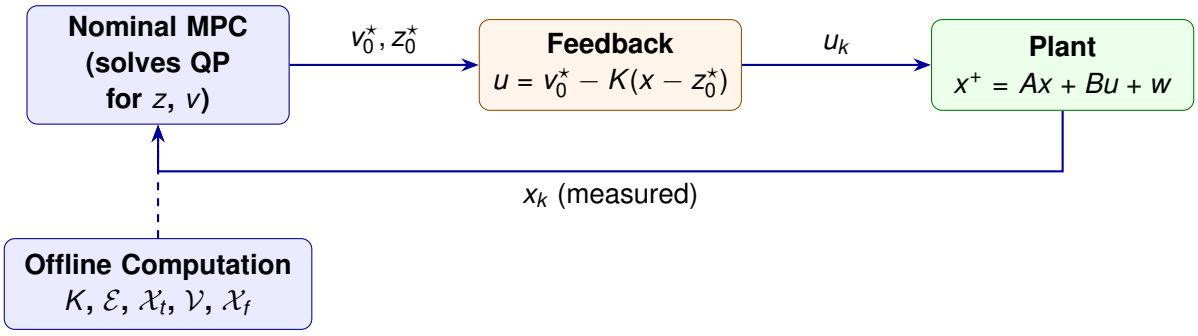
After solving (10.8), the control input applied to the real system is:

$$u_k = v_0^\star - K(x_k - z_0^\star) \tag{10.9}$$

At the next time step, we re-measure x_{k+1} and solve again.

10.7 Tube MPC Architecture

The complete Tube MPC controller has the following structure:



The offline computation (feedback gain K, mRPI set $\mathcal{E}$, tightened constraints $\mathcal{X}_t$ and $\mathcal{V}$, and the terminal set $\mathcal{X}_f$) is performed once during the design phase. At run time, the controller only needs to solve the nominal QP (same size as standard MPC) and add the feedback correction.

10.8 Worked Example: Tube MPC for a Scalar System

Example 10.3: Tube MPC: Scalar Integrator with Disturbance

Consider the system:

$$x_{k+1} = 1.2\, x_k + u_k + w_k$$

with $|u_k| \leq 1$, $|x_k| \leq 5$, $|w_k| \leq 0.1$, and initial condition $x_0 = 2$.

Design a Tube MPC controller with $Q = 1$, $R = 1$, $N = 5$.

Solution:

Step 1: Choose K and compute the closed-loop eigenvalue.

Using the DARE with $Q = 1$, $R = 1$: $K = 0.79$ (positive, as returned by `dlqr`), so $u = -Kx \approx -0.79x$, giving $a_{cl} = a - bK = 1.2 - 0.79 = 0.41$.

Step 2: Compute the mRPI set.

$$\bar{e} = \frac{w_{\max}}{1 - |a_{cl}|} = \frac{0.1}{1 - 0.41} = \frac{0.1}{0.59} \approx 0.169$$

So $\mathcal{E} = [-0.169,\ 0.169]$.

Step 3: Tighten constraints.

$$\begin{aligned} \text{State:} \quad & |z_k| \leq 5 - 0.169 = 4.831 \\ \text{Input:} \quad & |v_k| \leq 1 - |K| \cdot \bar{e} = 1 - 0.79 \times 0.169 \approx 0.867 \end{aligned}$$

Step 4: Solve the nominal MPC.

At each time step, solve the standard MPC problem (10.8) for the disturbance-free system $z_{k+1} = 1.2\, z_k + v_k$ with the tightened constraints $|z_k| \leq 4.831$ and $|v_k| \leq 0.867$.

Step 5: Apply the tube control law.

$$u_k = v_k^\star - K(x_k - z_k^\star) = v_k^\star - 0.79(x_k - z_k^\star)$$

Figure 10.4 shows 50 different disturbance realisations. The blue traces are the actual state trajectories; the red dashed line is the nominal trajectory. All realisations remain within the constraint bounds, despite the disturbances.

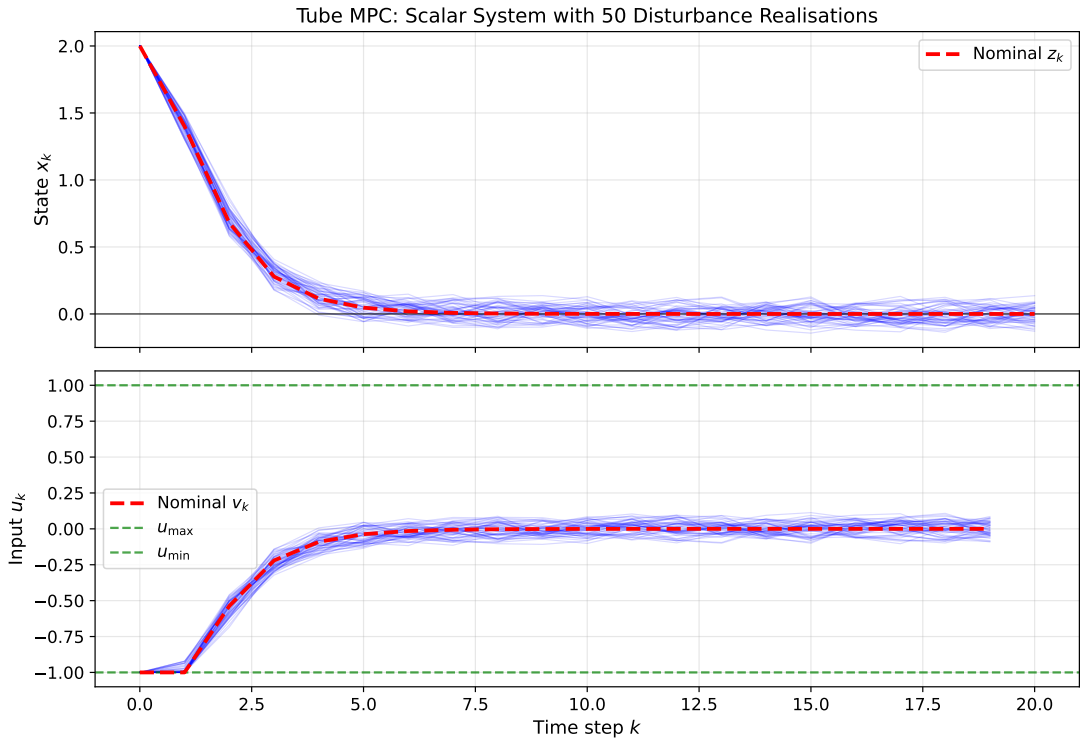


Figure 10.4: Tube MPC applied to the scalar system $x_{k+1} = 1.2\,x_k + u_k + w_k$ with $|w_k| \leq 0.1$. Fifty disturbance realisations (blue) cluster around the nominal trajectory (red dashed). All trajectories satisfy the input constraints $|u_k| \leq 1$.

10.9 Worked Example: Two-State System

Example 10.4: Tube MPC: Double Integrator with Disturbance

Consider the double integrator:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}, \quad w_k \in \mathcal{W} = \{w \mid \|w\|_\infty \leq 0.05\}$$

with $|u_k| \leq 1$, $|x_{1,k}| \leq 5$, $|x_{2,k}| \leq 3$, and $Q = I_2$, $R = 1$.

Solution:

Step 1: Compute LQR gain.

Solving the DARE gives $K = [0.4345 \quad 1.0285]$ (positive, as returned by `dlqr`) and $P = \begin{bmatrix} 2.3671 & 1.1180 \\ 1.1180 & 2.5875 \end{bmatrix}$.

The closed-loop matrix is:

$$A_{cl} = A - BK = \begin{bmatrix} 1 - 0.2172 & 1 - 0.5142 \\ 0 - 0.4345 & 1 - 1.0285 \end{bmatrix} = \begin{bmatrix} 0.7828 & 0.4858 \\ -0.4345 & -0.0285 \end{bmatrix}$$

The eigenvalues of A_{cl} are approximately $0.377 \pm 0.216j$, with $|\lambda| \approx 0.434$, well inside the unit circle.

Step 2: Compute the mRPI set $\mathcal{E}$.

This requires the iterative Minkowski sum (10.5). For a two-state system, this is not practical by hand. Using MPT3 in MATLAB:

```
A = [1 1; 0 1]; B = [0.5; 1];
Q = eye(2); R = 1;
[K_lqr, ~] = dlqr(A, B, Q, R);
Acl = A - B*K_lqr;

% Disturbance set
W = Polyhedron('lb', [-0.05; -0.05], 'ub', [0.05; 0.05]);

% Compute mRPI set (iterative Minkowski sum approximation)
% Note: invariantSet() computes the MAXIMAL positive invariant set,
% which is NOT the mRPI. The mRPI is computed via the iteration below.
Omega = W;
for i = 1:30
    Omega_new = Omega + (Acl^i) * W;  % Minkowski sum
    if Omega_new == Omega              % convergence check
        fprintf('mRPI converged at iteration %d\n', i);
        break;
    end
    Omega = Omega_new;
end
E = Omega;
```

Step 3: Tighten constraints.

```
X = Polyhedron('lb', [-5; -3], 'ub', [5; 3]);
U = Polyhedron('lb', -1, 'ub', 1);
KE = K_lqr * E;            % Image of E under K (set of |Ke| values)

Xt = X - E;                % Pontryagin difference
Vt = U - KE;               % Pontryagin difference
```

Step 4: Solve nominal MPC with tightened constraints.

The optimisation (10.8) is a standard QP with the same structure as Chapter 7. The only difference is that $\mathcal{X}, \mathcal{U}$ are replaced by $\mathcal{X}_t, \mathcal{V}$.

Step 5: Apply. $u_k = v_0^\star - K(x_k - z_0^\star)$.

10.10 Guarantees of Tube MPC

Tube MPC provides three key guarantees, under the same type of assumptions as the nominal MPC stability theory (Chapter 7, Section 7.7).

Theorem: Properties of Tube MPC

Suppose (A, B) is stabilisable, K is chosen so that $A_{cl} = A - BK$ is Schur stable, and the MPC formulation (10.8) includes a terminal cost P (DARE solution) and terminal set $\mathcal{X}_f$ for the nominal system. Then, for any disturbance sequence $\{w_k\} \subseteq \mathcal{W}$:

1. **Robust constraint satisfaction:** The actual state and input satisfy $x_k \in \mathcal{X}$ and $u_k \in \mathcal{U}$ for all $k \geq 0$.
2. **Recursive feasibility:** If the optimisation (10.8) is feasible at $k = 0$, it is feasible for all $k \geq 0$.
3. **Robust stability:** The actual state converges to a neighbourhood of the origin. Specifically, x_k converges to the mRPI set $\mathcal{E}$ (not to the origin itself, because the persistent disturbance prevents exact convergence).

The first property follows directly from constraint tightening. The second uses the same shifted-sequence argument as in Chapter 7, applied to the nominal system. The third uses the nominal optimal cost $J^\star(z)$ as a Lyapunov function for the nominal system and then bounds the deviation of the real state.

Note a subtle difference from the nominal case: the state does not converge to zero. It converges to the mRPI set $\mathcal{E}$ around the origin. This is the best we can hope for with persistent disturbances — we cannot drive the state to exactly zero when the disturbance keeps pushing it around.

10.11 Tube MPC vs Standard MPC: A Comparison

	Standard MPC	**Tube MPC**
Model	$x_{k+1} = Ax_k + Bu_k$	$x_{k+1} = Ax_k + Bu_k + w_k$
Disturbance	Ignored	Bounded: $w_k \in \mathcal{W}$
Constraint guarantee	Nominal only	Robust (for all $w \in \mathcal{W}$)
Online computation	Solve QP	Solve QP (same size) + feedback
Offline computation	$P, K, \mathcal{X}_f$	$P, K, \mathcal{E}, \mathcal{X}_t, \mathcal{V}, \mathcal{X}_f$
Convergence	$x_k \to 0$	$x_k \to \mathcal{E}$ (neighbourhood of 0)
Feasible region	Larger	Smaller (due to tightening)

The main takeaway: Tube MPC achieves robust guarantees at essentially no additional online cost. The extra work is in the offline design (computing $\mathcal{E}$ and tightening constraints), not in the real-time optimisation. The QP solved at each step has exactly the same size and structure as standard MPC.

10.12 Implementation in MATLAB

The following MATLAB code implements a complete Tube MPC simulation for the scalar system from Section 10.8, using YALMIP.

```
%% Tube MPC: Scalar System
% x_{k+1} = 1.2*x_k + u_k + w_k,  |w_k| <= 0.1
clear; clc;

%--- System parameters ---
a = 1.2;  b = 1;
Q = 1;    R = 1;    N = 5;

%--- Constraints ---
x_max = 5;   u_max = 1;   w_max = 0.1;

%--- Step 1: LQR gain ---
[K_lqr, P_dare] = dlqr(a, b, Q, R);
K = K_lqr;          % K > 0 (dlqr convention); control law: u = -K*x
a_cl = a - b*K;     % closed-loop eigenvalue: a_cl = a - b*K
fprintf('K = %.4f, a_cl = %.4f\n', K, a_cl);

%--- Step 2: mRPI set (scalar: geometric series) ---
e_bar = w_max / (1 - abs(a_cl));
fprintf('mRPI half-width: e_bar = %.4f\n', e_bar);

%--- Step 3: Tighten constraints ---
x_max_t = x_max - e_bar;              % tightened state
u_max_t = u_max - abs(K) * e_bar;     % tightened input
fprintf('Tightened: |z| <= %.3f, |v| <= %.3f\n', x_max_t, u_max_t);

```

```
%--- Step 4: Build nominal MPC with YALMIP ---
% z(1) is a decision variable (optimised), not fixed to x(k).
% Constraint: |x(k) - z(1)| <= e_bar (i.e., x(k) in {z(1)} + E).
z = sdpvar(1, N+1);   % nominal states (z(1) = z_0 is optimised)
v = sdpvar(1, N);      % nominal inputs
x_meas = sdpvar(1);   % parameter: measured state

constraints = [];
cost = 0;
for i = 1:N
    cost = cost + z(i)^2 * Q + v(i)^2 * R;
    constraints = [constraints, ...
        z(i+1) == a*z(i) + b*v(i), ...
        -x_max_t <= z(i) <= x_max_t, ...
        -u_max_t <= v(i) <= u_max_t];
end
cost = cost + z(N+1)^2 * P_dare;  % terminal cost
constraints = [constraints, -x_max_t <= z(N+1) <= x_max_t];

% Tube constraint: x(k) must lie in {z_0} + E
constraints = [constraints, -e_bar <= x_meas - z(1) <= e_bar];

params_in = x_meas;           % parameter: measured state
sols_out = [v(1), z(1)];      % solutions: first nominal input AND z_0

ops = sdpsettings('verbose', 0, 'solver', 'quadprog');
controller = optimizer(constraints, cost, ops, params_in, sols_out);

%--- Step 5: Simulate ---
T_sim = 30;
x = zeros(1, T_sim+1);
z_traj = zeros(1, T_sim+1);
u_applied = zeros(1, T_sim);

x(1) = 2.0;       % initial condition

rng(42);
for k = 1:T_sim
    % Solve nominal MPC (z_0 is optimised, not fixed to x(k))
    sol = controller(x(k));
    v0 = sol(1);
    z0 = sol(2);

    % Apply tube control law: u = v - K*(x - z)
    % Note: z0 ~= x(k) in general, so the feedback term is nonzero
    u_applied(k) = v0 - K * (x(k) - z0);
    u_applied(k) = max(-u_max, min(u_max, u_applied(k)));

    % Simulate real system with disturbance
    w = w_max * (2*rand - 1);   % uniform in [-w_max, w_max]
    x(k+1) = a * x(k) + b * u_applied(k) + w;

    % Store nominal trajectory for plotting
    z_traj(k) = z0;
    z_traj(k+1) = a * z0 + b * v0;
end

%--- Plot ---
figure;
subplot(2,1,1);
```

```
plot(0:T_sim, x, 'b-', 'LineWidth', 1.5); hold on;
plot(0:T_sim, z_traj, 'r--', 'LineWidth', 1.5);
yline(x_max, 'g--'); yline(-x_max, 'g--');
ylabel('State'); legend('Actual x_k', 'Nominal z_k', 'Constraint');
title('Tube MPC: Scalar System'); grid on;

subplot(2,1,2);
stairs(0:T_sim-1, u_applied, 'b-', 'LineWidth', 1.5);
yline(u_max, 'g--'); yline(-u_max, 'g--');
xlabel('Time step k'); ylabel('Input u_k');
legend('Applied input', 'Constraint'); grid on;
```

Listing 10.1: Tube MPC implementation for the scalar system

10.13 Computing the mRPI Set with MPT3

For multi-state systems, computing the mRPI set by hand is impractical. The MPT3 toolbox [5] provides automated tools for this. The key steps are:

```
%% Tube MPC Offline Design using MPT3
% System: x+ = Ax + Bu + w,  w in W

A = [1 1; 0 1];    B = [0.5; 1];
Q = eye(2);         R = 1;

% LQR gain
[K_lqr, P_dare] = dlqr(A, B, Q, R);
Acl = A - B*K_lqr;

% Define constraint sets as Polyhedra
X = Polyhedron('lb', [-5; -3], 'ub', [5; 3]);      % state
U = Polyhedron('lb', -1, 'ub', 1);                  % input
W = Polyhedron('lb', [-0.05; -0.05], ...
               'ub', [0.05; 0.05]);                  % disturbance

% --- Compute the mRPI set ---
% Iterative Minkowski sum: E = W + Acl*W + Acl^2*W + ...
E = W;
for i = 1:30
    E_new = E + Acl^i * W;    % Minkowski sum
    if E_new == E              % convergence check
        fprintf('mRPI converged at iteration %d\n', i);
        break;
    end
    E = E_new;
end

% --- Tighten constraints ---
KE  = K_lqr * E;          % image of E under gain K (set of |Ke| values)
Xt  = X - E;              % Pontryagin difference: X ominus E
Vt  = U - KE;             % Pontryagin difference: U ominus KE

% --- Compute terminal set for nominal system ---
sys_nom = LTISystem('A', A - B*K_lqr);
sys_nom.x.min = Xt.Internal.lb;
sys_nom.x.max = Xt.Internal.ub;
Xf = sys_nom.invariantSet();
```

```
% --- Extract half-space representation for QP ---
% Tightened state: Xt.A * z <= Xt.b
% Tightened input: Vt.A * v <= Vt.b
% Terminal:        Xf.A * z_N <= Xf.b

fprintf('Tightened state constraints: %d inequalities\n', size(Xt.A, 1));
fprintf('Tightened input constraints: %d inequalities\n', size(Vt.A, 1));
fprintf('Terminal set: %d inequalities\n', size(Xf.A, 1));
```

Listing 10.2: Computing the mRPI set and tightened constraints using MPT3

Approximate mRPI for large systems

For systems with more than 3–4 states, the exact mRPI computation can produce polytopes with a very large number of faces, making the resulting QP expensive. In practice, an outer approximation $\hat{\mathcal{E}} \supseteq \mathcal{E}_\infty$ is often used instead [16]. A common approach is to stop the iteration after a fixed number of steps s and scale the result slightly outward: $\hat{\mathcal{E}} = (1-\epsilon)^{-1}\,\Omega_s$ for a small $\epsilon > 0$ chosen so that $A_{cl}^{s+1}\mathcal{W} \subseteq \epsilon\,\Omega_s$. This guarantees that $\hat{\mathcal{E}}$ is a true RPI set while keeping the number of polytope faces manageable.

10.14 Visualising the Tube

Figure 10.5 illustrates the key concept. The nominal trajectory z_k (red dashed) is planned by the MPC optimiser. The actual trajectory x_k (blue solid) deviates due to disturbances but stays within the grey "tube" — the Minkowski sum $\{z_k\} \oplus \mathcal{E}$ at each step. The nominal trajectory respects the tightened constraints (orange lines), which guarantees the actual trajectory respects the original constraints (green dashed lines).

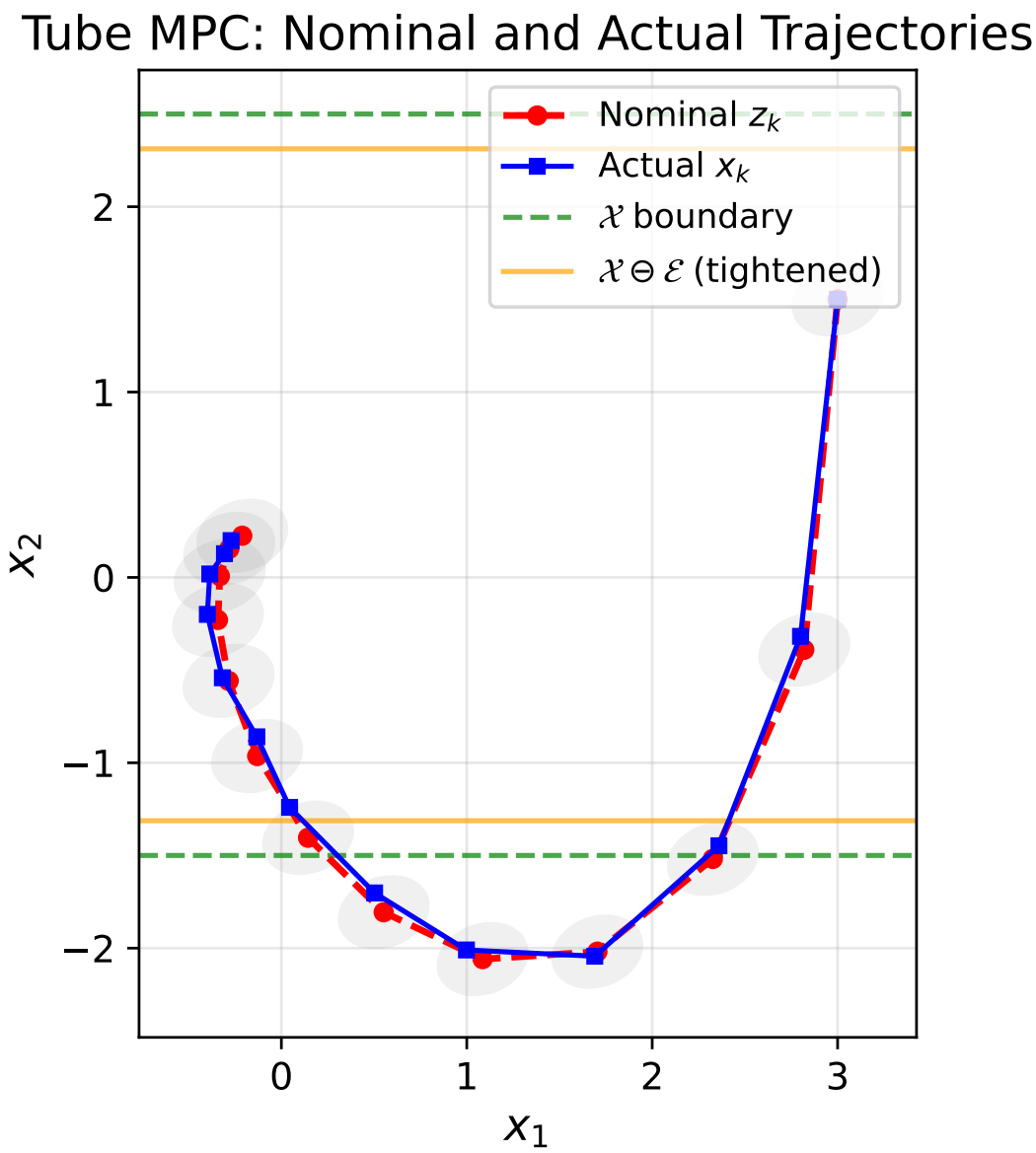


Figure 10.5: The tube concept in state space. Grey ellipses represent the cross-section $\mathcal{E}$ of the tube at each time step. The nominal trajectory (red) stays within the tightened constraints (orange), ensuring the actual trajectory (blue) stays within the original constraints (green dashed).

10.15 Summary and Practical Guidelines

Tube MPC Design Recipe

1. **Model the disturbance.** Identify the disturbance set $\mathcal{W}$ from physical knowledge or data. Larger $\mathcal{W}$ gives more robustness but tighter constraints.
2. **Choose the feedback gain K.** Use LQR or pole placement. Faster gains give a smaller tube (less constraint tightening) but require more control effort (more input constraint tightening). There is a design trade-off.
3. **Compute the mRPI set $\mathcal{E}$.** Use the iterative Minkowski sum (10.5) or MPT3. For scalar systems, use the geometric series formula.
4. **Tighten constraints.** State: $\mathcal{X}_t = \mathcal{X} \ominus \mathcal{E}$. Input: $\mathcal{V} = \mathcal{U} \ominus K\mathcal{E}$. Check that the tightened sets are non-empty; if they are empty, the disturbance is too large relative to the constraints, and robust constraint satisfaction is impossible.
5. **Design the nominal MPC.** Solve a standard MPC problem with the tightened constraints. All the techniques from Chapter 7 apply (terminal cost P, terminal set $\mathcal{X}_f$, YALMIP implementation).
6. **Apply the tube law.** At each step: $u_k = v_0^\star - K(x_k - z_0^\star)$.

When Tube MPC cannot help

If the disturbance bound $\mathcal{W}$ is so large that the tightened constraint sets $\mathcal{X}_t$ or $\mathcal{V}$ become empty, then no controller can guarantee robust constraint satisfaction. In this case, the system physically cannot respect the constraints under all possible disturbances. Options include: relaxing the constraints (soft constraints), reducing the disturbance (better actuators or shielding), or accepting occasional constraint violations and using a stochastic MPC approach instead.

Further Reading

The Tube MPC framework was introduced by Langson et al. [9] and further developed by Mayne et al. [12]. The efficient outer approximation of the mRPI set used in practice is due to Raković et al. [16]. For a comprehensive treatment of robust MPC including stochastic formulations, see Kouvaritakis and Cannon [8]. The broader MPC stability theory underpinning the guarantees discussed in this chapter is surveyed in Mayne et al. [11]. The textbooks by Rawlings et al. [17] and Borrelli et al. [2] provide detailed treatments of both nominal and robust MPC.

Exercises

1. Consider the scalar system $x_{k+1} = 1.5\,x_k + u_k + w_k$ with $|u_k| \leq 2$, $|w_k| \leq 0.2$, and $Q = R = 1$.
   (a) Compute the LQR gain K and the closed-loop eigenvalue a_{cl}.

(b) Using the geometric series formula, compute the mRPI half-width $\bar{e}$.

(c) What are the tightened input constraints for the nominal MPC?

(d) If $|x_k| \leq 10$, what are the tightened state constraints?

2. Compute the Minkowski sum $P \oplus Q$ and the Pontryagin difference $P \ominus Q$ for:

   (a) $P = [-3, 3]$, $Q = [-1, 1]$ (intervals in $\mathbb{R}$).

   (b) $P = \{(x_1, x_2) \mid |x_1| \leq 2, |x_2| \leq 2\}$ (a square) and $Q = \{(x_1, x_2) \mid |x_1| + |x_2| \leq 0.5\}$ (a diamond). Sketch the results.

3. A control engineer designs Tube MPC for a chemical process with $|w_k| \leq 0.5$ and input constraint $|u_k| \leq 3$. The LQR gain is $K = 1.8$ (positive, with $u = -Kx$) and the closed-loop eigenvalue is $a_{cl} = 0.3$.

   (a) Compute the tube width $\bar{e}$.

   (b) What is the tightened input constraint for v_k?

   (c) The engineer's colleague suggests using a more aggressive gain with $a_{cl} = 0.1$, which gives $K = 2.5$. Recompute the tube width and tightened input constraint. Is this gain better or worse? Discuss the trade-off.

4. Consider the double integrator from Section 10.9. Using MATLAB and MPT3:

   (a) Compute the mRPI set $\mathcal{E}$ for $\mathcal{W} = \{w \mid \|w\|_\infty \leq 0.05\}$.

   (b) Plot the original constraint set $\mathcal{X}$, the mRPI set $\mathcal{E}$, and the tightened constraint set $\mathcal{X}_t = \mathcal{X} \ominus \mathcal{E}$.

   (c) Repeat with $\mathcal{W} = \{w \mid \|w\|_\infty \leq 0.2\}$. How does the larger disturbance affect the feasible region?

   (d) Find the largest disturbance bound δ such that $\mathcal{X}_t$ is still non-empty (i.e., the tightened constraint set has positive volume).

5. Explain in your own words:

   (a) Why does Tube MPC guarantee convergence to a neighbourhood of the origin rather than to the origin itself?

   (b) What happens to the tube width $\mathcal{E}$ as the feedback gain K becomes more aggressive (eigenvalues of A_{cl} closer to zero)?

   (c) Under what conditions does Tube MPC reduce to standard MPC?

Chapter 11

Course Conclusion and Further Topics$^{(*)}$

This chapter concludes the introduction to Model Predictive Control. We began with the *Linear Quadratic Regulator (LQR)*, deriving the optimal feedback gain for unconstrained systems. We then introduced *Model Predictive Control (MPC)* to explicitly handle physical constraints (actuator saturation and safety limits). Finally, we bridged the "reality gap" by incorporating *State Estimation* and *Disturbance Modelling*, creating an architecture capable of offset-free tracking in the presence of noise and uncertainty.

The structure presented here—**MPC + Estimator + Target Calculator**—represents the standard architecture for modern industrial control. However, due to time constraints, we have focused on the *practical* application of these methods, necessarily omitting several advanced topics that define the forefront of control research. For students wishing to deepen their understanding, we outline the key areas and fundamental literature below.

11.1 Rigorous Stability Analysis

Throughout this course, we relied on the *Terminal Cost* ($x_N^\top P x_N$) to encourage stability. While effective in practice, this does not mathematically *guarantee* stability for constrained systems.

Rigorous MPC theory relies on Lyapunov analysis. To prove stability, one must often include a **Terminal Set Constraint** ($x_N \in \mathcal{X}_f$) in the optimisation. This forces the state to land in a *safe haven* (a Positive Invariant Set, meaning that once the trajectory enters the terminal set, it remains inside) at the end of the horizon, ensuring recursive feasibility.

- **Fundamental Reference:** Mayne, D. Q., Rawlings, J. B., Rao, C. V., & Scokaert, P. O. M. (2000). Survey: Constrained model predictive control: Stability and optimality. *Automatica*, 36(6), 789–814.

11.2 Robust MPC

Robust MPC addresses the problem of guaranteeing constraint satisfaction when the system model is not perfect. The most practical framework is **Tube MPC**, which separates the control into a nominal trajectory planner and a feedback controller that keeps the real state within a bounded "tube" around the plan. This topic is developed in full detail — including the required set operations, constraint tightening, worked examples, and MATLAB implementations — in Chapter 10.

11.3 Nonlinear MPC (NMPC)

In previous sections, we relied heavily on linearising the system dynamics ($x_{k+1} = Ax_k + Bu_k$). While effective near an equilibrium point, this approach may fail for highly nonlinear systems, such as chemical reactors with Arrhenius kinetics or agile fighter jets performing aggressive manoeuvres.

Nonlinear MPC [4] addresses this by solving the optimisation problem using the full nonlinear dynamic model directly. The general N-step finite horizon optimal control problem (OCP) solved at each time step t is formulated as:

$$\begin{aligned} \min_{u,x} \quad & J = \sum_{k=0}^{N-1} \ell(x_k, u_k) + V_f(x_N), \\ \text{subject to:} \quad & x_0 = x(t), \\ & x_{k+1} = f(x_k, u_k), && k = 0, \dots, N-1, \\ & h(x_k, u_k) \leq 0, && k = 0, \dots, N-1, \end{aligned} \tag{11.1}$$

where $\ell(\cdot)$ is the stage cost, $V_f(\cdot)$ is the terminal cost, and $f(x_k, u_k)$ represents the nonlinear discretised system dynamics (often obtained via numerical integration, such as RK4).

Challenges: Non-Convexity and Computation

Unlike linear MPC, which typically results in a Convex Quadratic Programme (QP) with a guaranteed unique global minimum, NMPC results in a **Nonlinear Programme (NLP)**. Because the equality constraints in (11.1) are nonlinear, the feasible set is often non-convex. This introduces two major difficulties:

- **Local Minima:** Solvers may converge to a local optimum rather than the global optimum, potentially resulting in suboptimal control performance.
- **Computational Cost:** Solving an NLP is iteratively expensive (requiring the computation of Jacobians and Hessians at every step). Since the optimisation must complete within the sampling time T_s, real-time feasibility is a critical bottleneck.

- **Fundamental Reference:** Chen, H., & Allgöwer, F. (1998). A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica*, 34(10), 1205–1217.

Numerical Methods and Tools

To solve these NLPs efficiently, modern NMPC implementations typically rely on *Direct Methods*, which transcribe the continuous control problem into a finite-dimensional NLP. Two popular approaches are:

- **Direct Single Shooting:** Only the control inputs u_k are optimisation variables; states are eliminated by simulating the model forward. This results in small but dense matrices.
- **Direct Multiple Shooting:** Both states x_k and controls u_k are treated as decision variables. This increases the problem size but preserves the sparsity structure and improves convergence stability, making it the preferred choice for stiff systems.

Standard solvers typically employ Sequential Quadratic Programming (SQP) or Interior Point Methods (IPM). In practice, engineers rely on specialised high-performance software frameworks rather than writing solvers from scratch:

- **CasADi:** An open-source tool for algorithmic differentiation (AD) and optimisation. It is widely used in academia for prototyping NMPC controllers in MATLAB and Python, interfacing directly with NLP solvers like **IPOPT**.
- **ACADOS:** A modular software package designed for embedded optimisation. It focuses on speed, utilising BLASFEO (linear algebra optimised for small matrices) to achieve microsecond-scale solution times suitable for robotics and automotive applications.
- **FORCES Pro:** A commercial code-generation tool tailored for high-speed embedded NMPC, often used in the automotive and aerospace industries.

Example 11.1: Case Study: Pendulum Swing-Up and Balance

In this scenario, the pendulum starts in the stable downward position ($\theta = \pi$). The NMPC controller must "pump" energy into the system to swing it up and then stabilise it at the unstable upright equilibrium ($\theta = 0$).

Problem Formulation:

The objective is to minimise the quadratic cost function subject to the nonlinear dynamics and constraints. We define the state vector $x = [\theta, \dot{\theta}]^\top = [x_1, x_2]^\top$ and input u.

The optimisation problem is formulated as:

$$\min_{x,u} \quad J = \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R u_k) + x_N^\top P x_N$$

$$\text{subject to: } \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ \frac{g}{L}\sin(x_1) + u \end{bmatrix},$$

$$-5 \leq u_k \leq 5,$$

$$-2\pi \leq x_{1,k} \leq 2\pi$$

where $Q = \text{diag}(50, 0.1)$, $R = 0.1$, and the terminal weight $P = \text{diag}(1000, 10)$. The notation RK4$(\cdot)$ denotes the explicit Runge-Kutta 4 integration of the continuous dynamics over the sampling interval Δt.

Implementation Details:

The horizon is selected as $N = 40$ (2.0s) to allow the solver to plan the momentum build-up required for the swing-up manoeuvre. Figure 11.1 shows the simulation results.

```
clear; clc;
import casadi.*

opti = casadi.Opti();

N  = 40;      % Horizon (2.0s)
dt = 0.05;    % Sampling time

X = opti.variable(2, N+1); % States x = [theta; theta_dot]

U = opti.variable(1, N);    % Input u

x_init = opti.parameter(2, 1);

g = 9.81; L = 1;

f_continuous = @(x,u) [x(2); (g/L)*sin(x(1)) + u];

rk4_step = @(x,u,h) rk4_integrator(x, u, h, f_continuous);

opti.subject_to(X(:,1) == x_init);

objective = 0;
Q = diag([50, 0.1]);  R = 0.1; P = diag([1000, 10]);

for k = 1:N
  x_next = rk4_step(X(:,k), U(k), dt);
  opti.subject_to(X(:,k+1) == x_next);
  opti.subject_to(-5 <= U(k) <= 5);
  opti.subject_to(-2*pi <= X(1,k) <= 2*pi);

  objective = objective + X(:,k)'*Q*X(:,k) + U(k)'*R*U(k);
end

```

```
    objective = objective + X(:,N+1)'*P*X(:,N+1);

    opti.minimize(objective);

    p_opts = struct('expand',true);
    s_opts = struct('max_iter',2000, 'print_level',0, 'tol',1e-4);
    opti.solver('ipopt', p_opts, s_opts);

    M = opti.to_function('M', {x_init}, {U(:,1)});

    T_sim = 200;
    x_curr = [pi; 0]; % Start hanging DOWN (pi)

    x_hist = zeros(2, T_sim);
    u_hist = zeros(1, T_sim);

    fprintf('Swinging up...\n');
    for t = 1:T_sim
      u_val = full(M(x_curr));
      x_curr = full(rk4_step(x_curr, u_val, dt));
      x_hist(:, t) = x_curr;
      u_hist(t) = u_val;
    end

    function x_next = rk4_integrator(x, u, h, f)
      k1 = f(x, u);
      k2 = f(x + h/2 * k1, u);
      k3 = f(x + h/2 * k2, u);
      k4 = f(x + h * k3, u);
      x_next = x + (h/6) * (k1 + 2*k2 + 2*k3 + k4);
    end

```

Listing 11.1: Pendulum Swing-Up using CasADi.

11.4 Real-Time Implementation and Embedded MPC

In our examples, we used solvers such as MATLAB's `quadprog` or the open-source solver IPOPT (accessed via CasADi or other interfaces), which are sufficient for offline simulations but insufficient for millisecond-level control on microcontrollers or DSPs. Deploying MPC on embedded hardware requires specialised techniques:

- **Explicit MPC (Multiparametric Programming) [1]:** For small systems, we can pre-calculate the solution offline. The result is a giant "lookup table" of linear gains. On the chip, the "optimisation" reduces to simple `if-else` checks to find the correct region, requiring microseconds to execute.
- **Code Generation:** Tools like ACADO, CVXGEN, or OSQP generate highly optimised, library-free C-code specifically tailored to the structure of the MPC problem, allowing convex MPC to run on standard automotive ECUs.

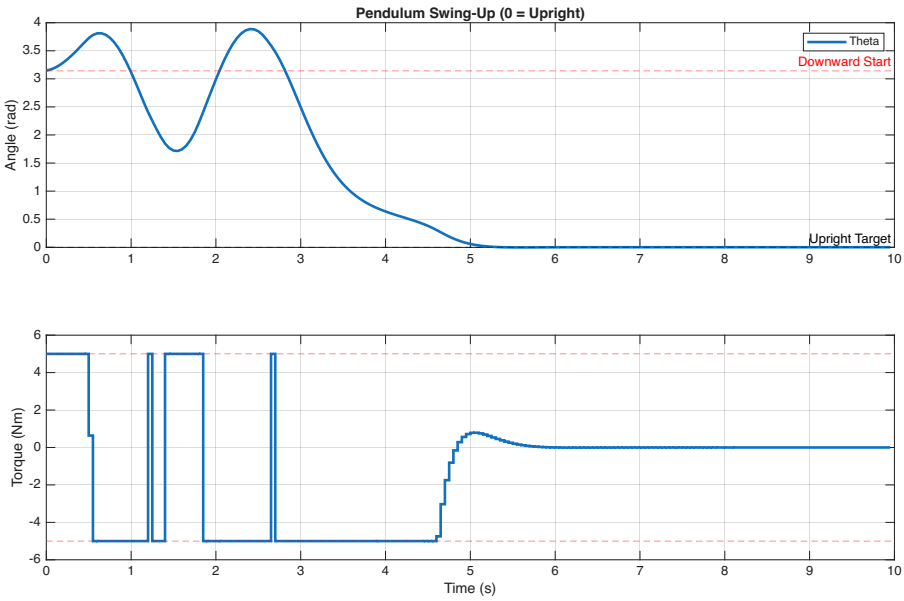


Figure 11.1: Simulation results for the NMPC pendulum swing-up manoeuvre. The top panel shows the angle trajectory $\theta(t)$ starting from the stable downward position ($\theta = \pi$ rad) and settling at the unstable upright equilibrium ($\theta = 0$ rad). The bottom panel displays the control torque $u(t)$. Note how the controller utilises the full actuator authority (saturating at ± 5 Nm) to 'pump' energy into the system before smoothly stabilising the pendulum after approximately 5 seconds.

- **Fundamental Reference:** Bemporad, A., Morari, M., Dua, V., & Pistikopoulos, E. N. (2002). The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1), 3–20.

11.5 Data-Driven and Learning-Based MPC

A major bottleneck in implementing MPC is obtaining an accurate mathematical model (A, B or $f(x, u)$). Deriving models from first principles (Newton's laws, thermodynamics) is time-consuming and often fails to capture complex phenomena like aerodynamic drag or tire friction.

Modern research addresses this through three primary paradigms: **Model Learning**, where the dynamic equations are approximated from data; **Direct Data-Driven Control**, where the controller relies directly on raw data trajectories; and **Iterative Learning**, where the control policy itself is refined over repeated trials.

11.5.1 Learning Dynamics: Gaussian Processes and Neural Networks

Instead of relying on a fixed physical model, we can approximate the system dynamics using function approximators [6]. A popular approach is to combine a simple nominal model with a learned residual term:

$$x_{k+1} = \underbrace{Ax_k + Bu_k}_{\text{Nominal Physics}} + \underbrace{g(x_k, u_k)}_{\text{Learned Residual}}$$

- **Gaussian Processes (GPs):** GP-MPC models $g(\cdot)$ as a distribution over functions. The key advantage is that a GP provides both a prediction mean $\mu(x, u)$ and a variance $\Sigma(x, u)$.

This uncertainty quantification allows the controller to be *cautious* in unexplored regions of the state space (linking naturally to Robust MPC).

- **Neural Networks (NNs):** Deep Neural Networks can approximate highly complex nonlinearities. However, embedding a non-convex NN directly into the OCP constraints makes the optimisation difficult.

- **Fundamental Reference:** Kocijan, J., Murray-Smith, R., Rasmussen, C. E., & Girard, A. (2004). Gaussian process model based predictive control. *Proceedings of the 2004 American Control Conference*, pp. 2214–2219, vol.3, Boston, MA.

11.5.2 Linear Embeddings: Koopman and DMD

To avoid the computational burden of non-convex optimisation required by NNs, techniques like **Dynamic Mode Decomposition (DMD)** and the **Koopman Operator** seek to *lift* the nonlinear system into a higher-dimensional space where the dynamics behave linearly.

Given a discrete-time nonlinear system $x_{k+1} = f(x_k)$, we define a set of observable functions $\psi(x)$ (e.g., $[x, x^2, \sin(x), \dots]^\top$). In this lifted space $z = \psi(x)$, the evolution is linear:

$$z_{k+1} = \mathcal{K} z_k$$

where $\mathcal{K}$ is the Koopman matrix identified from data. This allows the use of fast, convex Quadratic Programming (QP) solvers (standard Linear MPC tools) to control fundamentally nonlinear systems.

- **Fundamental Reference:** Korda, M., & Mezić, I. (2018). Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93, 149–160.

11.5.3 Direct Data-Driven Control (Data-Enabled MPC)

While the methods above use data to identify a model (indirect approach), **Data-Enabled MPC (DeePC)** [3] skips the system identification step entirely. It relies on *Willems' Fundamental Lemma*, which states that for a linear time-invariant system, any valid future trajectory can be represented as a linear combination of sufficiently rich past data trajectories.

Hankel Matrix Construction

To apply Willems' Lemma, we construct a Hankel matrix from a single, sufficiently rich pre-recorded trajectory of inputs u^d and outputs y^d of length T_D. We define a window length $L = T_{ini} + N$, where T_{ini} is the length of past data required to fix the initial state, and N is the prediction horizon.

The data is arranged into Hankel matrices and partitioned explicitly into **Past** (p) and **Future** (f) blocks:

$$\mathcal{H}(u^d) = \left[\begin{array}{cccc} u_0^d & u_1^d & \cdots & u_{T_D-L}^d \\ \vdots & \vdots & \ddots & \vdots \\ u_{T_{ini}-1}^d & u_{T_{ini}}^d & \cdots & u_{T_D-N-1}^d \\ \hline u_{T_{ini}}^d & u_{T_{ini}+1}^d & \cdots & u_{T_D-N}^d \\ \vdots & \vdots & \ddots & \vdots \\ u_{L-1}^d & u_L^d & \cdots & u_{T_D-1}^d \end{array}\right] = \left[\begin{array}{c} U_p \\ \hline U_f \end{array}\right]$$

The output matrix $\mathcal{H}(y^d)$ is partitioned identically into Y_p and Y_f.

The DeePC Optimisation

The optimisation variables are the future control sequence u and the operator vector g, which selects a linear combination of the columns to construct a valid trajectory. The Optimal Control Problem is:

$$\begin{aligned} \min_{u,g,\sigma_y} \quad & \sum_{k=0}^{N-1} \|y_{f,k} - y_{ref}\|_Q^2 + \|u_{f,k}\|_R^2 + \lambda_g \|g\|^2 + \lambda_\sigma \|\sigma_y\|^2 \\ \text{subject to:} \quad & \left[\begin{array}{c} U_p \\ Y_p \\ \hline U_f \\ Y_f \end{array}\right] g = \left[\begin{array}{c} u_{\text{ini}} \\ y_{\text{ini}} \\ \hline u \\ y_f \end{array}\right] + \left[\begin{array}{c} 0 \\ \sigma_y \\ \hline 0 \\ 0 \end{array}\right] \\ & u \in \mathbb{U}, \quad y_f \in \mathbb{Y} \end{aligned}$$

Here, u_{ini} and y_{ini} are the most recent T_{ini} measurements used to fix the initial condition. The slack variable σ_y accounts for measurement noise in the initial condition, and the regularisation $\lambda_g \|g\|^2$ prevents overfitting to specific noisy data samples in the library.

- **Fundamental Reference:** Coulson, J., Lygeros, J., & Dörfler, F. (2019). Data-Enabled Predictive Control: In the Shallows of the DeePC. *European Control Conference (ECC)*, 307–312.

11.5.4 Learning MPC (LMPC)

While the methods above focus on identifying the plant, **Learning MPC** focuses on improving performance for repetitive tasks (e.g., a robot arm picking up an object or a race car driving laps).

LMPC uses trajectories from previous successful iterations (indexed by j) to construct a safe set and a terminal cost. If the task is to minimise time or energy, the controller updates its terminal cost $V_f(x)$ based on the realised cost of previous runs:

$$V_f(x) = \min_{j<\text{current}} \{J_{\text{realised}}^{(j)}(x)\}$$

Mathematically, this guarantees *recursive feasibility* and *non-increasing cost*: the controller learns to perform at least as well as its best previous attempt, eventually converging to the optimal trajectory without needing a perfect model.

- **Fundamental Reference:** Rosolia, U., & Borrelli, F. (2018). Learning Model Predictive Control for Iterative Tasks: A Data-Driven Control Framework. *IEEE Transactions on Automatic Control*, 63(9), 3018–3033.

Exercises

1. **(Nonlinear MPC: Formulating an NLP for a Pendulum)** Consider a simple pendulum with continuous-time dynamics:

$$\dot{\theta} = \omega, \qquad \dot{\omega} = \frac{g}{L}\sin(\theta) + u,$$

where θ is the angle from the upright position, ω is the angular velocity, $g = 9.81$ m/s^2, $L = 1$ m, and u is the control torque.

   (a) Discretise the dynamics using a single Euler step with sampling time $\Delta t = 0.05$ s. Write out the resulting discrete-time equations $x_{k+1} = f_d(x_k, u_k)$, where $x = [\theta, \omega]^\top$.

   (b) Formulate the NMPC optimisation problem as a Nonlinear Programme (NLP) using the direct multiple shooting approach over a horizon $N = 20$. Use cost matrices $Q = \text{diag}(10, 0.1)$, $R = 0.01$, and $P = \text{diag}(100, 1)$. The input is constrained as $|u_k| \leq 5$. Write out all decision variables, the objective function, and the full set of equality and inequality constraints.

   (c) Explain why this NLP is non-convex and discuss the practical consequences: what difficulties might a solver encounter, and what strategies can mitigate them?

2. **(Linear MPC with Successive Linearisation vs. Full Nonlinear MPC)** Consider using MPC to stabilise the pendulum from Exercise 1 at the upright equilibrium $(\theta, \omega) = (0, 0)$.

   (a) Linearise the pendulum dynamics about the upright equilibrium to obtain $x_{k+1} = Ax_k + Bu_k$. State the matrices A and B for the Euler-discretised system with $\Delta t = 0.05$ s.

   (b) Describe the *successive linearisation* (also known as Real-Time Iteration) approach: at each time step, linearise the nonlinear model about the current predicted trajectory and solve the resulting QP. Compare this to solving the full NLP at each step in terms of (i) computational cost per iteration, (ii) optimality of the solution, and (iii) the domain of attraction.

   (c) For which initial conditions would the linear MPC from part (a) likely succeed? For which would it likely fail? Justify your answer by considering the validity of the linearisation.

3. **(When Is Nonlinear MPC Necessary?)** For each of the following systems and control tasks, discuss whether a linear MPC controller (possibly with successive linearisation) would be adequate, or whether a full nonlinear MPC formulation is required. Justify each answer by identifying the key nonlinearities and assessing whether they are significant within the expected operating range.

   (a) Temperature regulation of a room using an HVAC system, where the setpoint varies between 20 ℃ and 24 ℃.

   (b) Swing-up control of an inverted pendulum from the downward equilibrium to the upright equilibrium.

(c) Altitude hold of a quadrotor at a fixed hover point in calm conditions, subject to small perturbations.

(d) Autonomous racing of a ground vehicle near the limits of tyre grip, where lateral forces saturate nonlinearly.

(e) Controlling the concentration in a continuously stirred tank reactor (CSTR) with an exothermic reaction exhibiting multiple steady states.

Appendix A

Linear Algebra: A Self-Study Reference

This appendix provides a concise but complete review of the linear algebra concepts used throughout these lecture notes. Each topic is presented with a formal definition, worked examples by hand, and the corresponding MATLAB commands so that you can verify every result interactively. No prior knowledge beyond basic school-level algebra is assumed.

All MATLAB code in this appendix can be typed directly into the MATLAB Command Window or saved as a script. Matrices are entered row by row, with semicolons separating rows and commas (or spaces) separating elements within a row.

A.1 Scalars, Vectors, and Matrices

Scalar

A **scalar** is a single real (or complex) number, denoted by a lower-case italic letter, e.g. $a = 3.7$.

Vector

A **column vector** of size n is an ordered list of n scalars arranged vertically:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

A row vector is the transpose of a column vector: $x^\top = [x_1,\ x_2,\ \dots,\ x_n]$. Unless stated otherwise, *vector* means column vector in these notes.

Matrix

An $m \times n$ **matrix** has m rows and n columns:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

The element in row i and column j is written a_{ij} or $[A]_{ij}$. A **square matrix** has $m = n$.

MATLAB notation

MATLAB uses `(i,j)` indexing for elements, starting from 1: `A(i,j)` returns a_{ij}. Size is queried with `size(A)`, which returns `[m n]`.

Example A.1: Entering matrices and vectors in MATLAB

```
% Column vector
x = [2; -1; 4];        % 3x1, semicolons separate rows

% Row vector
y = [5, 0, -3];        % 1x3, commas separate elements

% 3x3 matrix
A = [1, 2, 3;
4, 5, 6;
7, 8, 9];

% Access element in row 2, column 3
a23 = A(2,3)           % returns 6

% Query dimensions
[m, n] = size(A)       % m=3, n=3
```

Solution:

Running the code above sets `a23 = 6` and confirms A is 3×3. Vectors are simply matrices where one dimension equals 1.

A.2 Special Matrices

Several matrix types appear repeatedly in MPC formulations and are worth knowing by name.

Important Special Matrices

- **Zero matrix** $0_{m\times n}$: all entries are zero.
- **Identity matrix** I_n: square, ones on the main diagonal, zeros elsewhere; $[I_n]_{ij} = 1$ if $i = j$, else 0.
- **Diagonal matrix**: square, all off-diagonal entries are zero; written $\mathrm{diag}(d_1, d_2, \dots, d_n)$.
- **Symmetric matrix**: square, $A = A^\top$, i.e. $a_{ij} = a_{ji}$ for all i, j.
- **Positive definite matrix**: symmetric A such that $x^\top Ax > 0$ for all $x \neq 0$. Written $A \succ 0$.
- **Positive semi-definite matrix**: symmetric A such that $x^\top Ax \geq 0$. Written $A \succeq 0$.

Why positive definiteness matters in MPC

The cost matrices Q and R in the MPC objective must satisfy $Q \succeq 0$ and $R \succ 0$ for the optimisation problem to be well-posed (bounded below with a unique minimiser).

Example A.2: Special matrices in MATLAB

```
n = 4;
Z  = zeros(n);            % 4x4 zero matrix
I  = eye(n);              % 4x4 identity matrix
D  = diag([1, 2, 3, 4]);% 4x4 diagonal matrix

% Check symmetry
A  = [4, 2; 2, 3];
issymmetric(A)            % returns 1 (true)

% Check positive definiteness via eigenvalues
ev = eig(A)               % all positive => A is positive definite

```

Solution:

`eig(A)` returns `[1.4384; 5.5616]`, both positive, confirming $A \succ 0$.

A.3 Matrix Addition and Subtraction

Addition and Subtraction

Two matrices $A, B \in \mathbb{R}^{m\times n}$ of the *same size* are added (or subtracted) element by element:

$$[A \pm B]_{ij} = a_{ij} \pm b_{ij}.$$

Addition with matrices of different sizes is **not defined**.

Properties of matrix addition:

- Commutativity: $A + B = B + A$
- Associativity: $(A + B) + C = A + (B + C)$
- Identity element: $A + 0 = A$
- Distributivity over scalars: $\alpha(A + B) = \alpha A + \alpha B$

Example A.3: Matrix addition and subtraction by hand and in MATLAB

Let

$$A = \begin{bmatrix} 1 & 3 \\ -2 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 5 & -1 \\ 0 & 2 \end{bmatrix}.$$

Compute $A + B$, $A - B$, and $3A$.

Solution:

$$A + B = \begin{bmatrix} 1+5 & 3-1 \\ -2+0 & 4+2 \end{bmatrix} = \begin{bmatrix} 6 & 2 \\ -2 & 6 \end{bmatrix},$$

$$A - B = \begin{bmatrix} 1-5 & 3+1 \\ -2-0 & 4-2 \end{bmatrix} = \begin{bmatrix} -4 & 4 \\ -2 & 2 \end{bmatrix},$$

$$3A = \begin{bmatrix} 3 & 9 \\ -6 & 12 \end{bmatrix}.$$

```
A = [1, 3; -2, 4];
B = [5, -1; 0, 2];

ApB = A + B       % [6  2; -2  6]
AmB = A - B       % [-4  4; -2  2]
threeA = 3*A      % [3  9; -6  12]

```

A.4 Matrix Transpose

Transpose

The **transpose** of $A \in \mathbb{R}^{m \times n}$ is the matrix $A^\top \in \mathbb{R}^{n \times m}$ obtained by swapping rows and columns: $[A^\top]_{ij} = a_{ji}$.

Useful transpose identities:

$$(A^\top)^\top = A \tag{A.1}$$

$$(A + B)^\top = A^\top + B^\top \tag{A.2}$$

$$(AB)^\top = B^\top A^\top \tag{A.3}$$

$$(\alpha A)^\top = \alpha A^\top \tag{A.4}$$

Identity (A.3) is the most frequently used and most easily forgotten: note the *reversed* order.

Example A.4: Transpose in MATLAB

```
A  = [1, 2, 3; 4, 5, 6];   % 2x3 matrix
At = A'                      % 3x2 transpose

% Verify (AB)' = B'A'
B  = [1, 0; -1, 2; 3, 1];  % 3x2 so that AB is defined
lhs = (A*B)'
rhs = B' * A'
isequal(lhs, rhs)           % returns 1 (true)
```

Solution:

`At` is the 3×2 matrix with rows `[1 4]`, `[2 5]`, `[3 6]`. MATLAB confirms the identity.

Warning: in MATLAB, `A'` computes the *conjugate* (Hermitian) transpose. For real matrices this makes no difference. For complex matrices use `A.'` (dot-transpose) if you want the non-conjugate transpose.

A.5 Matrix–Matrix Multiplication

Matrix Multiplication

If $A \in \mathbb{R}^{m\times p}$ and $B \in \mathbb{R}^{p\times n}$ (the inner dimensions must match), their product $C = AB \in \mathbb{R}^{m\times n}$ is defined by

$$c_{ij} = \sum_{k=1}^{p} a_{ik}\, b_{kj}, \qquad i = 1, \dots, m, \quad j = 1, \dots, n.$$

In words: the (i, j) entry of C is the *dot product* of the i-th row of A with the j-th column of B.

Matrix multiplication is NOT commutative

In general $AB \neq BA$. The product BA may not even be defined if A and B are not square. Always check dimension compatibility before multiplying: an $m \times p$ matrix times a $p \times n$ matrix gives an $m \times n$ result.

Properties:

- Associativity: $(AB)C = A(BC)$
- Distributivity: $A(B + C) = AB + AC$
- Identity: $AI = IA = A$

- Scalar: $(\alpha A)B = A(\alpha B) = \alpha(AB)$

Example A.5: Matrix–matrix multiplication by hand and in MATLAB

Let

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

Compute AB and BA and confirm they differ.

Solution:

$$AB = \begin{bmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix},$$

$$BA = \begin{bmatrix} 5\cdot 1 + 6\cdot 3 & 5\cdot 2 + 6\cdot 4 \\ 7\cdot 1 + 8\cdot 3 & 7\cdot 2 + 8\cdot 4 \end{bmatrix} = \begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}.$$

Clearly $AB \neq BA$.

```
A  = [1, 2; 3, 4];
B  = [5, 6; 7, 8];

AB = A * B            % [19 22; 43 50]
BA = B * A            % [23 34; 31 46]
isequal(AB, BA)       % returns 0 (false) -- not commutative

```

Example A.6: Non-square matrix multiplication

$$A = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 3 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 3}, \qquad B = \begin{bmatrix} 2 \\ -1 \\ 4 \end{bmatrix} \in \mathbb{R}^{3\times 1}.$$

Compute AB (which exists) and state why BA does not exist.

Solution:

A is 2×3 and B is 3×1: inner dimensions both equal 3, so $AB \in \mathbb{R}^{2\times 1}$:

$$AB = \begin{bmatrix} 1\cdot 2 + 0\cdot(-1) + (-1)\cdot 4 \\ 2\cdot 2 + 3\cdot(-1) + 1\cdot 4 \end{bmatrix} = \begin{bmatrix} -2 \\ 5 \end{bmatrix}.$$

BA would require B (3×1) times A (2×3): inner dimensions are $1 \neq 2$, so BA is **undefined**.

```
A = [1, 0, -1; 2, 3, 1];
B = [2; -1; 4];
AB = A * B            % [-2; 5]
BA = B * A            % Error: inner matrix dimensions must agree

```

A.6 Matrix–Vector Multiplication

Matrix–vector multiplication is a special case of matrix–matrix multiplication where $n = 1$, but it has a particularly important geometric interpretation: Ax transforms the vector x into a new vector in a (possibly different) space.

Matrix–Vector Product

For $A \in \mathbb{R}^{m\times n}$ and $x \in \mathbb{R}^n$:

$$y = Ax, \qquad y_i = \sum_{j=1}^{n} a_{ij}\, x_j, \quad i = 1, \dots, m.$$

Equivalently, Ax is a linear combination of the columns of A with coefficients given by the entries of x:

$$Ax = x_1 a_1 + x_2 a_2 + \cdots + x_n a_n,$$

where a_j denotes the j-th column of A.

In state-space control the plant update $x_{k+1} = Ax_k + Bu_k$ is nothing more than two matrix–vector products added together.

Example A.7: Matrix–vector multiplication by hand and in MATLAB

Given the state-space matrices

$$A = \begin{bmatrix} 0.9 & 0.1 \\ 0 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0.1 \\ 0.5 \end{bmatrix}, \quad x_k = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad u_k = 3,$$

compute the next state $x_{k+1} = Ax_k + Bu_k$.

Solution:

$$Ax_k = \begin{bmatrix} 0.9 \cdot 2 + 0.1 \cdot (-1) \\ 0 \cdot 2 + 0.8 \cdot (-1) \end{bmatrix} = \begin{bmatrix} 1.7 \\ -0.8 \end{bmatrix}, \qquad Bu_k = \begin{bmatrix} 0.3 \\ 1.5 \end{bmatrix},$$

$$x_{k+1} = \begin{bmatrix} 1.7 \\ -0.8 \end{bmatrix} + \begin{bmatrix} 0.3 \\ 1.5 \end{bmatrix} = \begin{bmatrix} 2.0 \\ 0.7 \end{bmatrix}.$$

```
A  = [0.9, 0.1; 0, 0.8];
B  = [0.1; 0.5];
xk = [2; -1];
uk = 3;

xk1 = A*xk + B*uk     % [2.0; 0.7]
```

A.7 The Inverse of a Matrix

Matrix Inverse

A square matrix $A \in \mathbb{R}^{n\times n}$ is **invertible** (or **non-singular**) if there exists a matrix $A^{-1} \in \mathbb{R}^{n\times n}$ such that

$$AA^{-1} = A^{-1}A = I_n.$$

If no such matrix exists, A is called **singular**.

Conditions for Invertibility

A square matrix A is invertible if and only if any one of the following equivalent conditions holds:

- $\det(A) \neq 0$ (see Section A.8);
- all eigenvalues of A are non-zero (see Section A.10);
- the columns (or rows) of A are linearly independent;
- the equation $Ax = 0$ has only the trivial solution $x = 0$.

Useful inverse identities:

$$(A^{-1})^{-1} = A \tag{A.5}$$

$$(AB)^{-1} = B^{-1}A^{-1} \quad \text{(reversed order)} \tag{A.6}$$

$$(A^\top)^{-1} = (A^{-1})^\top \tag{A.7}$$

$$(\alpha A)^{-1} = \tfrac{1}{\alpha}A^{-1} \tag{A.8}$$

Inverse of a 2×2 matrix by hand:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \implies A^{-1} = \frac{1}{ad - bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Example A.8: Computing the inverse by hand and in MATLAB

Let

$$A = \begin{bmatrix} 3 & 1 \\ 5 & 2 \end{bmatrix}.$$

Find A^{-1} and verify $AA^{-1} = I$.

Solution:

$\det(A) = 3 \cdot 2 - 1 \cdot 5 = 1$, so

$$A^{-1} = \frac{1}{1}\begin{bmatrix} 2 & -1 \\ -5 & 3 \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -5 & 3 \end{bmatrix}.$$

Verification:

$$AA^{-1} = \begin{bmatrix} 3 & 1 \\ 5 & 2 \end{bmatrix} \begin{bmatrix} 2 & -1 \\ -5 & 3 \end{bmatrix} = \begin{bmatrix} 6-5 & -3+3 \\ 10-10 & -5+6 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I. \checkmark$$

```
A     = [3, 1; 5, 2];
Ainv  = inv(A)              % [2 -1; -5 3]

% Verify
A * Ainv                    % should give eye(2)
norm(A*Ainv - eye(2))       % numerical error, should be ~1e-15
```

Avoid `inv(A)` in numerical computations

Although `inv(A)` is convenient for checking results, numerically it is almost always better to use the backslash operator `A\b` to solve $Ax = b$ directly, without forming A^{-1} explicitly. The backslash operator uses LU factorisation, which is faster and more numerically stable than computing the full inverse.

```
% Solve Ax = b
b = [1; 2];
x_inv = inv(A) * b     % slow and less accurate
x_bs  = A \ b          % fast, numerically recommended
```

Example A.9: Detecting a singular matrix

```
A_sing = [1, 2; 2, 4];      % row 2 = 2 * row 1

det(A_sing)                  % returns 0 => singular
rank(A_sing)                 % returns 1 (not full rank)
cond(A_sing)                 % returns Inf

inv(A_sing)                  % MATLAB warns: matrix is singular
```

Solution:

The determinant is zero and the rank is 1 (not 2), confirming singularity. MATLAB returns a matrix of `Inf` values from `inv` and issues a warning. The condition number `cond(A)` measures how close a matrix is to being singular; values much larger than 10^6 indicate near-singularity.

A.8 The Determinant

Determinant

The **determinant** $\det(A)$ (also written $|A|$) is a scalar function of a square matrix that encodes fundamental properties such as invertibility, volume scaling, and orientation.

For a 2×2 matrix:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

For a 3×3 matrix (cofactor expansion along the first row):

$$\det \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31}).$$

Key properties:

- $\det(I) = 1$
- $\det(AB) = \det(A)\det(B)$
- $\det(A^\top) = \det(A)$
- $\det(A^{-1}) = 1/\det(A)$
- $\det(\alpha A) = \alpha^n \det(A)$ for $A \in \mathbb{R}^{n\times n}$
- $\det(A) = 0$ if and only if A is singular

Example A.10: Determinant by hand and in MATLAB

Compute $\det(A)$ for

$$A = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 3 & -2 \\ 0 & 1 & 4 \end{bmatrix}.$$

Solution:

Expanding along the first row:

$$\det(A) = 2(3 \cdot 4 - (-2) \cdot 1) - (-1)(1 \cdot 4 - (-2) \cdot 0) + 0(\cdots) = 2(14) + 1(4) = 32.$$

```
A     = [2, -1, 0; 1, 3, -2; 0, 1, 4];
d     = det(A)            % returns 32

```

A.9 The Trace

Trace

The **trace** of a square matrix $A \in \mathbb{R}^{n\times n}$ is the sum of its diagonal elements:

$$\operatorname{tr}(A) = \sum_{i=1}^{n} a_{ii} = a_{11} + a_{22} + \cdots + a_{nn}.$$

Key properties:

- Linearity: $\operatorname{tr}(\alpha A + \beta B) = \alpha \operatorname{tr}(A) + \beta \operatorname{tr}(B)$
- Cyclic permutation: $\operatorname{tr}(ABC) = \operatorname{tr}(CAB) = \operatorname{tr}(BCA)$
- Transpose invariance: $\operatorname{tr}(A) = \operatorname{tr}(A^\top)$
- Equal to sum of eigenvalues: $\operatorname{tr}(A) = \sum_{i=1}^{n} \lambda_i$

The trace appears in control theory when expressing quadratic cost functions compactly. The MPC stage cost $x^\top Qx$ can be written as $\operatorname{tr}(Qxx^\top)$, which is useful in stochastic and robust formulations.

Example A.11: Trace in MATLAB

```
A = [1, 2, 3;
4, 5, 6;
7, 8, 9];

tr = trace(A)            % 1 + 5 + 9 = 15

% Verify trace equals sum of eigenvalues
lambda = eig(A);
sum(lambda)              % also 15 (up to numerical precision)

% Cyclic property: tr(ABC) = tr(CAB)
B = rand(3);  C = rand(3);
abs(trace(A*B*C) - trace(C*A*B))   % ~1e-14, effectively zero
```

Solution:

`trace(A) = 15 = 1+5+9`. The sum of eigenvalues agrees (numerically), confirming the identity.

A.10 Eigenvalues and Eigenvectors

Eigenvalues and eigenvectors are central to understanding the dynamic behaviour of linear systems. In discrete-time state-space control, the eigenvalues of the system matrix A determine

stability: the closed-loop system $x_{k+1} = A_{\text{cl}} x_k$ is stable if and only if all eigenvalues of A_{cl} lie strictly inside the unit circle.

Eigenvalue and Eigenvector

Let $A \in \mathbb{R}^{n \times n}$. A scalar $\lambda \in \mathbb{C}$ is an **eigenvalue** of A if there exists a non-zero vector $v \in \mathbb{C}^n$ such that

$$Av = \lambda v.$$

The vector v is the corresponding **eigenvector**. Geometrically, A does not rotate v; it only scales it by λ.

Finding eigenvalues — the characteristic equation:

Rearranging $Av = \lambda v$ gives $(A - \lambda I)v = 0$. For a non-trivial solution $v \neq 0$, the matrix $(A - \lambda I)$ must be singular:

$$\det(A - \lambda I) = 0.$$

This is the **characteristic equation**, a degree-n polynomial in λ whose n roots (counting multiplicity, possibly complex) are the eigenvalues.

Example A.12: Eigenvalues by hand for a 2×2 matrix

Find the eigenvalues of

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

Solution:

$$\det(A - \lambda I) = \det \begin{bmatrix} 3 - \lambda & 1 \\ 1 & 3 - \lambda \end{bmatrix} = (3 - \lambda)^2 - 1 = 0.$$

$$(3 - \lambda)^2 = 1 \implies 3 - \lambda = \pm 1 \implies \lambda_1 = 4, \quad \lambda_2 = 2.$$

Both eigenvalues are real and positive. For a discrete-time system matrix, we would check $|\lambda_i| < 1$; here both are > 1, so this A would represent an unstable open-loop plant.

```
A      = [3, 1; 1, 3];
lambda = eig(A)          % [2; 4]

```

Example A.13: Eigenvectors and the full eigendecomposition

Using the same matrix $A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$, find the eigenvectors corresponding to $\lambda_1 = 4$ and $\lambda_2 = 2$.

Solution:

For $\lambda_1 = 4$:

$$(A - 4I)v = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = 0 \implies -v_1 + v_2 = 0 \implies v_1 = v_2.$$

Choosing $v_1 = 1$: $v_1 = \frac{1}{\sqrt{2}}[1,\ 1]^\top$ (normalised).

For $\lambda_2 = 2$:

$$(A - 2I)v = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} v = 0 \implies v_1 + v_2 = 0 \implies v_2 = -v_1.$$

Choosing $v_1 = 1$: $v_2 = \frac{1}{\sqrt{2}}[1,\ -1]^\top$.

```
A = [3, 1; 1, 3];

% eig returns V (eigenvector matrix) and D (diagonal eigenvalue matrix)
[V, D] = eig(A)
% V: columns are eigenvectors
% D: diagonal entries are eigenvalues

% Verify A*v = lambda*v for first eigenvector
v1     = V(:,1);
lam1   = D(1,1);
norm(A*v1 - lam1*v1)    % ~1e-15, effectively zero

% The eigendecomposition: A = V * D * inv(V)
norm(A - V*D*inv(V))    % ~1e-15, effectively zero

```

Eigenvalue Properties for Stability Analysis

For a discrete-time system $x_{k+1} = Ax_k$:

- **Asymptotically stable:** all eigenvalues satisfy $|\lambda_i| < 1$ (strictly inside the unit circle).
- **Marginally stable:** all $|\lambda_i| \leq 1$ and those on the unit circle have distinct eigenvectors.
- **Unstable:** at least one eigenvalue satisfies $|\lambda_i| > 1$.

For a continuous-time system $\dot{x} = Ax$, replace the unit circle with the left half-plane: stable iff $\mathrm{Re}(\lambda_i) < 0$.

Example A.14: Stability check via eigenvalues in MATLAB

Check whether the following discrete-time system matrix is stable:

$$A = \begin{bmatrix} 0.98 & 0.01 \\ 0.02 & 0.97 \end{bmatrix}.$$

Solution:

```
A       = [0.98, 0.01; 0.02, 0.97];
lambda = eig(A)              % [0.96; 0.99]
abs_lambda = abs(lambda)     % [0.96; 0.99]
all(abs_lambda < 1)          % returns 1 (true) => stable

% Visualise on the unit circle
theta = linspace(0, 2*pi, 200);
figure; hold on;
plot(cos(theta), sin(theta), 'k--');     % unit circle
plot(real(lambda), imag(lambda), 'rx', 'MarkerSize', 10);
axis equal; grid on;
xlabel('Real'); ylabel('Imaginary');
title('Eigenvalues of A');

```

Both eigenvalues have magnitude less than 1, so the system is asymptotically stable. The plot shows both poles inside the unit circle.

A.11 Putting It All Together: A State-Space Example

This section demonstrates how all the operations above arise together in a single state-space simulation, mirroring the structure of the MPC plant model used throughout these notes.

Example A.15: Two-state discrete system: simulation and analysis

Consider the discrete-time system

$$x_{k+1} = Ax_k + Bu_k, \qquad y_k = Cx_k,$$

with

$$A = \begin{bmatrix} 0.9 & 0.1 \\ 0.0 & 0.8 \end{bmatrix}, \quad B = \begin{bmatrix} 0.1 \\ 0.3 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad x_0 = \begin{bmatrix} 5 \\ 2 \end{bmatrix}, \quad u_k = 0 \ \forall k.$$

Carry out the following tasks in MATLAB:

1. Confirm the system is stable.
2. Compute $\mathrm{tr}(A)$ and $\det(A)$ and relate them to the eigenvalues.
3. Simulate 20 steps and plot the output y_k.

Solution:

```
A   = [0.9, 0.1; 0.0, 0.8];
B   = [0.1; 0.3];
C   = [1, 0];
x   = [5; 2];
N   = 20;

%% 1. Stability
lambda = eig(A);
disp('Eigenvalues:'); disp(lambda)
disp('Magnitudes:');  disp(abs(lambda))
% Both < 1 => stable

%% 2. Trace and determinant
tr_A   = trace(A)          % 0.9 + 0.8 = 1.7
det_A  = det(A)            % 0.9*0.8 - 0.1*0 = 0.72
sum_lam   = sum(lambda)    % also 1.7 = trace
prod_lam  = prod(lambda)   % also 0.72 = det

%% 3. Simulation
y = zeros(1, N);
for k = 1:N
y(k) = C * x;
x    = A * x + B * 0;    % u=0
end

figure;
stem(1:N, y, 'filled');
xlabel('Time step k'); ylabel('y_k');
title('Output y_k (free response, u=0)');
grid on;
```

The output decays to zero, consistent with the stable eigenvalues $\lambda_1 \approx 0.9$ and $\lambda_2 = 0.8$. Notice that $\mathrm{tr}(A) = \sum \lambda_i = 1.7$ and $\det(A) = \prod \lambda_i = 0.72$, confirming the identities from Sections A.9 and A.8.

A.12 Quick Reference: MATLAB Commands

MATLAB command	Operation	Example
`A + B`	Addition	Element-wise, same size
`A - B`	Subtraction	Element-wise, same size
`alpha * A`	Scalar multiplication	Multiplies every entry
`A * B`	Matrix multiplication	Inner dims must match
`A'`	Transpose (conjugate)	Use `A.'` for real transpose
`A ^ n`	Matrix power	A^n, square A only
`inv(A)`	Matrix inverse	Avoid for solving $Ax = b$
`A \ b`	Solve $Ax = b$	Preferred over `inv(A)*b`
`det(A)`	Determinant	Scalar result
`trace(A)`	Trace	Sum of diagonal
`eig(A)`	Eigenvalues	Column vector λ
`[V,D] = eig(A)`	Eigenvectors + values	Columns of `V`, diag of `D`
`rank(A)`	Rank	Number of independent columns
`cond(A)`	Condition number	Large value $\Rightarrow$ near-singular
`norm(A)`	2-norm (default)	`norm(A,1)`, `norm(A,'fro')` also available
`size(A)`	Dimensions	Returns `[m n]`
`zeros(m,n)`	Zero matrix	All entries 0
`ones(m,n)`	Ones matrix	All entries 1
`eye(n)`	Identity matrix	I_n
`diag(v)`	Diagonal matrix	`diag(A)` extracts diagonal of A
`linspace(a,b,n)`	Equally spaced vector	n points from a to b

Linear Algebra Summary for MPC

1. **Dimensions matter:** always check that inner dimensions are compatible before multiplying; $A_{m\times p} \cdot B_{p\times n}$ gives $C_{m\times n}$.
2. **Multiplication is not commutative:** $AB \neq BA$ in general; the reversed-order rule $(AB)^\top = B^\top A^\top$ and $(AB)^{-1} = B^{-1}A^{-1}$ are essential.
3. **Invertibility:** A^{-1} exists iff $\det(A) \neq 0$ iff all eigenvalues are non-zero. Use `A\b` rather than `inv(A)*b`.
4. **Stability:** for discrete-time systems, check $|\lambda_i| < 1$ for all eigenvalues of the closed-loop matrix.
5. **Trace and determinant** equal the sum and product of eigenvalues respectively, providing a fast sanity check.
6. **Positive definiteness** of cost matrices $Q \succeq 0$, $R \succ 0$ is required for the MPC QP to be convex and bounded below; verify with `eig(Q)` and `eig(R)`.

Appendix B

Probability and Statistics: A Self-Study Reference

Modern control systems — from Kalman filters and Linear Quadratic Gaussian (LQG) regulators to stochastic MPC and Reinforcement Learning (RL) — are built on probability theory. This appendix introduces the core ideas from scratch, assuming only basic school-level mathematics. Every concept is illustrated with worked examples and MATLAB code so that you can develop intuition interactively before encountering these tools in the main text.

By the end of this appendix you should be comfortable with random variables, Gaussian distributions, expectations, covariance matrices, Bayes' theorem, and stochastic processes — the exact toolkit needed for Chapters on Kalman filtering, LQG, stochastic MPC, and RL.

B.1 Why Probability in Control?

Every real control system operates in an uncertain environment. Sensors do not report exact values — they add noise. Actuators do not deliver exactly the commanded force or voltage. The plant model is always an approximation of reality. Disturbances (wind, load changes, temperature) are not known in advance.

Deterministic control (classical PID, standard MPC) ignores this uncertainty or handles it implicitly through robustness margins. Probabilistic control treats uncertainty explicitly by modelling unknown quantities as *random variables* and designing controllers that perform well *on average* or with *guaranteed probability*. The four main applications you will encounter in these notes are:

- **Kalman filter**: optimal state estimator when process and measurement noise are Gaussian.
- **LQG control**: combines a Kalman filter with an LQR controller to minimise expected quadratic cost under Gaussian noise.
- **Stochastic MPC**: replaces hard constraints with chance constraints (e.g. satisfy a comfort bound with probability 0.95).
- **Reinforcement Learning**: the agent observes noisy states and learns an optimal policy by maximising expected cumulative reward.

B.2 Sample Spaces, Outcomes, and Events

Sample Space and Event

The **sample space** Ω is the set of all possible outcomes of a random experiment. An **event** A is any subset $A \subseteq \Omega$. The **probability** of event A, written $P(A)$, is a number in $[0, 1]$ that quantifies how likely A is to occur.

The three axioms of probability (Kolmogorov, 1933):

1. $P(A) \geq 0$ for every event A.
2. $P(\Omega) = 1$ (something always happens).
3. If A and B are *mutually exclusive* ($A \cap B = \emptyset$), then $P(A \cup B) = P(A) + P(B)$.

From these three axioms every other rule of probability can be derived. Key consequences:

- $P(\emptyset) = 0$
- $P(A^c) = 1 - P(A)$, where A^c is the complement of A
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (inclusion–exclusion)
- $0 \leq P(A) \leq 1$

Example B.1: Sensor failure probability

A temperature sensor fails independently on any given day with probability $p = 0.02$. Over a 5-day week, what is the probability that it fails at least once?

Solution:

Let A_i be the event "fails on day i". The complement is "never fails in 5 days":

$$P(\text{never fails}) = (1 - 0.02)^5 = 0.98^5 \approx 0.904.$$

Therefore:

$$P(\text{at least one failure}) = 1 - 0.904 = 0.096.$$

There is roughly a 9.6% chance of at least one failure per week.

```
p_fail    = 0.02;
n_days    = 5;
p_none    = (1 - p_fail)^n_days     % 0.9039
p_atleast1 = 1 - p_none            % 0.0961
```

B.3 Conditional Probability and Independence

Conditional Probability

The **conditional probability** of event A given that event B has occurred (with $P(B) > 0$) is:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Intuitively, conditioning on B restricts the sample space to B and rescales probabilities accordingly.

Statistical Independence

Events A and B are **independent** if knowing B occurred gives no information about A:

$$P(A \mid B) = P(A) \quad \Longleftrightarrow \quad P(A \cap B) = P(A)\,P(B).$$

In Kalman filtering, process noise and measurement noise are assumed to be mutually independent, which greatly simplifies the filter equations.

Example B.2: Conditional probability with a noisy sensor

A room is occupied 70% of the time. When occupied, a motion sensor triggers with probability 0.9. When empty, it still triggers (false alarm) with probability 0.05. Given that the sensor triggered, what is the probability the room is actually occupied?

Solution:

Let O = "room occupied", T = "sensor triggered".

$$P(O) = 0.7, \quad P(T \mid O) = 0.9, \quad P(T \mid O^c) = 0.05.$$

By the **law of total probability**:

$$P(T) = P(T \mid O)P(O) + P(T \mid O^c)P(O^c) = 0.9 \times 0.7 + 0.05 \times 0.3 = 0.645.$$

By conditional probability:

$$P(O \mid T) = \frac{P(T \mid O)\,P(O)}{P(T)} = \frac{0.63}{0.645} \approx 0.977.$$

If the sensor triggers, the room is occupied with about 97.7% confidence.

```
P_O    = 0.7;
P_T_O  = 0.9;     % true positive rate
P_T_Oc= 0.05;     % false alarm rate
P_Oc   = 1 - P_O;

P_T    = P_T_O*P_O + P_T_Oc*P_Oc;      % 0.6450
P_O_T  = P_T_O * P_O / P_T              % 0.9767
```

B.4 Bayes' Theorem

Bayes' Theorem

For events A and B with $P(B) > 0$:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}.$$

In words: the **posterior** probability of A given evidence B equals the **likelihood** $P(B \mid A)$ times the **prior** $P(A)$, divided by the **marginal likelihood** (evidence) $P(B)$.

Bayes' theorem is the mathematical foundation of the Kalman filter. At every time step the filter:

1. Uses the prior state estimate (before the measurement).
2. Updates it with the likelihood of the new measurement.
3. Produces a posterior estimate that combines both sources of information optimally.

This three-step pattern — prior $\rightarrow$ likelihood $\rightarrow$ posterior — repeats at every time step and is precisely the *predict–correct* structure of the Kalman filter described in the main text.

Bayes' theorem and the Kalman filter

The Kalman filter is the exact, closed-form solution to Bayes' theorem when all distributions are Gaussian and the system is linear. For non-linear or non-Gaussian systems, approximate methods (Extended Kalman Filter, Unscented Kalman Filter, Particle Filter) are needed — all of which still implement the same Bayesian predict–correct cycle.

B.5 Random Variables

Random Variable

A **random variable** (RV) X is a function that assigns a numerical value to every outcome in the sample space Ω. It is **discrete** if it takes countably many values and **continuous** if it takes values in a continuous range.

B.5.1 Discrete Random Variables

A discrete RV is characterised by its **probability mass function** (PMF):

$$p_X(x) = P(X = x), \qquad \sum_{\text{all } x} p_X(x) = 1.$$

Example B.3: PMF of a discrete sensor reading

A quantised position sensor reports integer values in $\{-2, -1, 0, 1, 2\}$ with equal probability. Plot the PMF.

Solution:

$p_X(x) = 1/5$ for each $x \in \{-2, -1, 0, 1, 2\}$, zero elsewhere (a uniform discrete distribution).

```
x   = -2:2;
pmf = ones(1,5) / 5;

figure;
stem(x, pmf, 'filled', 'LineWidth', 1.5);
xlabel('Sensor reading x');
ylabel('P(X = x)');
title('PMF of quantised sensor');
ylim([0, 0.4]); grid on;
```

B.5.2 Continuous Random Variables

A continuous RV is characterised by its **probability density function** (PDF) $f_X(x)$, defined so that:

$$P(a \leq X \leq b) = \int_a^b f_X(x)\,\mathrm{d}x, \qquad \int_{-\infty}^{\infty} f_X(x)\,\mathrm{d}x = 1.$$

PDF values are NOT probabilities

$f_X(x)$ can exceed 1. Only *areas* under the PDF curve represent probabilities. $P(X = x) = 0$ for any single point x (a continuous RV never takes an exact value).

The **cumulative distribution function** (CDF) collects probability up to x:

$$F_X(x) = P(X \leq x) = \int_{-\infty}^{x} f_X(t)\,\mathrm{d}t.$$

B.6 Key Probability Distributions

B.6.1 Uniform Distribution

Uniform Distribution $\mathcal{U}(a, b)$

$X \sim \mathcal{U}(a, b)$ has constant density on $[a, b]$:

$$f_X(x) = \frac{1}{b-a}, \quad a \leq x \leq b; \qquad f_X(x) = 0 \text{ otherwise.}$$

Mean: $(a+b)/2$. Variance: $(b-a)^2/12$.

B.6.2 The Gaussian (Normal) Distribution

The Gaussian distribution is the most important distribution in control engineering. Its central role follows from the **Central Limit Theorem**: the sum of many independent random variables tends to a Gaussian, regardless of their individual distributions. Sensor noise, modelling errors, and thermal fluctuations are all well approximated as Gaussian.

Gaussian Distribution $\mathcal{N}(\mu, \sigma^2)$

A scalar RV $X \sim \mathcal{N}(\mu, \sigma^2)$ has PDF:

$$f_X(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R},$$

where μ is the **mean** (centre) and σ^2 is the **variance** (spread). The **standard deviation** is $\sigma = \sqrt{\sigma^2}$.

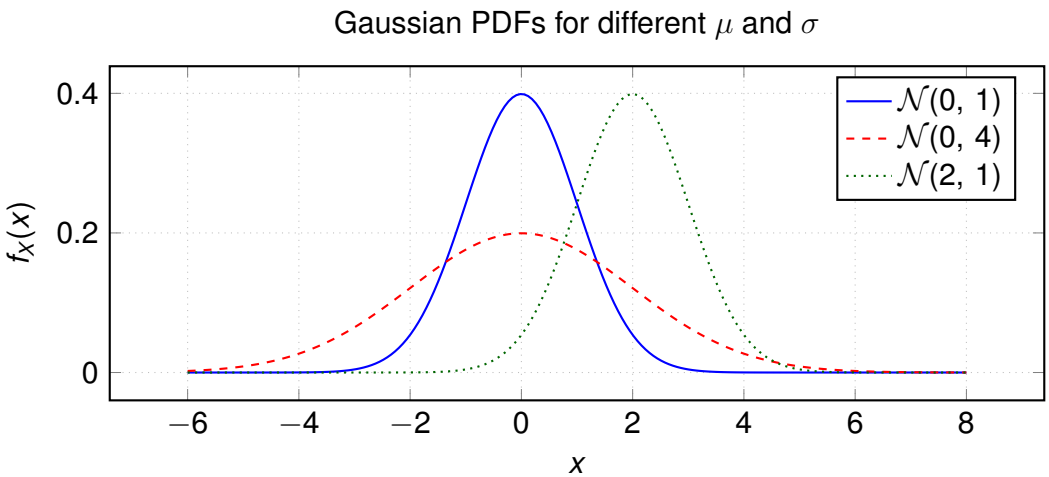


Figure B.1: Gaussian PDFs. Increasing σ^2 widens the bell curve (more uncertainty); shifting μ moves its centre (different expected value).

The 68–95–99.7 rule (essential for engineering):

$$P(\mu-\sigma \leq X \leq \mu+\sigma) \approx 0.683, \quad P(\mu-2\sigma \leq X \leq \mu+2\sigma) \approx 0.954, \quad P(\mu-3\sigma \leq X \leq \mu+3\sigma) \approx 0.997.$$

Example B.4: Gaussian sensor noise in MATLAB

A temperature sensor measures a true value of 20 °C with additive Gaussian noise of standard deviation $\sigma = 0.5$ °C. Simulate 10 000 measurements and verify the 68–95–99.7 rule.

```
mu    = 20;
sigma = 0.5;
N     = 10000;
rng(42);                % fix random seed for reproducibility

meas  = mu + sigma * randn(1, N);

% Verify 68-95-99.7 rule
in1sig = mean(abs(meas - mu) <= 1*sigma)   % ~0.683
in2sig = mean(abs(meas - mu) <= 2*sigma)   % ~0.954
in3sig = mean(abs(meas - mu) <= 3*sigma)   % ~0.997

% Plot histogram vs true PDF
figure;
histogram(meas, 60, 'Normalization', 'pdf', ...
          'FaceColor', [0.6 0.8 1], 'EdgeColor', 'none');
hold on;
x_range = linspace(17, 23, 300);
pdf_true = normpdf(x_range, mu, sigma);
plot(x_range, pdf_true, 'r-', 'LineWidth', 2);
xlabel('Measurement (deg C)');
ylabel('Probability density');
title('Simulated Gaussian sensor noise');
legend('Histogram','True PDF'); grid on;
```

Solution:

Running the code confirms all three percentages to within 1% of their theoretical values. `normpdf(x,mu,sigma)` evaluates the Gaussian PDF; `normcdf` evaluates the CDF.

B.6.3 Exponential and Other Useful Distributions

Distribution	PDF / PMF	Mean	Variance
Uniform $\mathcal{U}(a, b)$	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
Gaussian $\mathcal{N}(\mu, \sigma^2)$	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2
Exponential Exp(λ)	$\lambda e^{-\lambda x}$, $x \geq 0$	$1/\lambda$	$1/\lambda^2$
Bernoulli Bern(p)	$p^x(1-p)^{1-x}$, $x \in \{0, 1\}$	p	$p(1-p)$
Poisson Poi(λ)	$e^{-\lambda}\lambda^k/k!$, $k = 0, 1, \ldots$	λ	λ

B.7 Expectation, Variance, and Standard Deviation

Expected Value (Mean)

The **expected value** (or **mean**) of a random variable X is its probability-weighted average:

$$\mu = \mathbb{E}[X] = \begin{cases} \sum_x x\, p_X(x) & \text{(discrete)}, \\ \int_{-\infty}^{\infty} x\, f_X(x)\, \mathrm{d}x & \text{(continuous)}. \end{cases}$$

Variance and Standard Deviation

The **variance** measures the average squared deviation from the mean:

$$\sigma^2 = \mathrm{Var}(X) = \mathbb{E}\left[(X-\mu)^2\right] = \mathbb{E}[X^2] - \mu^2.$$

The **standard deviation** $\sigma = \sqrt{\mathrm{Var}(X)}$ has the same units as X and is the most intuitive measure of spread.

Linearity of expectation (always true, regardless of dependence):

$$\mathbb{E}[aX + bY + c] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y] + c.$$

Variance under linear transformation:

$$\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X).$$

Example B.5: Mean and variance of measurement noise

The measurement noise in a Kalman filter is modelled as $v_k \sim \mathcal{N}(0, R)$ with $R = 0.25$. What fraction of measurements lie within $\pm 1\,^\circ\mathrm{C}$ of the true value?

Solution:

$\sigma = \sqrt{0.25} = 0.5$. We need $P(-1 \le v_k \le 1) = P(-2\sigma \le v_k \le 2\sigma) \approx 0.954$.

```
R     = 0.25;
sigma = sqrt(R);          % 0.5

% Probability within +/- 1 degree
prob = normcdf(1, 0, sigma) - normcdf(-1, 0, sigma)  % 0.9545

% Alternatively, using the standard normal
z = 1 / sigma;            % 2 standard deviations
prob2 = 2*normcdf(z) - 1                             % 0.9545

% Mean and variance via Monte Carlo
v = sigma * randn(1, 1e6);
fprintf('Sample mean:     %.4f\n', mean(v))     % ~0
fprintf('Sample variance: %.4f\n', var(v))      % ~0.25
```

B.8 Covariance and Correlation

Covariance

The **covariance** of two random variables X and Y measures how they vary together:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\,\mathbb{E}[Y].$$

Note that $\mathrm{Cov}(X, X) = \mathrm{Var}(X)$. If X and Y are independent, $\mathrm{Cov}(X, Y) = 0$ (but the converse is not generally true).

Pearson Correlation Coefficient

The **correlation coefficient** normalises covariance to the range $[-1, 1]$:

$$\rho_{XY} = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y}.$$

$\rho = 1$: perfect positive linear relationship; $\rho = -1$: perfect negative; $\rho = 0$: uncorrelated (no linear relationship).

Example B.6: Covariance and correlation in MATLAB

```
rng(0);
N  = 1000;
x  = randn(N,1);
y1 = 2*x + 0.5*randn(N,1);   % highly correlated with x
y2 = randn(N,1);               % independent of x

% Covariance matrices
C1 = cov(x, y1)    % large off-diagonal entries
C2 = cov(x, y2)    % near-zero off-diagonal

% Correlation coefficients
rho1 = corr(x, y1)   % close to +1
rho2 = corr(x, y2)   % close to  0

% Scatter plots
figure;
subplot(1,2,1); scatter(x, y1, 5, '.'); grid on;
xlabel('x'); ylabel('y_1'); title(sprintf('\\rho = %.2f', rho1));
subplot(1,2,2); scatter(x, y2, 5, 'r.'); grid on;
xlabel('x'); ylabel('y_2'); title(sprintf('\\rho = %.2f', rho2));
```

B.9 The Multivariate Gaussian Distribution

State vectors in control systems are multi-dimensional. The Kalman filter propagates a full *joint* probability distribution over all state components simultaneously, which requires the multivariate generalisation of the Gaussian.

Multivariate Gaussian $\mathcal{N}(\mu, \Sigma)$

A random vector $x \in \mathbb{R}^n$ follows a **multivariate Gaussian** distribution with mean vector $\mu \in \mathbb{R}^n$ and **covariance matrix** $\Sigma \in \mathbb{R}^{n \times n}$ if its PDF is:

$$f(x) = \frac{1}{(2\pi)^{n/2}\sqrt{\det(\Sigma)}} \exp\left(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\right).$$

The covariance matrix Σ must be **symmetric positive semi-definite**: $\Sigma = \Sigma^\top$ and $\Sigma \succeq 0$.

The diagonal entries of Σ are the individual variances: $\Sigma_{ii} = \mathrm{Var}(x_i)$. The off-diagonal entries are covariances: $\Sigma_{ij} = \mathrm{Cov}(x_i, x_j)$.

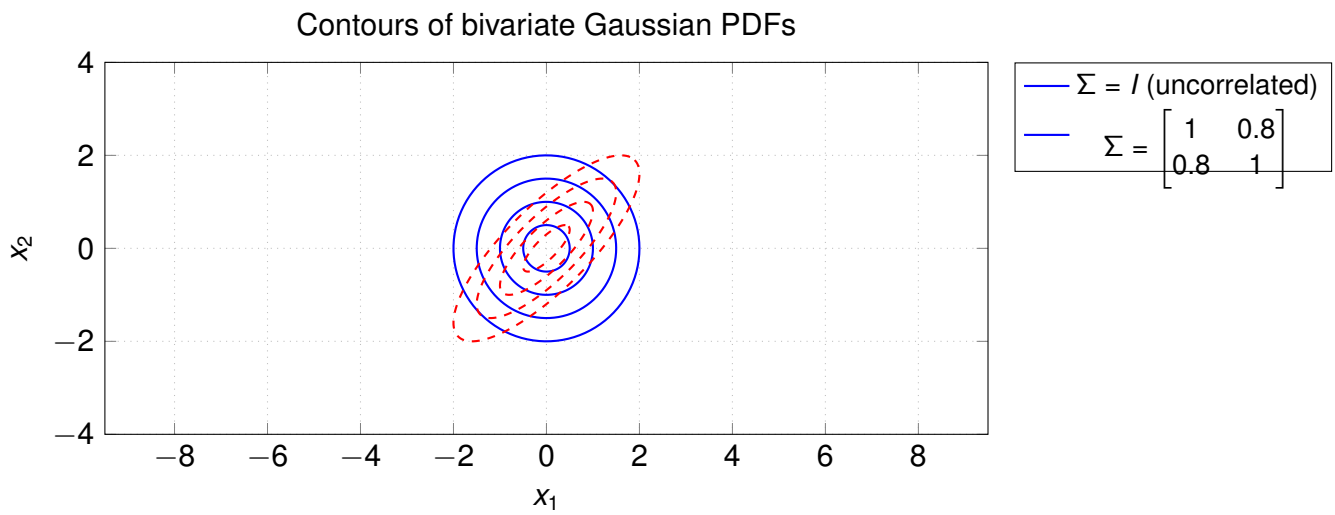


Figure B.2: Equal-probability contours of two bivariate Gaussian distributions. When $\Sigma = I$ the contours are circles (independent components). When off-diagonal covariance is large the contours tilt into ellipses aligned with the correlated direction.

Covariance matrices in the Kalman filter

The Kalman filter maintains the state estimate as $x_k \sim \mathcal{N}(\hat{x}_k, P_k)$, where P_k is the **error covariance matrix**. At each step:

- **Prediction**: P grows (uncertainty increases because the model is imperfect).
- **Correction**: P shrinks (the measurement reduces uncertainty).

The Kalman gain K is precisely the factor that balances how much to trust the model prediction versus the new measurement.

Example B.7: Multivariate Gaussian samples in MATLAB

Generate samples from a bivariate Gaussian with $\mu = [1,\ -2]^\top$ and $\Sigma = \begin{bmatrix} 2 & 0.8 \\ 0.8 & 1 \end{bmatrix}$.

```
mu    = [1; -2];
Sigma = [2, 0.8; 0.8, 1];
```

```
% Check Sigma is positive definite
eig(Sigma)            % both positive: [0.4781; 2.5219]

% Draw 2000 samples
rng(1);
N        = 2000;
samples = mvnrnd(mu', Sigma, N);  % N x 2 matrix

% Plot
figure;
scatter(samples(:,1), samples(:,2), 8, '.', ...
        'MarkerEdgeColor', [0.2 0.4 0.8]);
hold on;
plot(mu(1), mu(2), 'r+', 'MarkerSize', 15, 'LineWidth', 2);
xlabel('x_1'); ylabel('x_2');
title('Samples from \mathcal{N}(\mu, \Sigma)');
legend('Samples','Mean'); grid on; axis equal;

% Verify sample statistics
fprintf('Sample mean:  [%.3f, %.3f]\n', mean(samples));
disp('Sample covariance:'); disp(cov(samples))
```

B.10 Linear Transformations of Gaussian Variables

A crucial property that makes Gaussian distributions so convenient for control is that they are *closed under linear transformations*: a linear function of a Gaussian is still Gaussian.

Linear Transformation of a Gaussian Vector

If $x \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and $y = Ax + b$, then:

$$y \sim \mathcal{N}\left(A\boldsymbol{\mu} + b,\ A\Sigma A^\top\right).$$

This is exactly the **predict step** of the Kalman filter. If the state $x_k \sim \mathcal{N}(\hat{x}_k, P_k)$ and the system evolves as $x_{k+1} = Ax_k + w_k$ with process noise $w_k \sim \mathcal{N}(0, Q)$, then the predicted state distribution is:

$$x_{k+1} \sim \mathcal{N}\left(A\hat{x}_k,\ AP_kA^\top + Q\right).$$

No approximation is involved — the result is exact because all operations are linear and all distributions are Gaussian.

Example B.8: Propagating uncertainty through a linear system

A one-dimensional system has $x_{k+1} = 0.9\,x_k + w_k$ with $w_k \sim \mathcal{N}(0, 0.01)$. The initial state is $x_0 \sim \mathcal{N}(5, 1)$. Propagate the mean and variance for 10 steps.

Solution:

At each step: $\mu_{k+1} = 0.9\,\mu_k$ and $\sigma^2_{k+1} = 0.81\,\sigma^2_k + 0.01$. The variance decays towards the steady-state value $\sigma^2_\infty = 0.01/(1-0.81) \approx 0.053$.

```
a      = 0.9;
Q      = 0.01;     % process noise variance
mu     = 5;
P      = 1;        % initial variance
N      = 10;

mu_log = zeros(1,N+1);  P_log = zeros(1,N+1);
mu_log(1) = mu;          P_log(1) = P;

for k = 1:N
    mu = a * mu;
    P  = a^2 * P + Q;
    mu_log(k+1) = mu;
    P_log(k+1)  = P;
end

figure;
subplot(2,1,1);
plot(0:N, mu_log, 'b-o', 'LineWidth', 1.5);
ylabel('\mu_k'); title('Propagated mean'); grid on;

subplot(2,1,2);
plot(0:N, P_log, 'r-o', 'LineWidth', 1.5);
yline(Q/(1-a^2), 'k--', 'Steady state', 'LineWidth', 1);
ylabel('\sigma^2_k'); xlabel('Step k');
title('Propagated variance'); grid on;
```

B.11 Stochastic Processes

A **stochastic process** is a sequence of random variables indexed by time: $\{X_k\}_{k=0}^{\infty}$ (discrete time) or $\{X(t)\}_{t\geq 0}$ (continuous time). In control, the key stochastic processes are:

- **White noise**: uncorrelated sequence where each sample is drawn independently from the same distribution. Used to model process noise w_k and measurement noise v_k in state-space models.
- **Random walk**: $X_{k+1} = X_k + w_k$ where w_k is white noise. The variance grows without bound.
- **Markov process**: the future depends only on the present state, not the past. State-space models are (first-order) Markov by construction: $x_{k+1} = f(x_k, u_k, w_k)$.

White Noise Assumptions in State-Space Control

The standard Kalman filter assumptions are:

$$\begin{aligned} w_k &\sim \mathcal{N}(0, Q), && \text{(process noise)} \\ v_k &\sim \mathcal{N}(0, R), && \text{(measurement noise)} \\ \mathbb{E}[w_k w_j^\top] &= Q\,\delta_{kj}, && \text{(uncorrelated across time)} \\ \mathbb{E}[v_k v_j^\top] &= R\,\delta_{kj}, && \mathbb{E}[w_k v_j^\top] = 0, \end{aligned}$$

where $\delta_{kj} = 1$ if $k = j$, else 0 (Kronecker delta). Matrices $Q \succeq 0$ and $R \succ 0$ are the noise covariance matrices that appear explicitly in the Kalman filter equations.

Example B.9: Simulating white noise and a random walk

```
N      = 200;
Q      = 0.5;       % process noise variance
rng(7);
w      = sqrt(Q) * randn(1, N);    % white noise sequence

% Verify whiteness: autocorrelation should be near zero for lag > 0
figure;
subplot(3,1,1);
plot(w); xlabel('k'); ylabel('w_k');
title('White noise sequence'); grid on;

subplot(3,1,2);
autocorr(w, 20);    % should spike only at lag 0
title('Autocorrelation of white noise');

% Random walk: x_{k+1} = x_k + w_k
x = cumsum(w);
subplot(3,1,3);
plot(x, 'r'); xlabel('k'); ylabel('x_k');
title('Random walk (cumulative sum of white noise)'); grid on;
```

Solution:

The autocorrelation plot confirms that white noise has zero correlation at all non-zero lags (within statistical fluctuation). The random walk drifts unpredictably because its variance grows as $k \cdot Q$.

B.12 The Law of Large Numbers and the Central Limit Theorem

These two theorems are the bedrock of statistical intuition.

Law of Large Numbers (LLN)

Let $X_1, X_2, \dots, X_N$ be independent, identically distributed (i.i.d.) random variables with mean μ. The sample mean converges to the true mean as the sample size grows:

$$\bar{X}_N = \frac{1}{N}\sum_{i=1}^{N} X_i \xrightarrow{N\to\infty} \mu.$$

Central Limit Theorem (CLT)

Under the same conditions (i.i.d., finite variance σ^2), the *standardised* sample mean converges in distribution to a standard Gaussian:

$$\frac{\bar{X}_N - \mu}{\sigma/\sqrt{N}} \xrightarrow{d} \mathcal{N}(0,1) \quad \text{as } N \to \infty.$$

This is why Gaussian noise is such a universal model: it emerges naturally as the aggregate effect of many small, independent perturbations.

Example B.10: Demonstrating the CLT in MATLAB

Show that the sample mean of N uniform random variables approaches a Gaussian as N increases.

```
n_trials = 5000;
figure;

for idx = 1:4
    N = 10^idx;                              % N = 10, 100, 1000, 10000
    % Sum N uniform[0,1] samples, then standardise
    raw  = rand(n_trials, N);
    xbar = mean(raw, 2);                     % n_trials x 1 sample means
    mu_u = 0.5;  sig_u = 1/sqrt(12*N);  % theoretical mean and std
    z    = (xbar - mu_u) / sig_u;

    subplot(2,2,idx);
    histogram(z, 40, 'Normalization','pdf', ...
              'FaceColor',[0.6 0.8 1], 'EdgeColor','none');
    hold on;
    x_range = linspace(-4, 4, 200);
    plot(x_range, normpdf(x_range), 'r-', 'LineWidth', 2);
    title(sprintf('N = %d', N));
    xlabel('z'); ylabel('PDF'); grid on;
    xlim([-4 4]);
end
sgtitle('Central Limit Theorem: convergence to Gaussian');
```

Solution:

Even for $N = 10$ the histogram already resembles a bell curve. By $N = 1000$ the agreement with the standard Gaussian (red curve) is almost perfect.

B.13 Monte Carlo Methods

Monte Carlo methods approximate expectations and probabilities by drawing many random samples. They are used in stochastic MPC to evaluate chance constraints (e.g. "what fraction of scenarios violate a temperature bound?") and in RL to estimate policy value functions.

Monte Carlo Estimation

To estimate $\mathbb{E}[g(X)]$ for some function g and random variable X:

1. Draw N i.i.d. samples $x^{(1)}, x^{(2)}, \dots, x^{(N)}$ from the distribution of X.
2. Compute the sample average: $\hat{\mathbb{E}}[g(X)] = \frac{1}{N}\sum_{i=1}^{N} g(x^{(i)})$.

By the LLN this estimate converges to the true expectation. The standard error of the estimate is $\sigma_{g(X)}/\sqrt{N}$, so accuracy improves as $1/\sqrt{N}$.

Example B.11: Monte Carlo chance constraint evaluation

In a stochastic MPC application, room temperature at the next step is $x_1 \sim \mathcal{N}(21.5,\ 0.04)$ (mean 21.5°C, variance 0.04). Estimate the probability that the comfort constraint $x_1 \geq 21\,°\text{C}$ is satisfied.

```
mu_x   = 21.5;
var_x  = 0.04;
N      = 100000;
rng(3);
x_samples = mu_x + sqrt(var_x) * randn(1, N);

% Monte Carlo probability estimate
p_mc      = mean(x_samples >= 21)     % ~0.9938

% Exact answer via CDF
p_exact   = 1 - normcdf(21, mu_x, sqrt(var_x))  % 0.9938

fprintf('Monte Carlo: %.4f\n', p_mc);
fprintf('Exact:       %.4f\n', p_exact);
```

Solution:

Both methods give $P(x_1 \geq 21) \approx 0.9938$, i.e. the comfort constraint is satisfied 99.4% of the time. In stochastic MPC this would be expressed as the chance constraint $P(x_1 \geq 21) \geq 0.95$.

B.14 Connecting Probability to the Kalman Filter

With the tools of this appendix, the Kalman filter equations can now be understood probabilistically rather than just algorithmically.

The **state-space model** is:

$$x_{k+1} = Ax_k + Bu_k + w_k, \qquad w_k \sim \mathcal{N}(0, Q), \tag{B.1}$$
$$y_k = Cx_k + v_k, \qquad v_k \sim \mathcal{N}(0, R). \tag{B.2}$$

The Kalman filter maintains a Gaussian belief over the state:

$$p(x_k \mid y_{0:k}) = \mathcal{N}(\hat{x}_k, P_k).$$

Predict step (propagate using the system model):

$$\hat{x}_{k|k-1} = A\hat{x}_{k-1} + Bu_{k-1}, \tag{B.3}$$
$$P_{k|k-1} = AP_{k-1}A^\top + Q. \tag{B.4}$$

This is the linear transformation rule from Section B.10.

Correct step (update using the new measurement):

$$K_k = P_{k|k-1}C^\top(CP_{k|k-1}C^\top + R)^{-1}, \quad \text{(Kalman gain)} \tag{B.5}$$
$$\hat{x}_k = \hat{x}_{k|k-1} + K_k(y_k - C\hat{x}_{k|k-1}), \tag{B.6}$$
$$P_k = (I - K_kC)\,P_{k|k-1}. \tag{B.7}$$

The Kalman gain K_k is the optimal Bayesian weighting between the model prediction and the measurement. When R is large (noisy sensor), $K_k \to 0$ and the filter trusts the model. When Q is large (uncertain model), K_k increases and the filter trusts the measurement.

Example B.12: One-dimensional Kalman filter in MATLAB

Estimate the position of a slowly drifting signal with process noise $Q = 0.1$ and measurement noise $R = 2$.

```
A = 1;  C = 1;  Q = 0.1;  R = 2;
N = 60;  rng(5);

% Simulate true state and measurements
x_true = cumsum([10, sqrt(Q)*randn(1,N-1)]);
y_meas = x_true + sqrt(R)*randn(1,N);

% Kalman filter
x_hat = zeros(1,N);  P = 10;  x_est = 10;

for k = 1:N
    % Predict
    x_pred = A * x_est;
    P_pred = A^2 * P + Q;

    % Correct
```

```
    K       = P_pred * C / (C^2 * P_pred + R);
    x_est   = x_pred + K * (y_meas(k) - C*x_pred);
    P       = (1 - K*C) * P_pred;
    x_hat(k) = x_est;
end

% Plot
figure;
plot(x_true, 'k-',  'LineWidth', 1.5); hold on;
plot(y_meas, 'b.',  'MarkerSize', 8);
plot(x_hat,  'r-',  'LineWidth', 2);
legend('True state','Measurements','Kalman estimate');
xlabel('Time step k'); ylabel('Position');
title('1-D Kalman Filter'); grid on;
```

Solution:

The Kalman estimate (red) tracks the true state (black) much more closely than the raw noisy measurements (blue dots), demonstrating the filter's noise-rejection capability.

B.15 Probability in Reinforcement Learning

Reinforcement Learning frames the control problem as a **Markov Decision Process** (MDP), which is a direct application of the probabilistic tools developed in this appendix.

Markov Decision Process

An MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$:

- $\mathcal{S}$: state space (e.g. room temperatures)
- $\mathcal{A}$: action space (e.g. heater power levels)
- $\mathcal{P}(s' \mid s, a)$: **transition probability** — probability of reaching state s' from state s under action a
- $r(s, a)$: reward function (e.g. negative energy cost)
- $\gamma \in [0, 1)$: discount factor for future rewards

The agent's goal is to find a policy $\pi(a \mid s)$ (a conditional probability distribution over actions given states) that maximises the **expected discounted return**:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r(s_k, a_k) \right].$$

Every concept used in this definition — conditional probability, expectation, Markov property — has been covered in the earlier sections of this appendix. The connection between RL and MPC is that both seek to maximise a sum of future rewards/costs; RL does so by *learning* the optimal

policy from data rather than by solving an explicit optimisation problem at each step.

B.16 Quick Reference: MATLAB Commands for Probability

MATLAB command	Operation	Notes
`randn(m,n)`	Standard Gaussian samples	$\mathcal{N}(0,1)$, size $m \times n$
`rand(m,n)`	Uniform [0, 1] samples	Size $m \times n$
`rng(seed)`	Set random seed	For reproducibility
`normpdf(x,mu,sig)`	Gaussian PDF	Scalar or vector x
`normcdf(x,mu,sig)`	Gaussian CDF	$P(X \leq x)$
`norminv(p,mu,sig)`	Inverse CDF (quantile)	x such that $P(X \leq x) = p$
`mvnrnd(mu,Sigma,N)`	Multivariate Gaussian samples	N rows
`mvnpdf(X,mu,Sigma)`	Multivariate Gaussian PDF	
`mean(x)`	Sample mean	Along first non-singleton dim
`var(x)`	Sample variance	Unbiased ($N-1$ denominator)
`std(x)`	Sample standard deviation	$\sqrt{\texttt{var}(x)}$
`cov(X)`	Sample covariance matrix	`X` is $N \times p$ data matrix
`corr(x,y)`	Pearson correlation	Returns scalar in $[-1, 1]$
`autocorr(x,n)`	Sample autocorrelation	Up to lag n
`histogram(x,k)`	Histogram	Use `'Normalization','pdf'`
`eig(Sigma)`	Eigenvalues of Σ	Check positive definiteness
`chol(Sigma)`	Cholesky factorisation	For sampling multivariate Gaussian

Probability Summary for Control Engineers

1. **Everything uncertain is a random variable.** Sensor noise, model errors, and disturbances are all modelled as random variables with known distributions.
2. **The Gaussian is central.** Sensor noise and process disturbances are almost always modelled as Gaussian because of the CLT, and because Gaussians are closed under linear transformations.
3. **Mean and covariance matrix** fully characterise a Gaussian distribution. The Kalman filter propagates exactly these two quantities.
4. **Bayes' theorem is the theoretical backbone** of all state estimation. The Kalman filter is its exact solution under linearity and Gaussianity.
5. **Q and R are design choices.** The process noise covariance Q and measurement noise covariance R in the Kalman filter determine how much the filter trusts the model versus the sensor. Larger $R \Rightarrow$ trust the model more; larger $Q \Rightarrow$ trust the sensor more.
6. **Chance constraints in stochastic MPC** replace hard bounds with probabilistic ones: $P(Cx_k \leq d) \geq 1 - \epsilon$. Monte Carlo or analytical Gaussian CDF methods evaluate these constraints.
7. **RL uses expectations everywhere.** Value functions, policy gradients, and Bellman equations are all defined in terms of expected rewards under a probability distribution over future trajectories.

Bibliography

[1] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos. The explicit linear quadratic regulator for constrained systems. *Automatica*, 38(1):3–20, 2002. doi: 10.1016/S0005-1098(01)00174-1.

[2] F. Borrelli, A. Bemporad, and M. Morari. *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.

[3] J. Coulson, J. Lygeros, and F. Dörfler. Data-enabled predictive control: In the shallows of the DeePC. In *European Control Conference (ECC)*, pages 307–312, 2019. doi: 10.23919/ECC.2019.8795639.

[4] L. Grüne and J. Pannek. *Nonlinear Model Predictive Control: Theory and Algorithms*. Springer, 2nd edition, 2017. doi: 10.1007/978-3-319-46024-6.

[5] M. Herceg, M. Kvasnica, C. N. Jones, and M. Morari. Multi-parametric toolbox 3.0. In *European Control Conference (ECC)*, pages 502–510, 2013. doi: 10.23919/ECC.2013.6669862.

[6] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger. Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3:269–296, 2020. doi: 10.1146/annurev-control-090419-075625.

[7] R. E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME – Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.

[8] B. Kouvaritakis and M. Cannon. *Model Predictive Control: Classical, Robust and Stochastic*. Springer, 2016. doi: 10.1007/978-3-319-24853-0.

[9] W. Langson, I. Chryssochoos, S. V. Raković, and D. Q. Mayne. Robust model predictive control using tubes. *Automatica*, 40(1):125–133, 2004. doi: 10.1016/j.automatica.2003.08.009.

[10] J. Löfberg. YALMIP: A toolbox for modeling and optimization in MATLAB. In *IEEE International Symposium on Computer Aided Control Systems Design (CACSD)*, pages 284–289, 2004. doi: 10.1109/CACSD.2004.1393890.

[11] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000. doi: 10.1016/S0005-1098(99)00214-9.

[12] D. Q. Mayne, M. M. Seron, and S. V. Raković. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005. doi: 10.1016/j.automatica.2004.08.019.

[13] F. Oldewurtel, A. Parisio, C. N. Jones, D. Gyalistras, M. Gwerder, V. Stauch, B. Lehmann, and M. Morari. Use of model predictive control and weather forecasts for energy efficient building climate control. *Energy and Buildings*, 45:15–27, 2012. doi: 10.1016/j.enbuild.2011.09.022.

[14] G. Pannocchia and J. B. Rawlings. Disturbance models for offset-free model-predictive control. *AIChE Journal*, 49(2):426–437, 2003. doi: 10.1002/aic.690490213.

[15] S. J. Qin and T. A. Badgwell. A survey of industrial model predictive control technology. *Control Engineering Practice*, 11(7):733–764, 2003. doi: 10.1016/S0967-0661(02)00186-7.

[16] S. V. Raković, E. C. Kerrigan, K. I. Kouramas, and D. Q. Mayne. Invariant approximations of the minimal robust positively invariant set. *IEEE Transactions on Automatic Control*, 50(3): 406–410, 2005. doi: 10.1109/TAC.2005.843854.

[17] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2nd edition, 2017.